HELMHOLTZ AI

Transformers Workshop

Helmholtz AI Consultants @ HEITEC, IHRS BioSoft, BIF-IGS & HIDSS4Health

26.03.2026

Agenda

09:00 - 10:30	Introduction to Language Modeling
10:30 - 10:45	Break
10:45 - 11:45	Blablador: concept, models description. Optional: integration in VScode
11:45 - 12:00	Q&A
12:00 - 13:00	Lunch Break
13:00 - 14:00	Attention, self-attention, transformer architecture
14:00 - 14:15	Pre-training vs fine-tuning and foundation models (intro to the hands-on session)
14:15 - 14:30	Break
14:30 - 15:30	Hands-on: fine-tune a model on a downstream task
15:30 - 16:00	Q&A, Wrap-up, and Conclusion

Outline

1. Overview
2. Preprocessing
3. Attention mechanism
4. Putting it all together
5. Variants of Transformer

Transformer

- NeurIPS 2017
- Originally for machine translation
  - Nowadays, also for many more things
    - Chatbots
    - Vision
    - Etc.

Attention Is All You Need

Ashish Vaswani* Google Brain avaswani@google.com

Noam Shazeer* Google Brain noam@google.com

Niki Parmar* Google Research nikip@google.com

Jakob Uszkoreit* Google Research usz@google.com

Llion Jones* Google Research llion@google.com

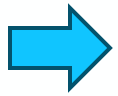
Aidan N. Gomez* † University of Toronto aidan@cs.toronto.edu

Łukasz Kaiser* Google Brain lukaszkaiser@google.com

Illia Polosukhin* ‡ illia.polosukhin@gmail.com

Our example

- German-English translation

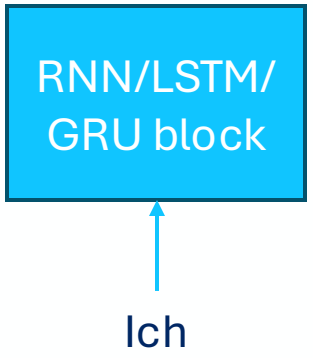
”Ich habe Hunger”  ”I am hungry”

Before Transformer

- Sequential models (RNNs/LSTMs/GRUs)

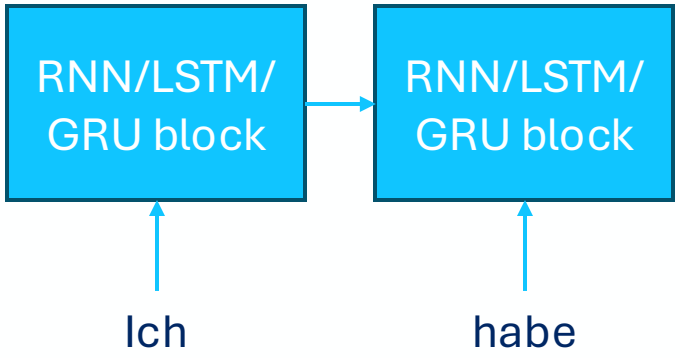
Before Transformer

- Sequential models (RNNs/LSTMs/GRUs)



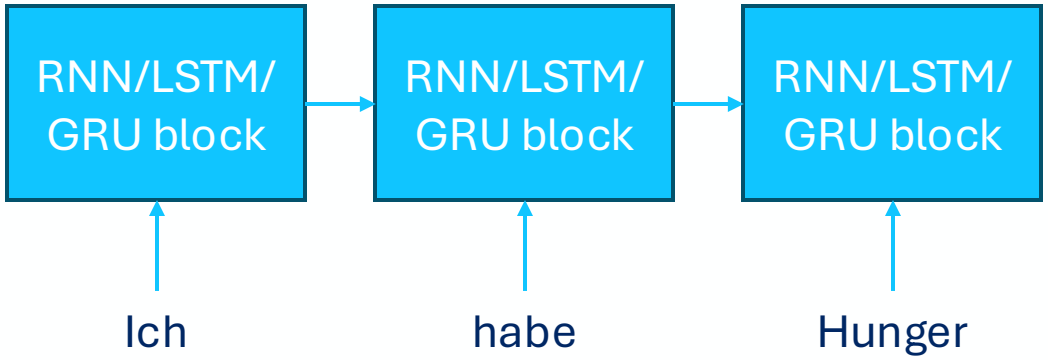
Before Transformer

- Sequential models (RNNs/LSTMs/GRUs)



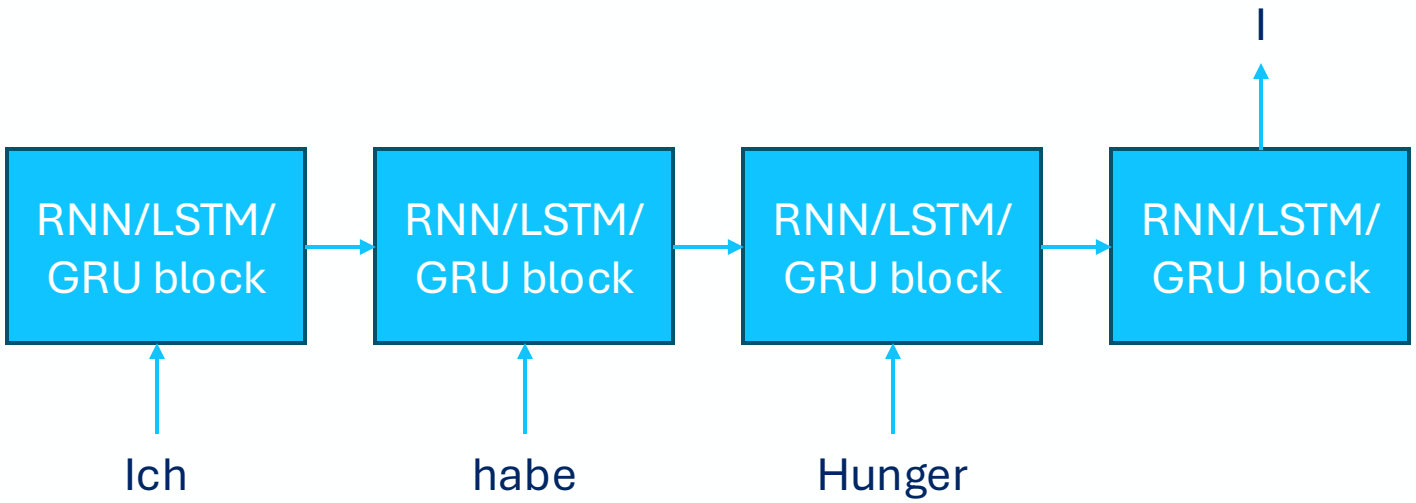
Before Transformer

- Sequential models (RNNs/LSTMs/GRUs)



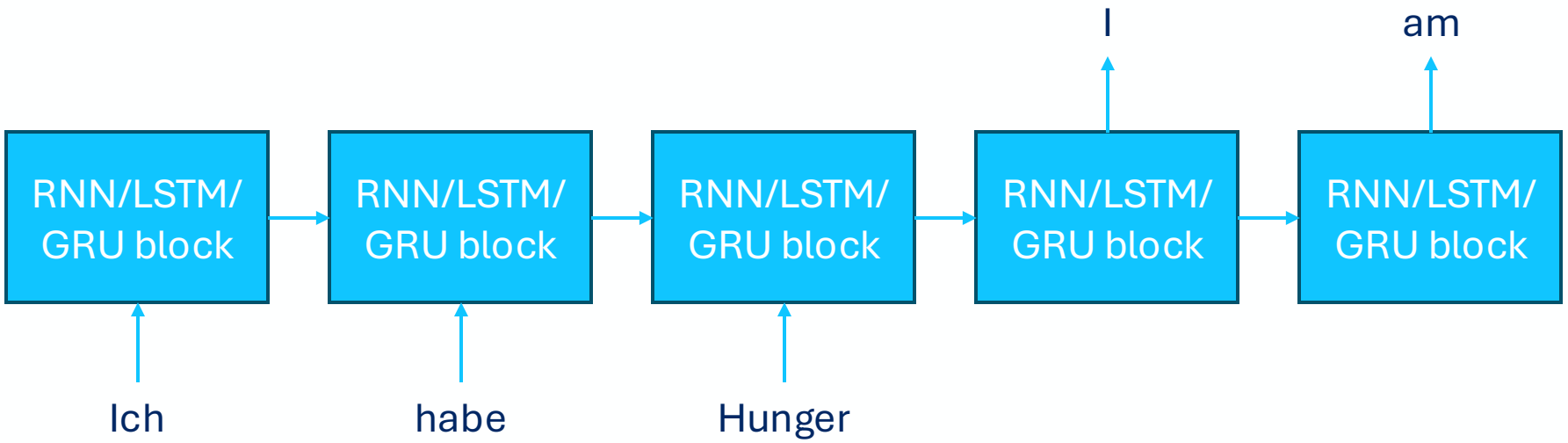
Before Transformer

- Sequential models (RNNs/LSTMs/GRUs)



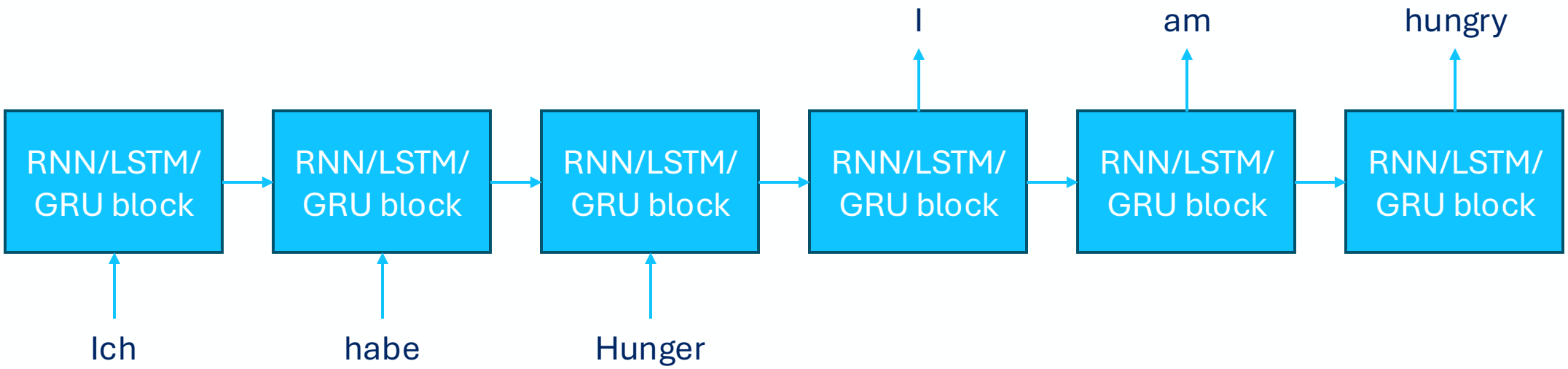
Before Transformer

- Sequential models (RNNs/LSTMs/GRUs)



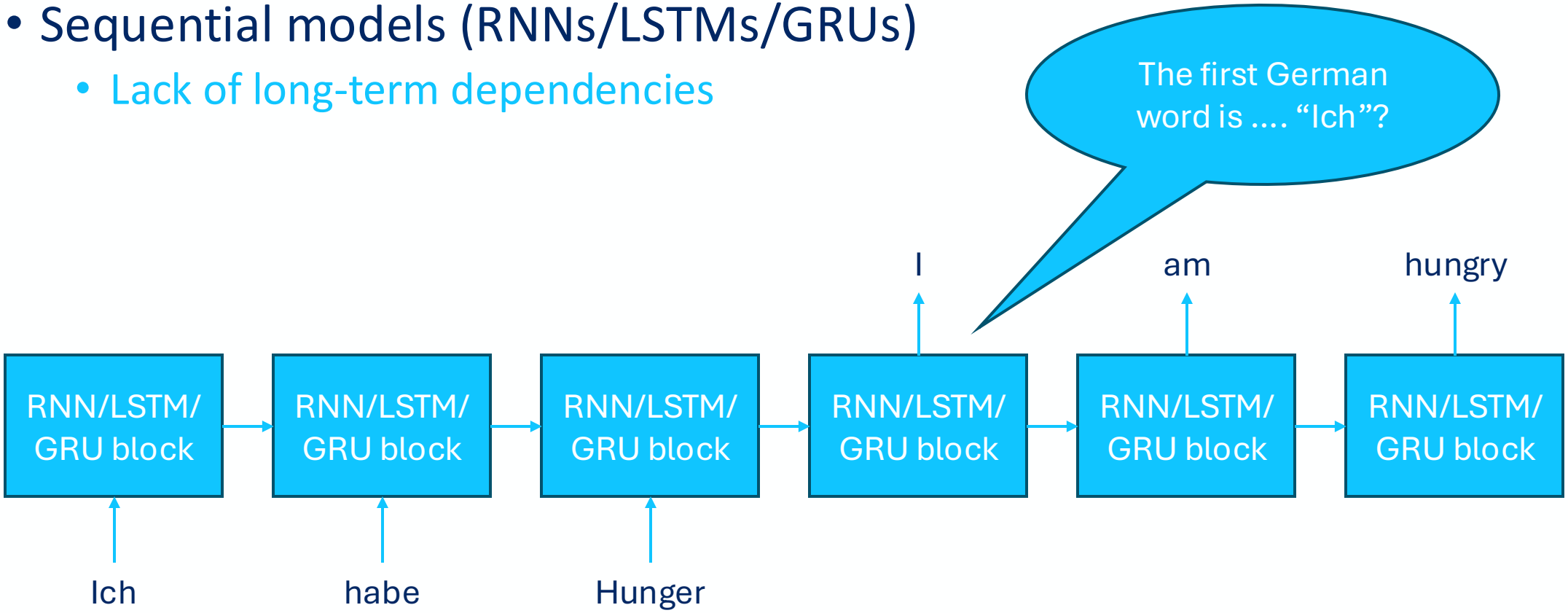
Before Transformer

- Sequential models (RNNs/LSTMs/GRUs)



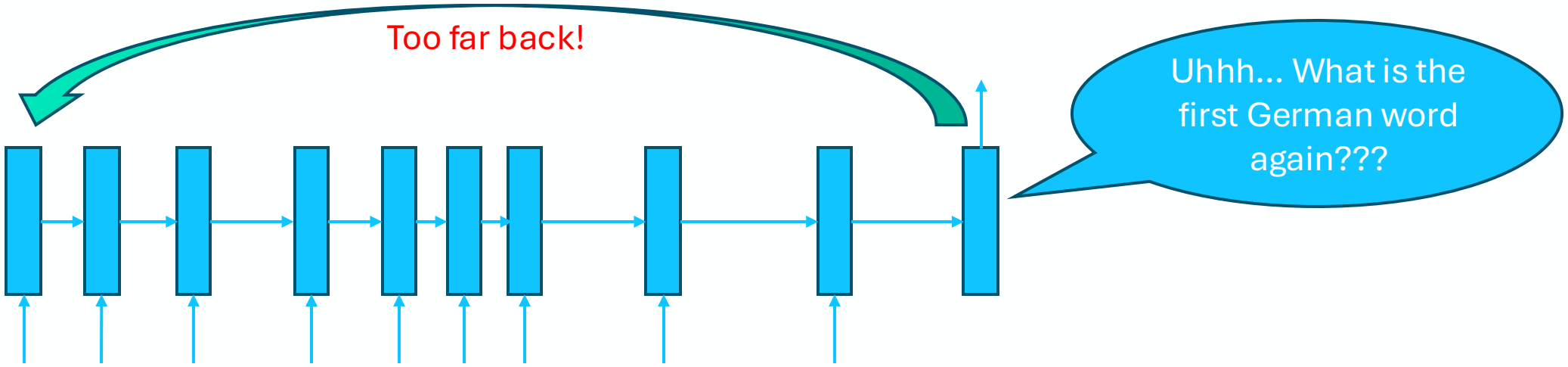
Before Transformer

- Sequential models (RNNs/LSTMs/GRUs)
  - Lack of long-term dependencies



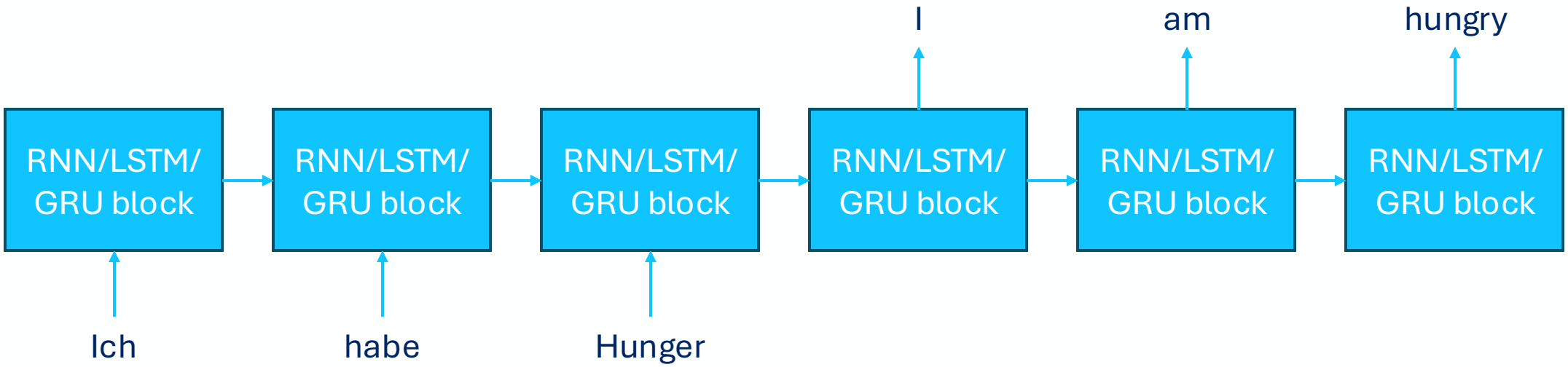
Before Transformer

- Sequential models (RNNs/LSTMs/GRUs)
  - Lack of long-term dependencies
    - Especially when the input is long



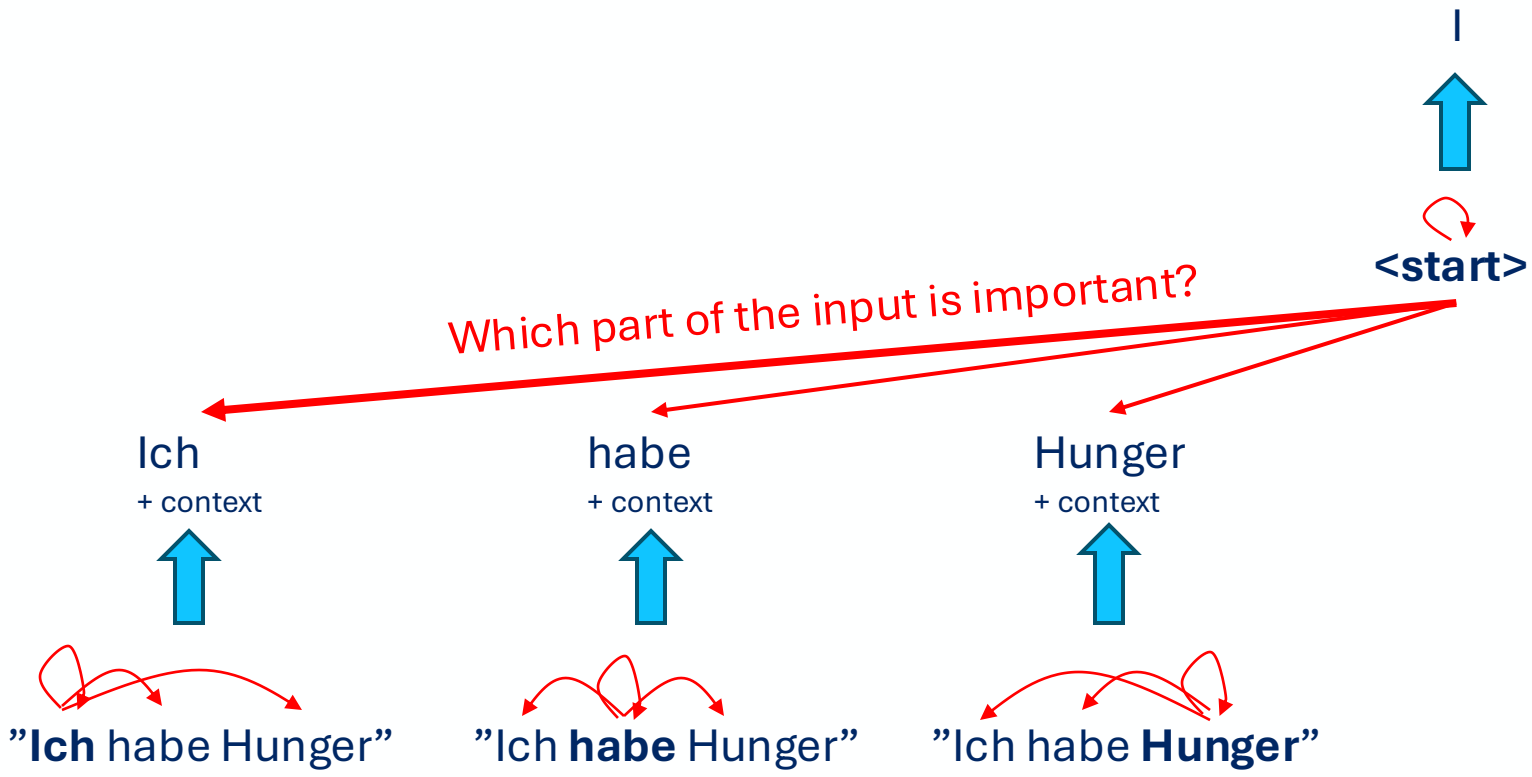
Before Transformer

- Sequential models (RNNs/LSTMs/GRUs)
  - Lack of long-term dependencies
  - Training is slow → sequential, no parallel



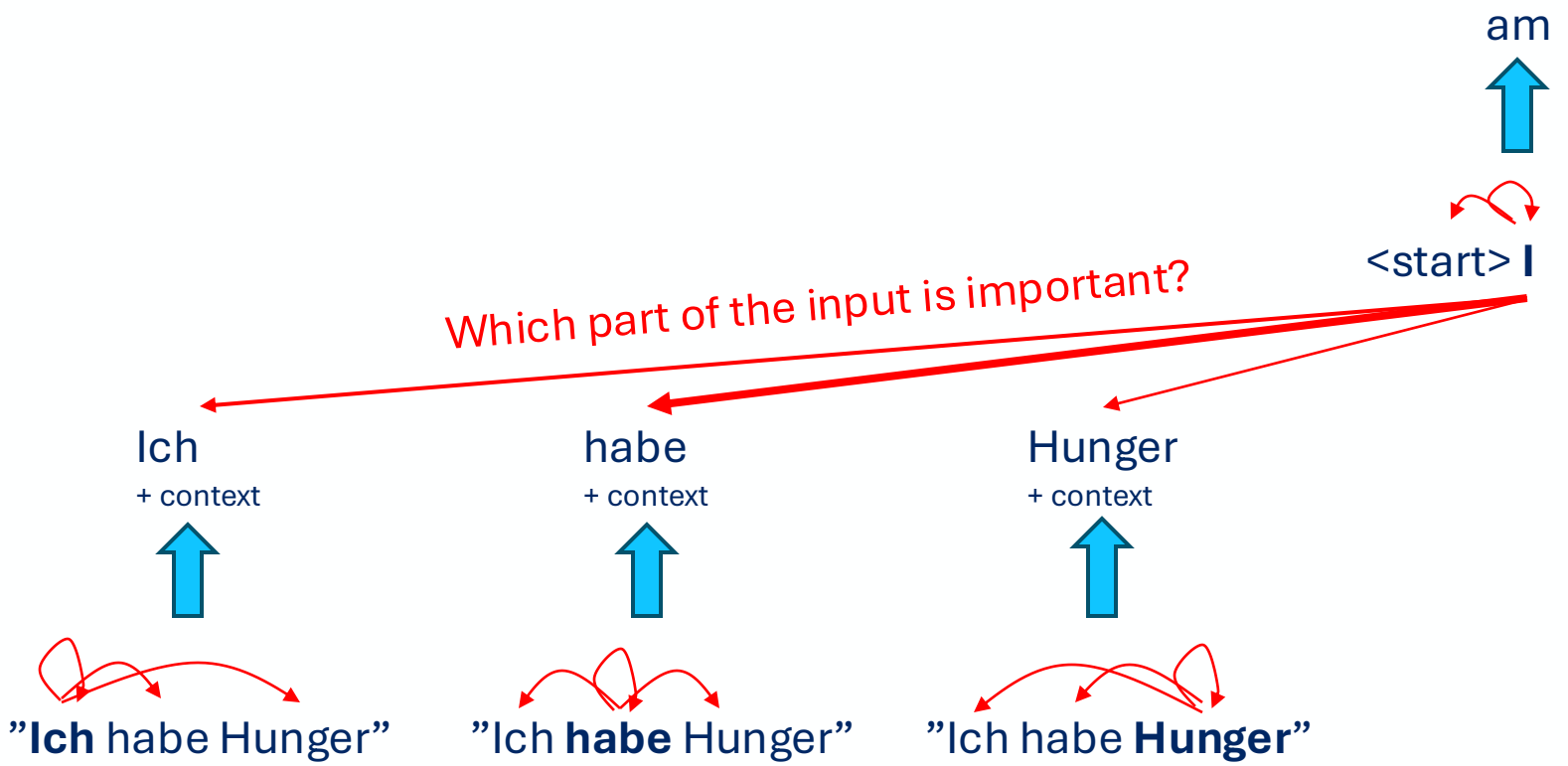
Transformer

- Attention mechanism → Transformer



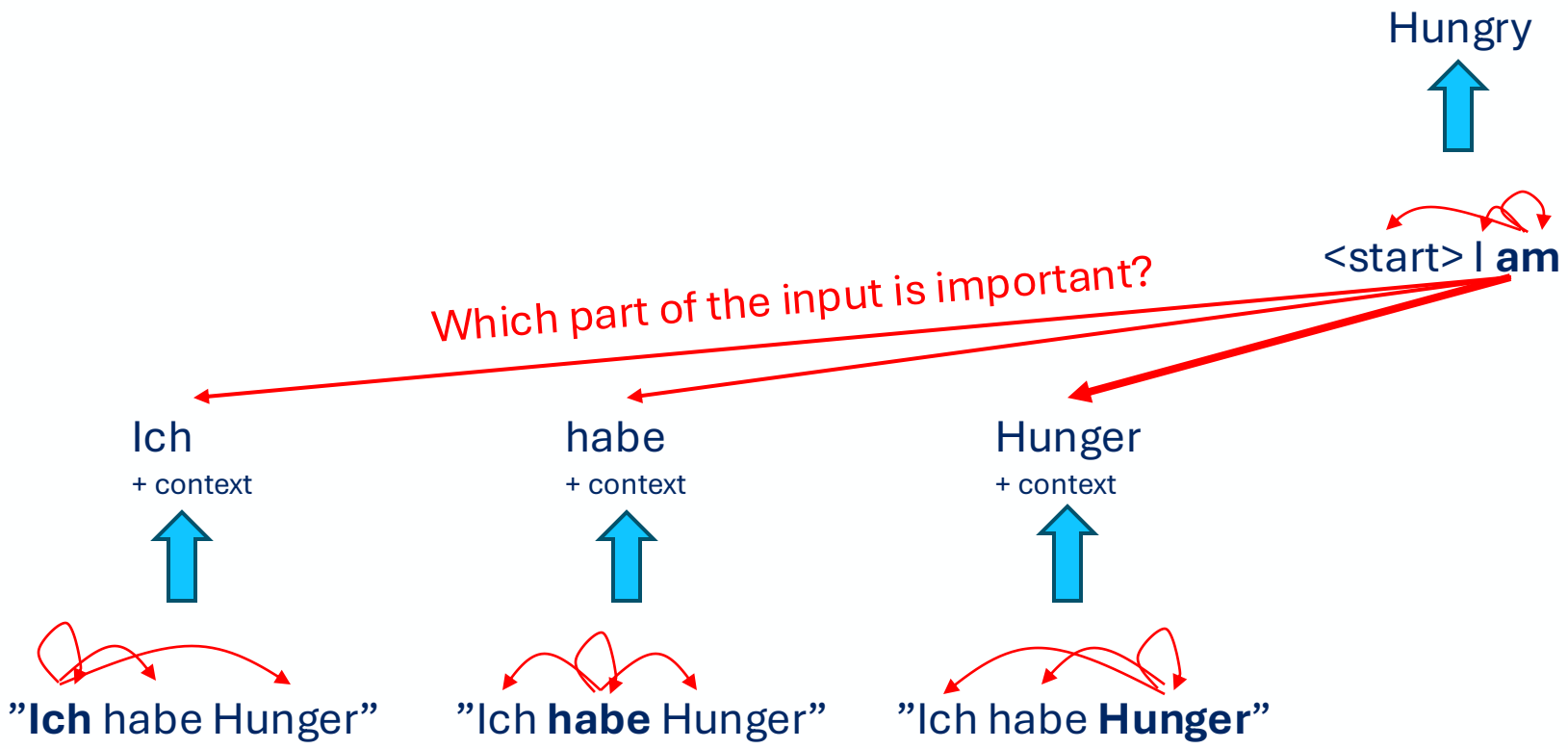
Transformer

- Attention mechanism → Transformer



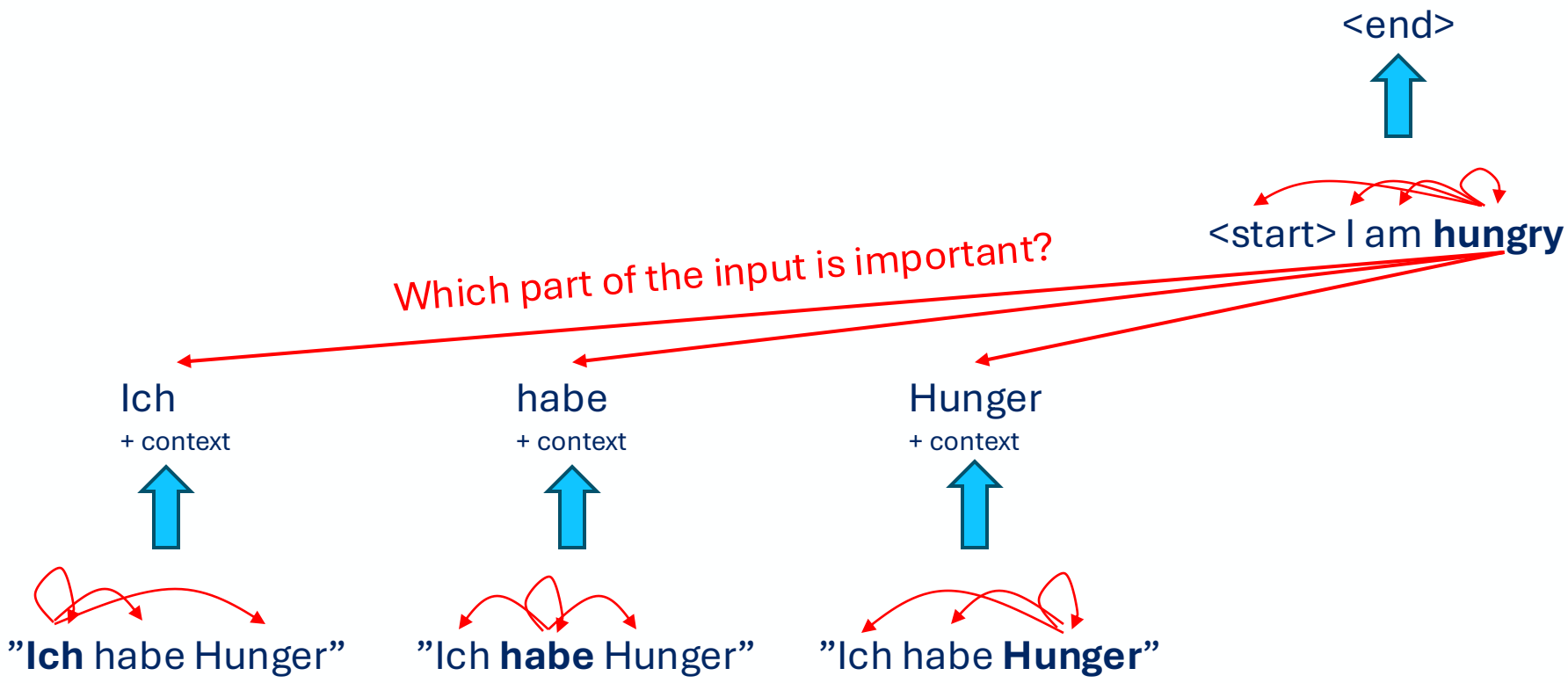
Transformer

- Attention mechanism → Transformer



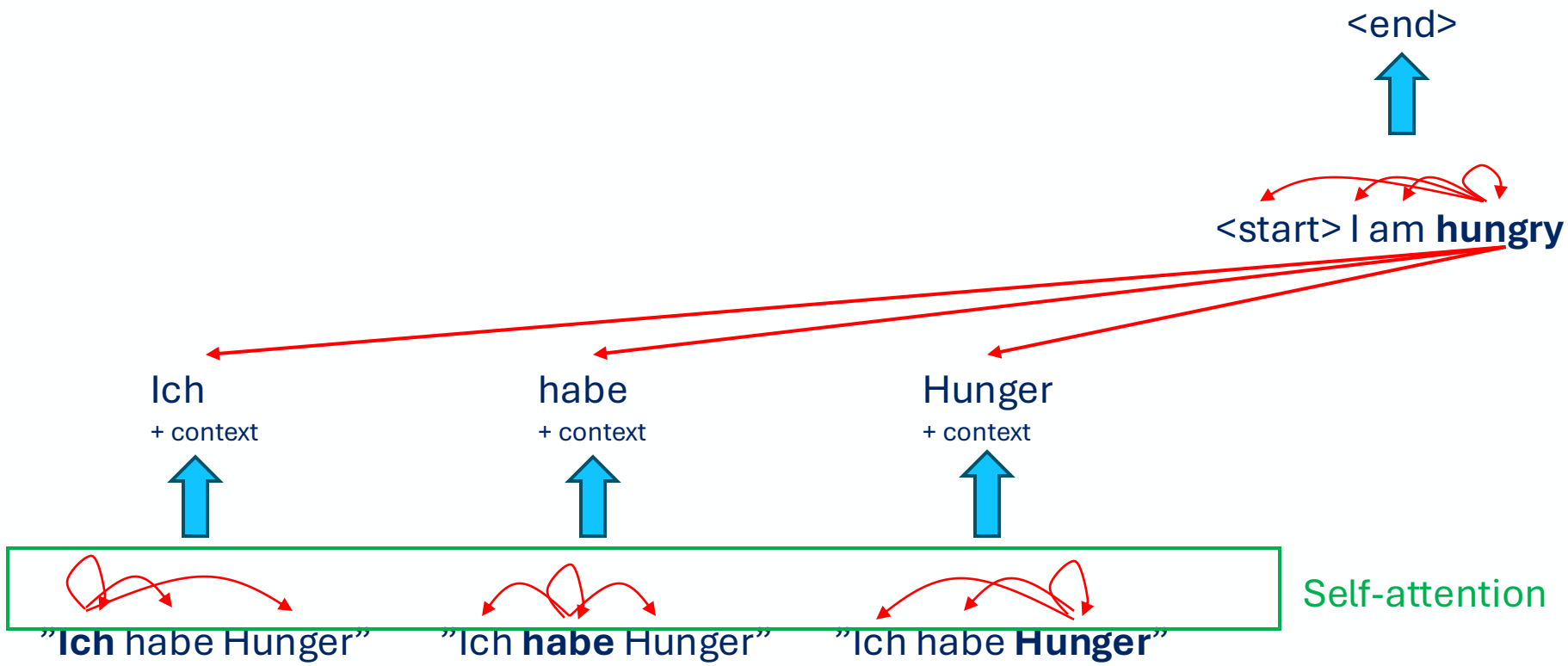
Transformer

- Attention mechanism → Transformer



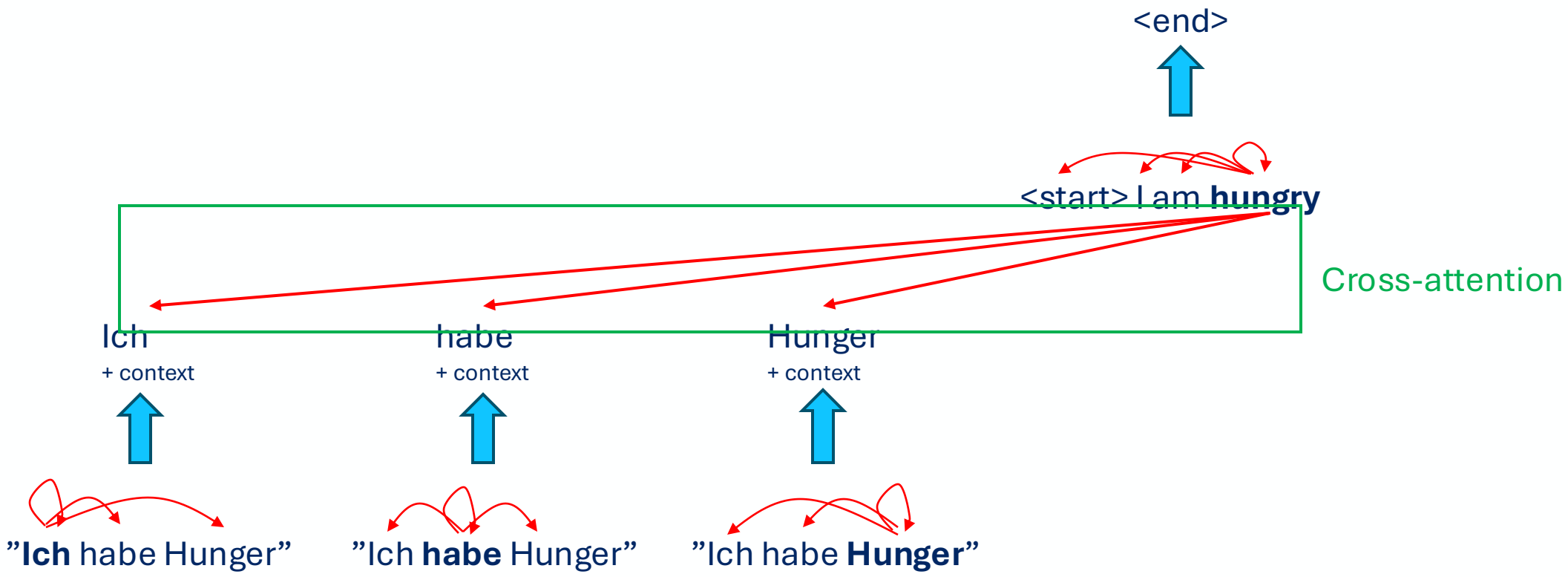
Transformer

- Attention mechanism → Transformer



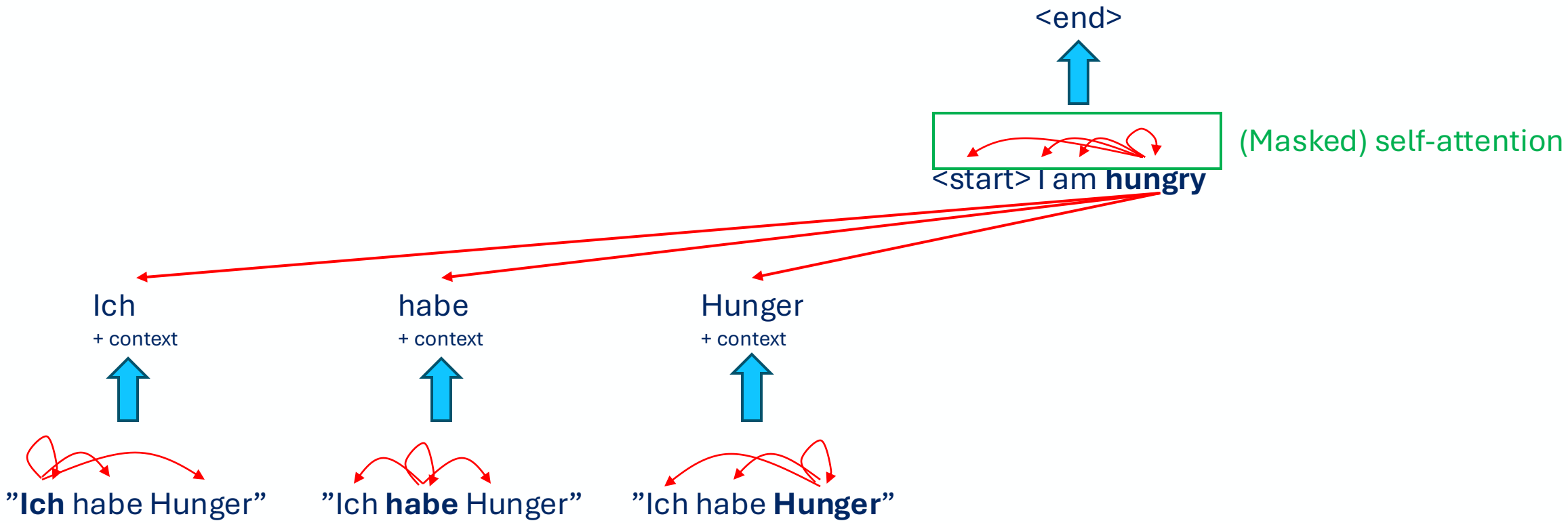
Transformer

- Attention mechanism → Transformer



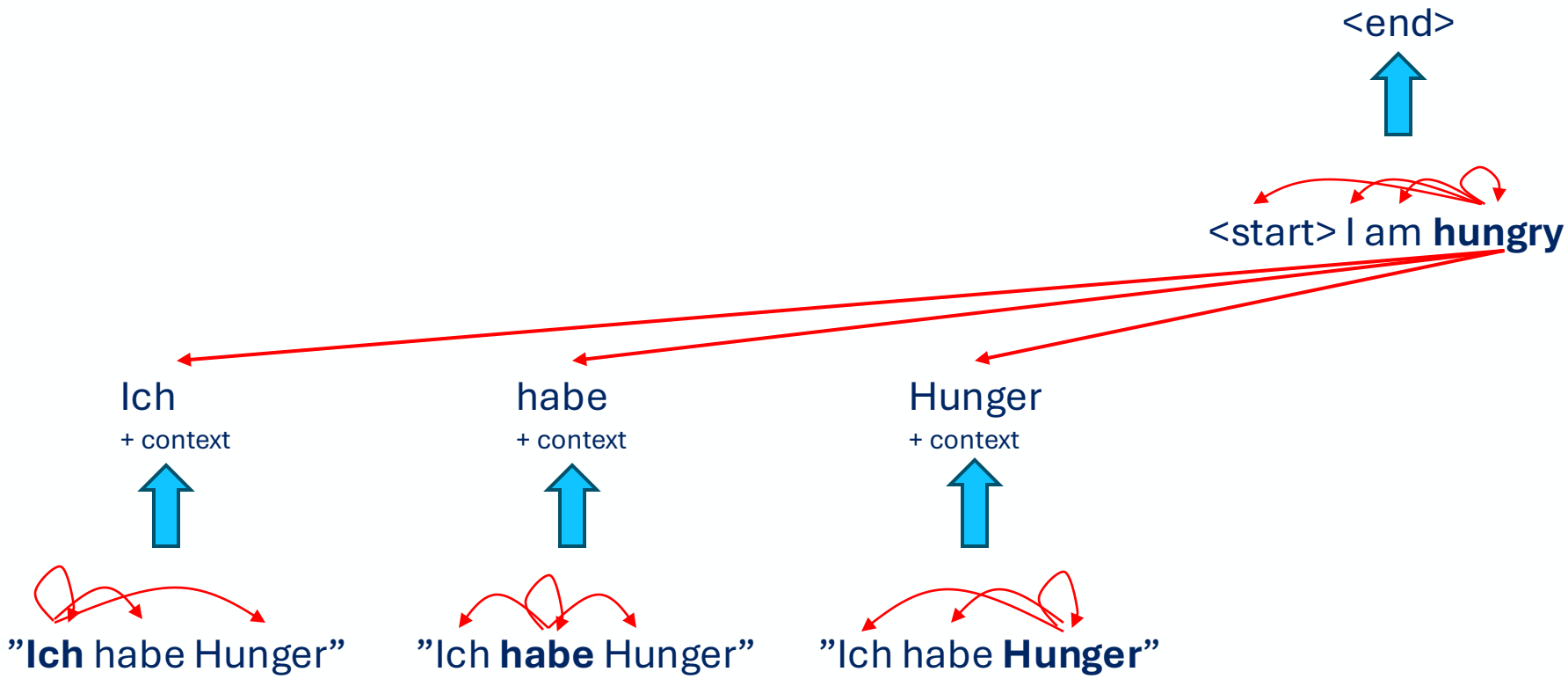
Transformer

- Attention mechanism → Transformer



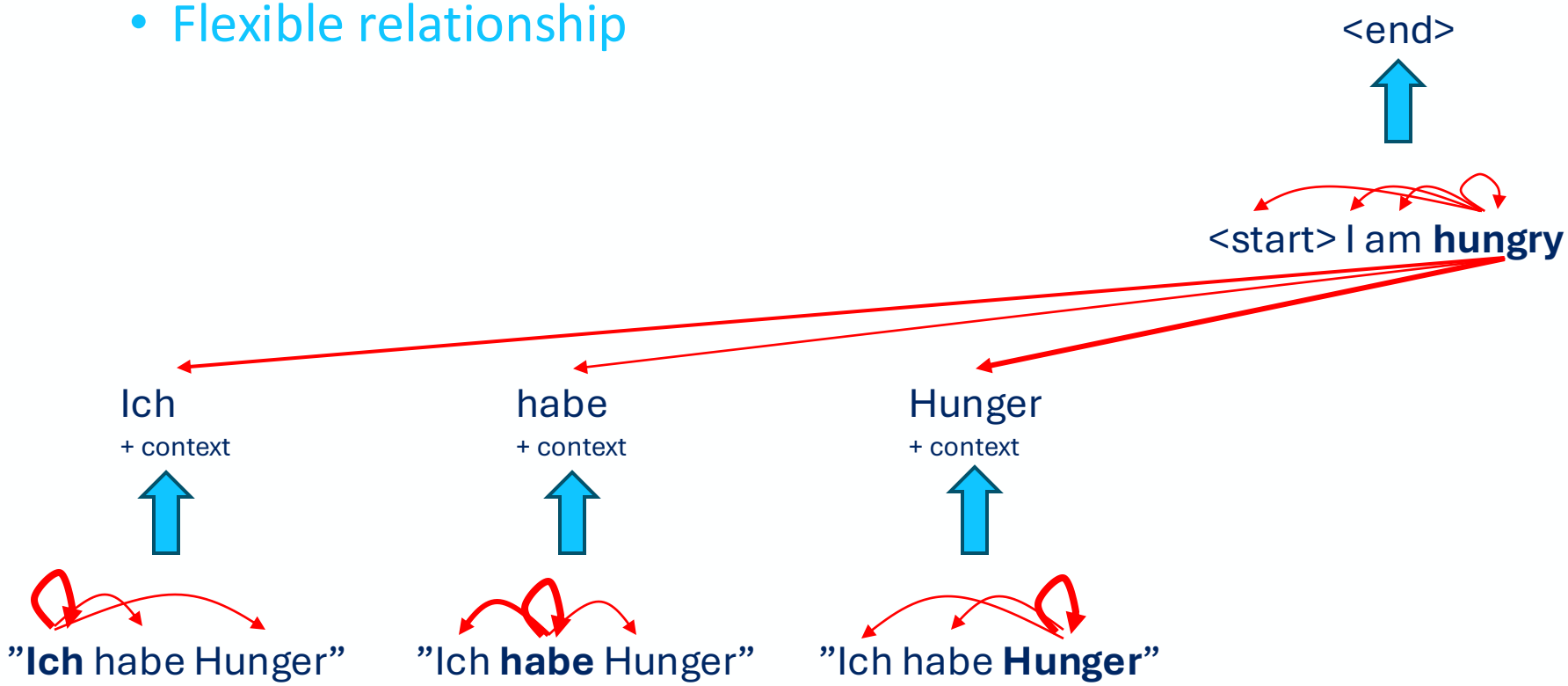
Transformer

- Attention mechanism → Transformer
  - Extract global view



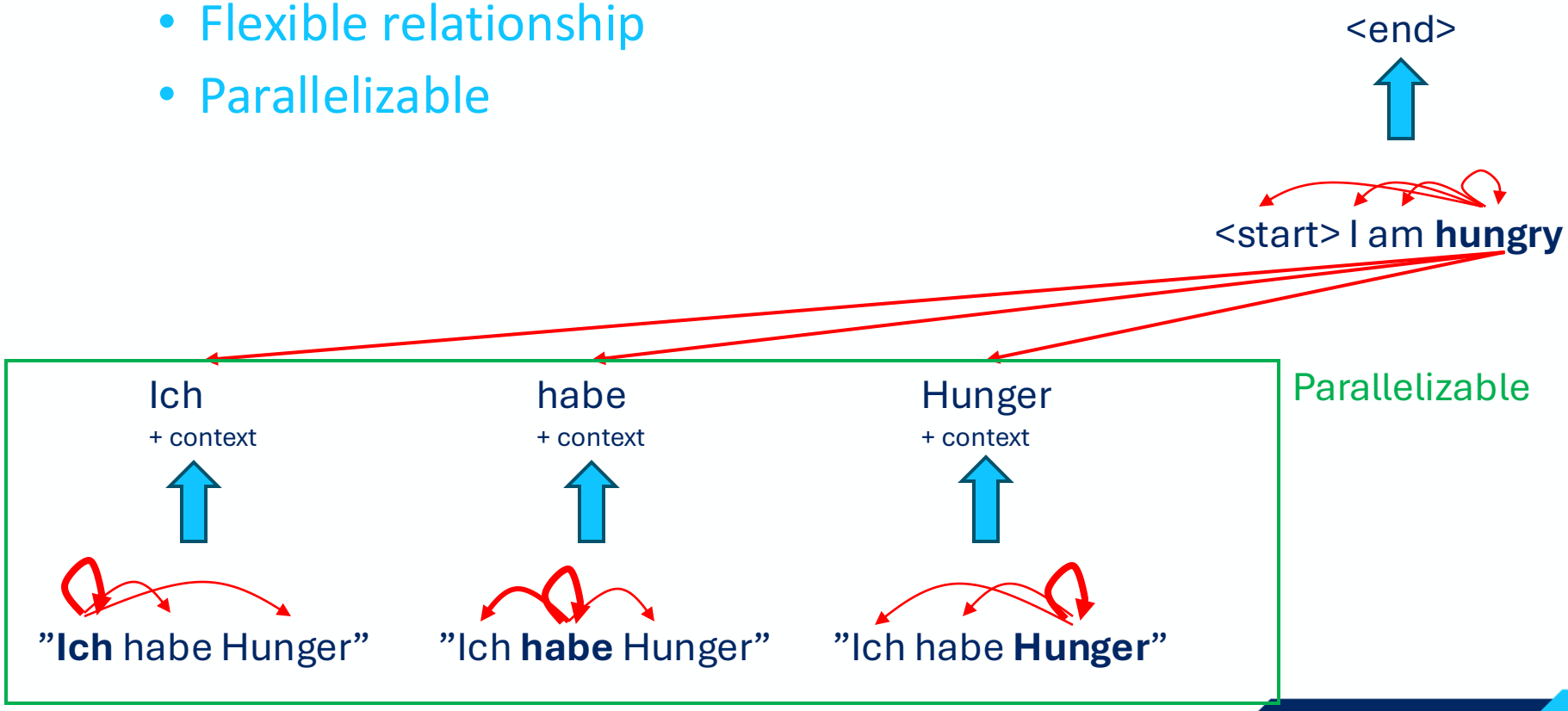
Transformer

- Attention mechanism → Transformer
  - Extract global view
  - Flexible relationship



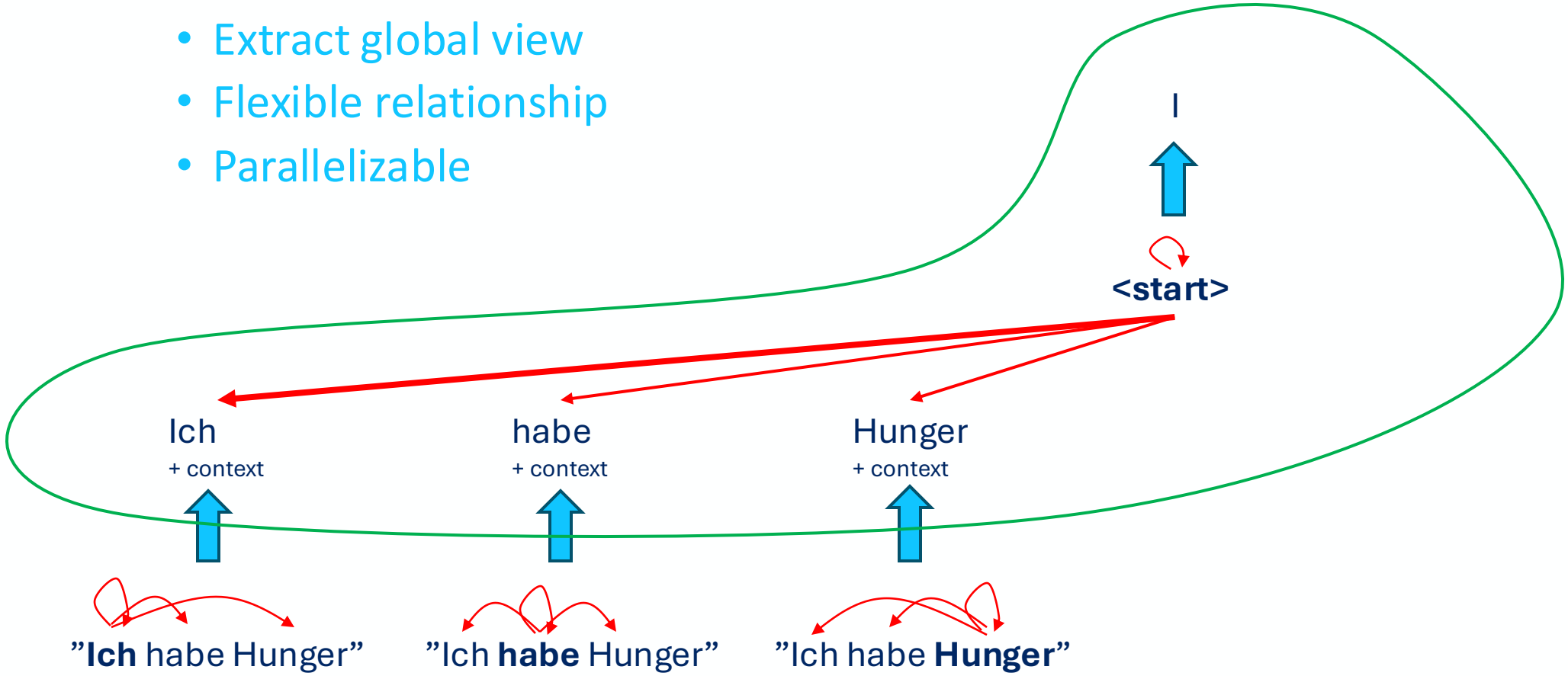
Transformer

- Attention mechanism → Transformer
  - Extract global view
  - Flexible relationship
  - Parallelizable



Transformer

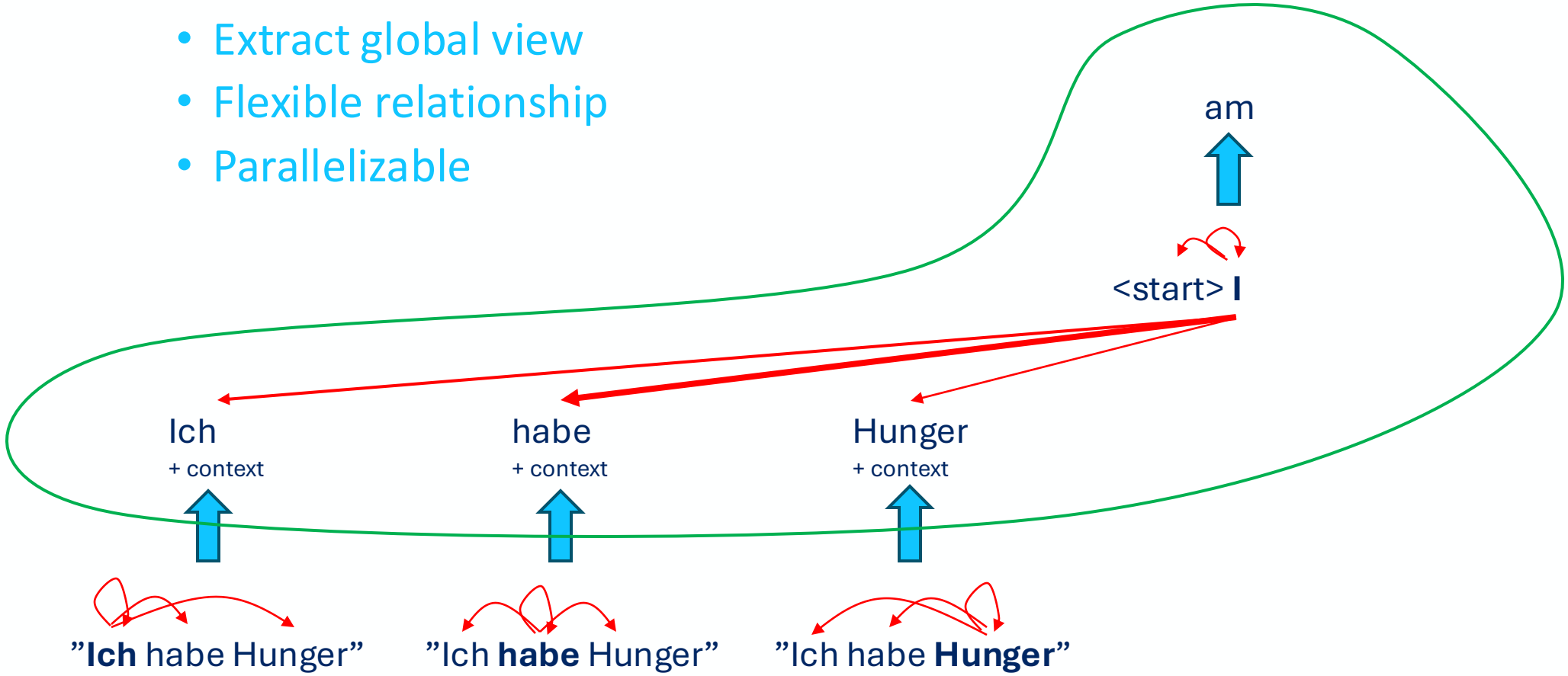
- Attention mechanism → Transformer
  - Extract global view
  - Flexible relationship
  - Parallelizable



This does not look parallelizable???
→ **masked** self attention trick (during training)

Transformer

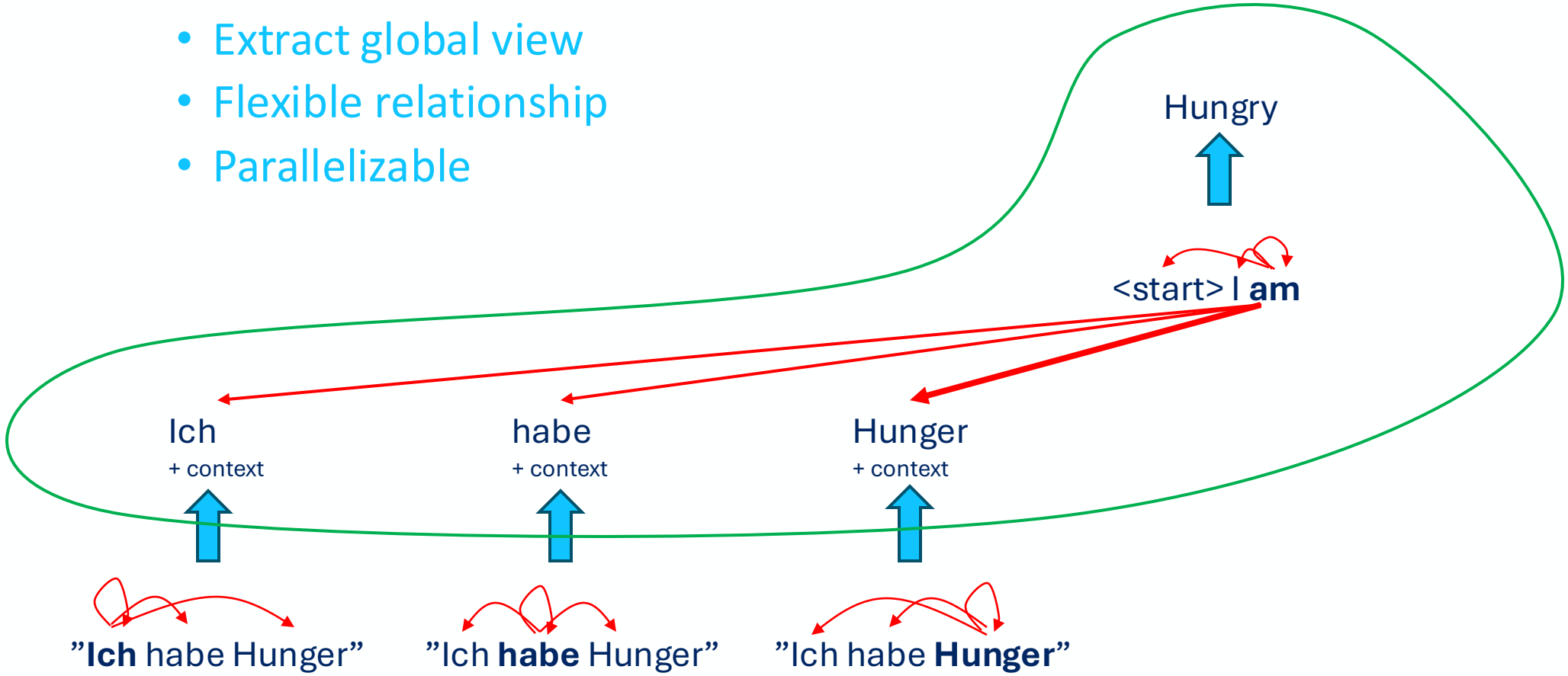
- Attention mechanism → Transformer
  - Extract global view
  - Flexible relationship
  - Parallelizable



This does not look parallelizable???
→ **masked** self attention trick (during training)

Transformer

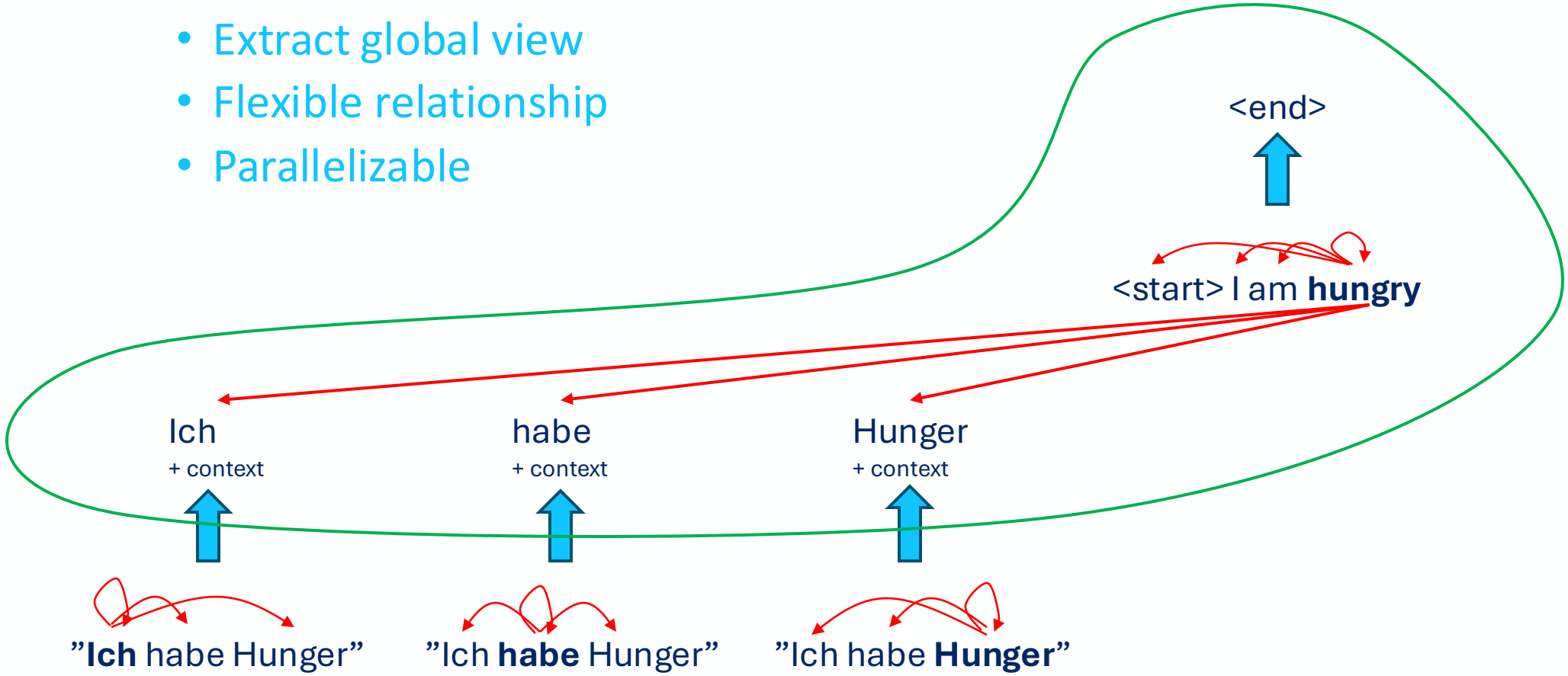
- Attention mechanism → Transformer
  - Extract global view
  - Flexible relationship
  - Parallelizable



This does not look parallelizable???
→ **masked** self attention trick (during training)

Transformer

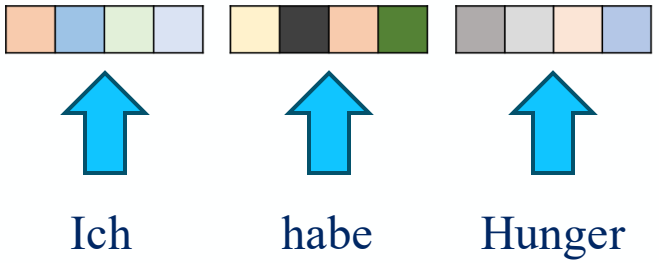
- Attention mechanism → Transformer
  - Extract global view
  - Flexible relationship
  - Parallelizable



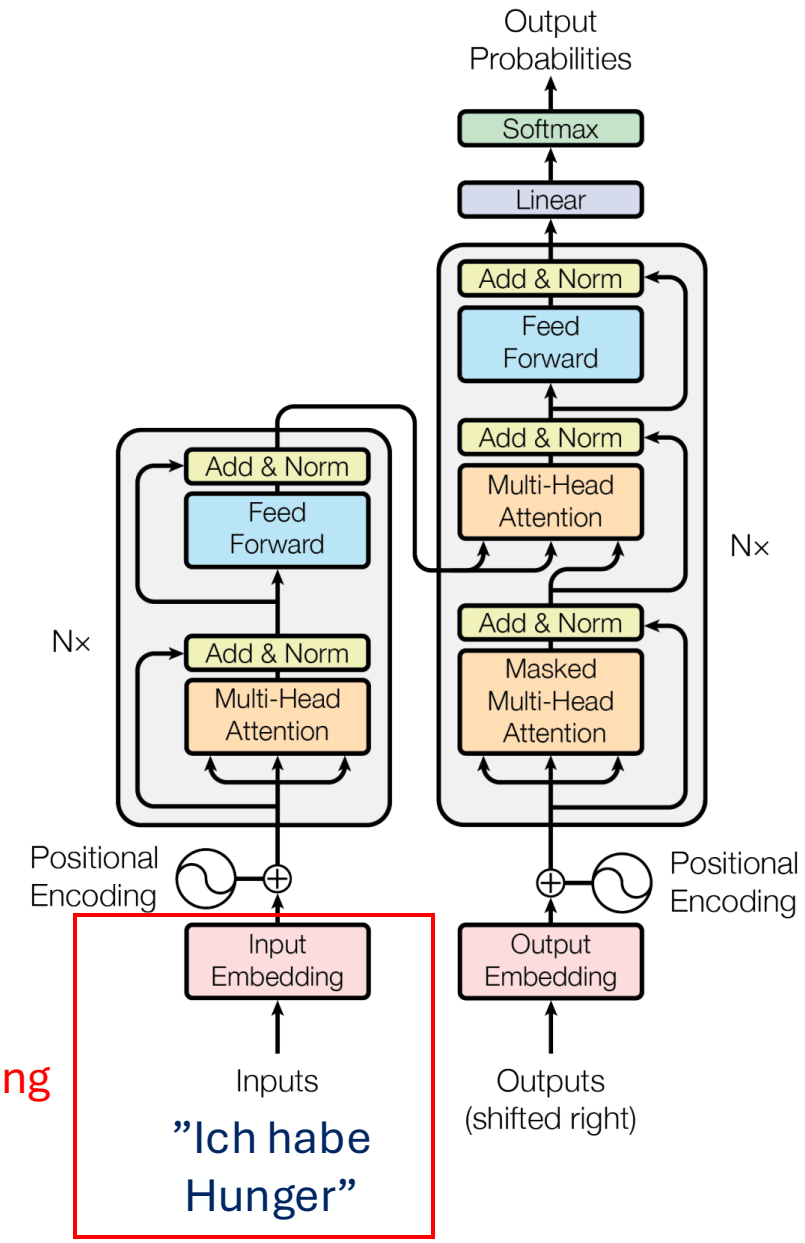
This does not look parallelizable???
→ **masked** self attention trick (during training)

Transformer

- Components:
  - Word embedding

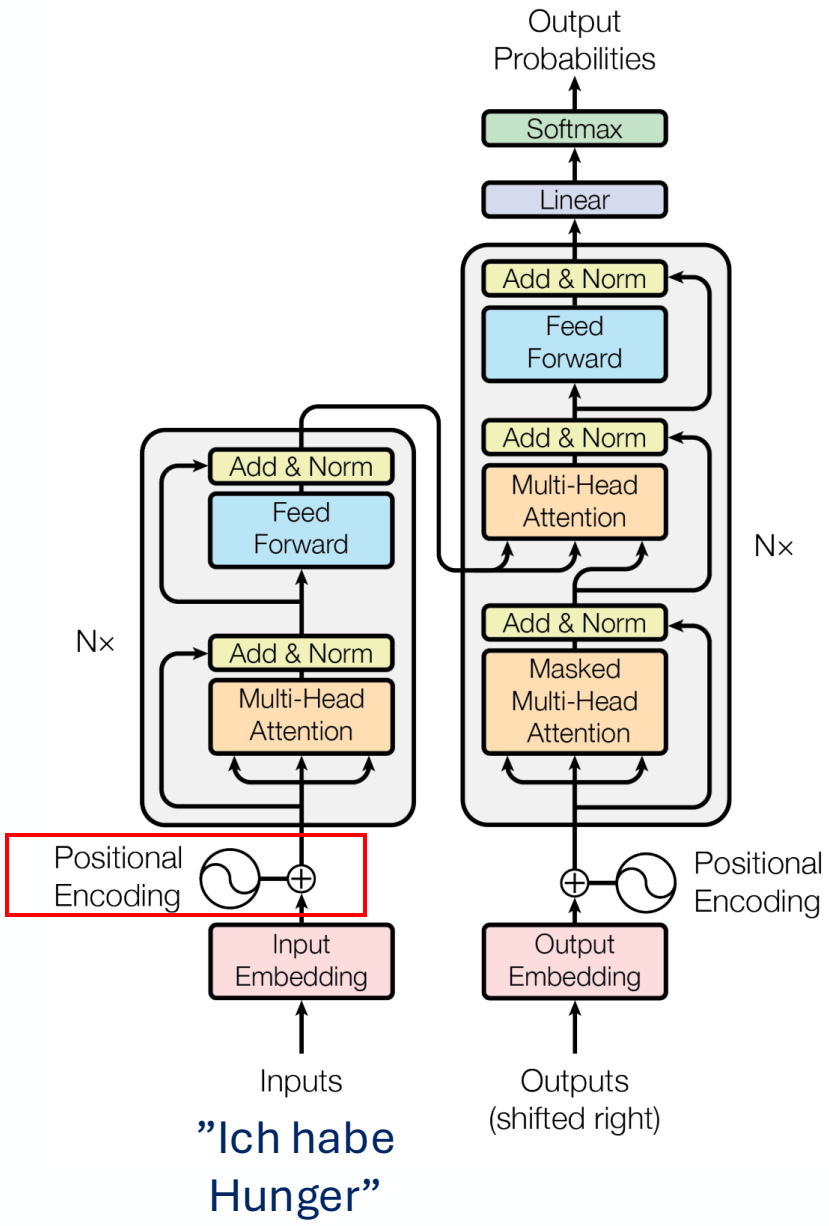


Word embedding



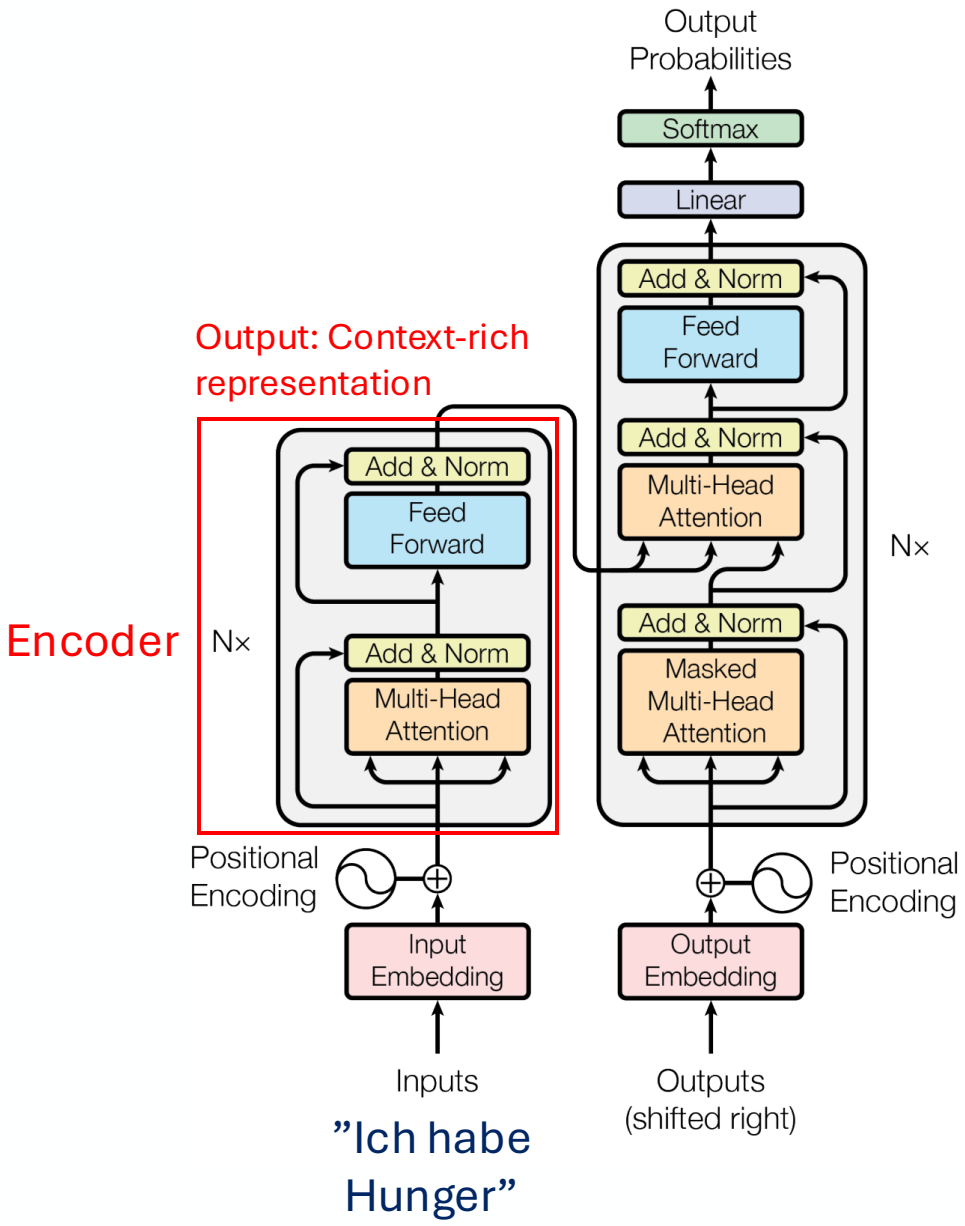
Transformer

- Components:
  - Word embedding
  - Positional encoding



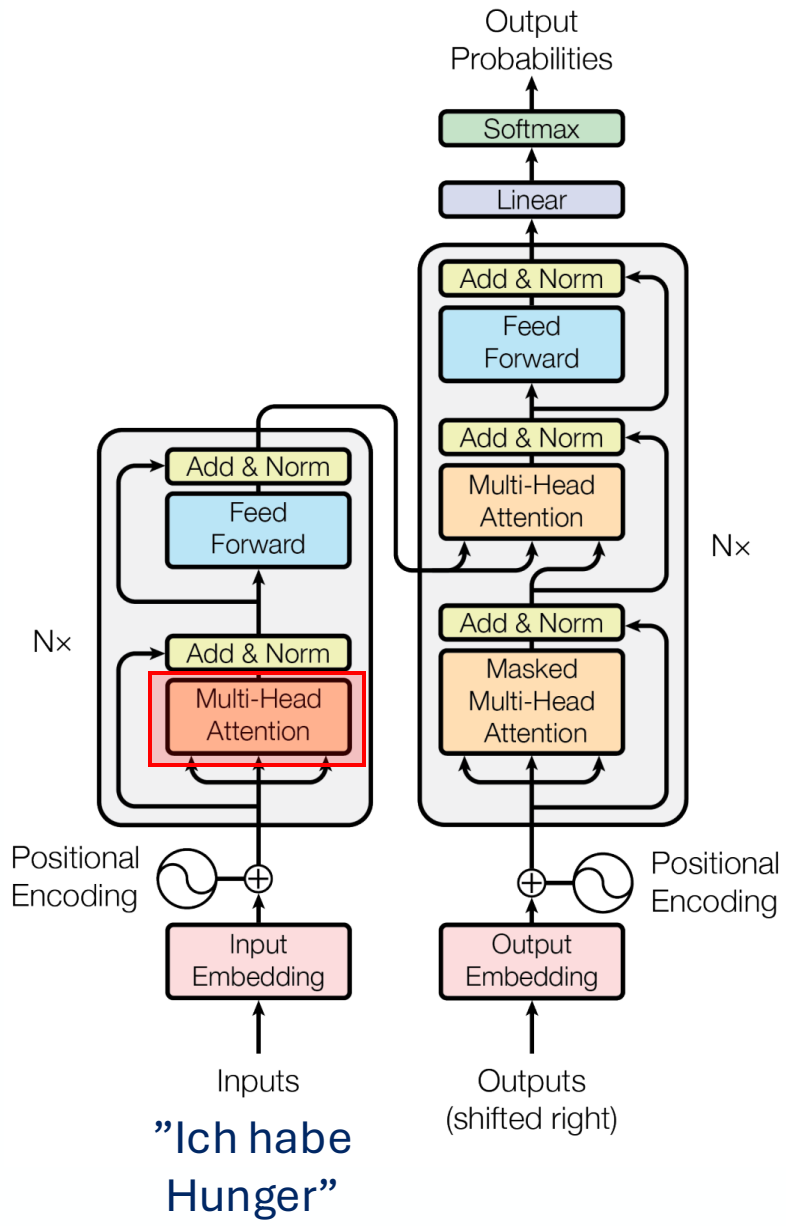
Transformer

- Components:
  - Word embedding
  - Positional encoding
  - Encoder



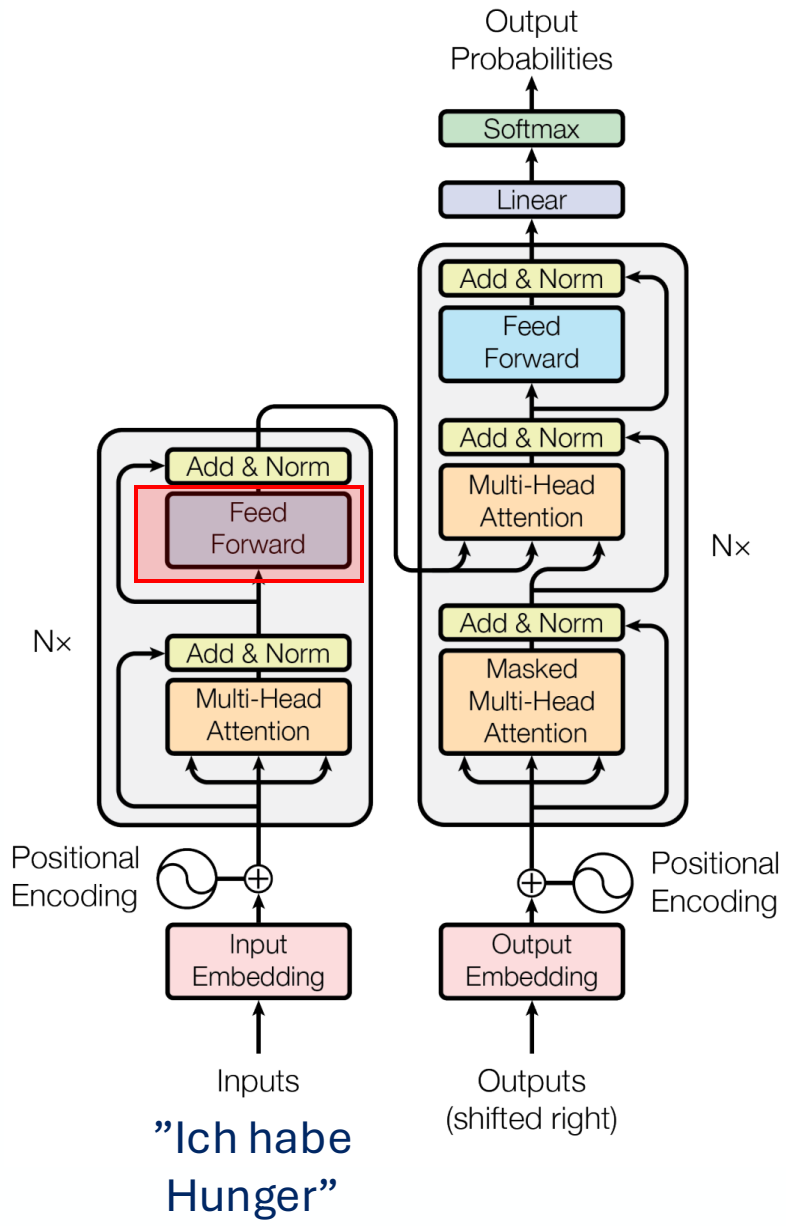
Transformer

- Components:
  - Word embedding
  - Positional encoding
  - Encoder
    - Multi-head self-attention



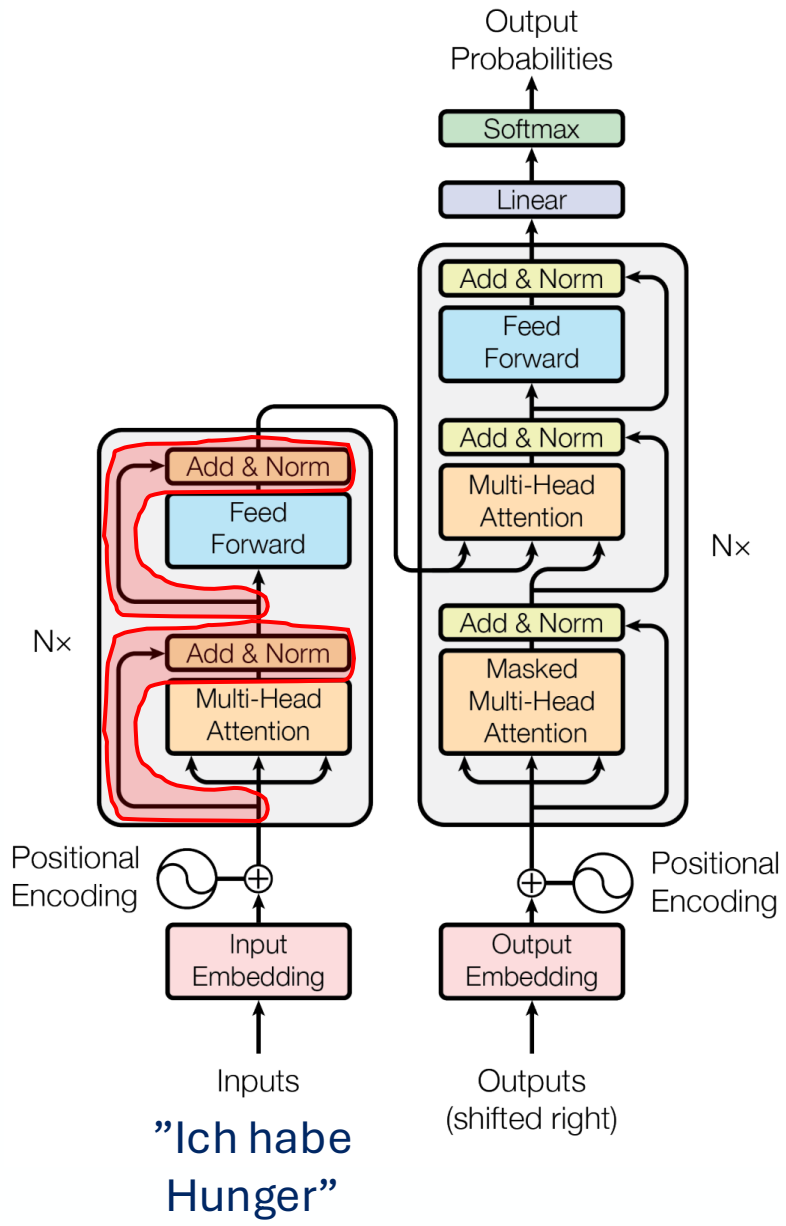
Transformer

- Components:
  - Word embedding
  - Positional encoding
  - Encoder
    - Multi-head self-attention
    - Fully-connected layer



Transformer

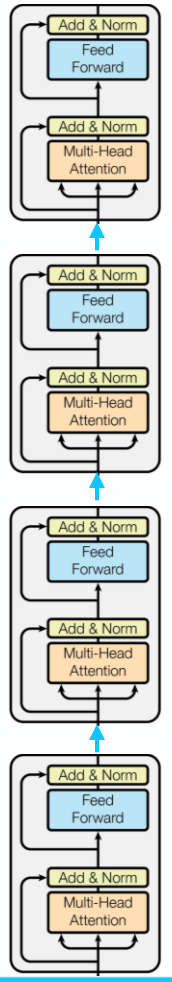
- Components:
  - Word embedding
  - Positional encoding
  - Encoder
    - Multi-head self-attention
    - Fully-connected layer
    - Skip connection with normalization

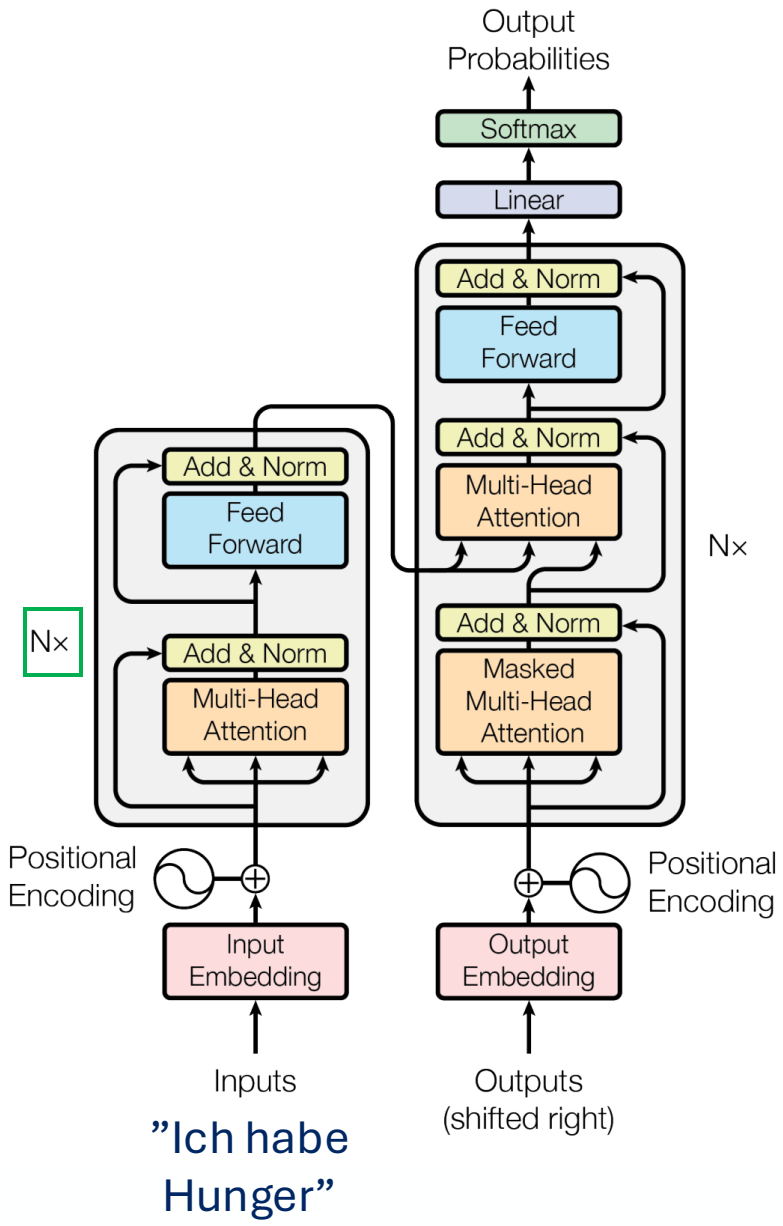


Transformer

- Components:
  - Word embedding
  - Positional encoding
  - Encoder
    - N blocks

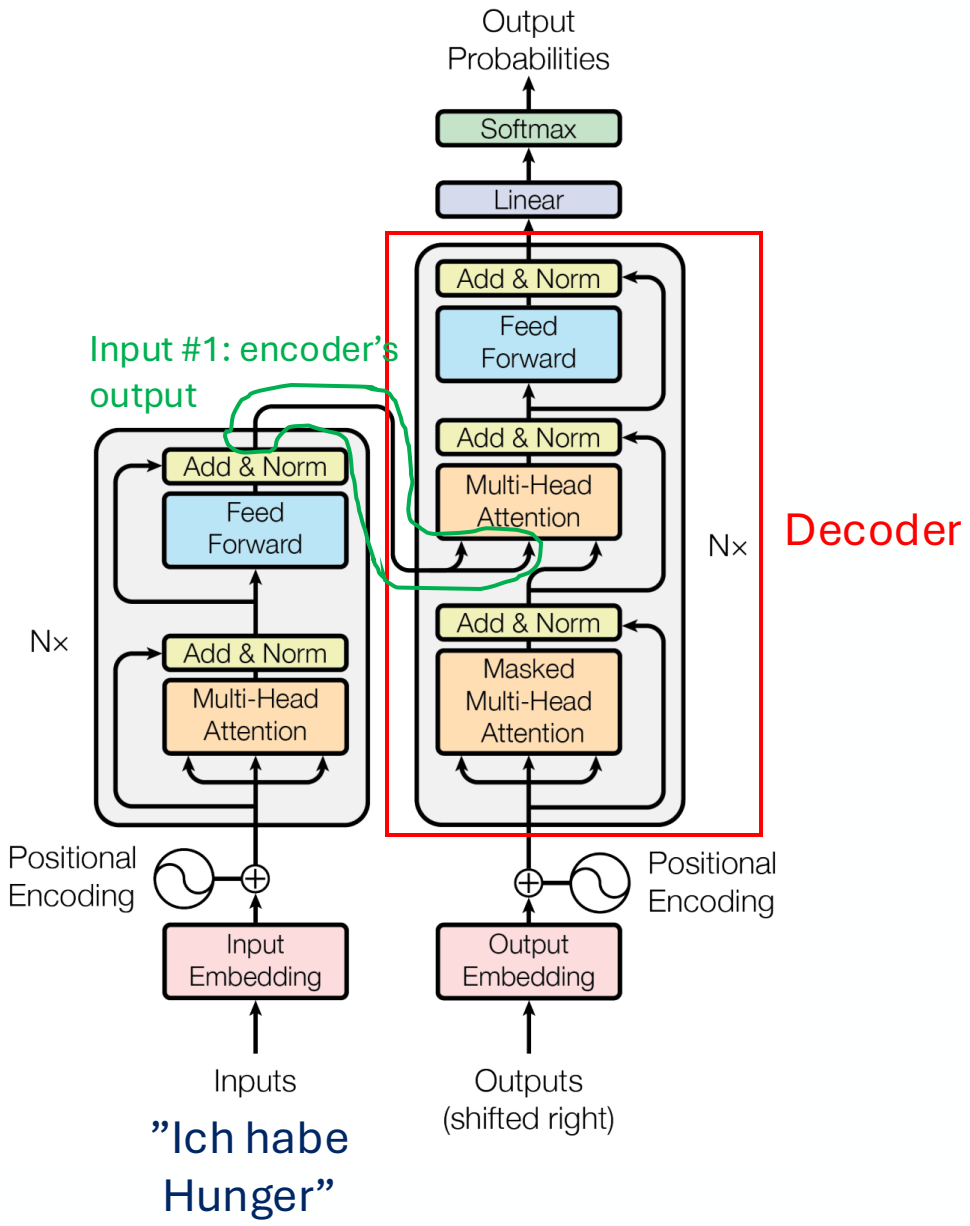
N=4 looks like





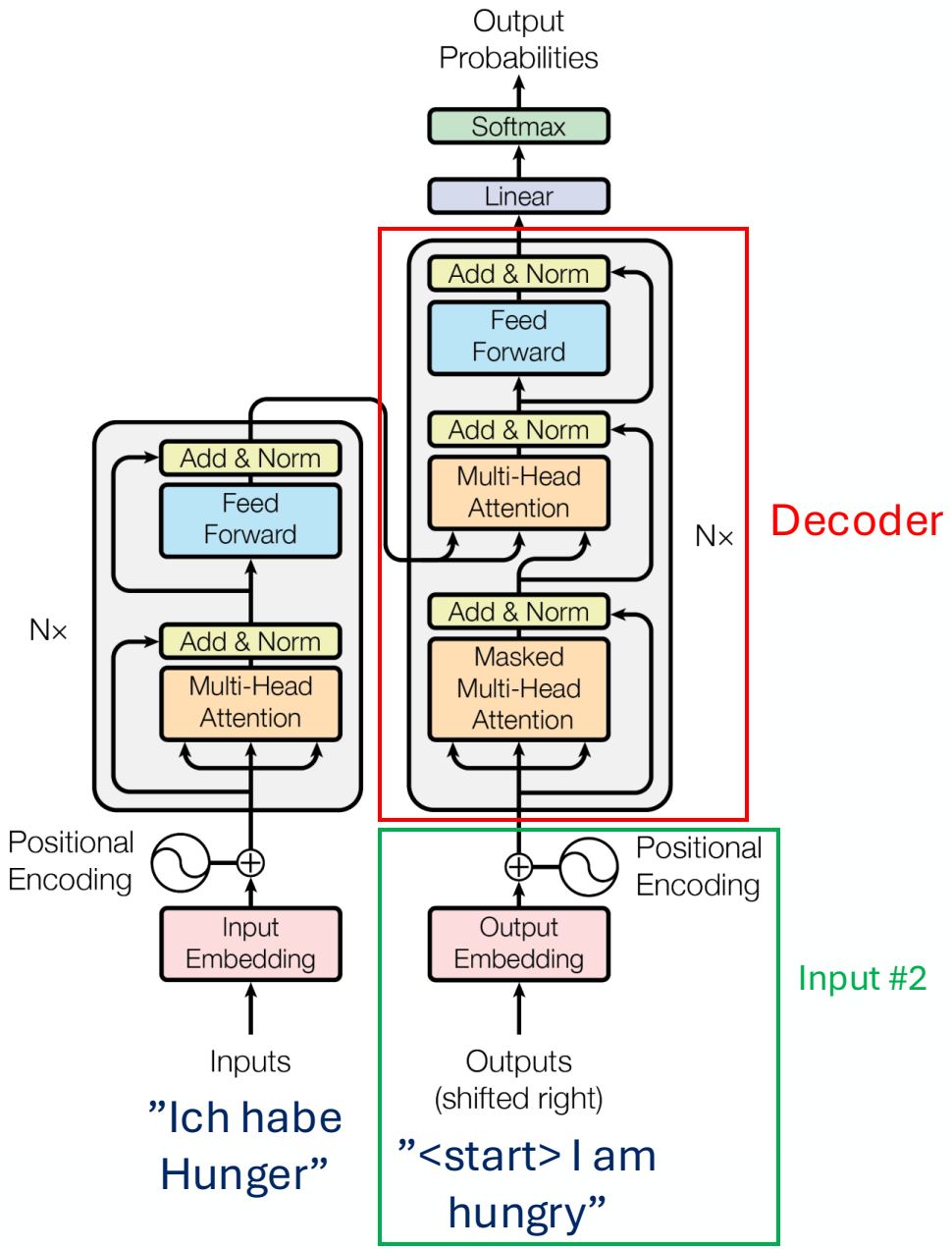
Transformer

- Components:
  - Word embedding
  - Positional encoding
  - Encoder
  - Decoder



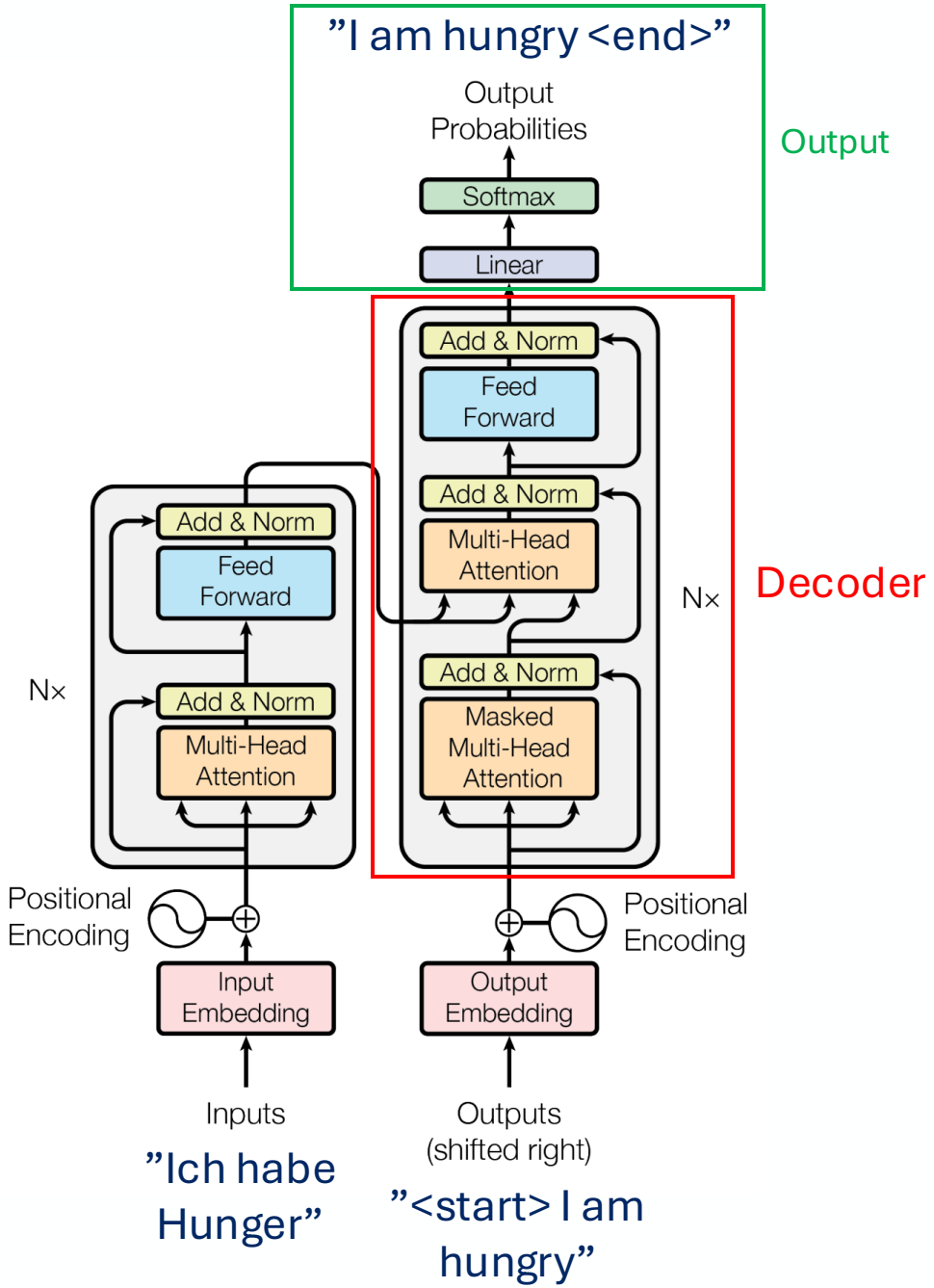
Transformer

- Components:
  - Word embedding
  - Positional encoding
  - Encoder
  - Decoder



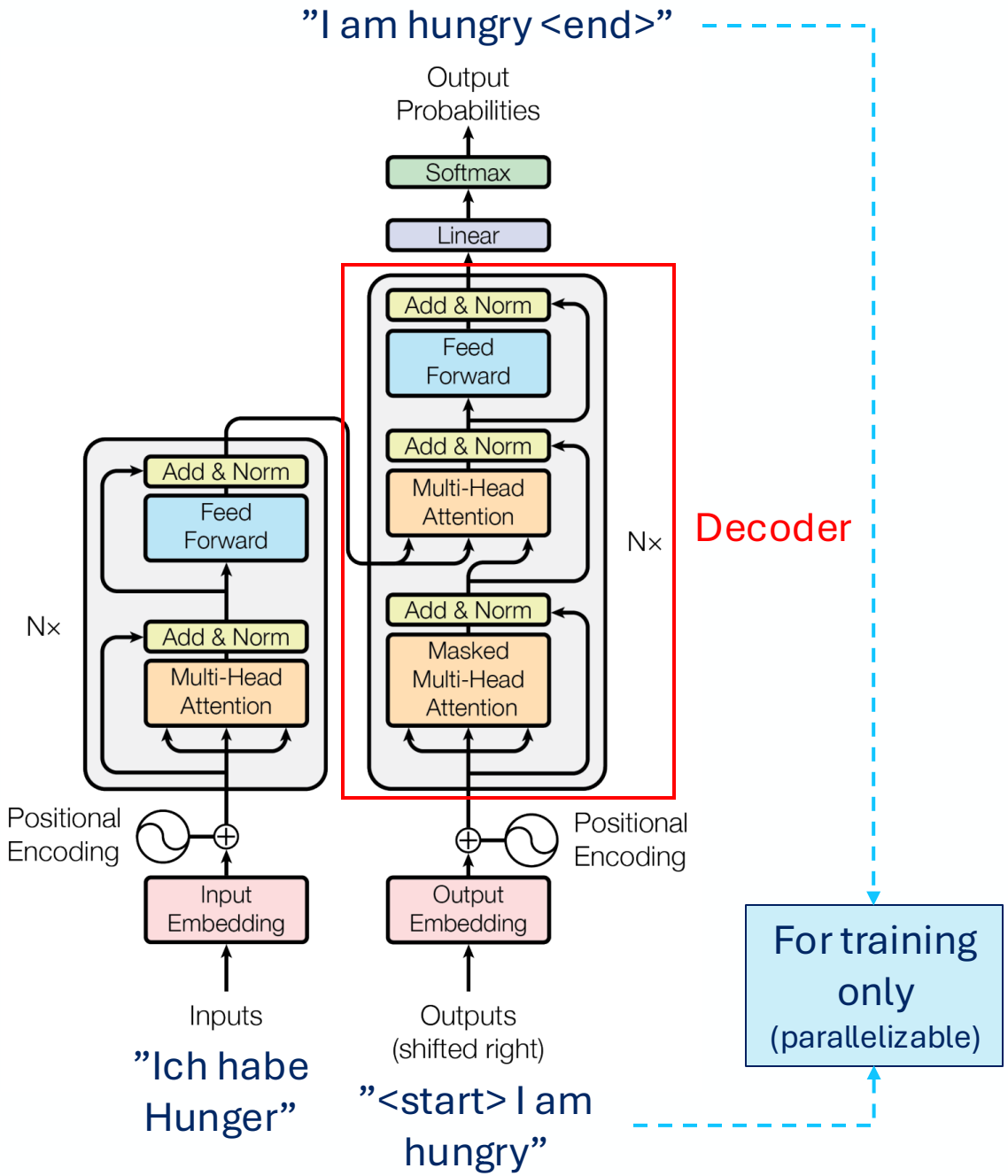
Transformer

- Components:
  - Word embedding
  - Positional encoding
  - Encoder
  - Decoder



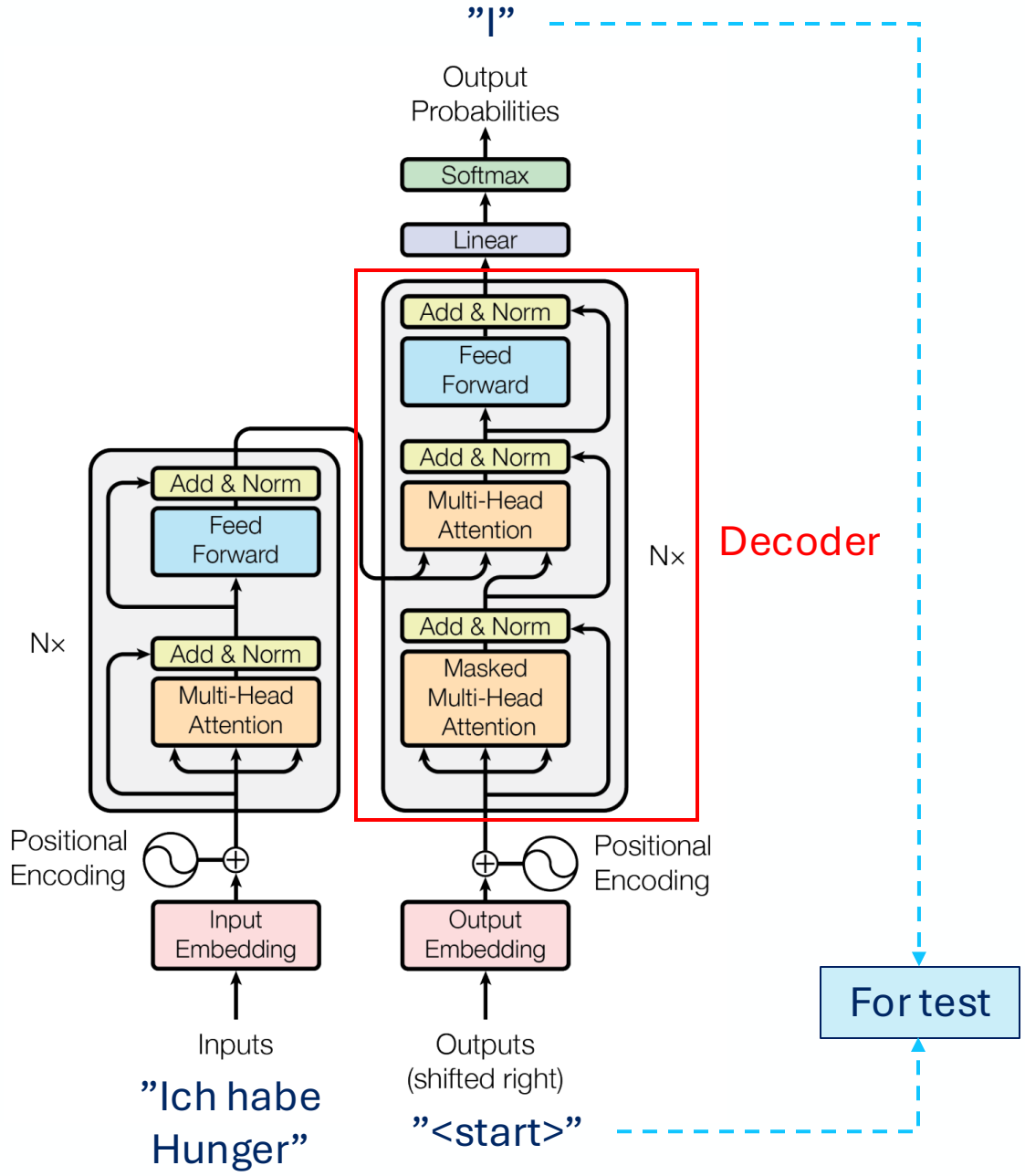
Transformer

- Components:
  - Word embedding
  - Positional encoding
  - Encoder
  - Decoder



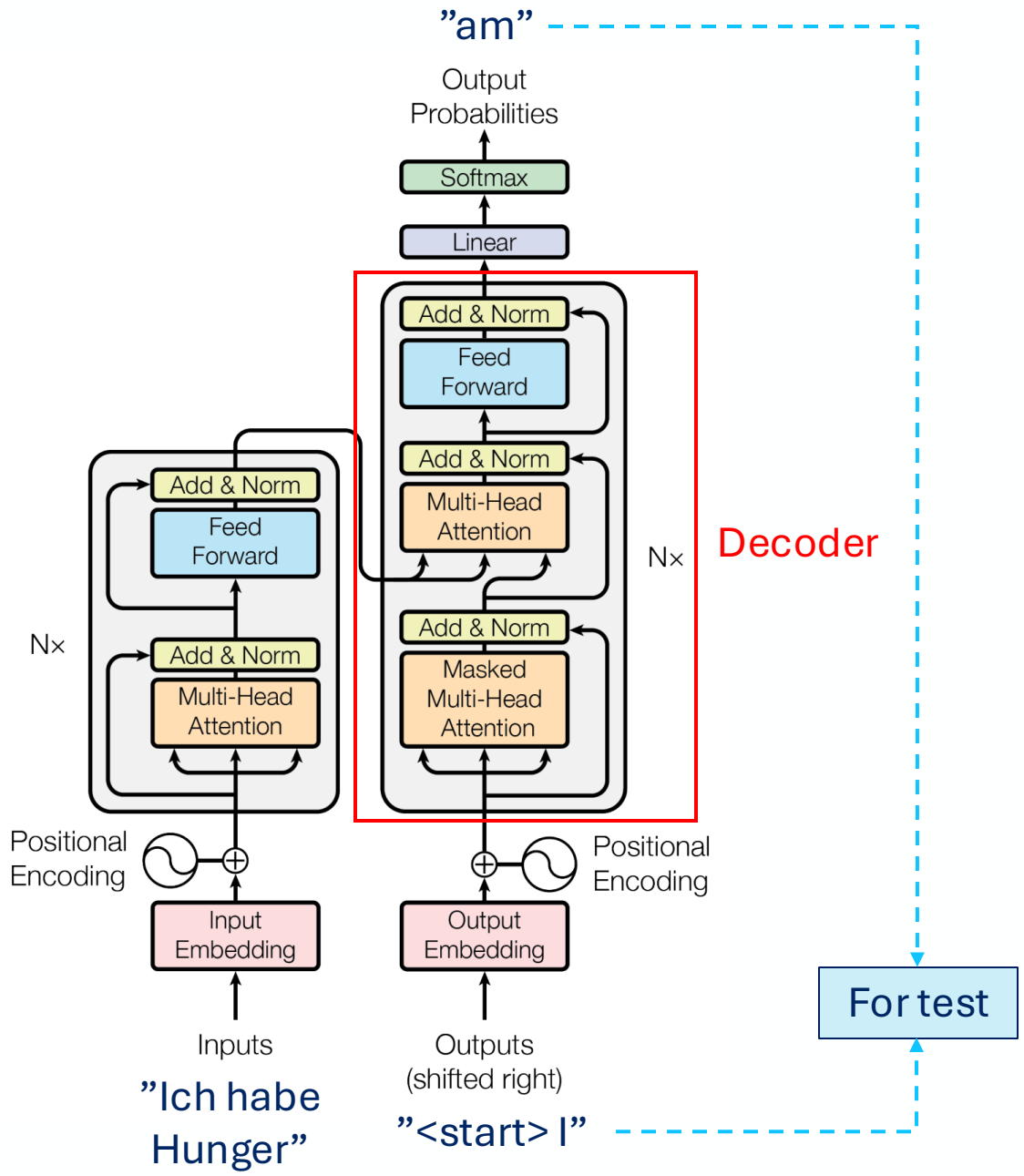
Transformer

- Components:
  - Word embedding
  - Positional encoding
  - Encoder
  - Decoder



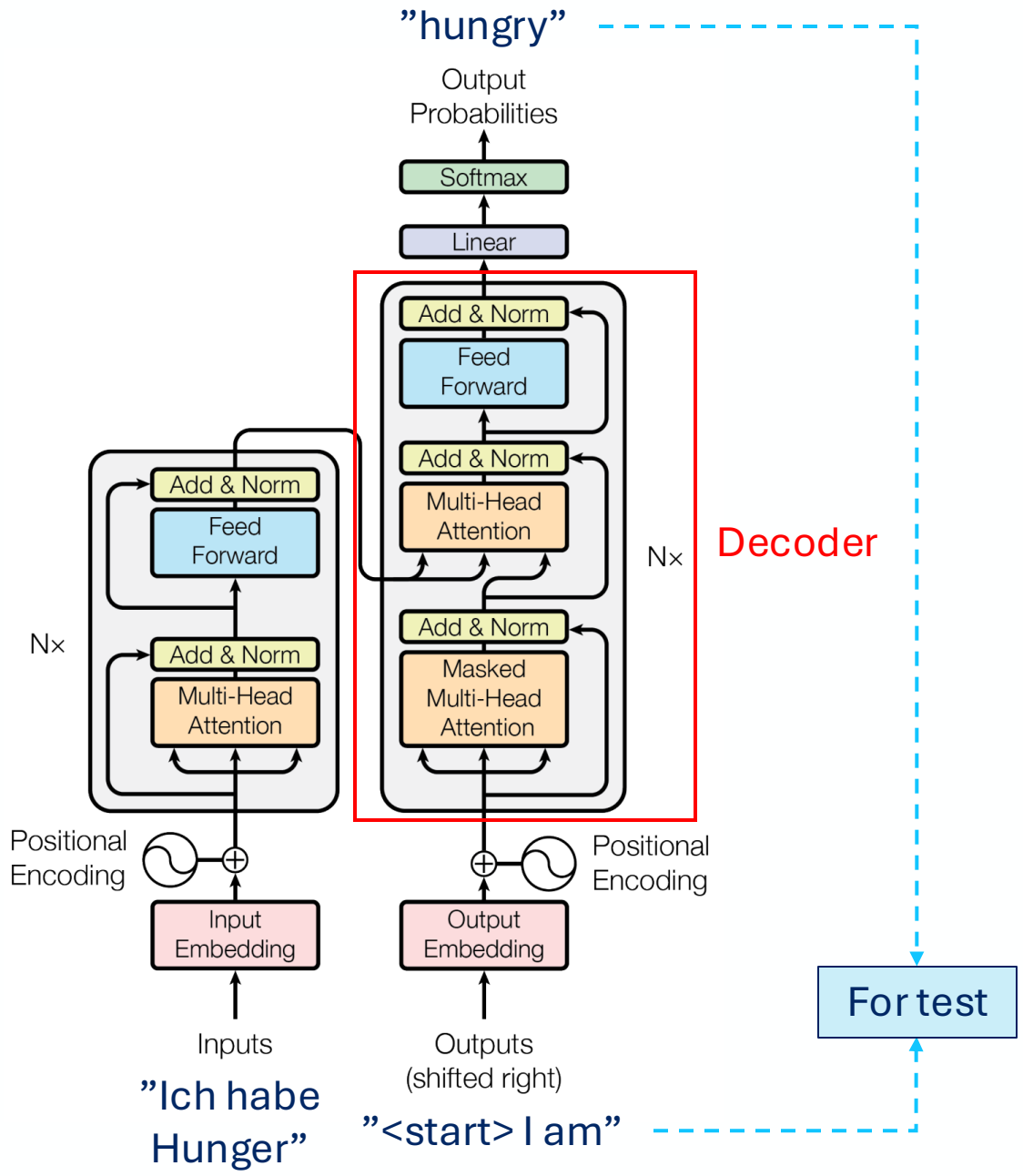
Transformer

- Components:
  - Word embedding
  - Positional encoding
  - Encoder
  - Decoder



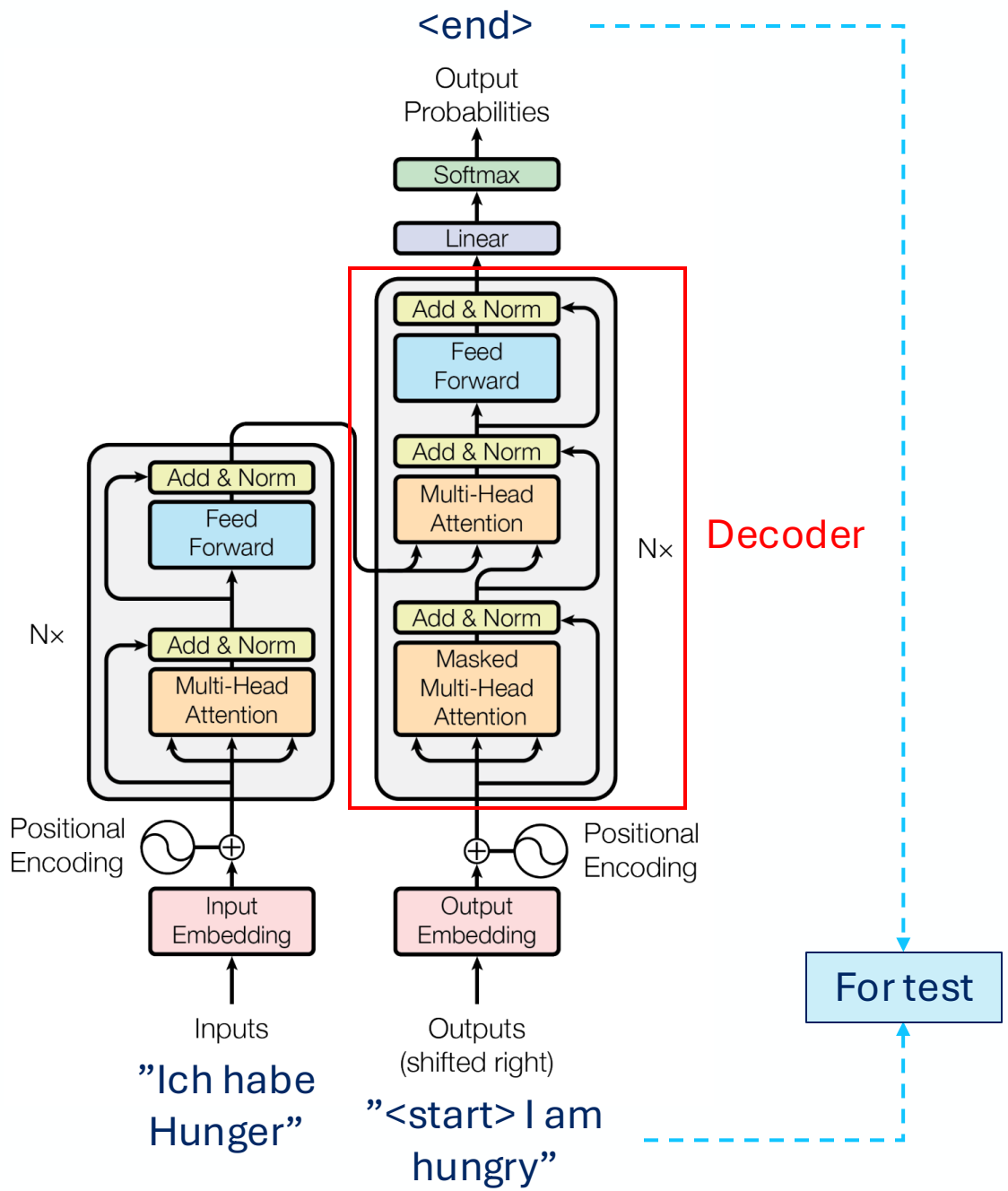
Transformer

- Components:
  - Word embedding
  - Positional encoding
  - Encoder
  - Decoder



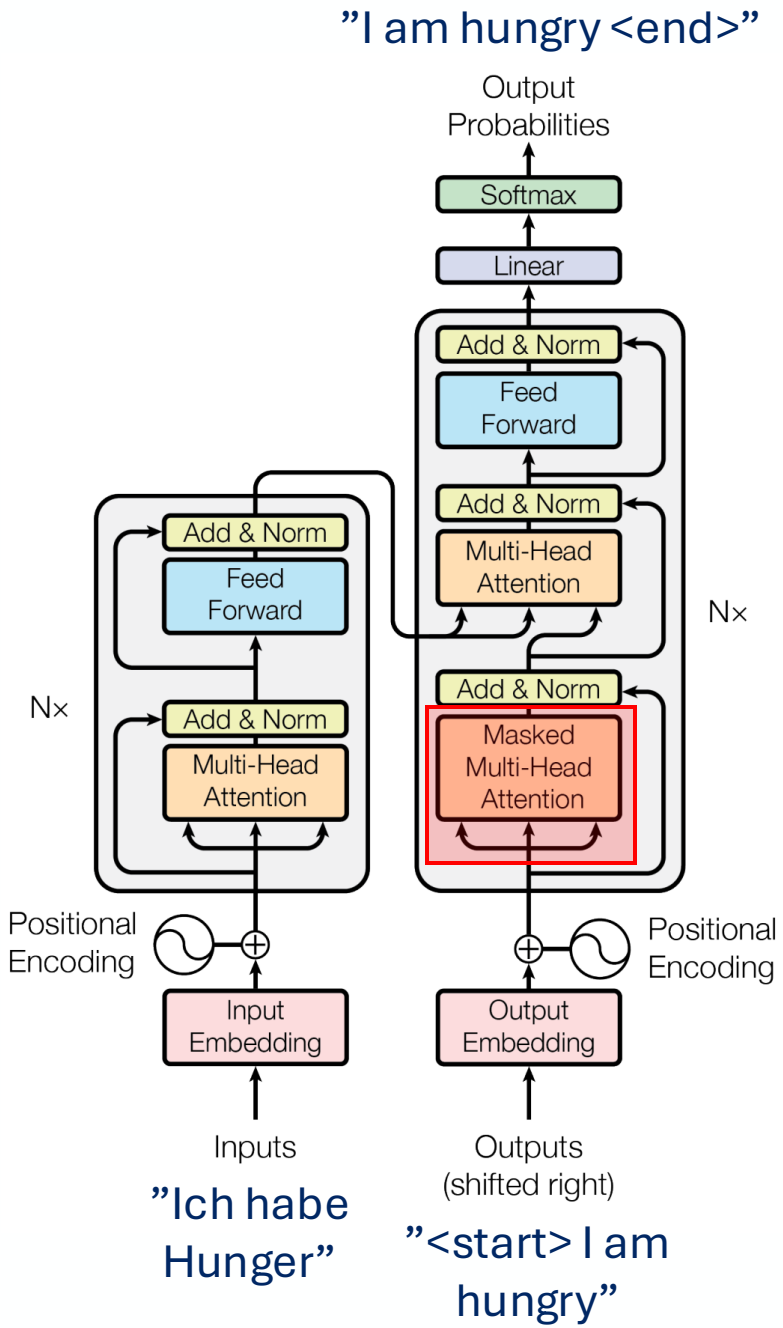
Transformer

- Components:
  - Word embedding
  - Positional encoding
  - Encoder
  - Decoder



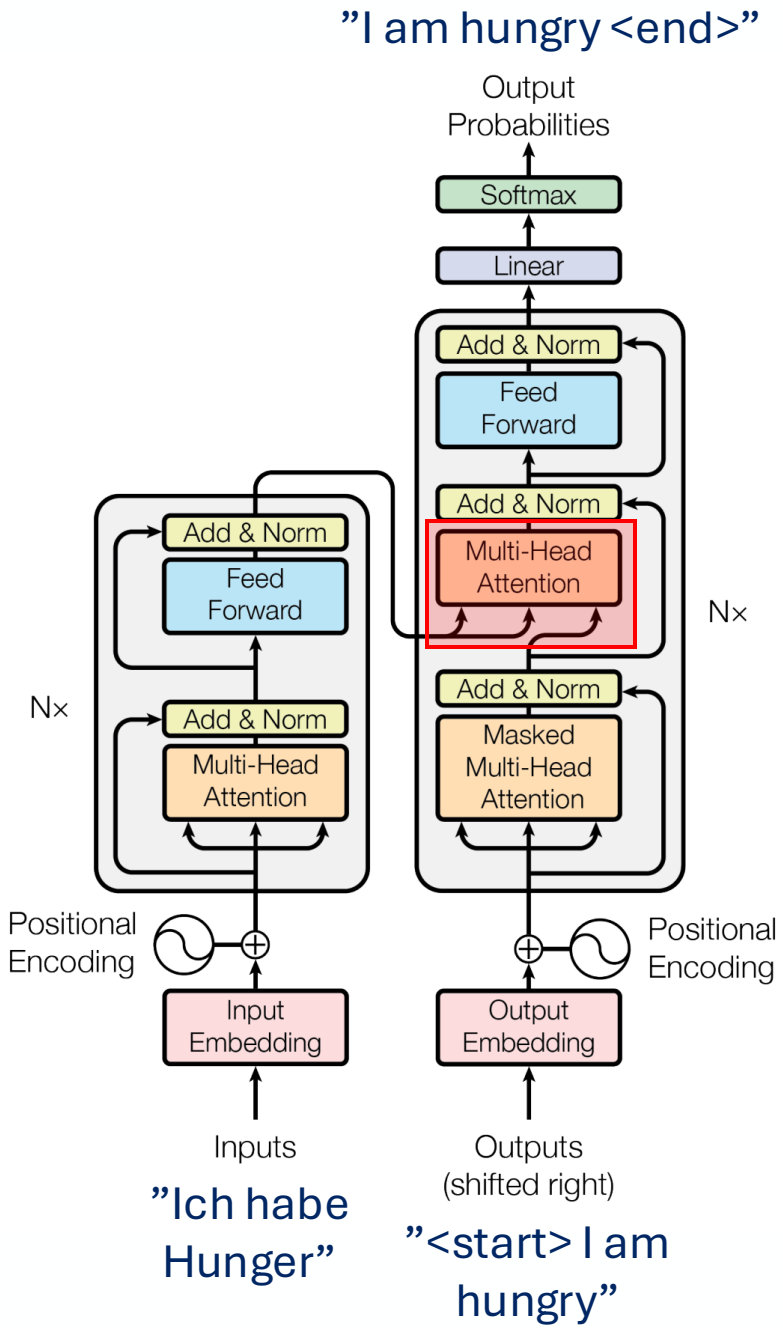
Transformer

- Components:
  - Word embedding
  - Positional encoding
  - Encoder
  - Decoder
    - Masked self-attention



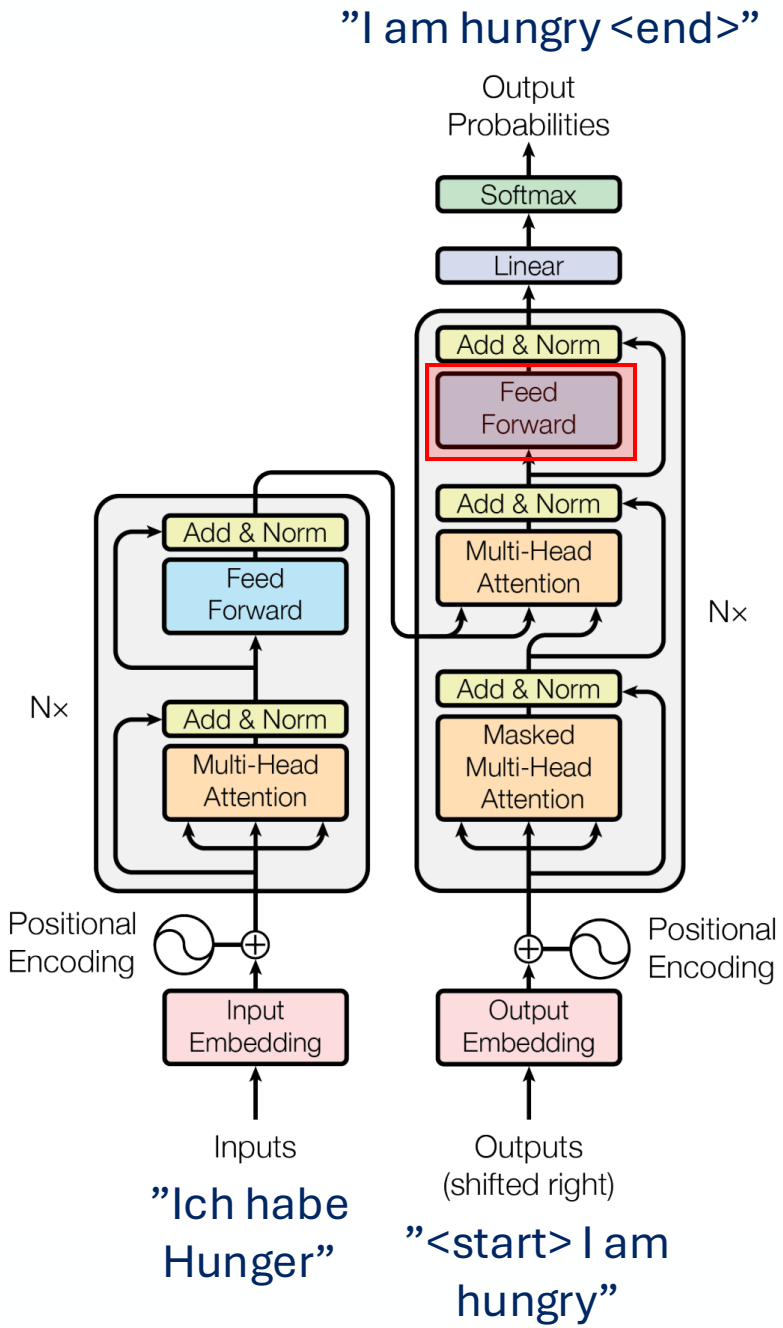
Transformer

- Components:
  - Word embedding
  - Positional encoding
  - Encoder
  - Decoder
    - Masked self-attention
    - Multi-head cross-attention



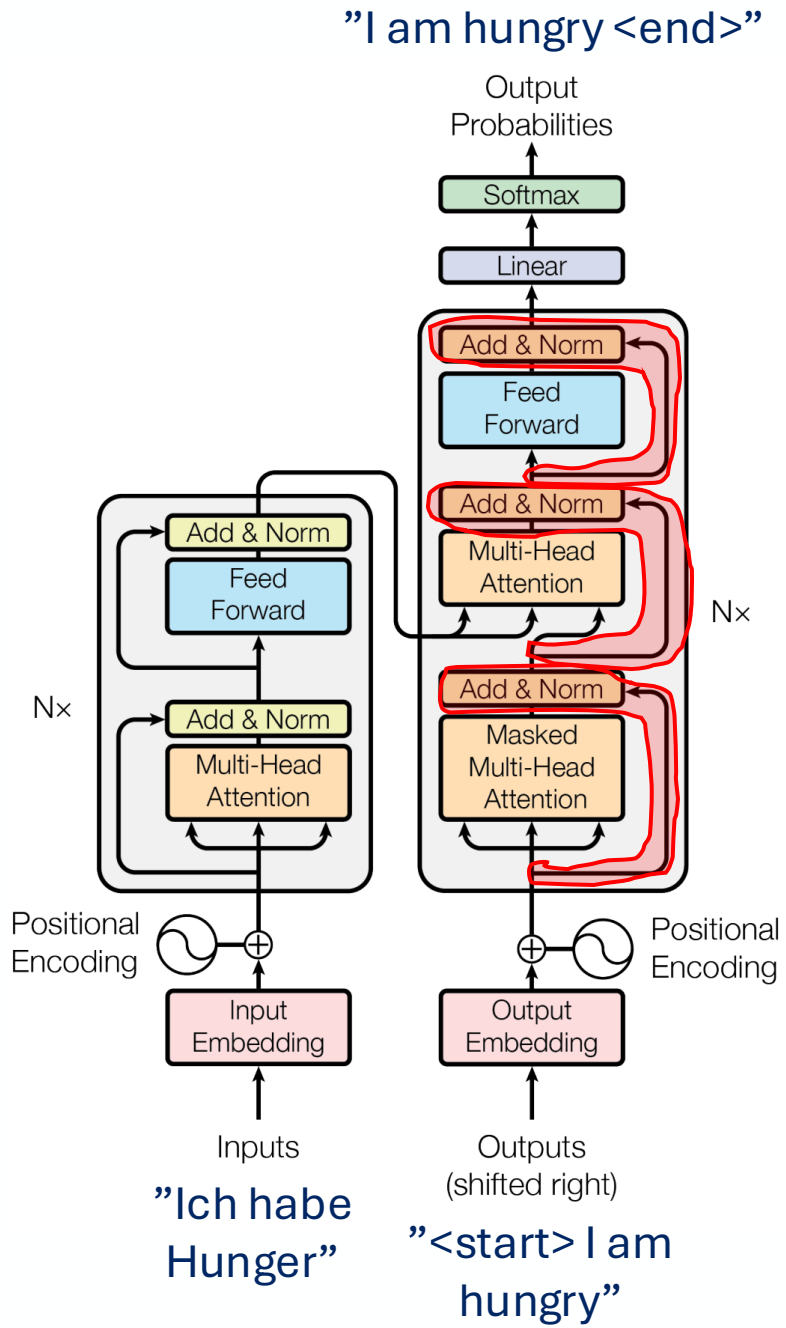
Transformer

- Components:
  - Word embedding
  - Positional encoding
  - Encoder
  - Decoder
    - Masked self-attention
    - Multi-head cross-attention
    - Fully-connected layer



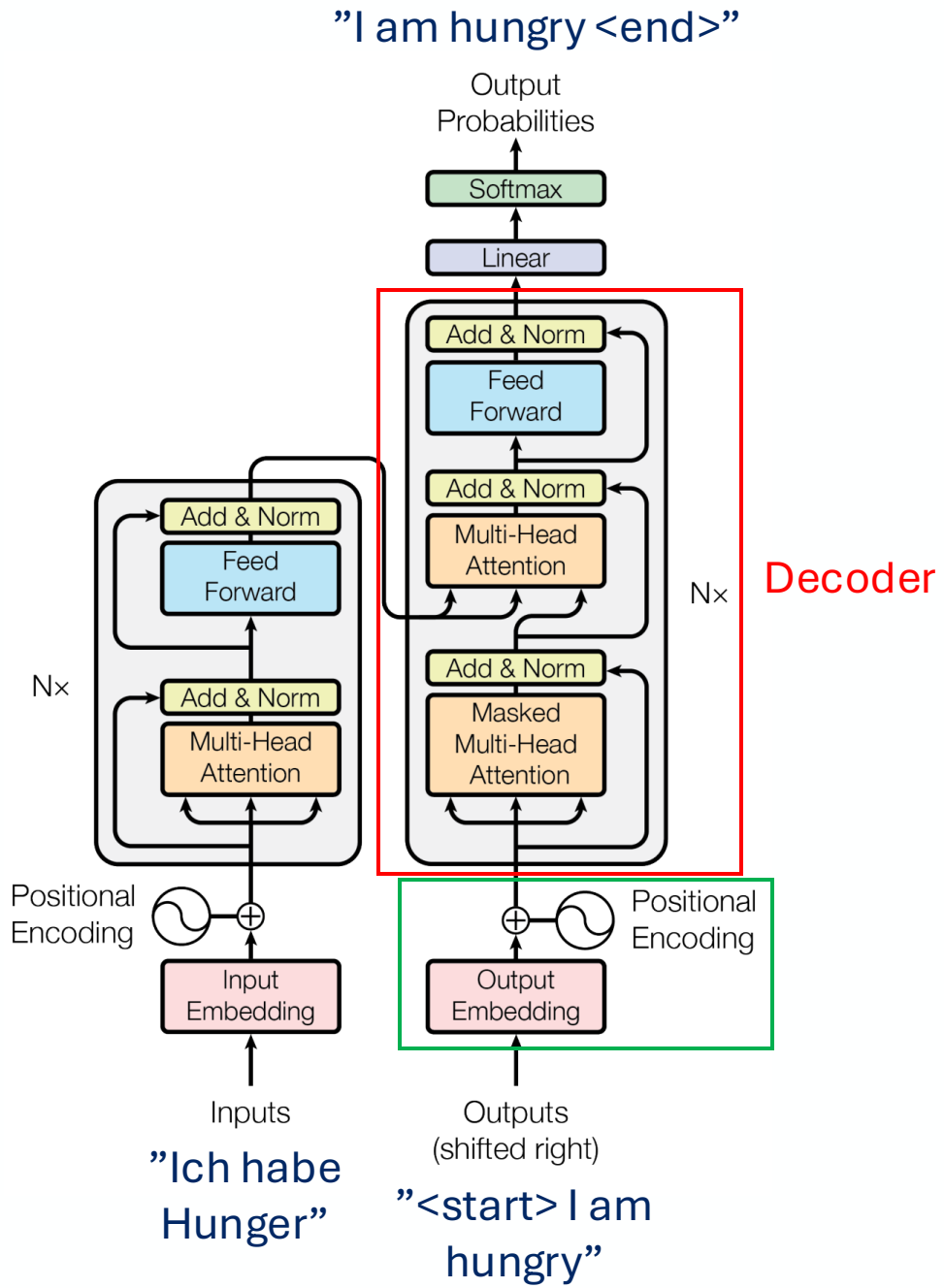
Transformer

- Components:
  - Word embedding
  - Positional encoding
  - Encoder
  - Decoder
    - Masked self-attention
    - Multi-head cross-attention
    - Fully-connected layer
    - Skip connection with normalization



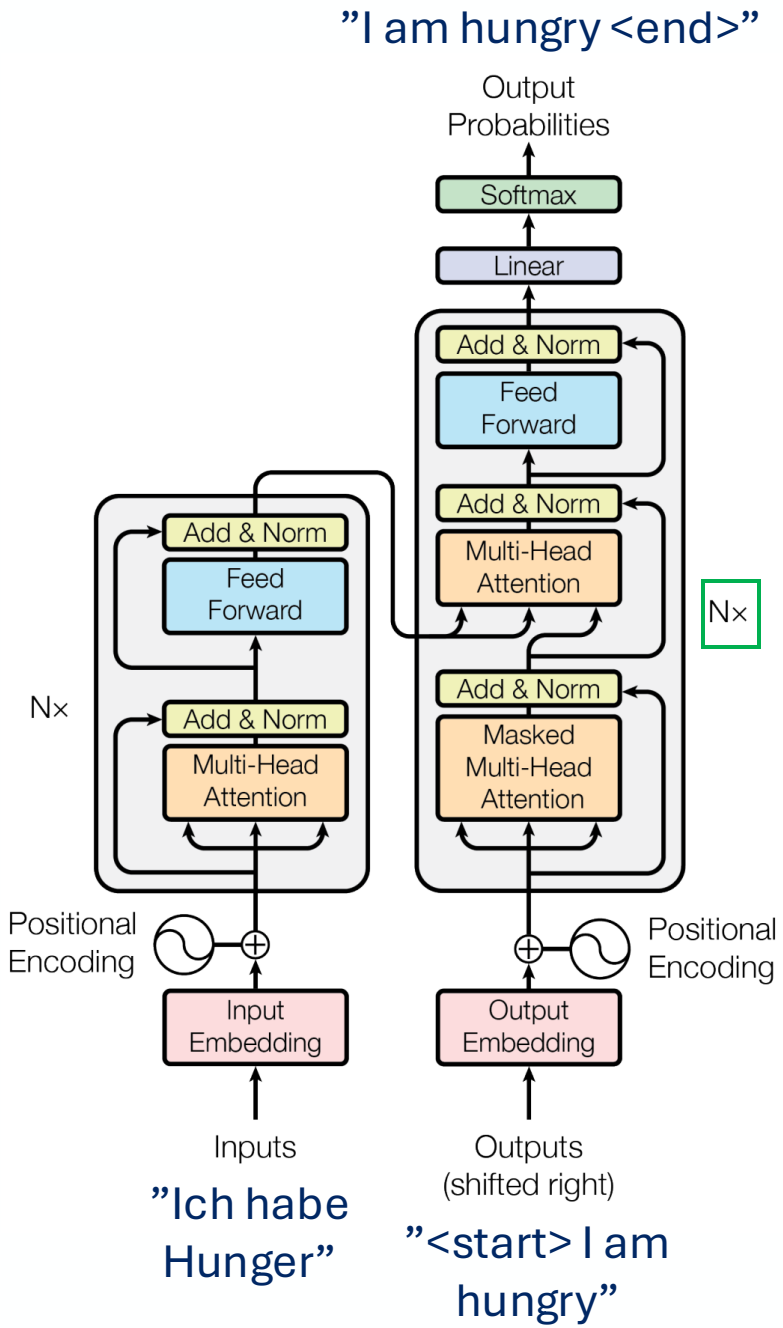
Transformer

- Components:
  - Word embedding
  - Positional encoding
  - Encoder
  - Decoder



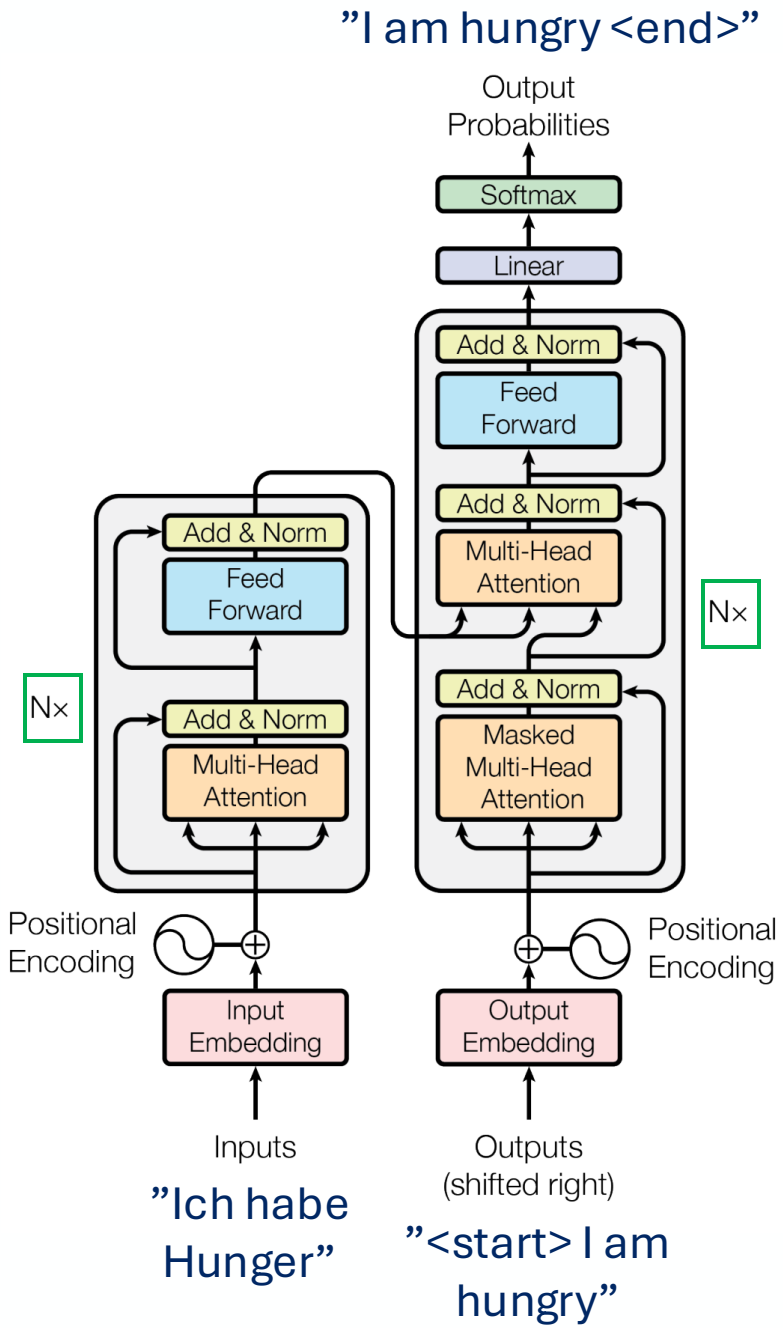
Transformer

- Components:
  - Word embedding
  - Positional encoding
  - Encoder
  - Decoder
    - N blocks



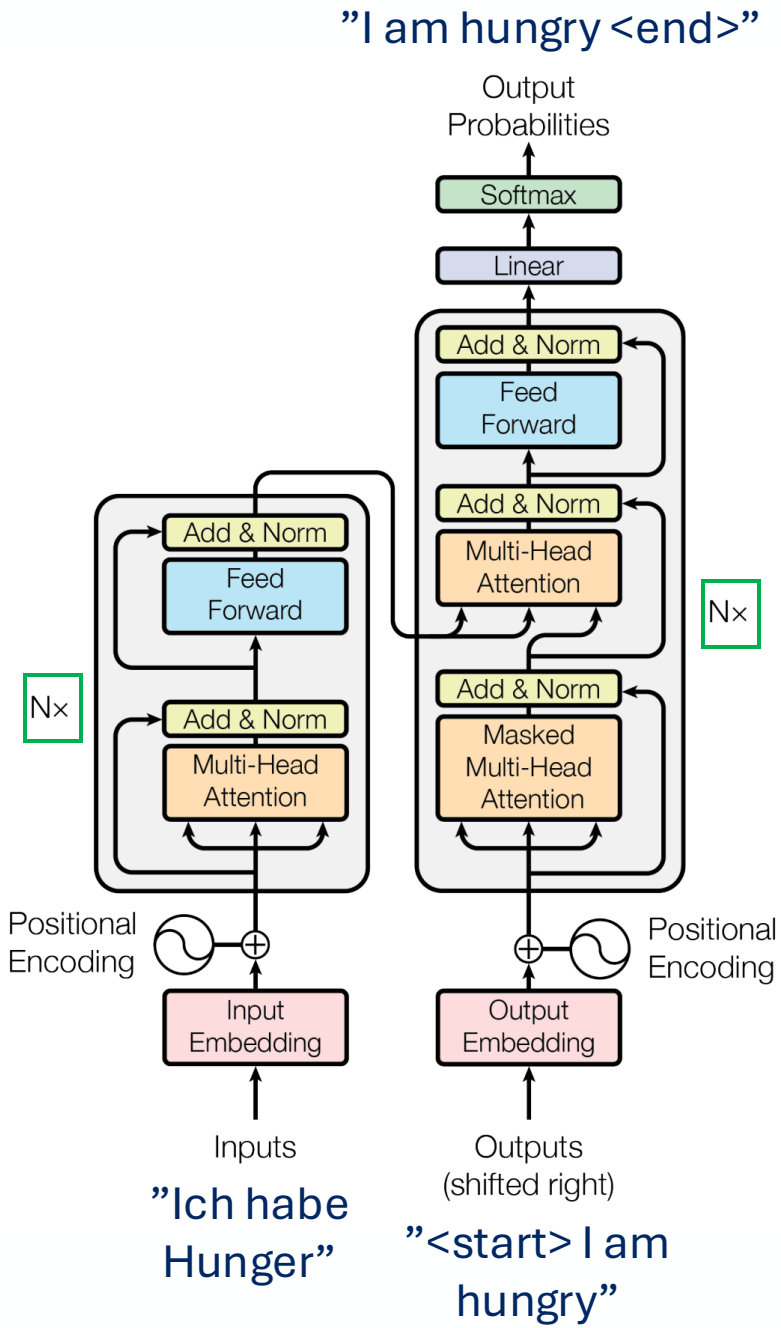
Transformer

- Components:
  - Word embedding
  - Positional encoding
  - Encoder
  - Decoder
- Originally, N = 6



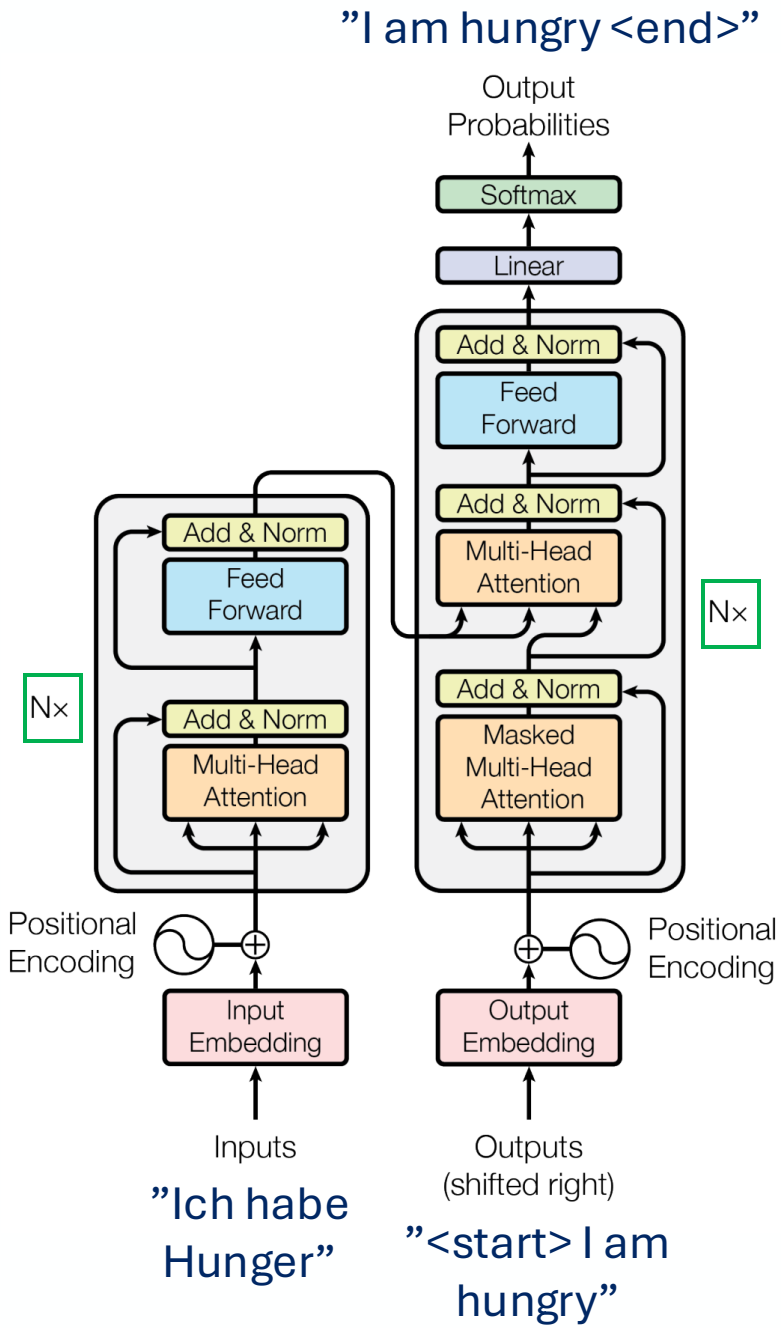
Transformer

- Components:
  - Word embedding
  - Positional encoding
  - Encoder
  - Decoder
- Originally, N = 6, but we can have
  - $N_{encoder} \neq N_{decoder}$



Transformer

- Components:
  - Word embedding
  - Positional encoding
  - Encoder
  - Decoder
- Originally, N = 6, but we can have
  - $N_{encoder} \neq N_{decoder}$
  - $N \neq 6$



Transformer Variants

- Different kinds of input

Words

Ich habe Hunger

Transformer Variants

- Different kinds of input

Words

Ich habe Hunger

Image

Transformer Variants

- Different kinds of input

Words

Ich habe Hunger

Image

Sound

Transformer Variants

- Different kinds of input

Words

Ich habe Hunger

Image

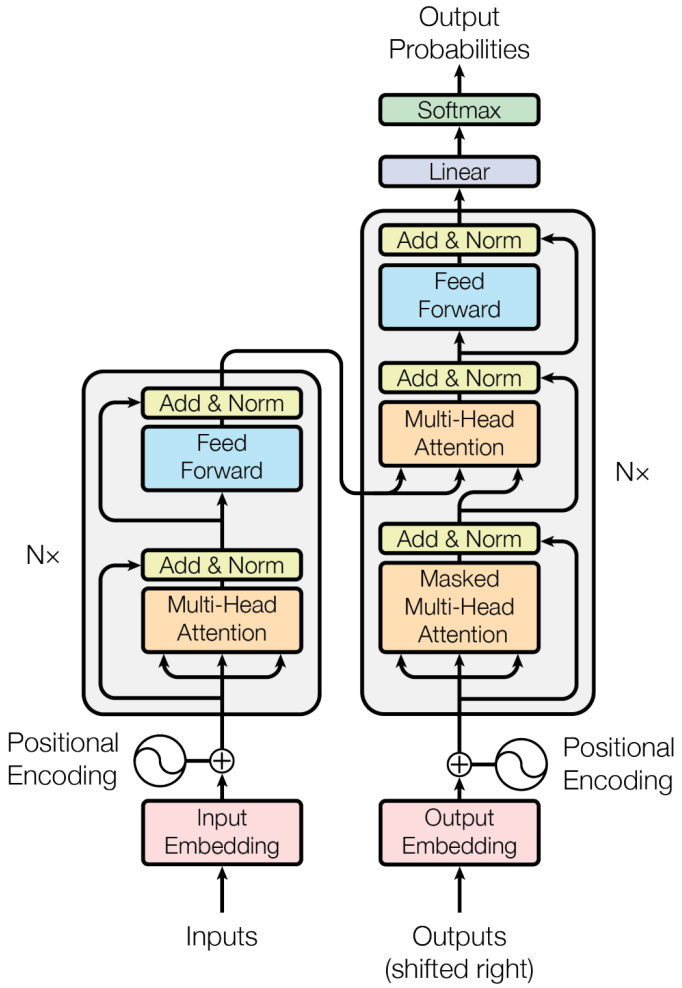
Sound

DNA sequence

ATA GCA TGA

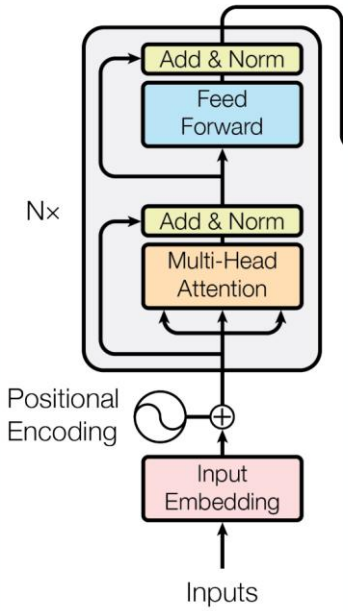
Transformer Variants

- Different kinds of architecture



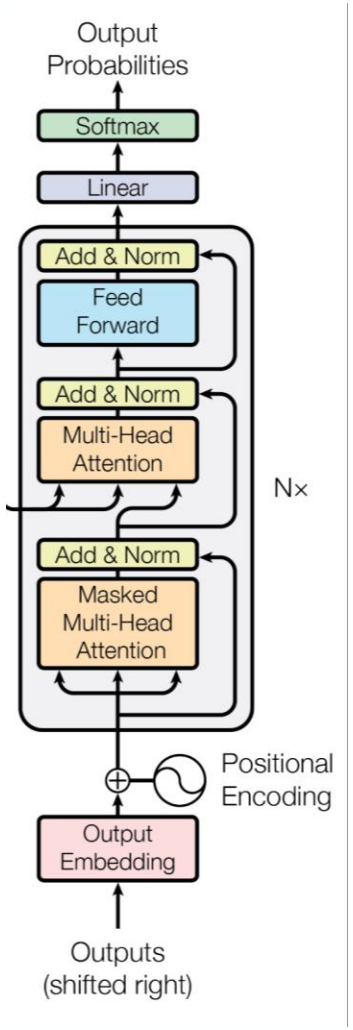
Transformer Variants

- Different kinds of architecture
  - Only encoder
    - E.g., BERT



Transformer Variants

- Different kinds of architecture
  - Only encoder
    - E.g., BERT
  - Only decoder
    - E.g., GPT

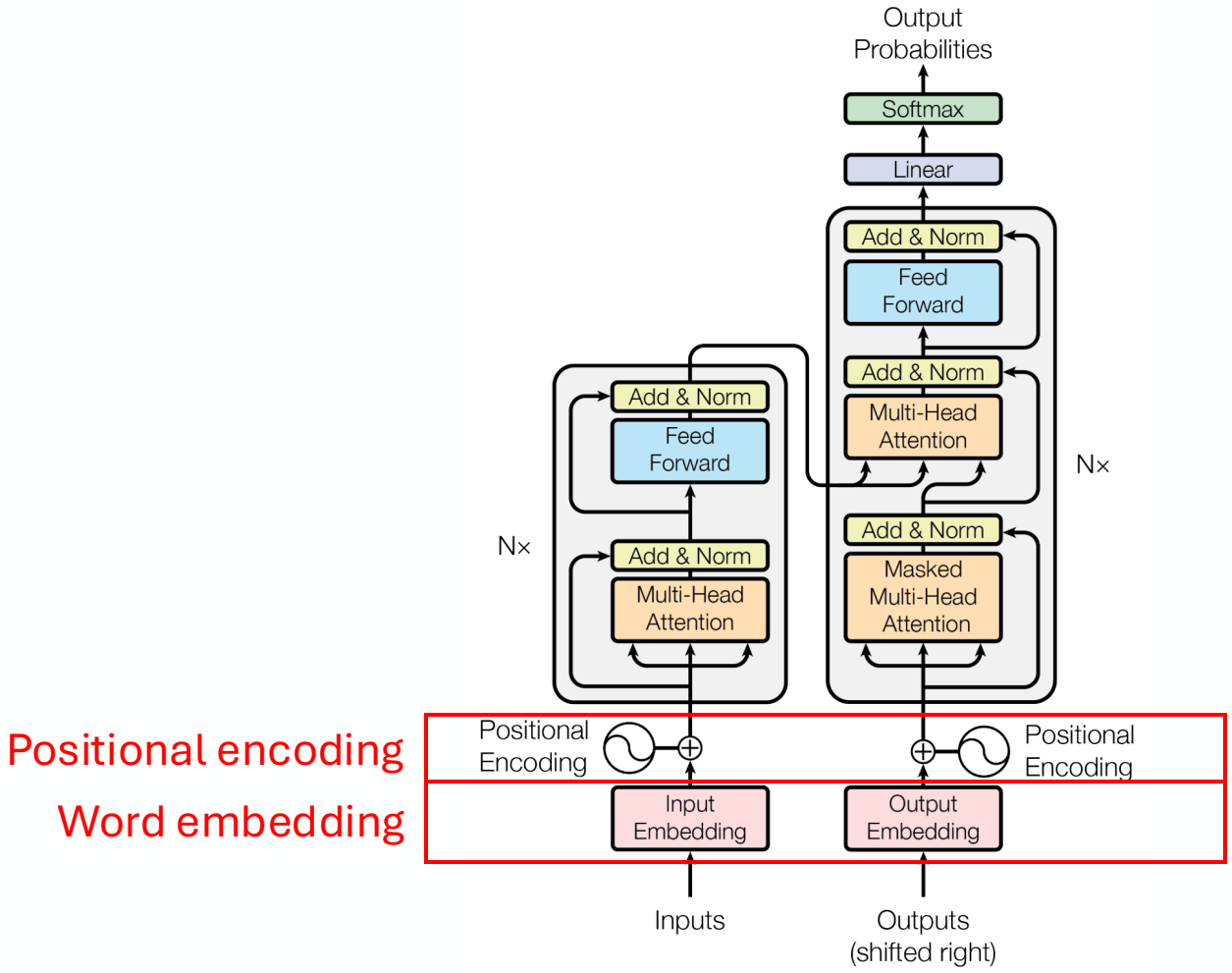


Outline

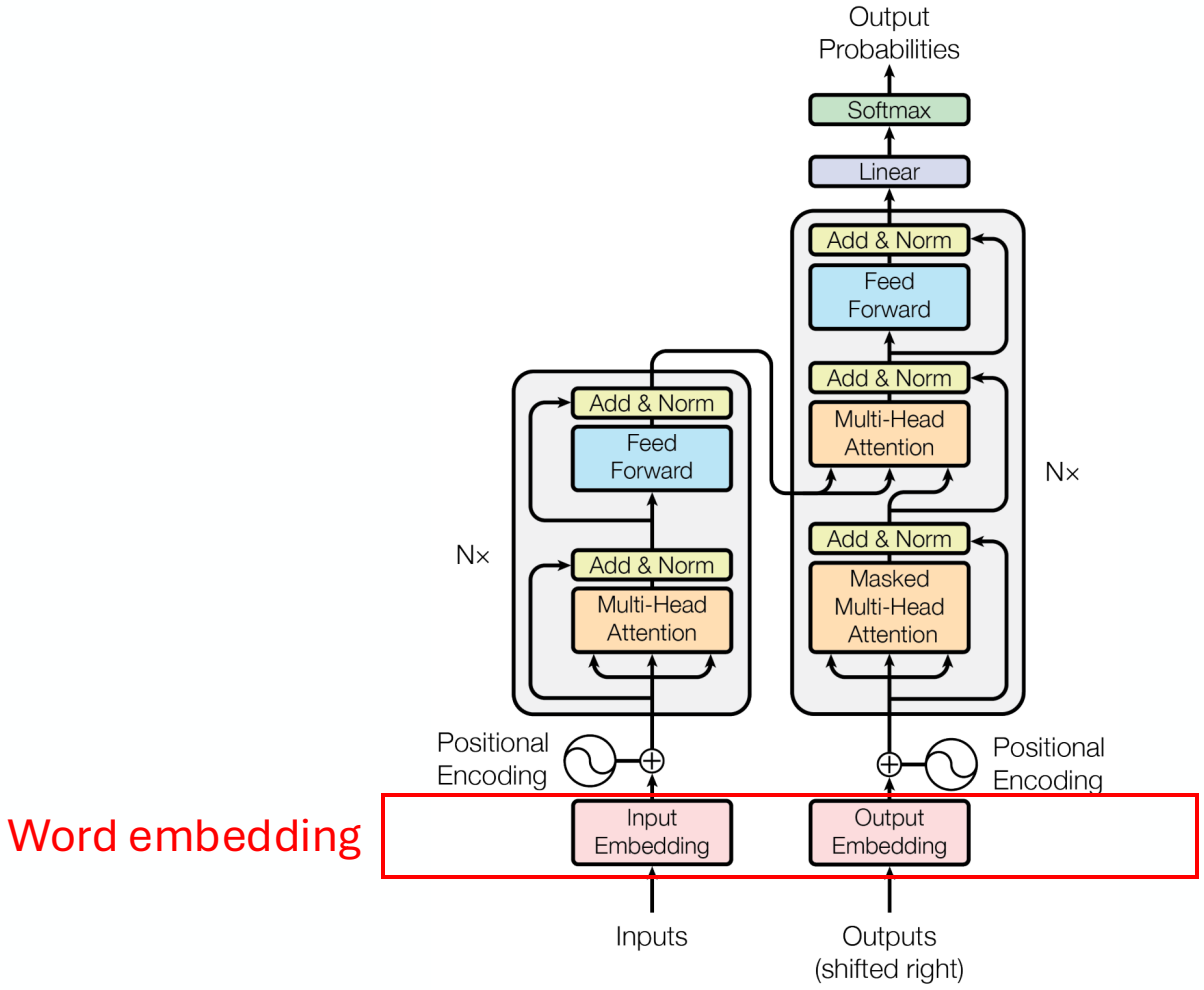
1. Overview
2. Preprocessing
3. Attention mechanism
4. Putting it all together
5. Variants of Transformer

Transformer Input

- Word embedding
- Positional encoding

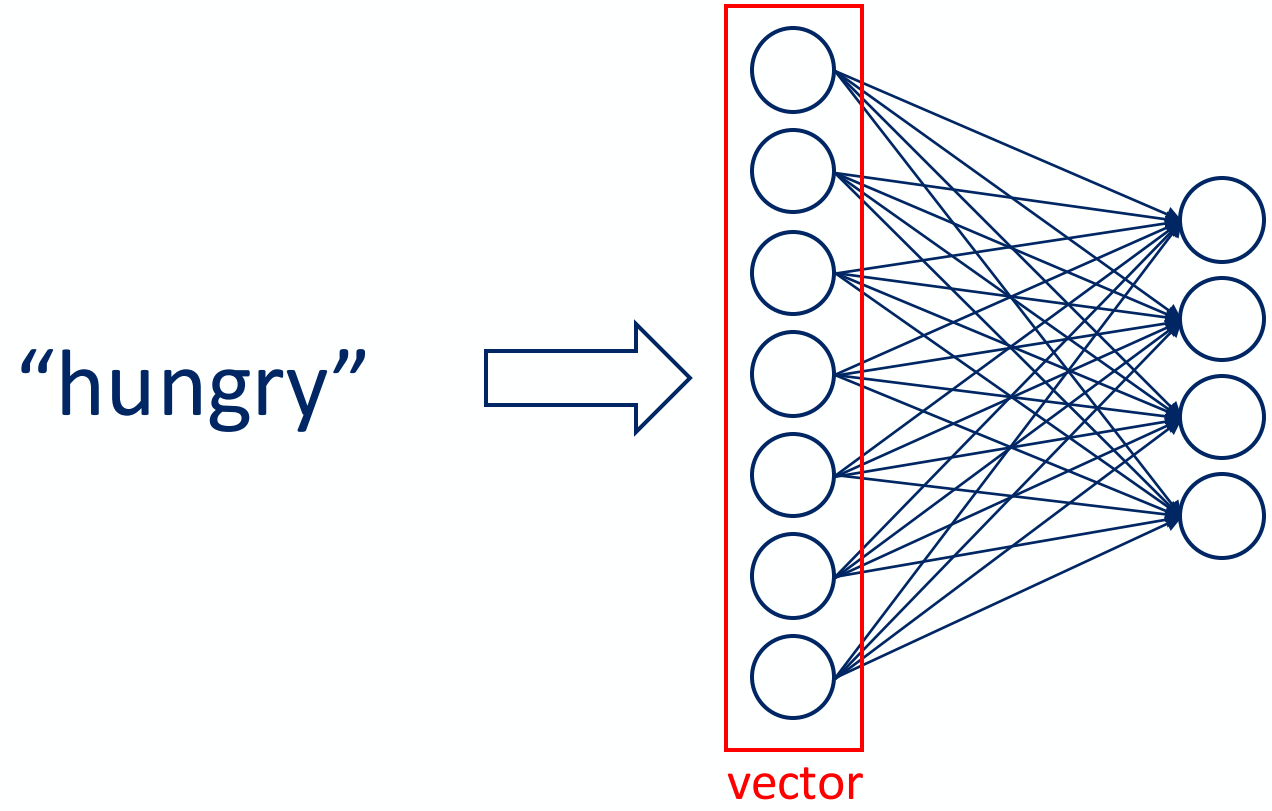


Word Embedding



Word Embedding

- Goal: How to represent a word as a vector



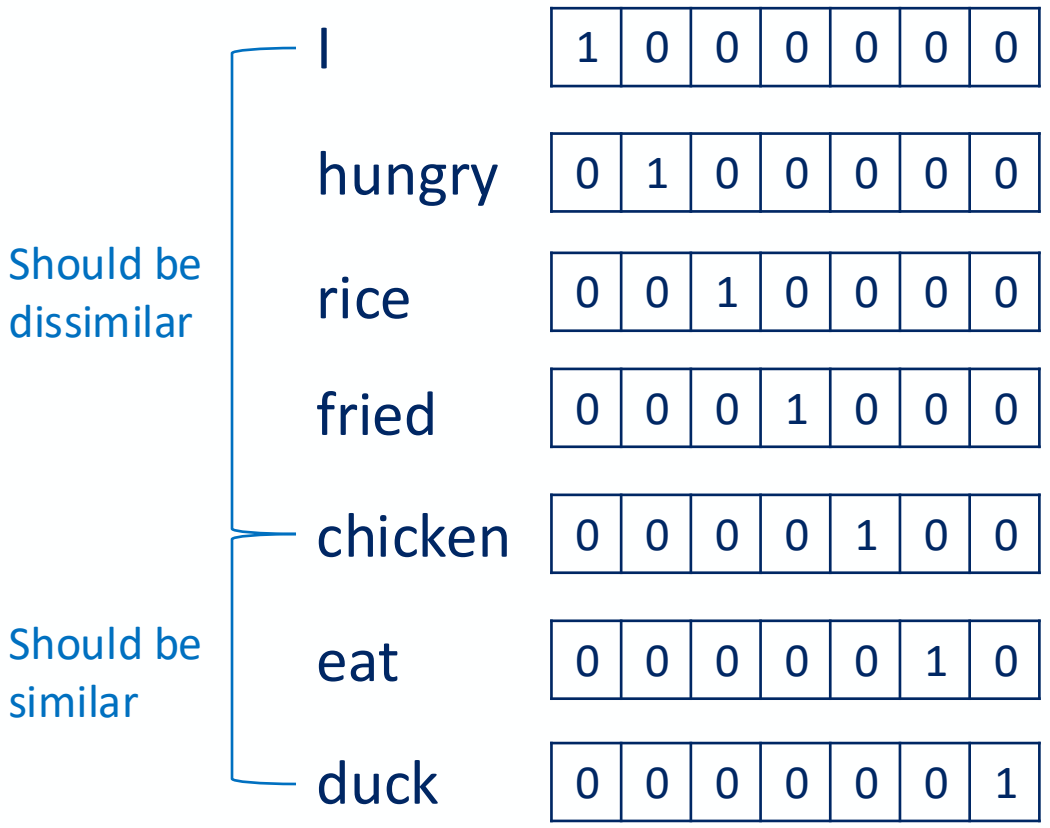
Word Embedding

- The simplest solution: one-hot vector
  - Length vector = length dictionary
  - 1 element as 1, others 0

I	1	0	0	0	0	0	0
hungry	0	1	0	0	0	0	0
rice	0	0	1	0	0	0	0
fried	0	0	0	1	0	0	0
chicken	0	0	0	0	1	0	0
eat	0	0	0	0	0	1	0
duck	0	0	0	0	0	0	1

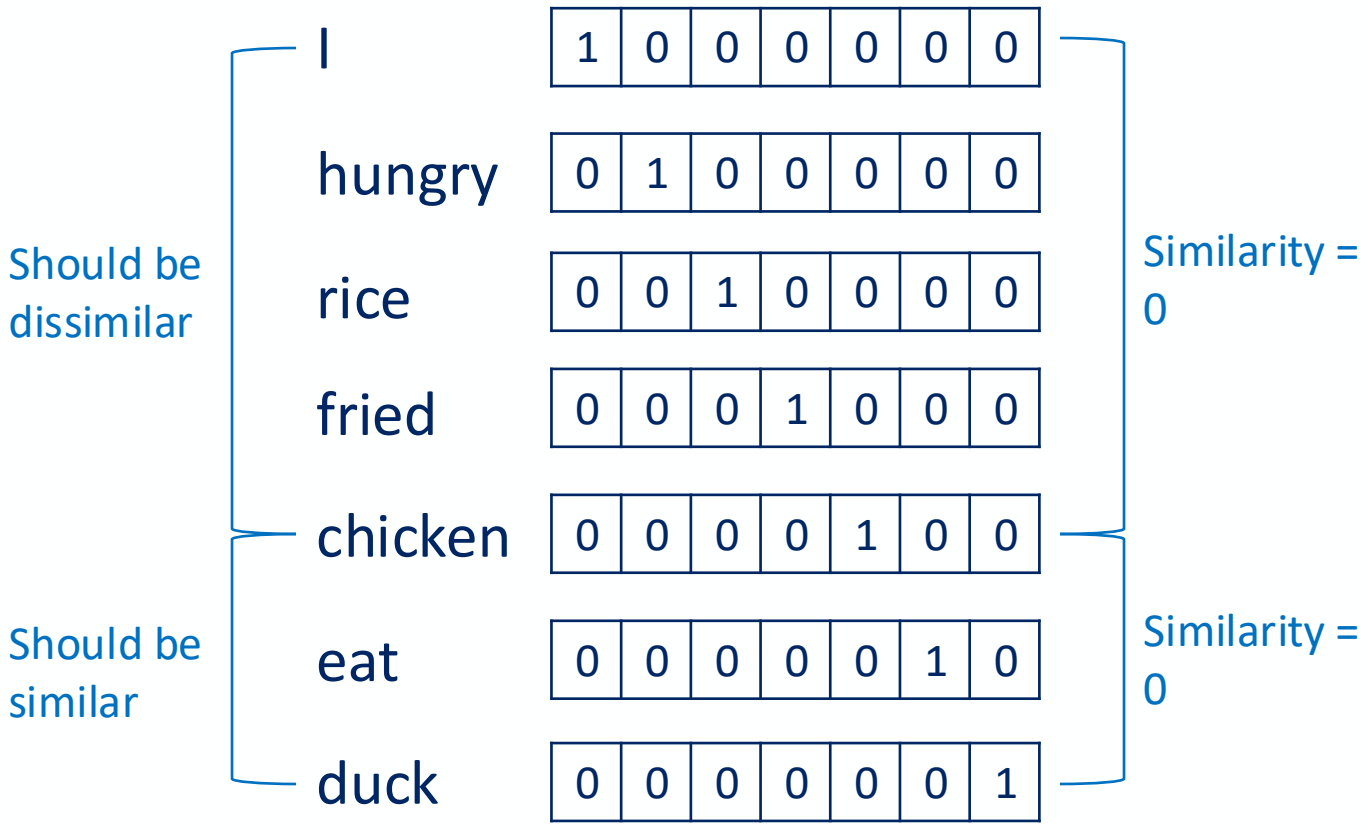
Word Embedding

- The simplest solution: one-hot vector
  - Length vector = length dictionary
  - 1 element as 1, others 0
  - Problem: not that meaningful



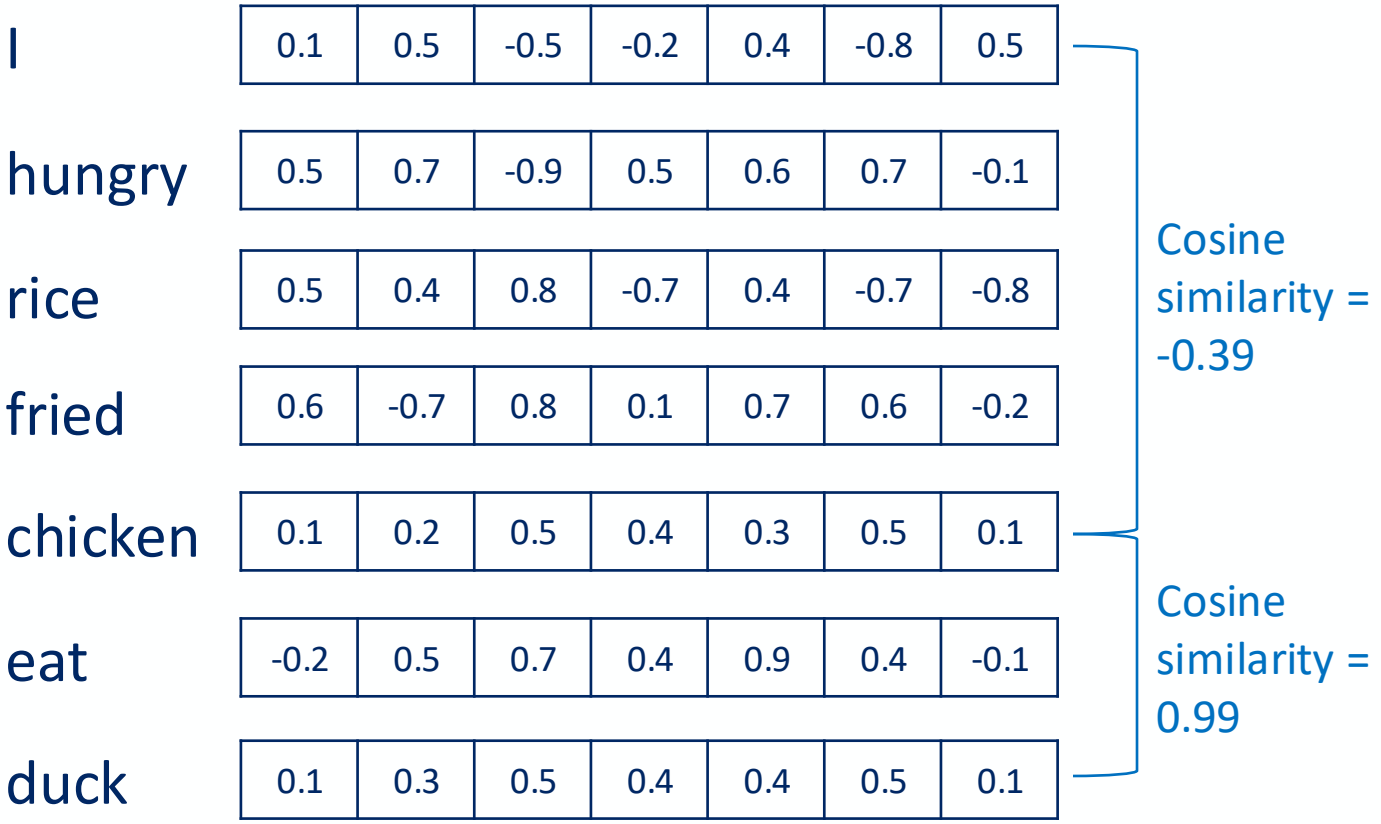
Word Embedding

- The simplest solution: one-hot vector
  - Length vector = length dictionary
  - 1 element as 1, others 0
  - Problem: not that meaningful
    - dot product or cosine similarity between each word 0



Word Embedding

- Should be meaningful
- Length vector doesn't have to be same as length of dictionary



Word Embedding

- Other ways:
  - Word2Vec
  - GloVe
  - fastText
  - ELMo
  - BERT
  - PCA

We will not discuss each of this further…

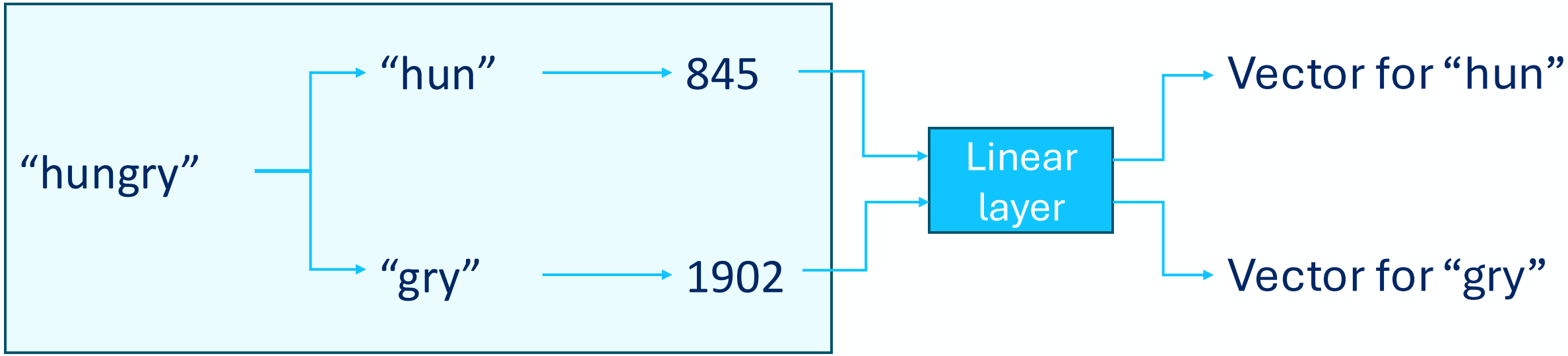
Good Word Embedding

- Ideally, can do vector operation to see relation between words

$$\overrightarrow{queen} - \overrightarrow{woman} + \overrightarrow{man} = \overrightarrow{king}$$

Word Embedding

- What original transformer does: Byte pair encoding + linear layer
  - Byte pair encoding: splitting words and map to a number
  - Linear layer: learnable



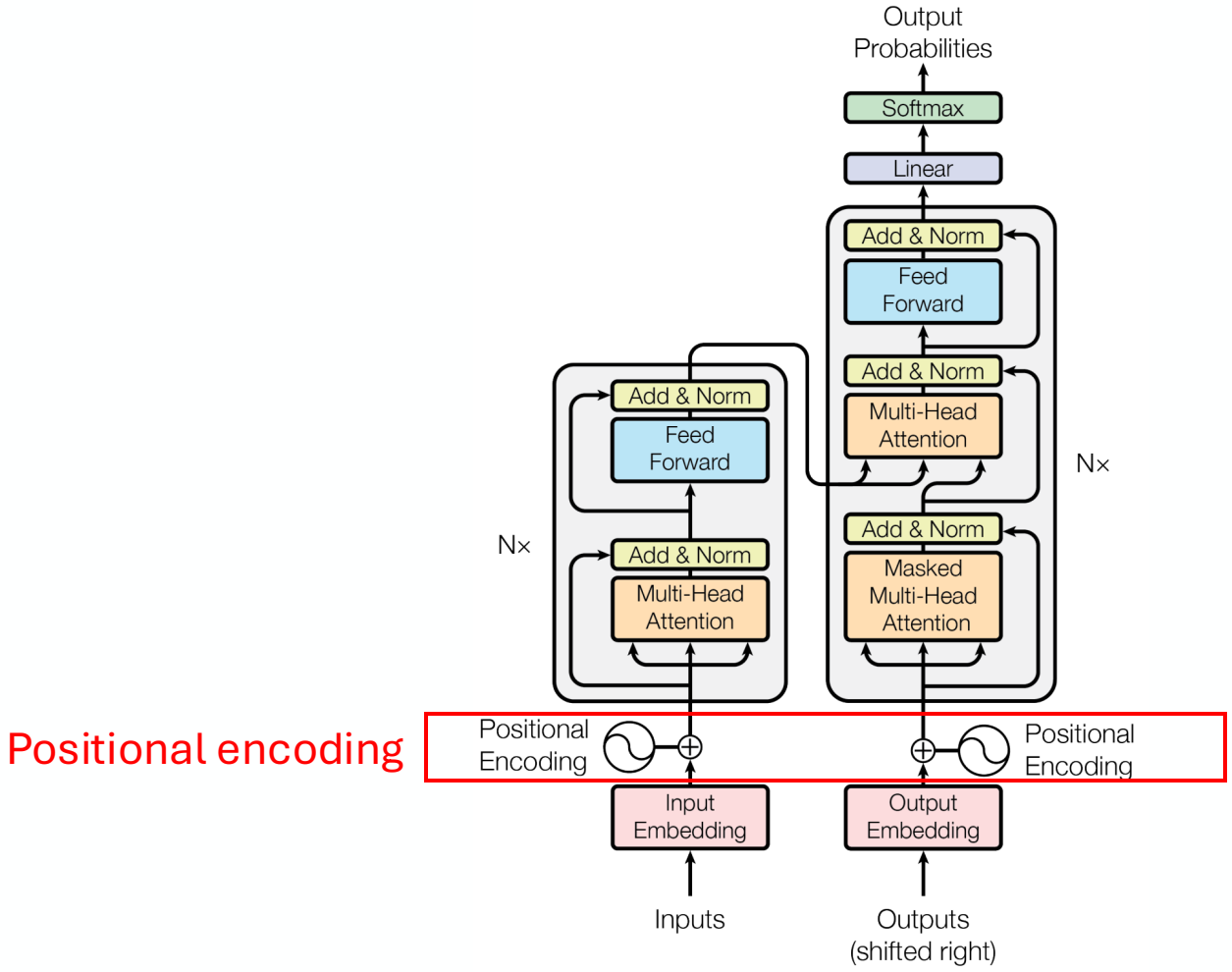
Byte pair encoding

Word Embedding

- What original transformer does: Byte pair encoding + linear layer
  - Probably not the best word embedding
    - “hun-ter” is not similar to “hun-gry”
    - But the transformer will know later with self-attention if the ”hun” refers to ”hunter” or “hungry”
  - And it still offers data efficiency and generalization
    - “walk”, “walk-ed”, “walk-ing”
  - Technically, can also still use other word-embedding methods
  - In this presentation, for simplicity, we will consider vector for each word
    - Not vector for split word

Positional Encoding

- Positional encoding



Positional encoding

Positional Encoding

- Motivation: words position is important
  - The order of the words is important for the meaning

"man bites dog"

"dog bites man"

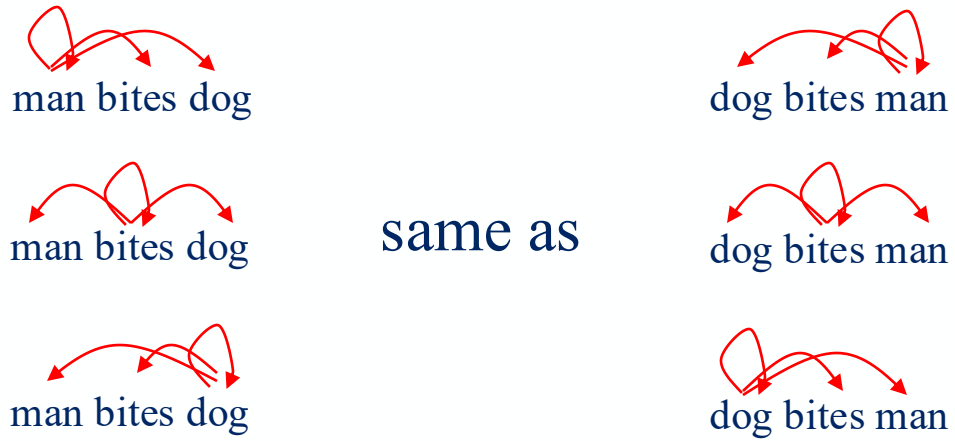
Positional Encoding

- Motivation: Transformer by itself can’t encode the position
  - Not sequential

Positional Encoding

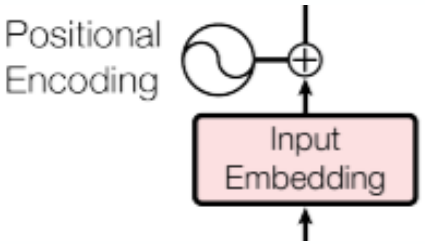
- Motivation: Transformer by itself can't encode the position
  - Not sequential

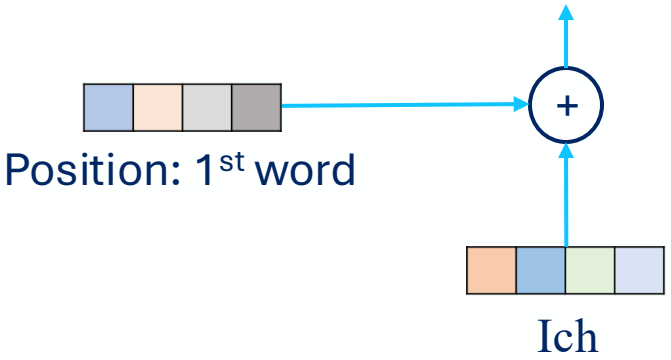
  - Self attention can't distinguish word order



Positional Encoding

- Position vector added to word embedding
  - Same size with the word embedding (e.g., size = 4)





Positional Encoding

- Simplest example: binary
  - Limited number of word
    - {number of bits}^{2}

position	Binary encoding			
0:	0	0	0	0
1:	0	0	0	1
2:	0	0	1	0
3:	0	0	1	1
4:	0	1	0	0
5:	0	1	0	1
6:	0	1	1	0
7:	0	1	1	1
8:	1	0	0	0
9:	1	0	0	1
10:	1	0	1	0
11:	1	0	1	1
12:	1	1	0	0
13:	1	1	0	1
14:	1	1	1	0
15:	1	1	1	1

Positional Encoding

- Original transformer way

$$\mathrm{PE}_{(pos,2i)} = \sin\left(\frac{\mathrm{pos}}{10000^{\frac{2i}{d}}}\right) \qquad \mathrm{PE}_{(pos,2i+1)} = \cos\left(\frac{\mathrm{pos}}{10000^{\frac{2i}{d}}}\right)$$

pos

0: [ ]

1: [ ]

2: [ ]

⋮

Positional Encoding

- Original transformer way

$$\mathrm{PE}_{(pos,2i)} = \sin\left(\frac{\mathrm{pos}}{10000^{\frac{2i}{d}}}\right) \qquad \mathrm{PE}_{(pos,2i+1)} = \cos\left(\frac{\mathrm{pos}}{10000^{\frac{2i}{d}}}\right)$$

$$\begin{array}{lcc} & \overset{i=0}{\mathrm{PE}_{(pos,0)}} & \overset{i=1}{\mathrm{PE}_{(pos,2)}} \\ \mathrm{pos} & & \\ 0: & \left[\sin\left(\frac{\mathrm{pos}}{10000^{\frac{2i}{d}}}\right), \right. & \left., \sin\left(\frac{\mathrm{pos}}{10000^{\frac{2i}{d}}}\right), \right] \\ 1: & \left[\sin\left(\frac{\mathrm{pos}}{10000^{\frac{2i}{d}}}\right), \right. & \left., \sin\left(\frac{\mathrm{pos}}{10000^{\frac{2i}{d}}}\right), \right] \\ 2: & \left[\sin\left(\frac{\mathrm{pos}}{10000^{\frac{2i}{d}}}\right), \right. & \left., \sin\left(\frac{\mathrm{pos}}{10000^{\frac{2i}{d}}}\right), \right] \\ \vdots & \vdots & \vdots \end{array}$$

sin for the even index

Positional Encoding

- Original transformer way

$$\mathrm{PE}_{(pos,2i)} = \sin\left(\frac{\mathrm{pos}}{10000^{\frac{2i}{d}}}\right) \qquad \mathrm{PE}_{(pos,2i+1)} = \cos\left(\frac{\mathrm{pos}}{10000^{\frac{2i}{d}}}\right)$$

pos	$i=0$ $\mathrm{PE}_{(pos,0)}$	$i=0$ $\mathrm{PE}_{(pos,1)}$	$i=1$ $\mathrm{PE}_{(pos,2)}$	$i=1$ $\mathrm{PE}_{(pos,3)}$
0:	$\sin\left(\frac{\mathrm{pos}}{10000^{\frac{2i}{d}}}\right)$	$\cos\left(\frac{\mathrm{pos}}{10000^{\frac{2i}{d}}}\right)$	$\sin\left(\frac{\mathrm{pos}}{10000^{\frac{2i}{d}}}\right)$	$\cos\left(\frac{\mathrm{pos}}{10000^{\frac{2i}{d}}}\right)$
1:	$\sin\left(\frac{\mathrm{pos}}{10000^{\frac{2i}{d}}}\right)$	$\cos\left(\frac{\mathrm{pos}}{10000^{\frac{2i}{d}}}\right)$	$\sin\left(\frac{\mathrm{pos}}{10000^{\frac{2i}{d}}}\right)$	$\cos\left(\frac{\mathrm{pos}}{10000^{\frac{2i}{d}}}\right)$
2:	$\sin\left(\frac{\mathrm{pos}}{10000^{\frac{2i}{d}}}\right)$	$\cos\left(\frac{\mathrm{pos}}{10000^{\frac{2i}{d}}}\right)$	$\sin\left(\frac{\mathrm{pos}}{10000^{\frac{2i}{d}}}\right)$	$\cos\left(\frac{\mathrm{pos}}{10000^{\frac{2i}{d}}}\right)$
⋮	⋮	⋮	⋮	⋮

cos for the odd index

Positional Encoding

- Original transformer way

$$\mathrm{PE}_{(pos,2i)} = \sin\left(\frac{\mathrm{pos}}{10000^{\frac{2i}{d}}}\right) \qquad \mathrm{PE}_{(pos,2i+1)} = \cos\left(\frac{\mathrm{pos}}{10000^{\frac{2i}{d}}}\right)$$

pos	$\overset{i=0}{\mathrm{PE}_{(pos,0)}}$	$\overset{i=0}{\mathrm{PE}_{(pos,1)}}$	$\overset{i=1}{\mathrm{PE}_{(pos,2)}}$	$\overset{i=1}{\mathrm{PE}_{(pos,3)}}$
0:	$\left[\sin\left(\frac{0}{10000^{\frac{2i}{d}}}\right)\right.$	$\cos\left(\frac{0}{10000^{\frac{2i}{d}}}\right)$	$\sin\left(\frac{0}{10000^{\frac{2i}{d}}}\right)$	$\left.\cos\left(\frac{0}{10000^{\frac{2i}{d}}}\right)\right]$
1:	$\left[\sin\left(\frac{1}{10000^{\frac{2i}{d}}}\right)\right.$	$\cos\left(\frac{1}{10000^{\frac{2i}{d}}}\right)$	$\sin\left(\frac{1}{10000^{\frac{2i}{d}}}\right)$	$\left.\cos\left(\frac{1}{10000^{\frac{2i}{d}}}\right)\right]$
2:	$\left[\sin\left(\frac{2}{10000^{\frac{2i}{d}}}\right)\right.$	$\cos\left(\frac{2}{10000^{\frac{2i}{d}}}\right)$	$\sin\left(\frac{2}{10000^{\frac{2i}{d}}}\right)$	$\left.\cos\left(\frac{2}{10000^{\frac{2i}{d}}}\right)\right]$
⋮	⋮	⋮	⋮	⋮

pos: word position

Positional Encoding

- Original transformer way

$$\mathrm{PE}_{(pos,2i)} = \sin\left(\frac{\mathrm{pos}}{10000^{\frac{2i}{d}}}\right) \qquad \mathrm{PE}_{(pos,2i+1)} = \cos\left(\frac{\mathrm{pos}}{10000^{\frac{2i}{d}}}\right)$$

$$\begin{array}{c|cccc} \mathrm{pos} & \overset{i=0}{\mathrm{PE}_{(pos,0)}} & \overset{i=0}{\mathrm{PE}_{(pos,1)}} & \overset{i=1}{\mathrm{PE}_{(pos,2)}} & \overset{i=1}{\mathrm{PE}_{(pos,3)}} \\ 0: & \sin\left(\frac{0}{10000^{\frac{2i}{4}}}\right) & \cos\left(\frac{0}{10000^{\frac{2i}{4}}}\right) & \sin\left(\frac{0}{10000^{\frac{2i}{4}}}\right) & \cos\left(\frac{0}{10000^{\frac{2i}{4}}}\right) \\ 1: & \sin\left(\frac{1}{10000^{\frac{2i}{4}}}\right) & \cos\left(\frac{1}{10000^{\frac{2i}{4}}}\right) & \sin\left(\frac{1}{10000^{\frac{2i}{4}}}\right) & \cos\left(\frac{1}{10000^{\frac{2i}{4}}}\right) \\ 2: & \sin\left(\frac{2}{10000^{\frac{2i}{4}}}\right) & \cos\left(\frac{2}{10000^{\frac{2i}{4}}}\right) & \sin\left(\frac{2}{10000^{\frac{2i}{4}}}\right) & \cos\left(\frac{2}{10000^{\frac{2i}{4}}}\right) \\ \vdots & \vdots & \vdots & \vdots & \vdots \end{array}$$

d: vector size (our example = 4)

Positional Encoding

- Original transformer way

$$\mathrm{PE}_{(pos,2i)} = \sin\left(\frac{\mathrm{pos}}{10000^{\frac{2i}{d}}}\right) \qquad \mathrm{PE}_{(pos,2i+1)} = \cos\left(\frac{\mathrm{pos}}{10000^{\frac{2i}{d}}}\right)$$

pos	$\mathrm{PE}_{(pos,0)}$ ($i=0$)	$\mathrm{PE}_{(pos,1)}$ ($i=0$)	$\mathrm{PE}_{(pos,2)}$ ($i=1$)	$\mathrm{PE}_{(pos,3)}$ ($i=1$)
0:	$\sin\left(\frac{0}{10000^{\frac{2*0}{4}}}\right)$	$\cos\left(\frac{0}{10000^{\frac{2*0}{4}}}\right)$	$\sin\left(\frac{0}{10000^{\frac{2*1}{4}}}\right)$	$\cos\left(\frac{0}{10000^{\frac{2*1}{4}}}\right)$
1:	$\sin\left(\frac{1}{10000^{\frac{2*0}{4}}}\right)$	$\cos\left(\frac{1}{10000^{\frac{2*0}{4}}}\right)$	$\sin\left(\frac{1}{10000^{\frac{2*1}{4}}}\right)$	$\cos\left(\frac{1}{10000^{\frac{2*1}{4}}}\right)$
2:	$\sin\left(\frac{2}{10000^{\frac{2*0}{4}}}\right)$	$\cos\left(\frac{2}{10000^{\frac{2*0}{4}}}\right)$	$\sin\left(\frac{2}{10000^{\frac{2*1}{4}}}\right)$	$\cos\left(\frac{2}{10000^{\frac{2*1}{4}}}\right)$
⋮	⋮	⋮	⋮	⋮

i: vector index, which will define the frequency of the sin/cos function

Positional Encoding

- Original transformer way

$$\mathrm{PE}_{(pos,2i)} = \sin\left(\frac{\mathrm{pos}}{10000^{\frac{2i}{d}}}\right) \qquad \mathrm{PE}_{(pos,2i+1)} = \cos\left(\frac{\mathrm{pos}}{10000^{\frac{2i}{d}}}\right)$$

$$\begin{array}{c|cccc} \mathrm{pos} & \mathrm{PE}_{(pos,0)}\ (i=0) & \mathrm{PE}_{(pos,1)}\ (i=0) & \mathrm{PE}_{(pos,2)}\ (i=1) & \mathrm{PE}_{(pos,3)}\ (i=1) \\ 0: & \sin\left(\frac{0}{10000^{\frac{0}{4}}}\right) & \cos\left(\frac{0}{10000^{\frac{0}{4}}}\right) & \sin\left(\frac{0}{10000^{\frac{2}{4}}}\right) & \cos\left(\frac{0}{10000^{\frac{2}{4}}}\right) \\ 1: & \sin\left(\frac{1}{10000^{\frac{0}{4}}}\right) & \cos\left(\frac{1}{10000^{\frac{0}{4}}}\right) & \sin\left(\frac{1}{10000^{\frac{2}{4}}}\right) & \cos\left(\frac{1}{10000^{\frac{2}{4}}}\right) \\ 2: & \sin\left(\frac{2}{10000^{\frac{0}{4}}}\right) & \cos\left(\frac{2}{10000^{\frac{0}{4}}}\right) & \sin\left(\frac{2}{10000^{\frac{2}{4}}}\right) & \cos\left(\frac{2}{10000^{\frac{2}{4}}}\right) \\ \vdots & \vdots & \vdots & \vdots & \vdots \end{array}$$

Positional Encoding

- Original transformer way

$$\mathrm{PE}_{(pos,2i)} = \sin\left(\frac{\mathrm{pos}}{10000^{\frac{2i}{d}}}\right) \qquad \mathrm{PE}_{(pos,2i+1)} = \cos\left(\frac{\mathrm{pos}}{10000^{\frac{2i}{d}}}\right)$$

$$\begin{array}{c|cccc} \mathrm{pos} & \overset{i=0}{\mathrm{PE}_{(pos,0)}} & \overset{i=0}{\mathrm{PE}_{(pos,1)}} & \overset{i=1}{\mathrm{PE}_{(pos,2)}} & \overset{i=1}{\mathrm{PE}_{(pos,3)}} \\ 0: & \sin\left(\frac{0}{10000^{\frac{0}{4}}}\right) & \cos\left(\frac{0}{10000^{\frac{0}{4}}}\right) & \sin\left(\frac{0}{10000^{\frac{2}{4}}}\right) & \cos\left(\frac{0}{10000^{\frac{2}{4}}}\right) \\ 1: & \sin\left(\frac{1}{10000^{\frac{0}{4}}}\right) & \cos\left(\frac{1}{10000^{\frac{0}{4}}}\right) & \sin\left(\frac{1}{10000^{\frac{2}{4}}}\right) & \cos\left(\frac{1}{10000^{\frac{2}{4}}}\right) \\ 2: & \sin\left(\frac{2}{10000^{\frac{0}{4}}}\right) & \cos\left(\frac{2}{10000^{\frac{0}{4}}}\right) & \sin\left(\frac{2}{10000^{\frac{2}{4}}}\right) & \cos\left(\frac{2}{10000^{\frac{2}{4}}}\right) \\ \vdots & \vdots & \vdots & \vdots & \vdots \end{array}$$

- No maximum limitation on the position

Positional Encoding

- Like having different signal & frequency for each vector position

$$\mathrm{PE}_{(pos,2i)} = \sin\left(\frac{\mathrm{pos}}{10000^{\frac{2i}{d}}}\right) \qquad \mathrm{PE}_{(pos,2i+1)} = \cos\left(\frac{\mathrm{pos}}{10000^{\frac{2i}{d}}}\right)$$

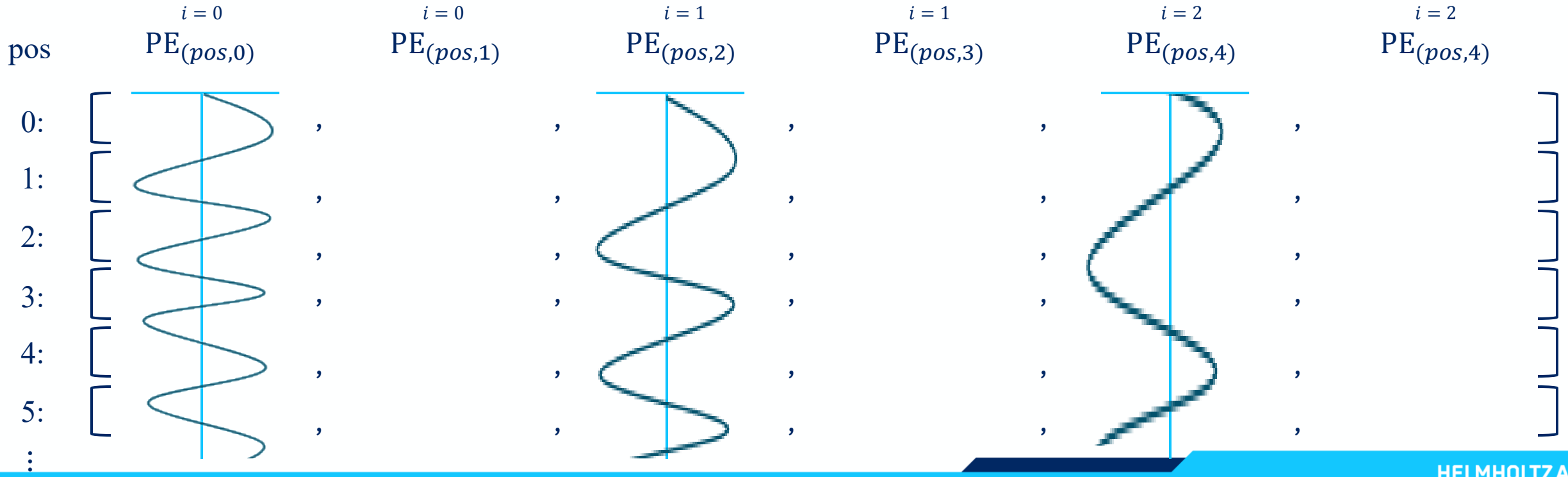
pos	$i=0$ $\mathrm{PE}_{(pos,0)}$	$i=0$ $\mathrm{PE}_{(pos,1)}$	$i=1$ $\mathrm{PE}_{(pos,2)}$	$i=1$ $\mathrm{PE}_{(pos,3)}$	$i=2$ $\mathrm{PE}_{(pos,4)}$	$i=2$ $\mathrm{PE}_{(pos,4)}$
0:	[,	,	,	,	,	]
1:	[,	,	,	,	,	]
2:	[,	,	,	,	,	]
3:	[,	,	,	,	,	]
4:	[,	,	,	,	,	]
5:	[,	,	,	,	,	]
⋮						

Positional Encoding

- Like having different signal & frequency for each vector position

$$\mathrm{PE}_{(pos,2i)} = \sin\left(\frac{\mathrm{pos}}{10000^{\frac{2i}{d}}}\right) \qquad \mathrm{PE}_{(pos,2i+1)} = \cos\left(\frac{\mathrm{pos}}{10000^{\frac{2i}{d}}}\right)$$

For sin...

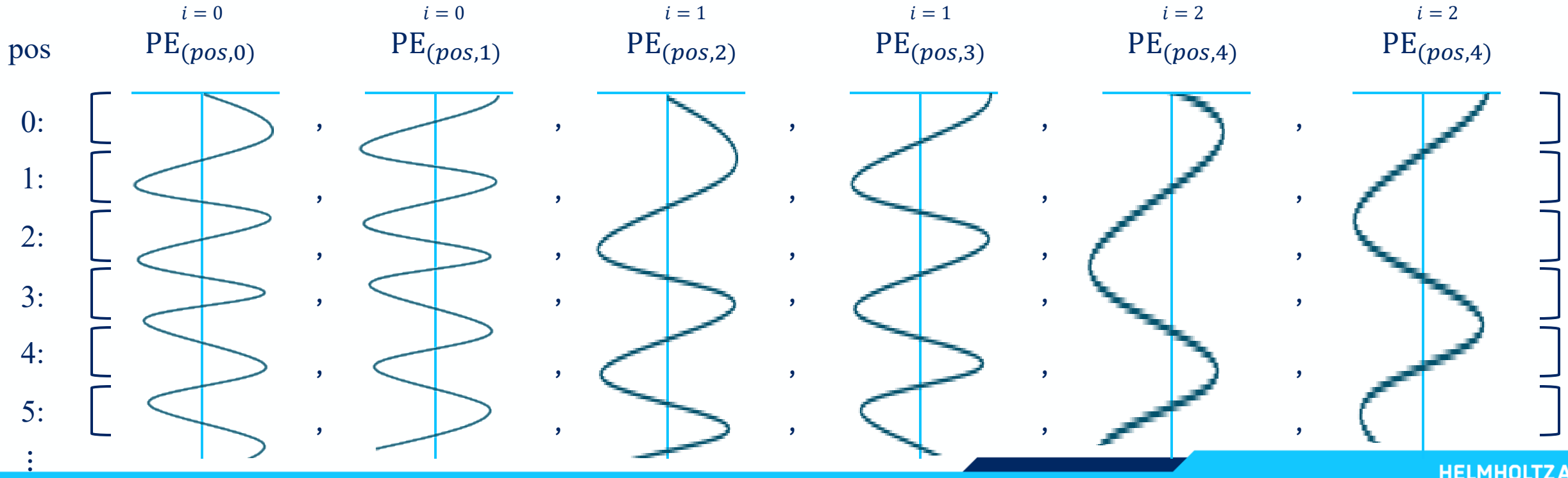


Positional Encoding

- Like having different signal & frequency for each vector position

$$\mathrm{PE}_{(pos,2i)} = \sin\left(\frac{\mathrm{pos}}{10000^{\frac{2i}{d}}}\right) \qquad \mathrm{PE}_{(pos,2i+1)} = \cos\left(\frac{\mathrm{pos}}{10000^{\frac{2i}{d}}}\right)$$

The rest, cos...



Positional Encoding

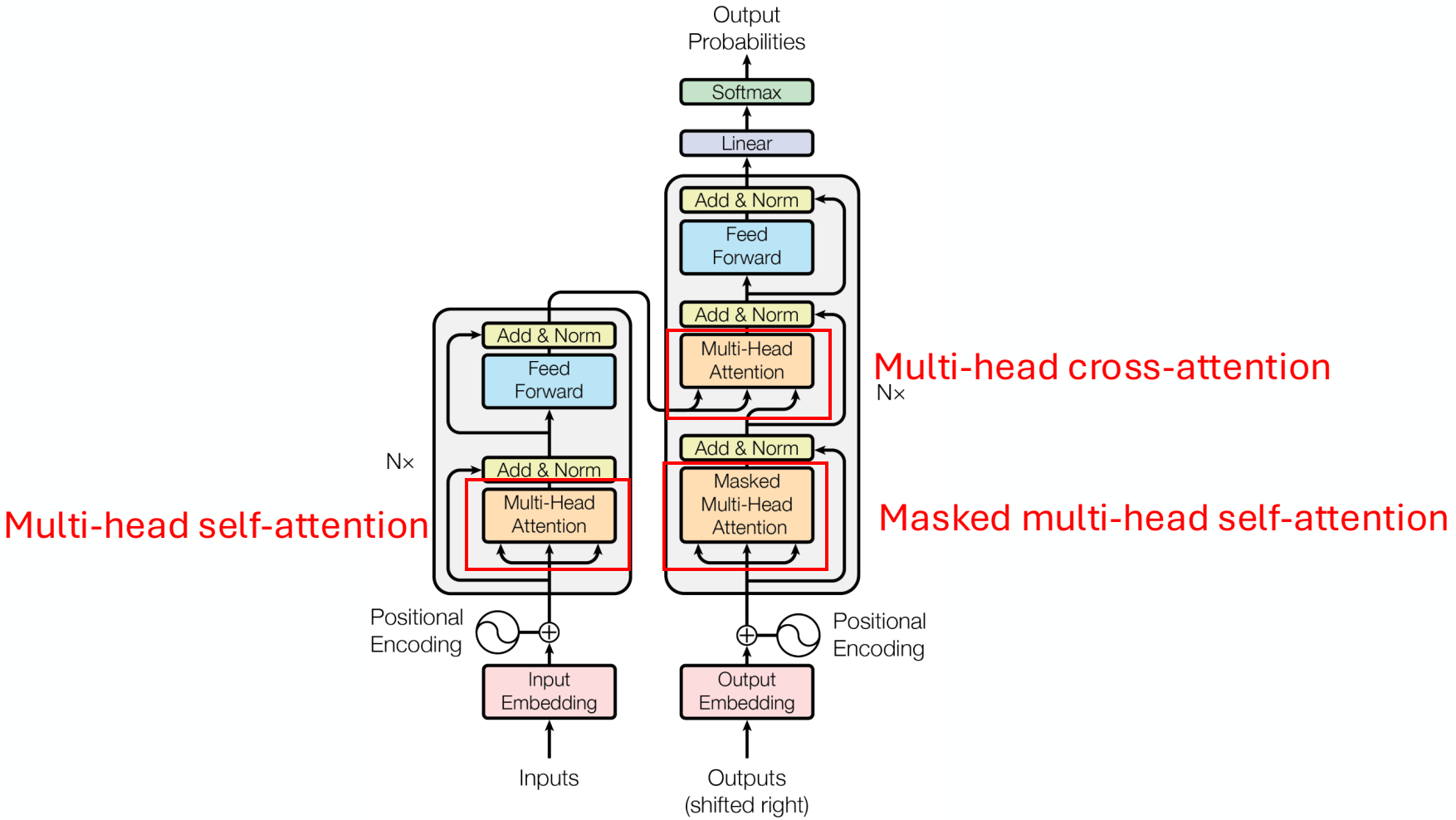
- Other ways
  - Learnable embedding → learn the position vectors
    - Done in Vision Transformer (ViT)
  - Relative positional encoding
  - Rotary Positional Embedding (RoPE)
  - Attention with Linear Biases (ALiBi)
  - etc.

We will not discuss further…

Outline

1. Overview
2. Preprocessing
3. Attention mechanism
4. Putting it all together
5. Variants of Transformer

Attention Mechanism



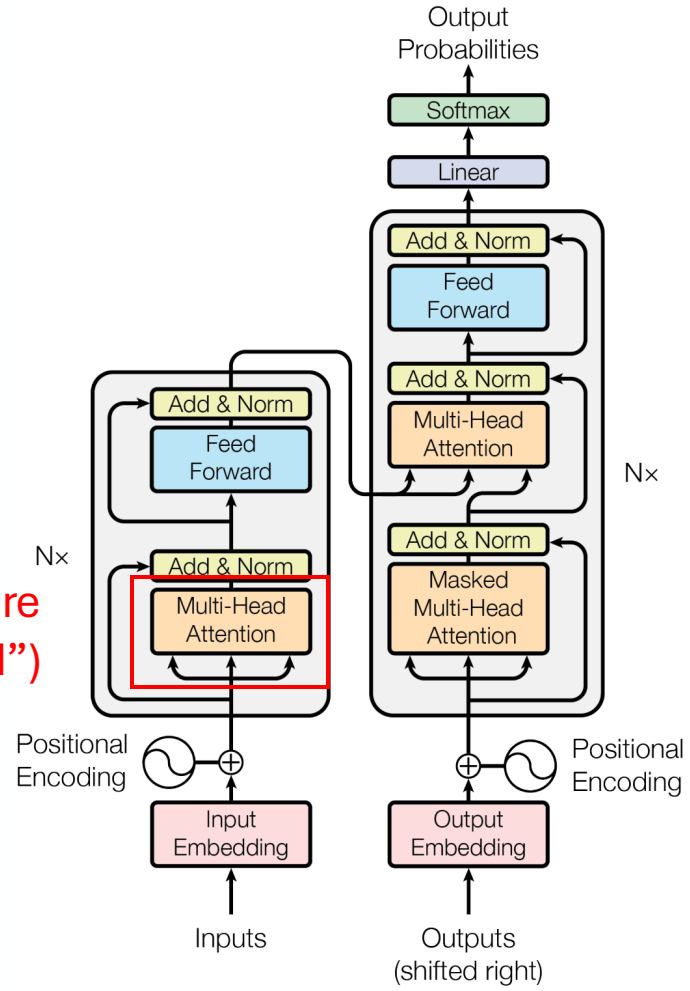
Attention Mechanism

- In this presentation:
  - **Self**-attention
  - **Masked** self-attention
  - **Cross**-attention
  - **Multi-head** self-attention

Self-Attention

- Input:
  input word embedding +
  positional encoding;
  OR
  the previous encoder block's
  output

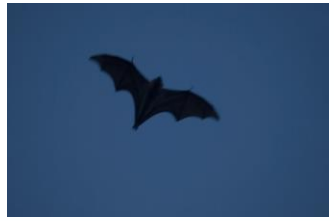
Self-attention is here
(we will discuss first without “multi-head”)



Self-Attention

- Motivation: context is important
  - Need to see other words to know context.

“bat”



“The bat flies in the cave.”

“He swings the bat.”

“right”

“Turn right”

“The right to give opinions”

“I am right, you are wrong.”

Self-Attention

- Intuition

We can know which “bat” because of ”flies” and “cave”

We can know which “bat” because of ”swings” and “he” (a person do something)

Self-Attention

- Intuition

“I poured water from the bottle into the cup until **it** was full.”

"it" refers to what?

“I poured water from the bottle into the cup until **it** was empty.”

Self-Attention

- Intuition

“I poured water from the bottle into the cup until **it** was full.”

"it" refers to cup

“I poured water from the bottle into the cup until **it** was empty.”

"it" refers to bottle

Self-Attention

- Intuition

How can we know?

"I **poured** water from the bottle **into** the **cup** until **it** was **full**."

We can know "it" refers to "cup" because something is **poured into cup** until it was **full**

"I **poured** water **from** the **bottle** into the cup until **it** was **empty**."

We can know "it" refers to "bottle" to because something is **poured from bottle** until it was **empty**

Self-Attention

- Use ”key”, ”query”, “value”. The interpretation:

“The **bat** flies in the cave”

Self-Attention

- Use ”key”, ”query”, “value”. The interpretation:

“The **bat** flies in the cave”

Query: bat

Keys: the, bat, flies, in, the, cave

Self-Attention

- Use ”key”, ”query”, “value”. The interpretation:

“The **bat** flies in the cave”

Query: bat

Which word is relevant to me?

Keys: the, bat, flies, in, the, cave

Self-Attention

- Use ”key”, ”query”, “value”. The interpretation:

“The **bat** flies in the cave”

Query: bat

Which word is relevant to me?

Keys: the, bat, flies, in, the, cave

ME ME ME ME ME ME

Keys compete for attention

Self-Attention

- Use ”key”, ”query”, “value”. The interpretation:

“The **bat** flies in the cave”

Query: bat

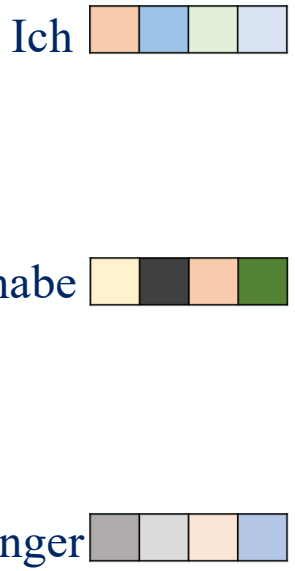
Keys: the, bat, **flies**, in, the, **cave**

Values: the, bat, flies, in, the, cave

If I got the attention, this is the value/information I will give

Self-Attention

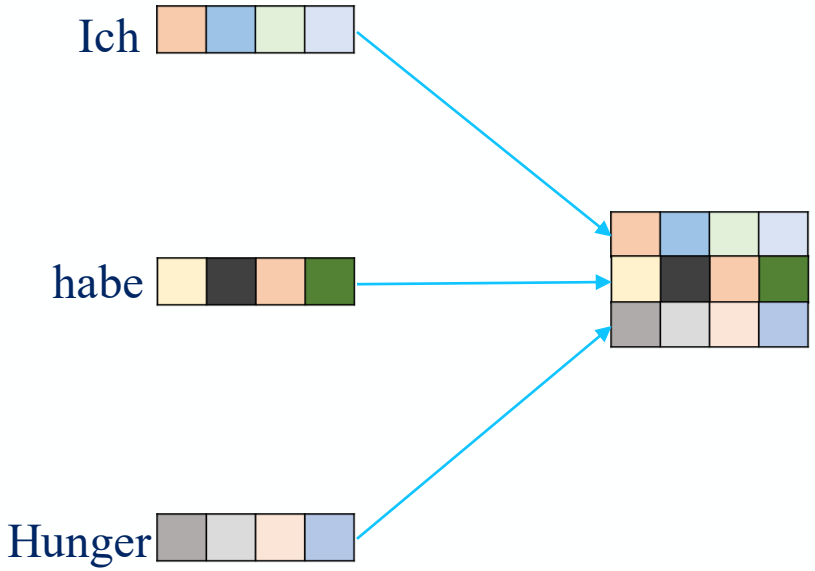
- Step 1: create query, key, value



Vectors from word embedding and positional encoding

Self-Attention

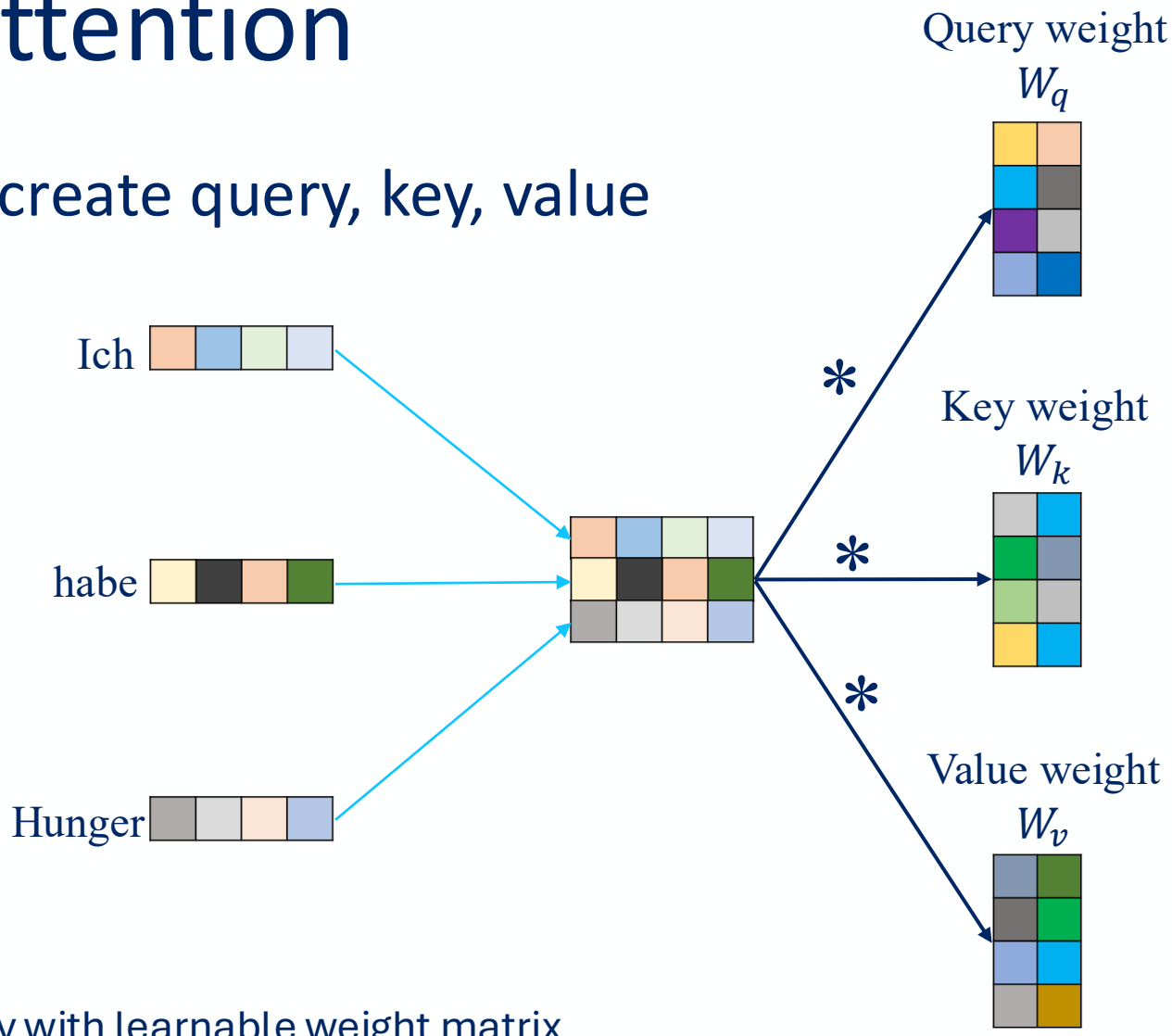
- Step 1: create query, key, value



Combine them to matrix

Self-Attention

- Step 1: create query, key, value

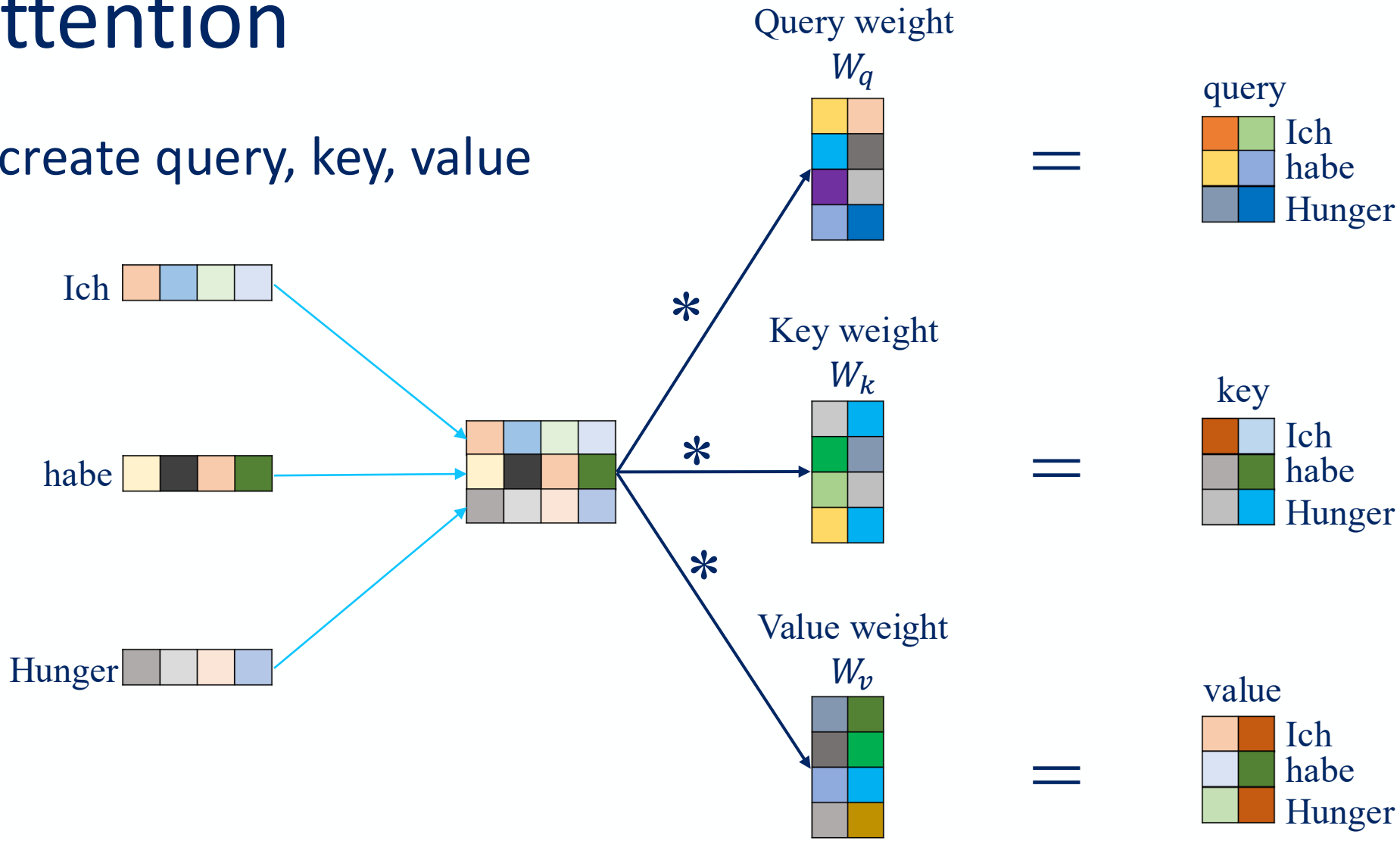


Multiply with learnable weight matrix

Self-Attention

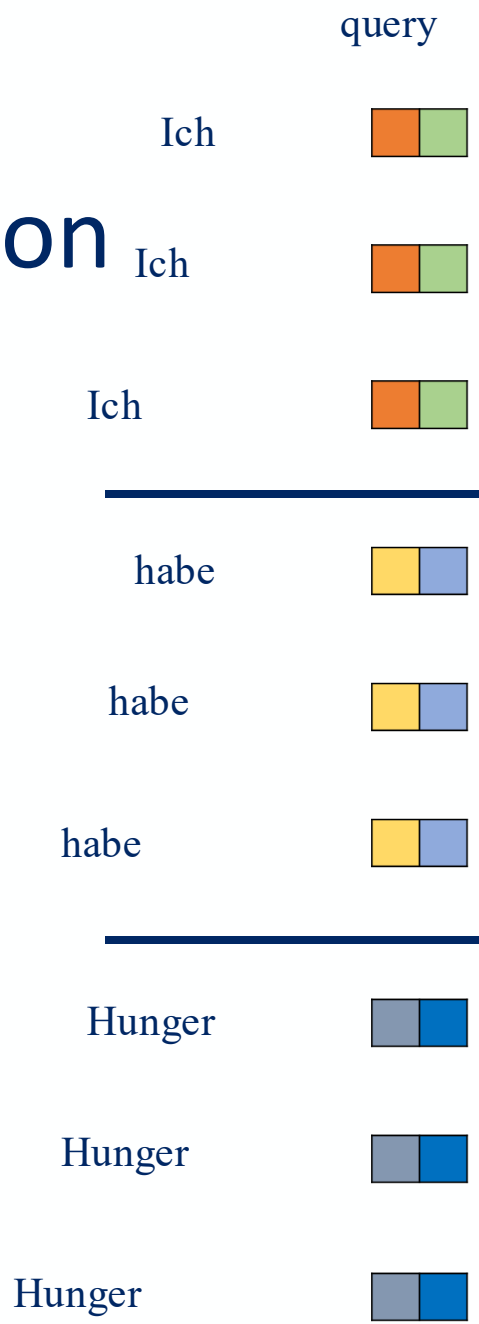
- Step 1: create query, key, value

Each word has its own query, key, value



Self-Attention

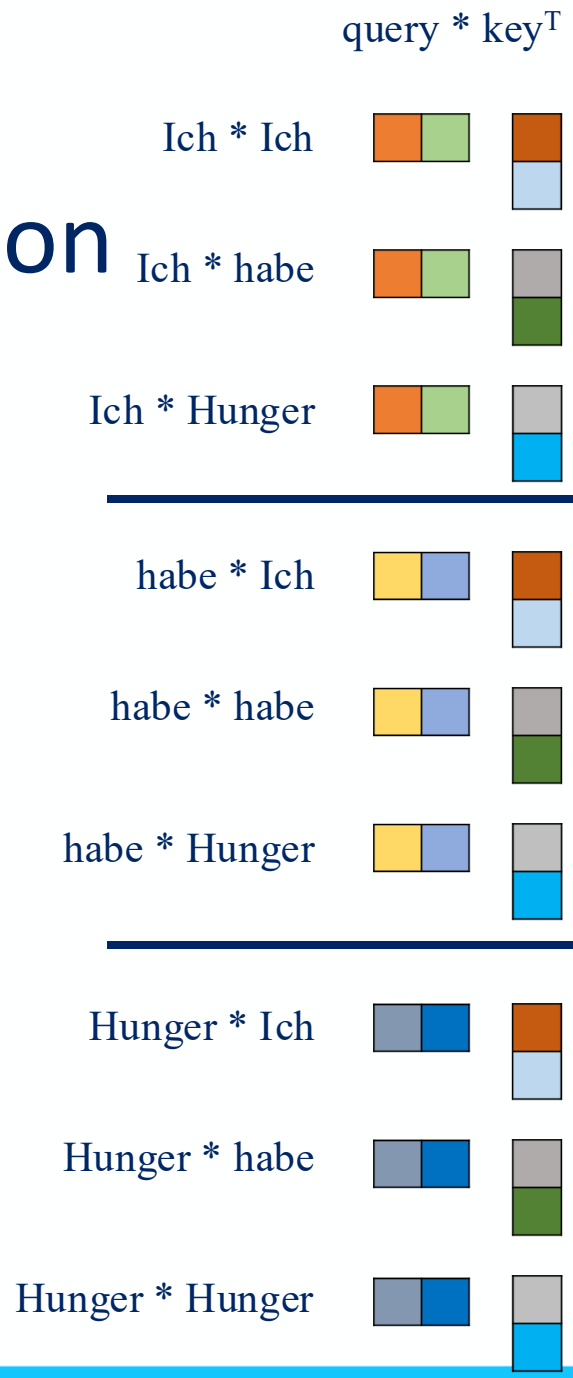
- Step 2: Finding attention between words



Query of each word

Self-Attention

- Step 2: Finding attention between words



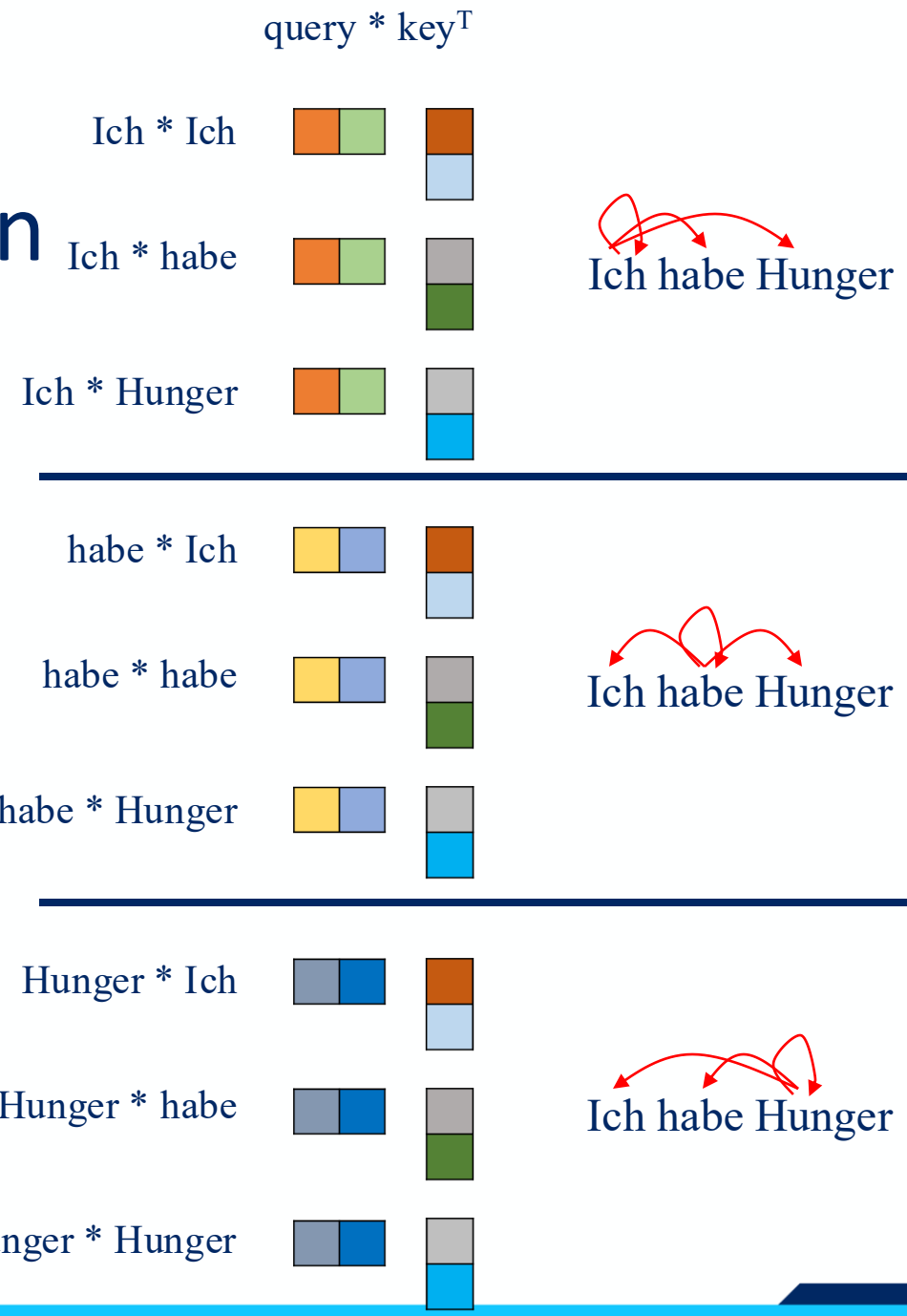
Multiply query of each word with the transpose of key of all words

Self-Attention

- Step 2: Finding attention between words

Self-Attention

- Step 2: Finding attention between words



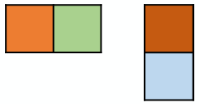
The dot product will build the “connection" between words

The keys compete for query’s attention

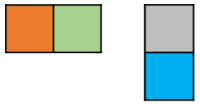
Self-Attention

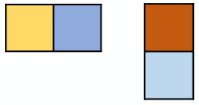
- Step 2: Finding attention between words

query * key^T

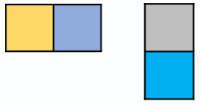
Ich * Ich  = 69

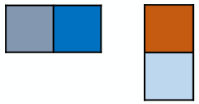
Ich * habe  = 10

Ich * Hunger  = 12

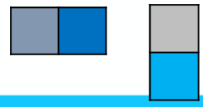
habe * Ich  = 21

habe * habe  = 70

habe * Hunger  = 3

Hunger * Ich  = 11

Hunger * habe  = 5

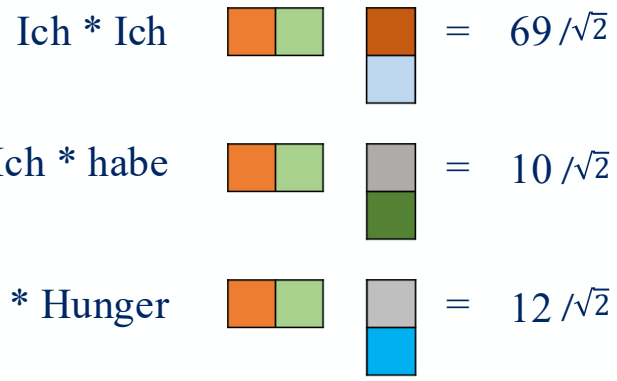
Hunger * Hunger  = 90

*Disclaimer: not real numbers from a real model. Only for illustration purpose

Self-Attention

- Step 2: Finding attention between words

query * key^T



$$\frac{QK^T}{\sqrt{d_k}}$$

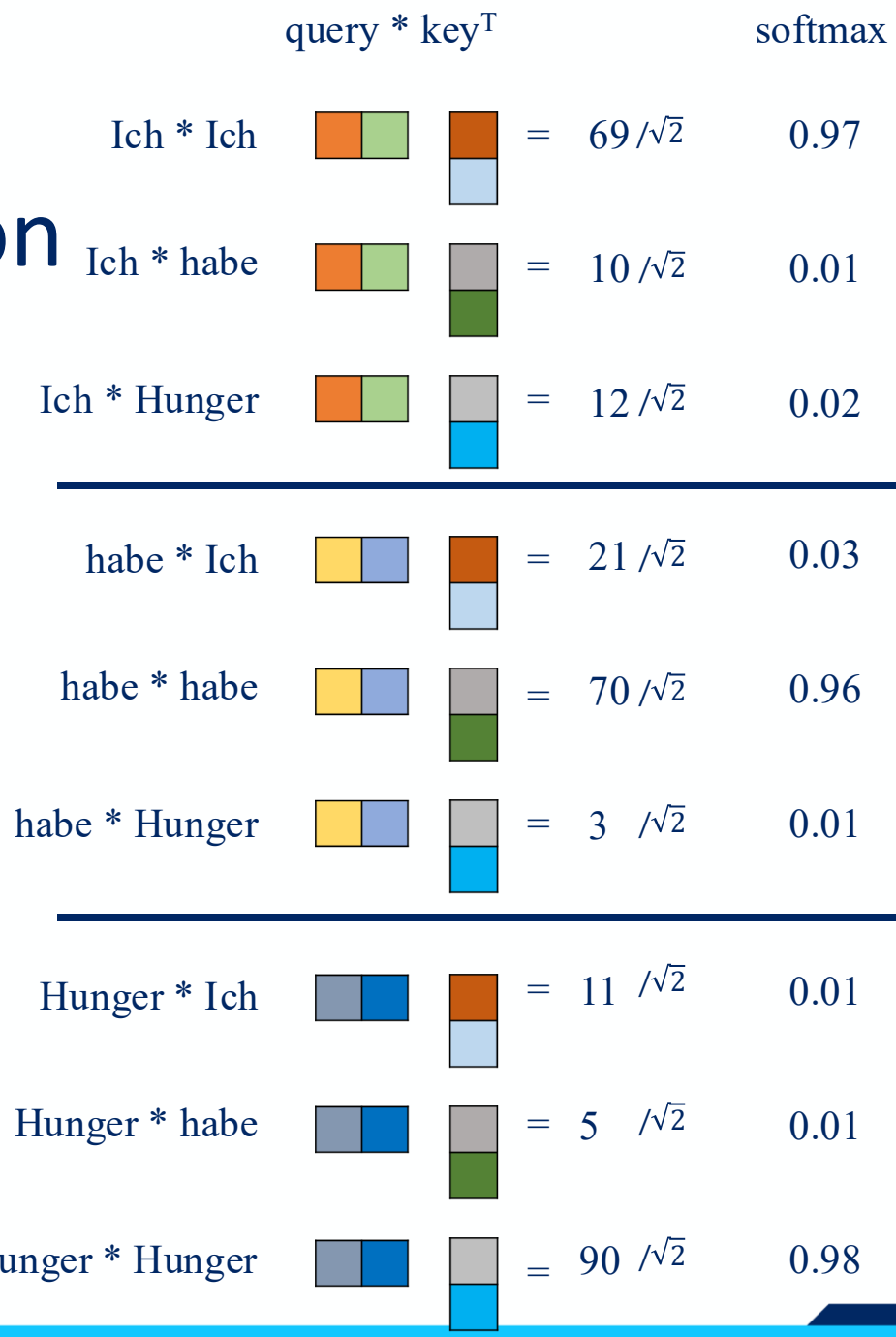
Query or key's vector size = 2

Normalize the dot product with vector size because dot product value depend on the vector size

*Disclaimer: not real numbers from a real model. Only for illustration purpose

Self-Attention

- Step 2: Finding attention between words

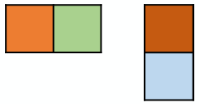
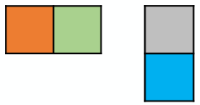
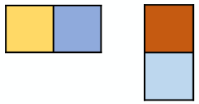
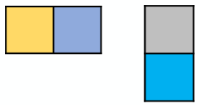
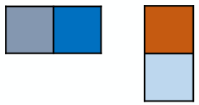
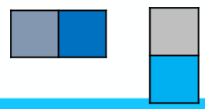


Softmax to get the connection's "probability"

*Disclaimer: not real numbers from a real model. Only for illustration purpose

Self-Attention

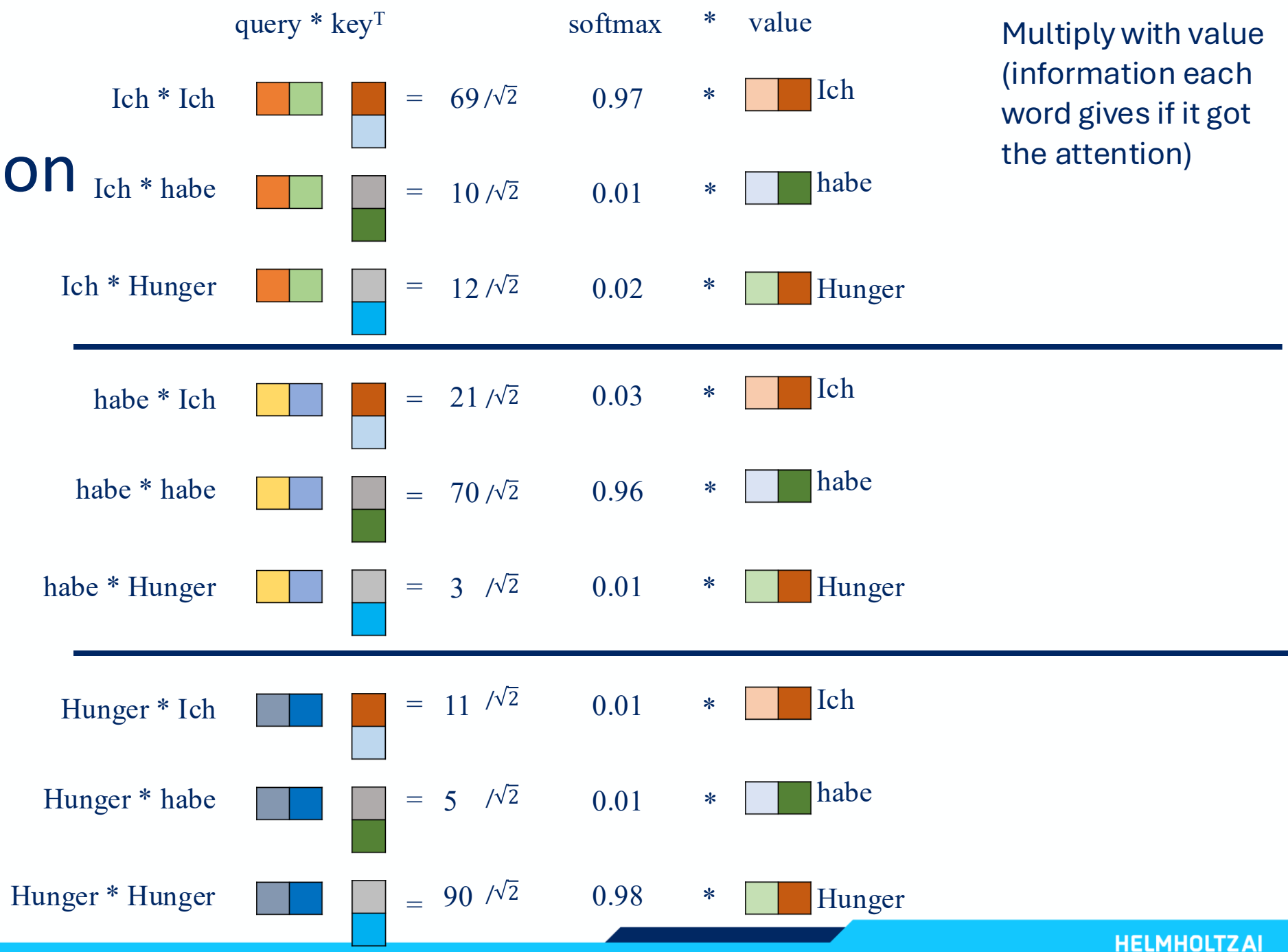
- Step 2: Finding attention between words

	query * key^T		softmax
Ich * Ich		$= 69 / \sqrt{2}$	0.97
Ich * habe		$= 10 / \sqrt{2}$	0.01
Ich * Hunger		$= 12 / \sqrt{2}$	0.02
habe * Ich		$= 21 / \sqrt{2}$	0.03
habe * habe		$= 70 / \sqrt{2}$	0.96
habe * Hunger		$= 3 / \sqrt{2}$	0.01
Hunger * Ich		$= 11 / \sqrt{2}$	0.01
Hunger * habe		$= 5 / \sqrt{2}$	0.01
Hunger * Hunger		$= 90 / \sqrt{2}$	0.98

The softmax value represents the connection strength between words

Self-Attention

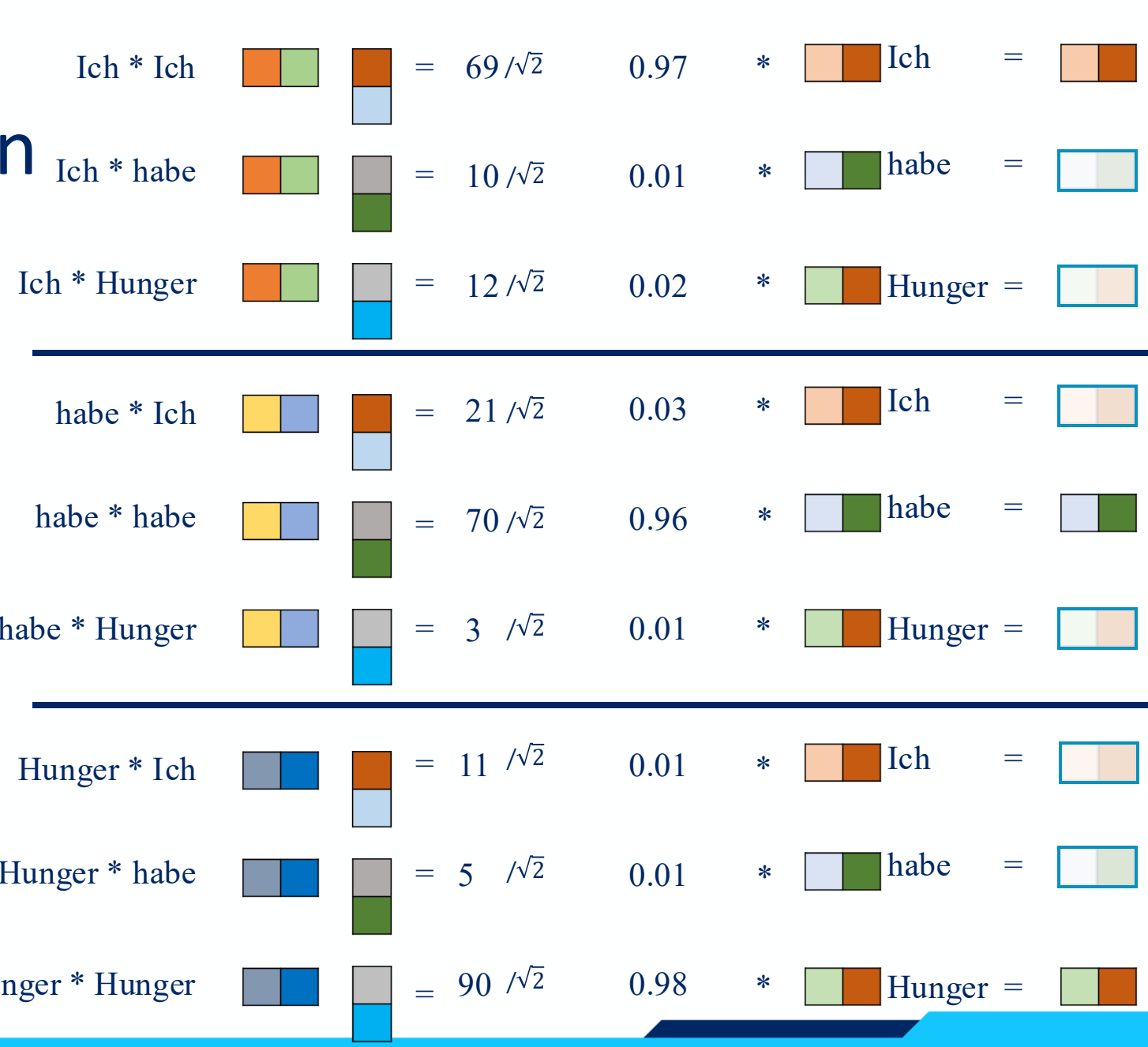
- Step 2: Finding attention between words



Multiply with value (information each word gives if it got the attention)

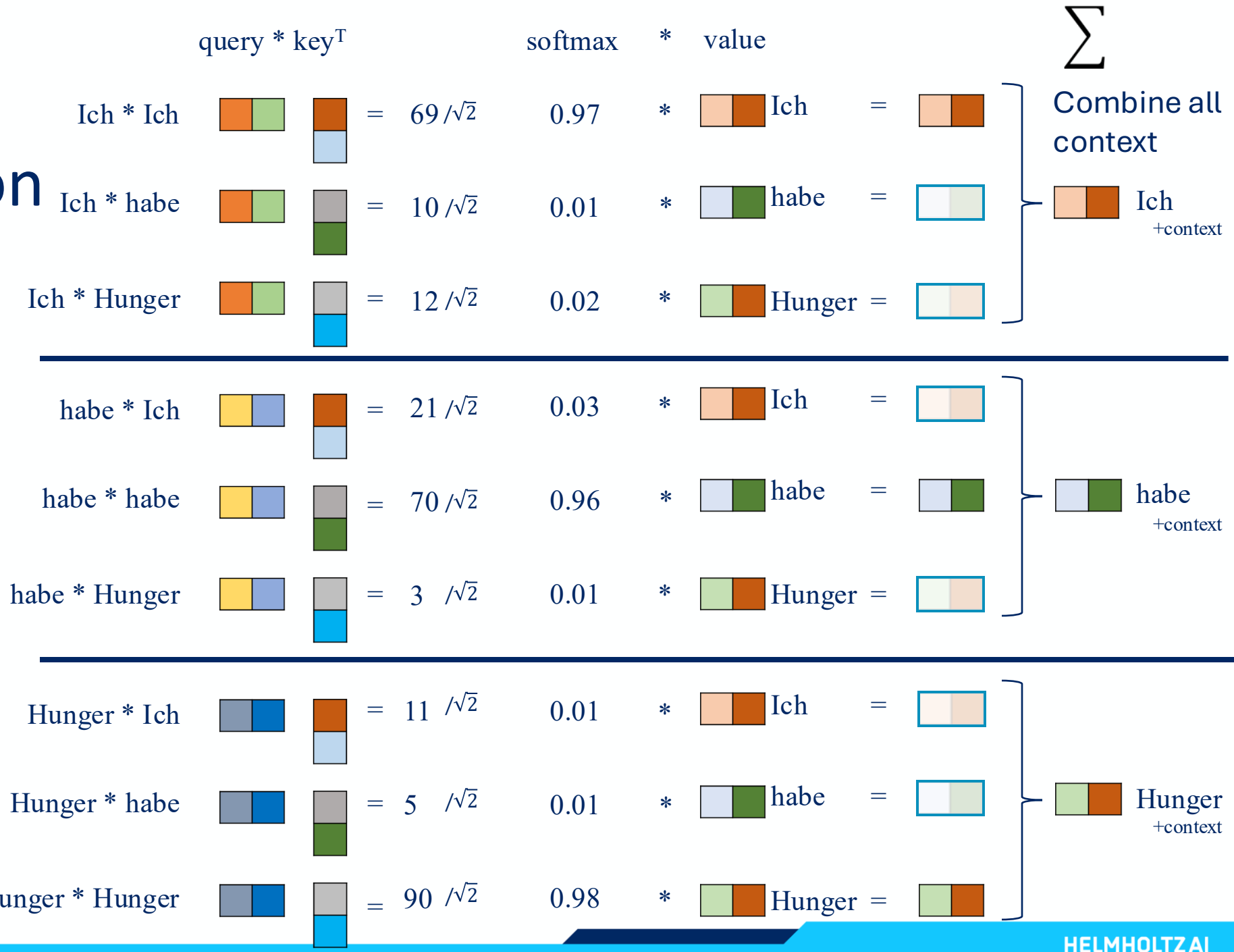
Self-Attention

- Step 2: Finding attention between words



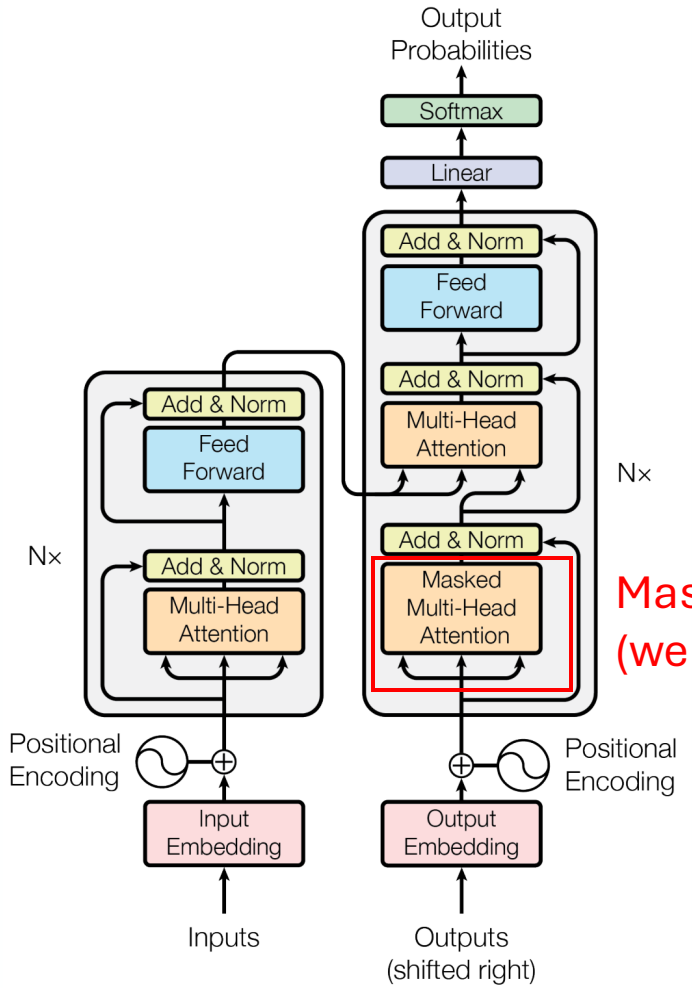
Self-Attention

- Step 2: Finding attention between words



Masked Self-Attention

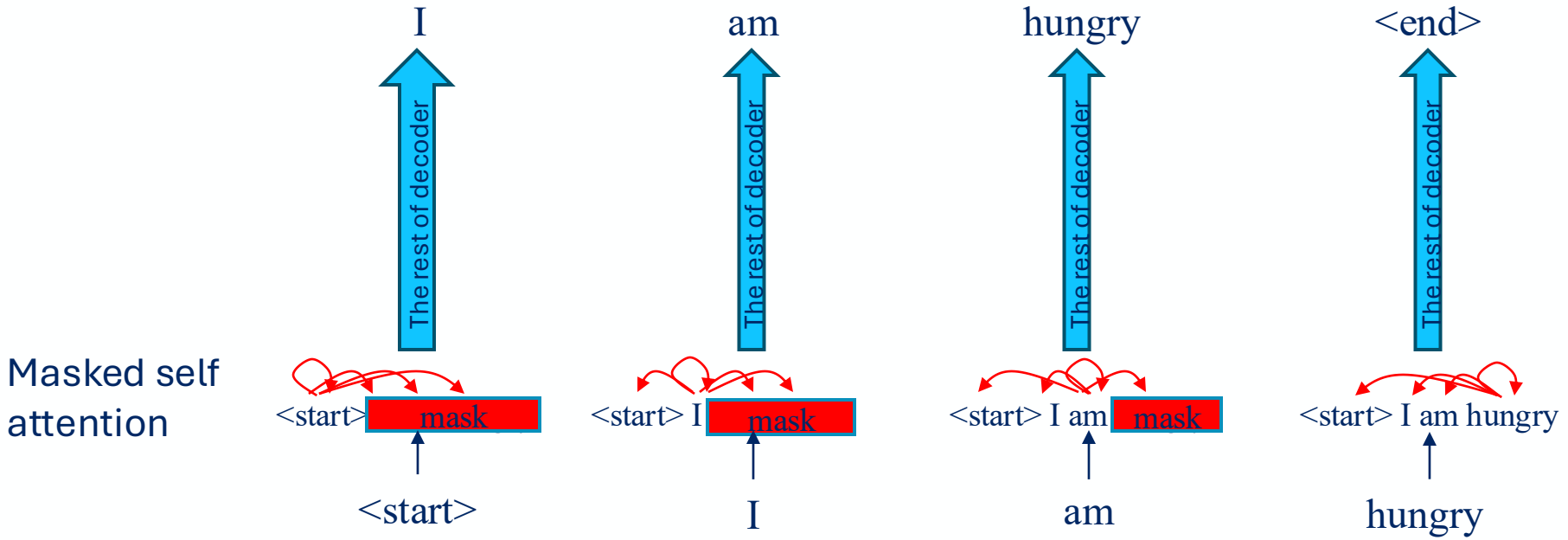
- Input: word embedding of output (ground truth) shifted to right + positional encoding;
  OR
  previous decoder block's output



Masked self-attention is here
(we will discuss first without “multi-head”)

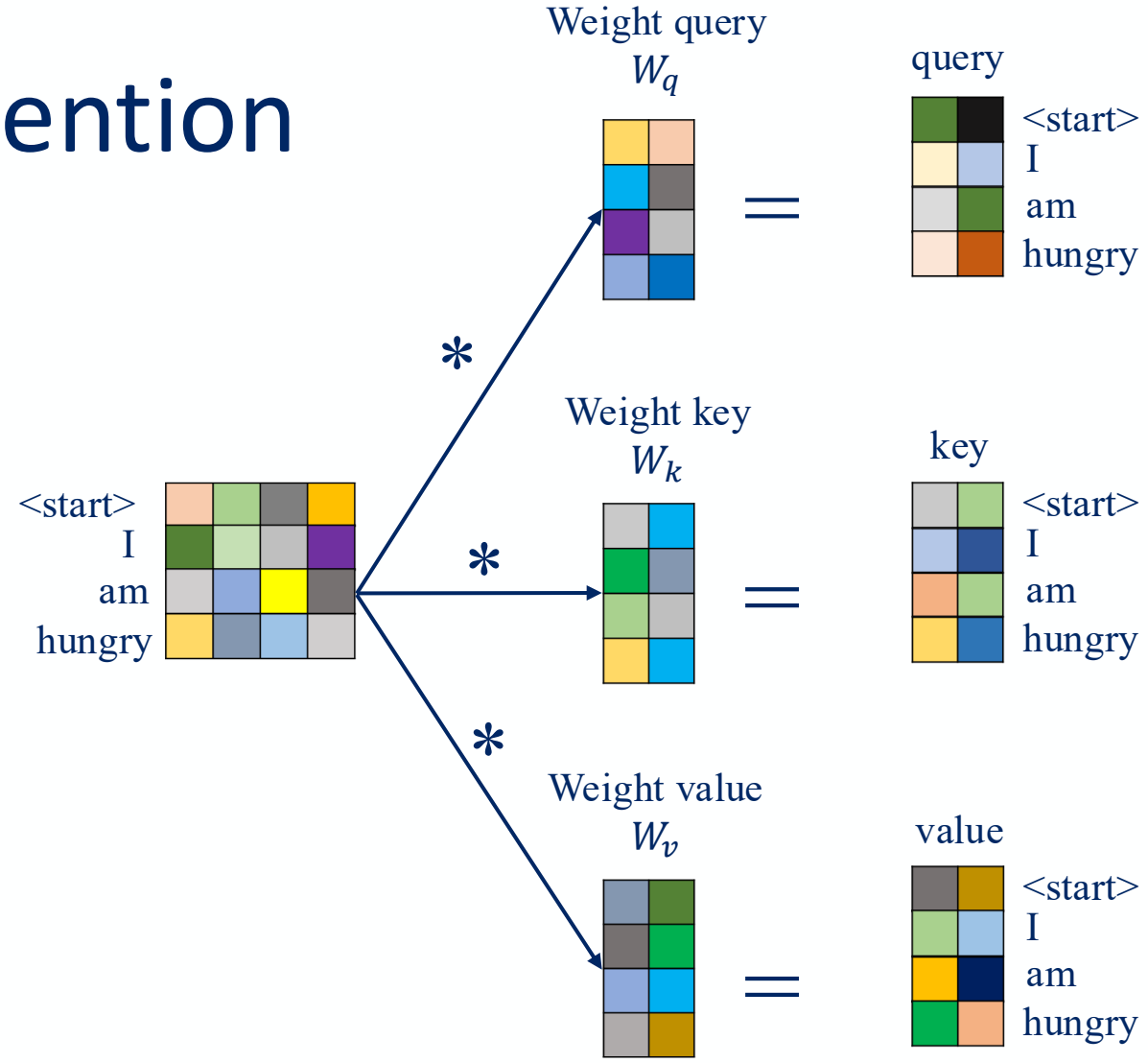
Masked Self-Attention

- Motivation of mask:
  - Decoder's self attention should not see the future words
    - No future words during test time.
  - Still parallelizable during training



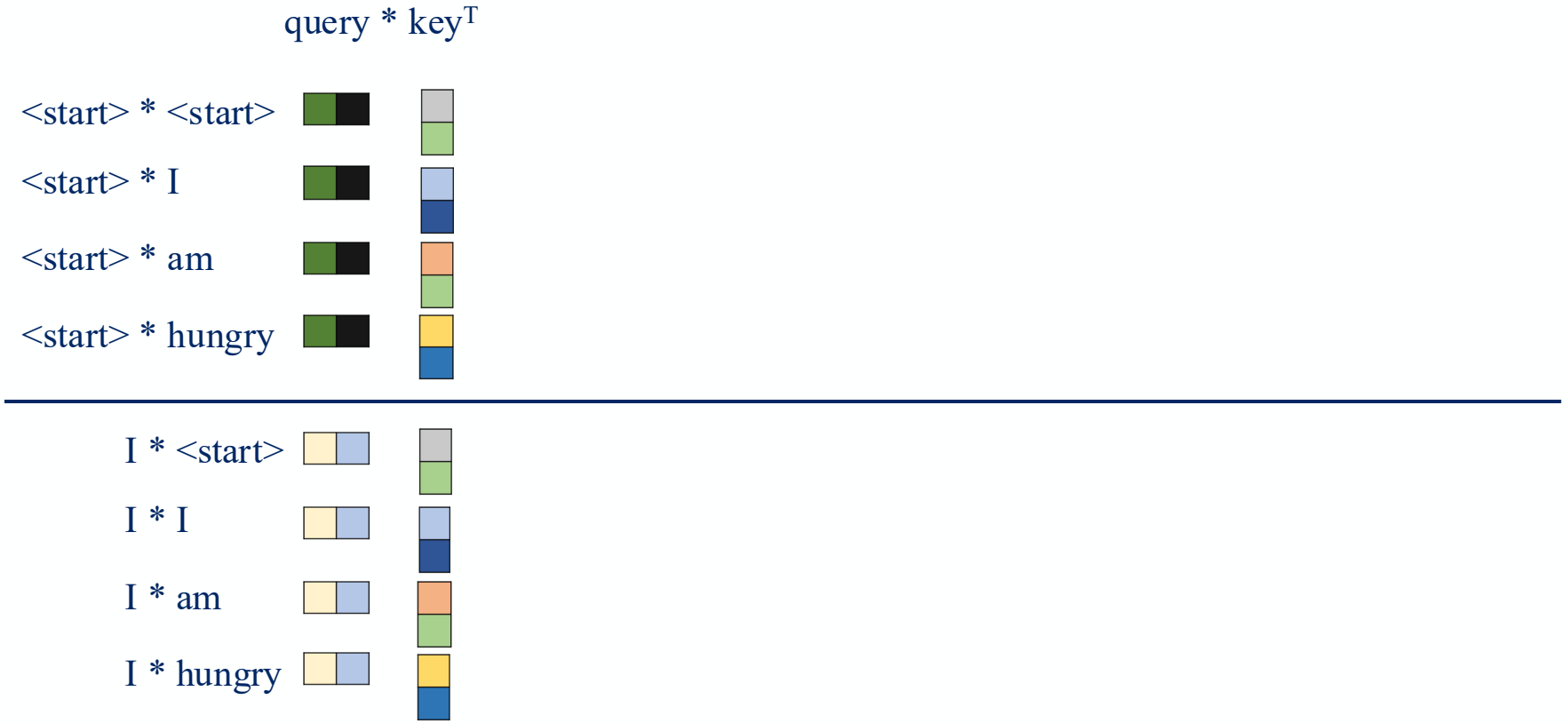
Masked Self-Attention

- Step 1: Query, key, value calculation is same as self-attention



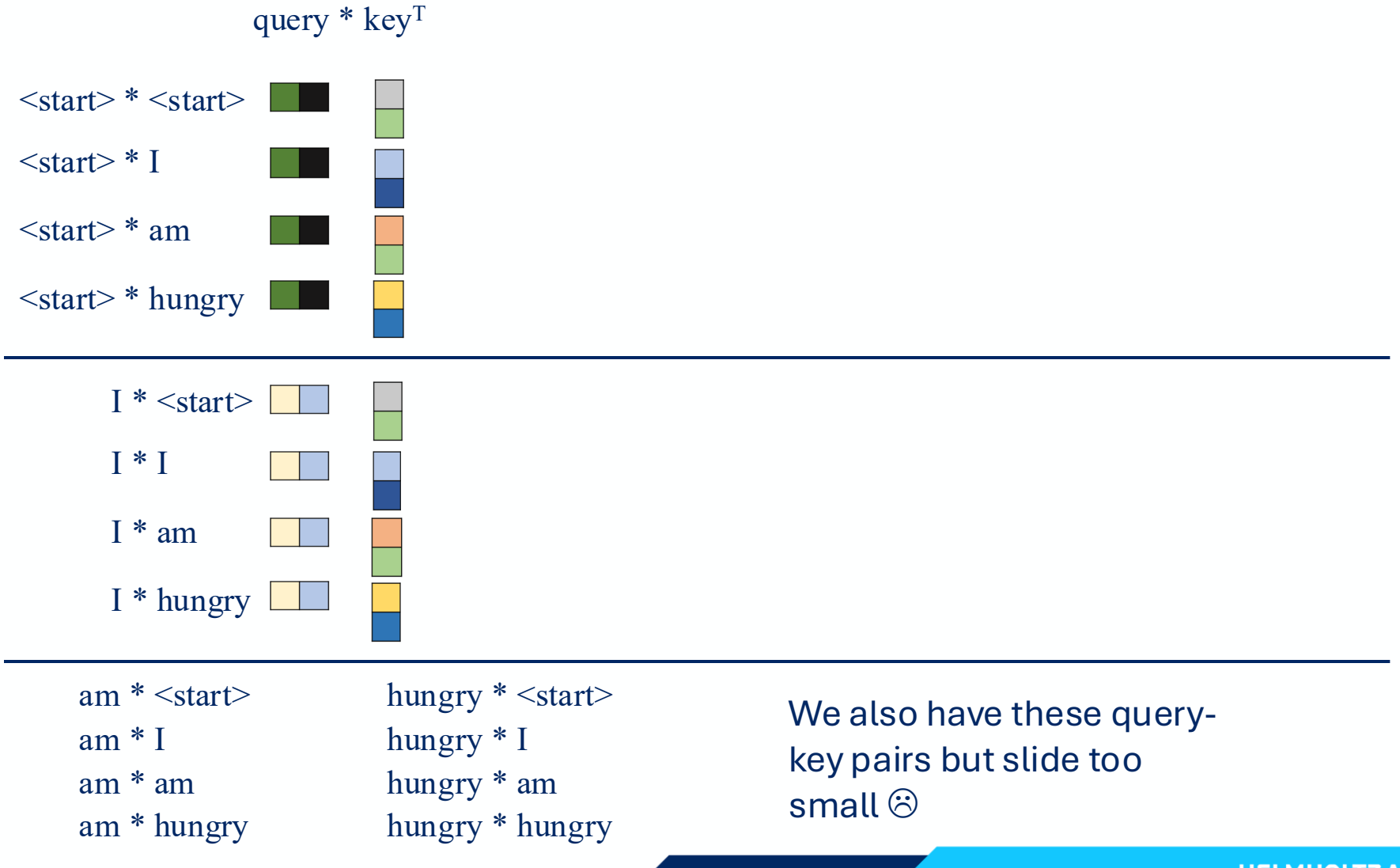
Masked Self-Attention

- Step 2: Dot product query and key is same as self-attention



Masked Self-Attention

- Step 2: Dot product query and key is same as self-attention



We also have these query-key pairs but slide too small ☹

Masked Self-Attention

- Step 2: Dot product query and key is same as self-attention

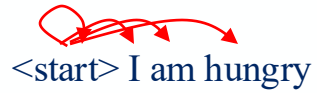
query * key^T

<start> * <start> = 69

<start> * I = 13

<start> * am = 12

<start> * hungry = 19



I * <start> = 10

I * I = 43

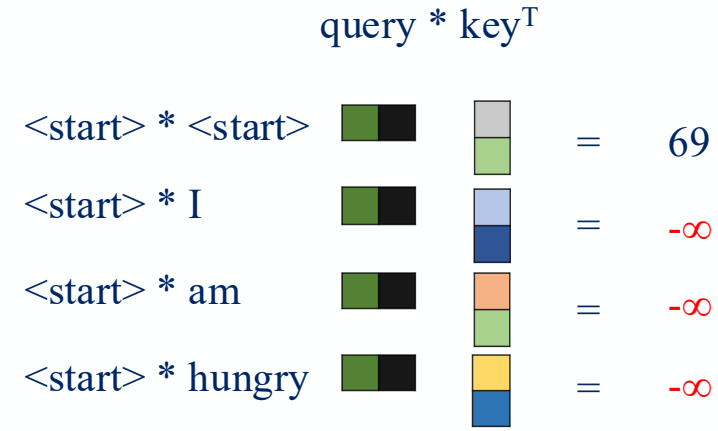
I * am = 11

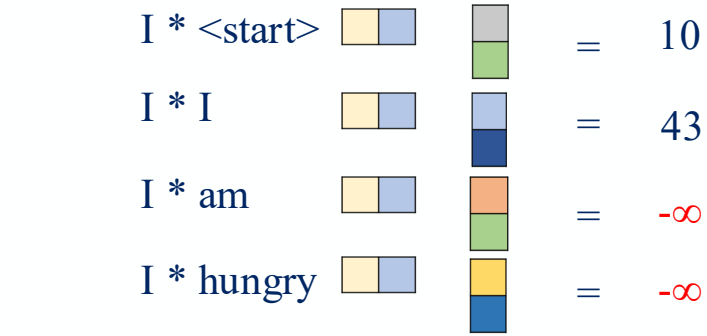
I * hungry = 8

Disclaimer: the number is not number from a real model.

Masked Self-Attention

- Step 3: For masked words:
  - Replace the dot product results with -∞





Disclaimer: the number is not number from a real model.

Masked Self-Attention

- Step 3: For masked words:
  - Replace the dot product results with -∞
  - Their softmax results will be 0

	query * key^T		softmax
<start> * <start>	=	69	1
<start> * I	=	-∞	0
<start> * am	=	-∞	0
<start> * hungry	=	-∞	0
I * <start>	=	10	0.02
I * I	=	43	0.98
I * am	=	-∞	0
I * hungry	=	-∞	0

Masked words have 0 attention

Disclaimer: the number is not number from a real model.

Masked Self-Attention

The rest is same as self-attention

- Step 3: For masked words:
  - Replace the dot product results with -∞
  - Their softmax results will be 0

Masked Self-Attention

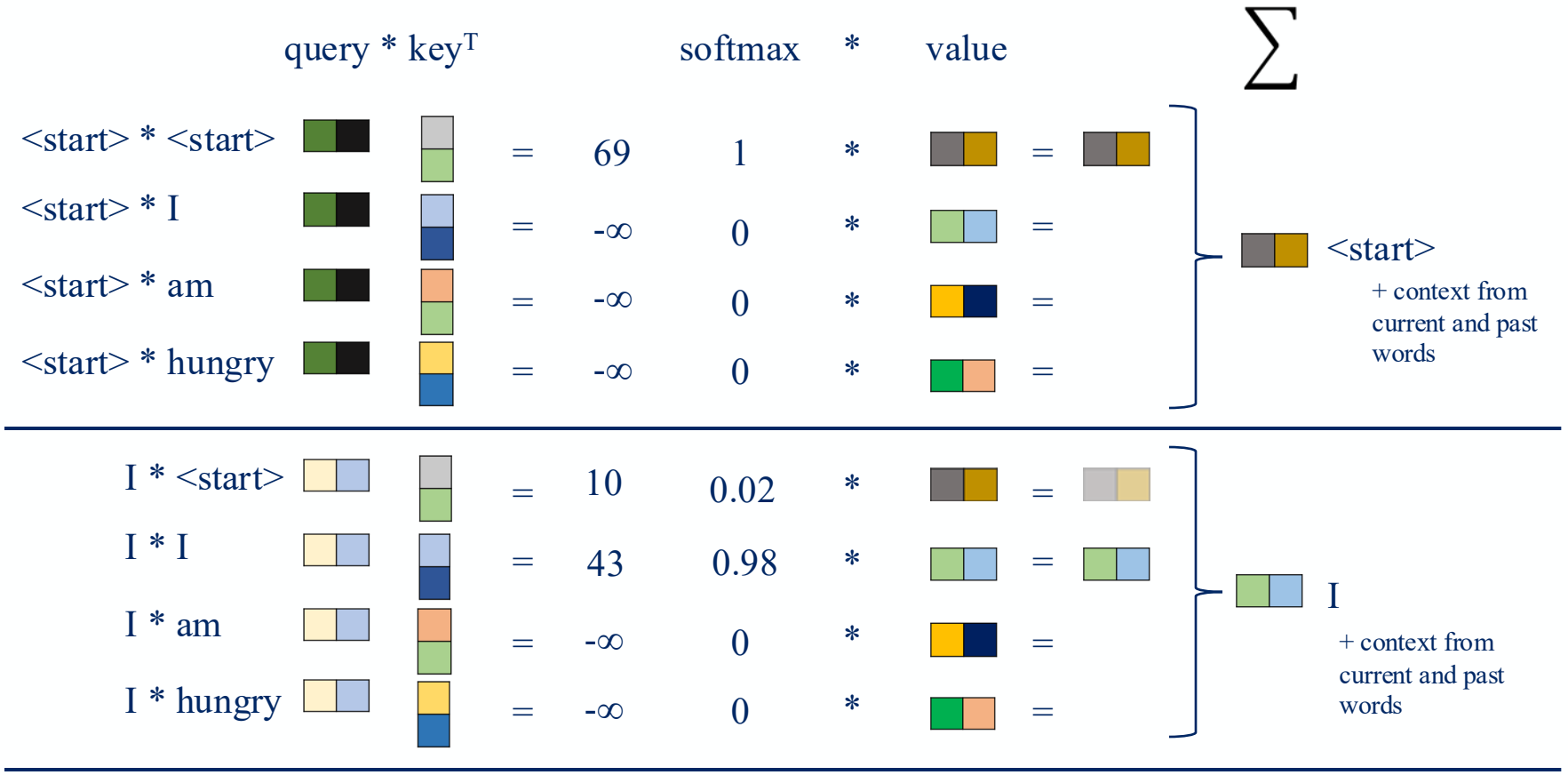
The rest is same as self-attention

- Step 3: For masked words:
  - Replace the dot product results with -∞
  - Their softmax results will be 0

Masked Self-Attention

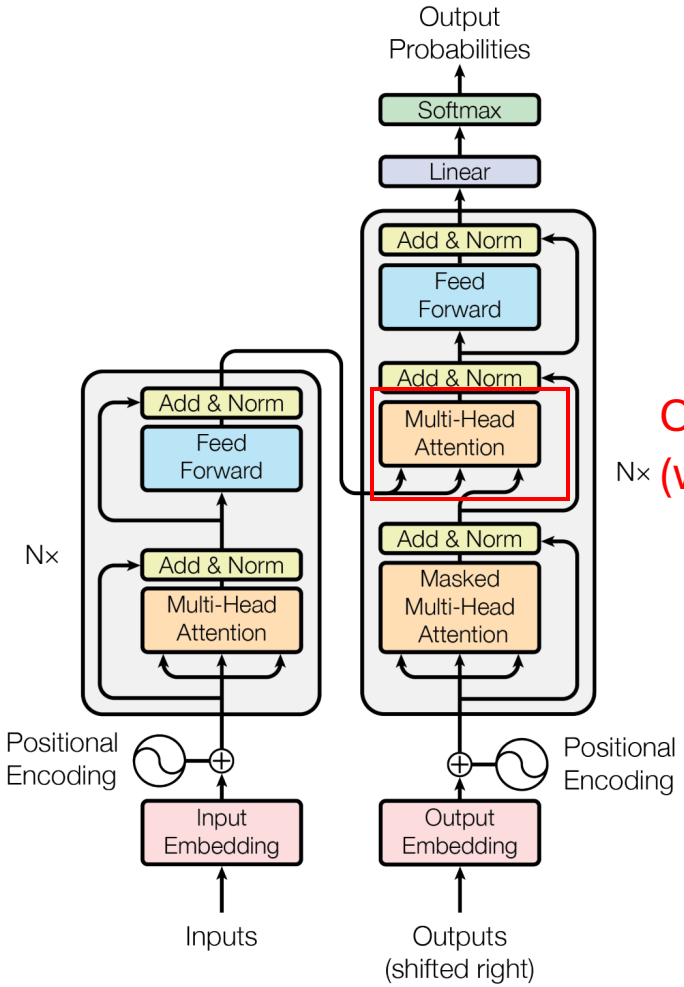
The rest is same as self-attention

- Step 3: For masked words:
  - Replace the dot product results with -∞
  - Their softmax results will be 0



Cross-Attention

- Input:
  - Encoder's final output
  - Output of masked self-attention



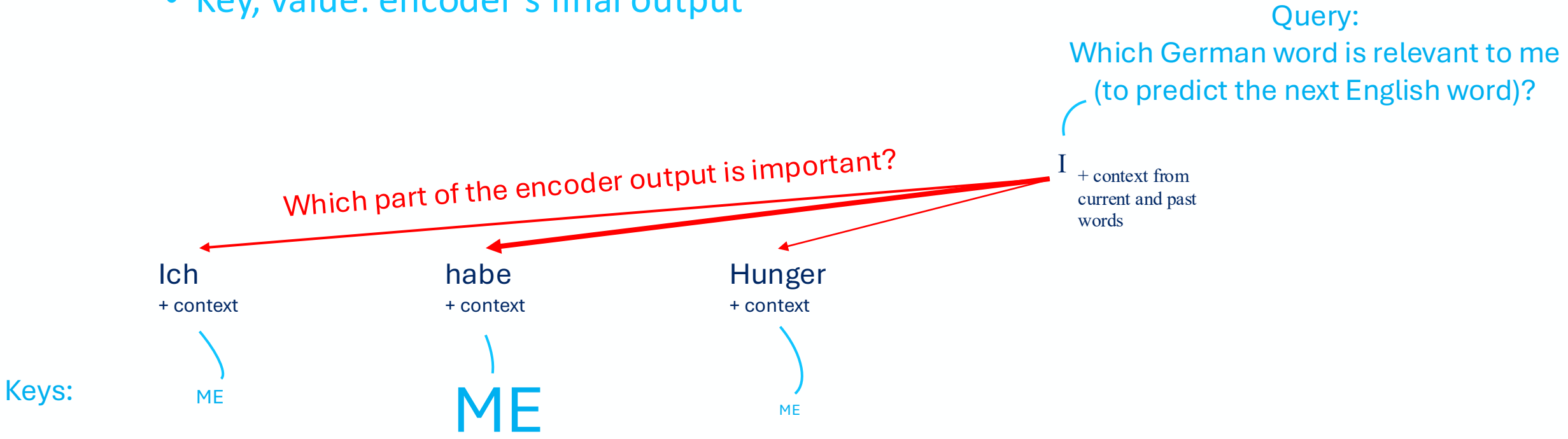
Cross-attention is here
(we will discuss first without “multi-head”)

Cross-Attention

- Similar with self-attention, but
  - Query: results of decoder's masked attention
  - Key, value: encoder's final output

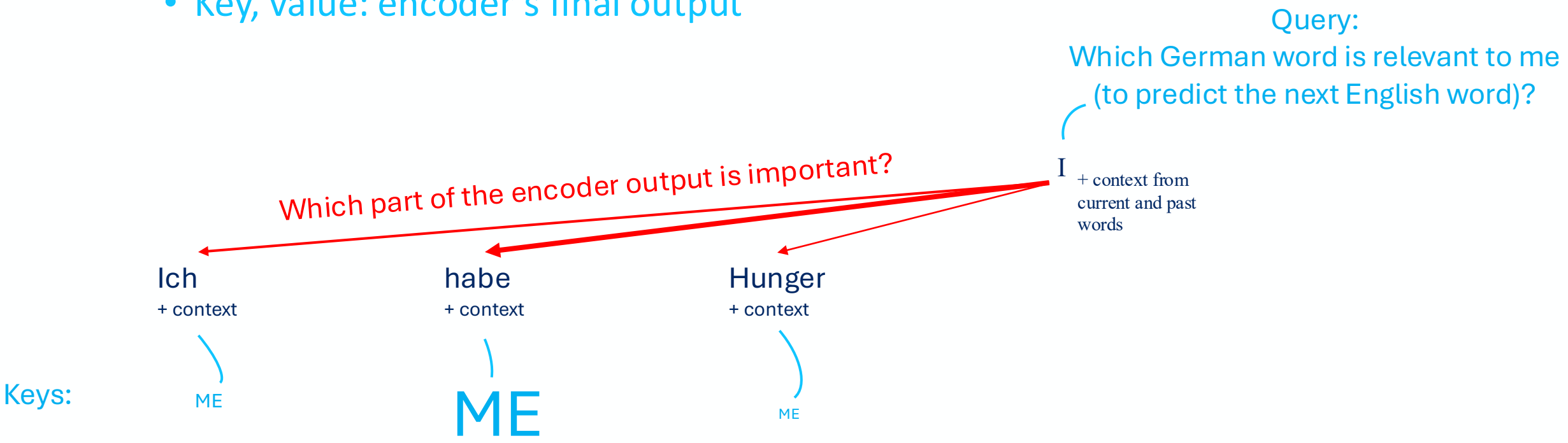
Cross-Attention

- Similar with self-attention, but
  - Query: results of decoder's masked attention
  - Key, value: encoder's final output



Cross-Attention

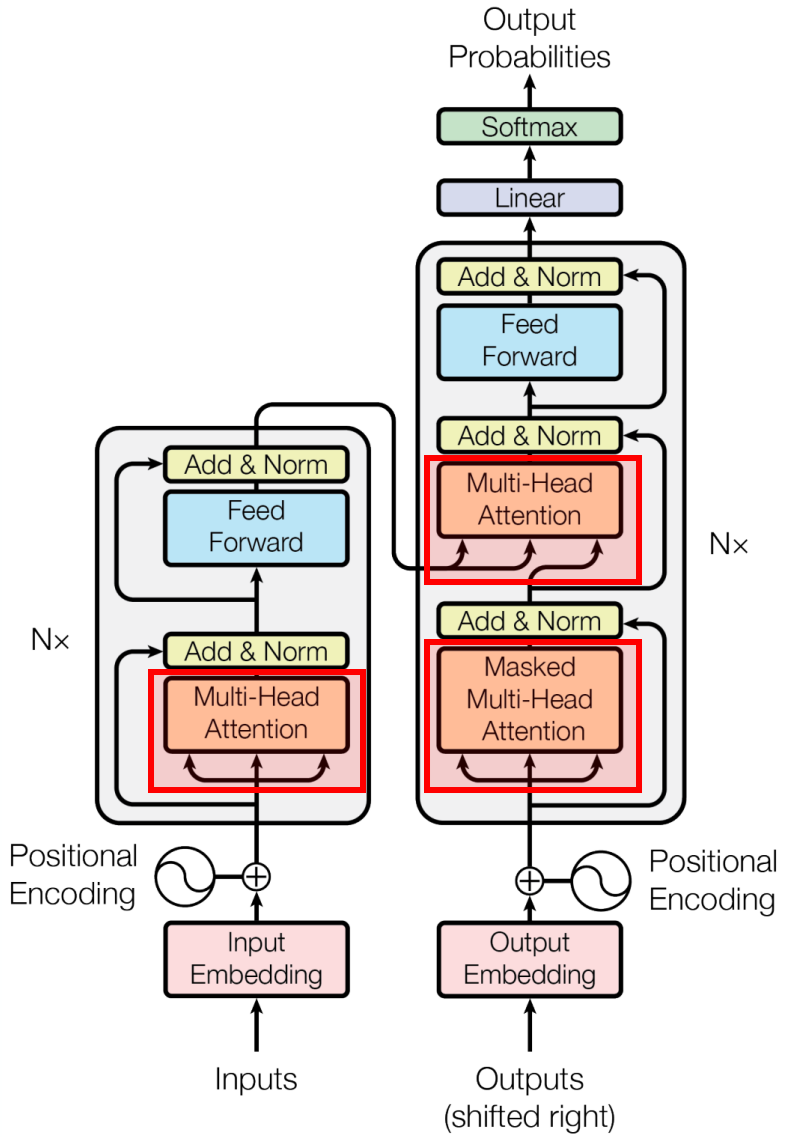
- Similar with self-attention, but
  - Query: results of decoder's masked attention
  - Key, value: encoder's final output



Values: "and here is the information I give if I got the attention"

Multi-Head Attention

- Multi-head self attention in encoder
- Multi-head cross attention in decoder
- Masked multi-head self attention in decoder

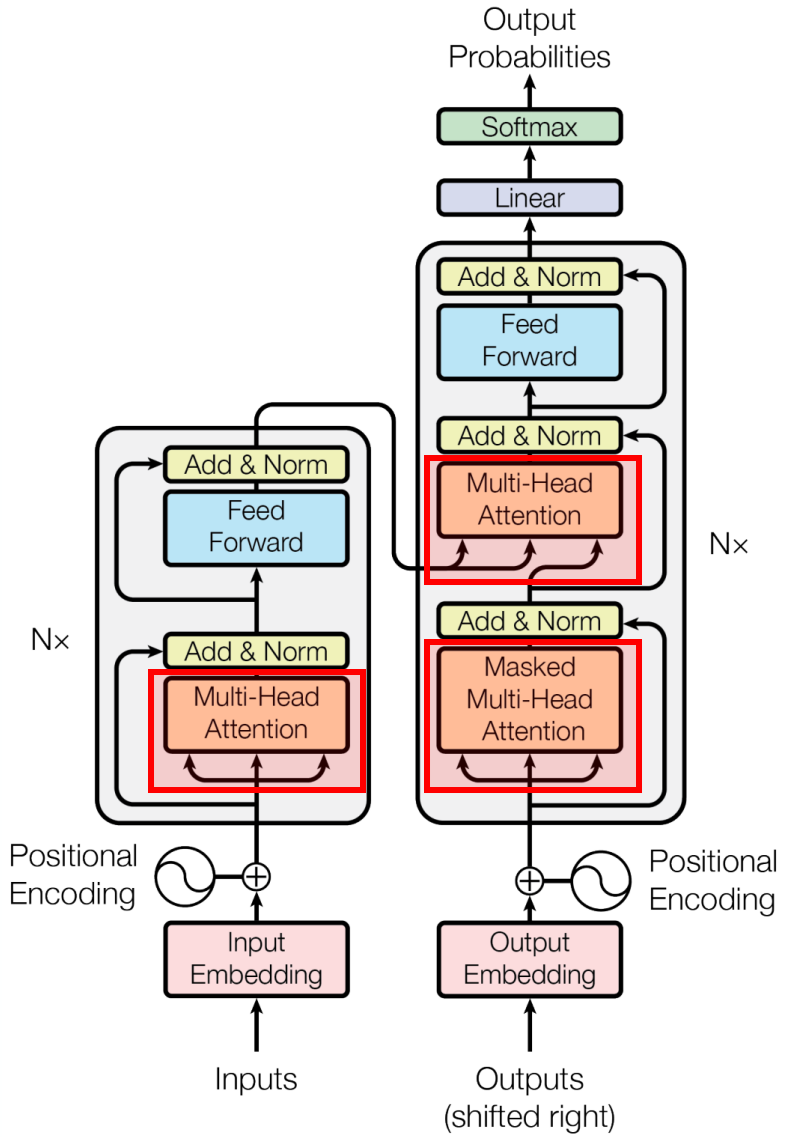


Multi-Head Attention

as an example…

- Multi-head self attention in encoder
- Multi-head cross attention in decoder
- Masked multi-head self attention in decoder

The others will have the same concept



Multi-head Attention

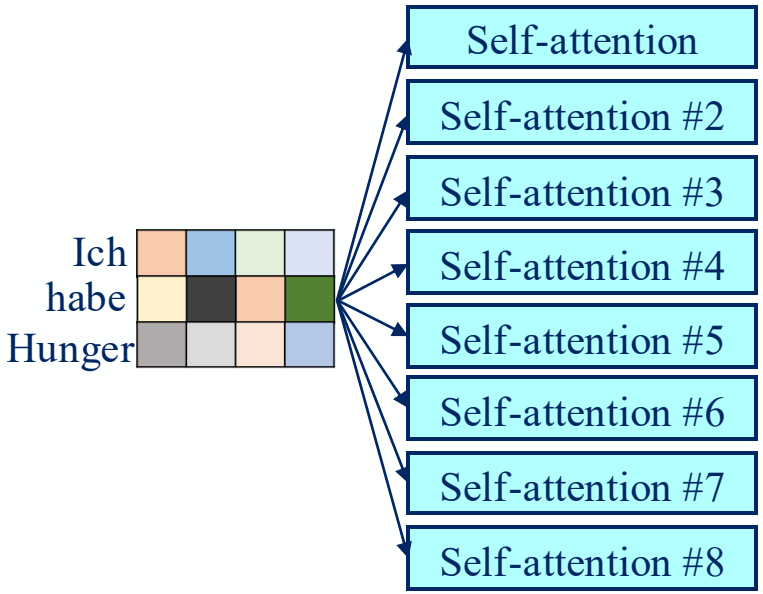
- Motivation of multi-head: to cover different kinds of attention/connection

Head 1 focuses on subject-verb connection

Head 2 focuses on subject-location connection

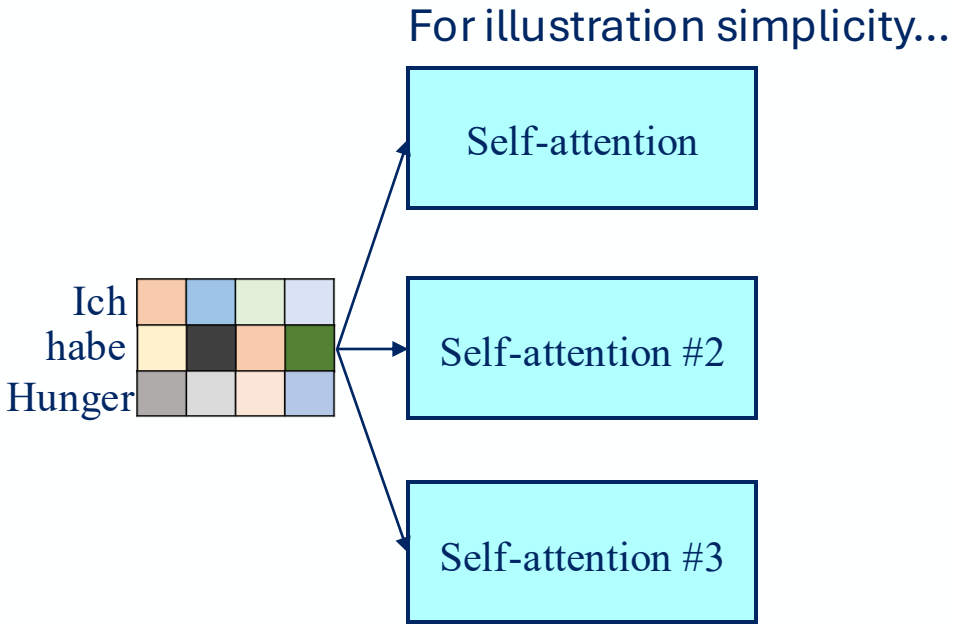
Multi-Head Attention

- More than 1 attentions, done in parallel
  - Originally, 8 attentions



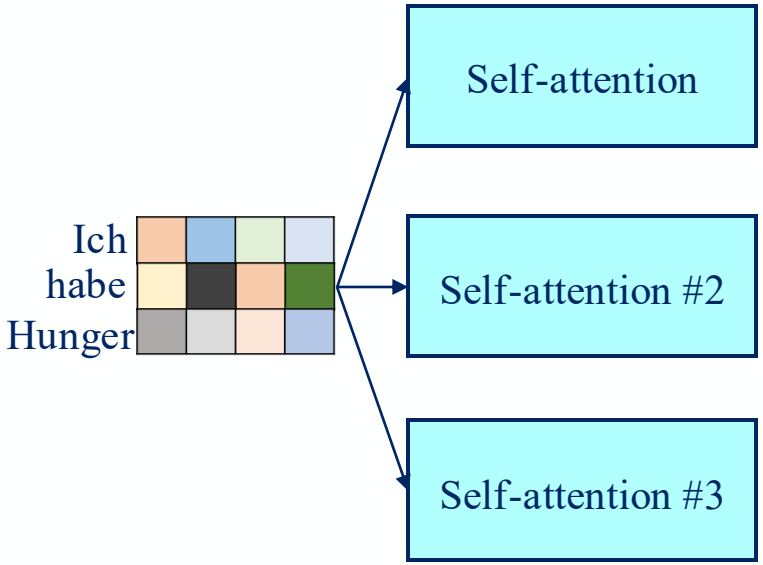
Multi-Head Attention

- More than 1 attentions, done in parallel
  - Originally, 8 attentions



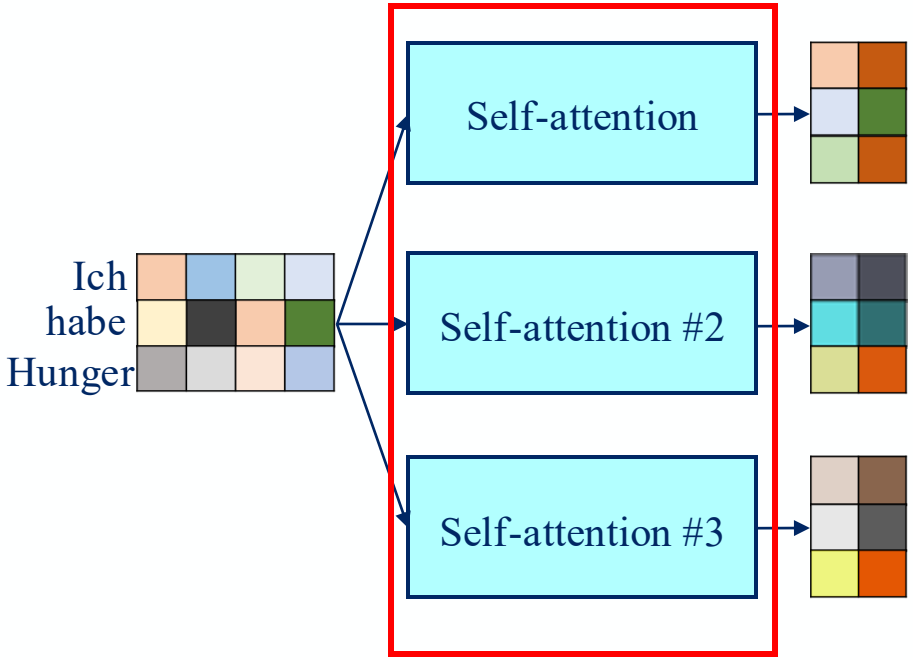
Multi-Head Attention

- More than 1 attentions, done in parallel
  - Originally, 8 attentions
  - Multiple attentions complementing each other



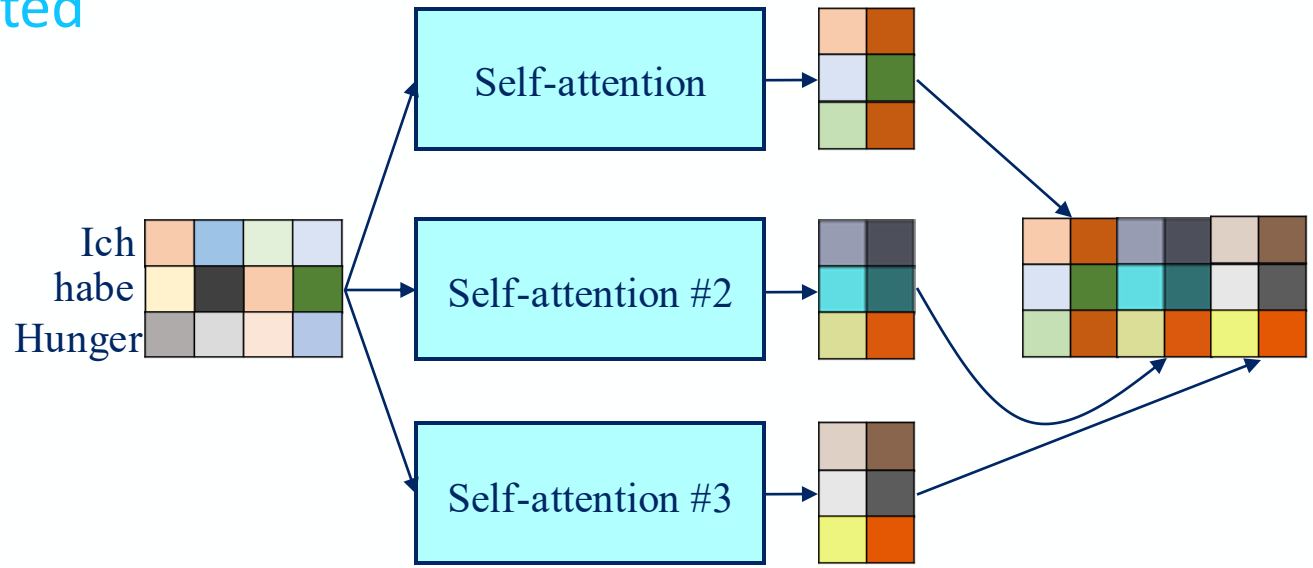
Multi-Head Attention

- More than 1 attentions, done in parallel
  - Originally, 8 attentions
  - Multiple attentions complementing each other → no shared weight



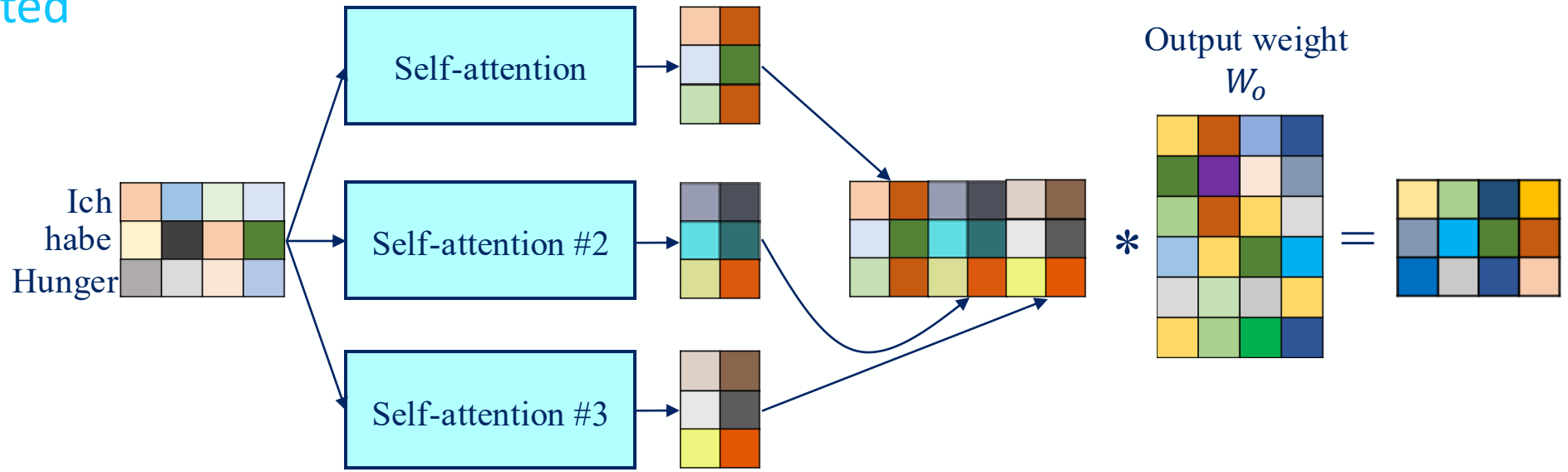
Multi-Head Attention

- More than 1 attentions, done in parallel
  - Originally, 8 attentions
  - Multiple attentions complementing each other → no shared weight
  - Concatenated



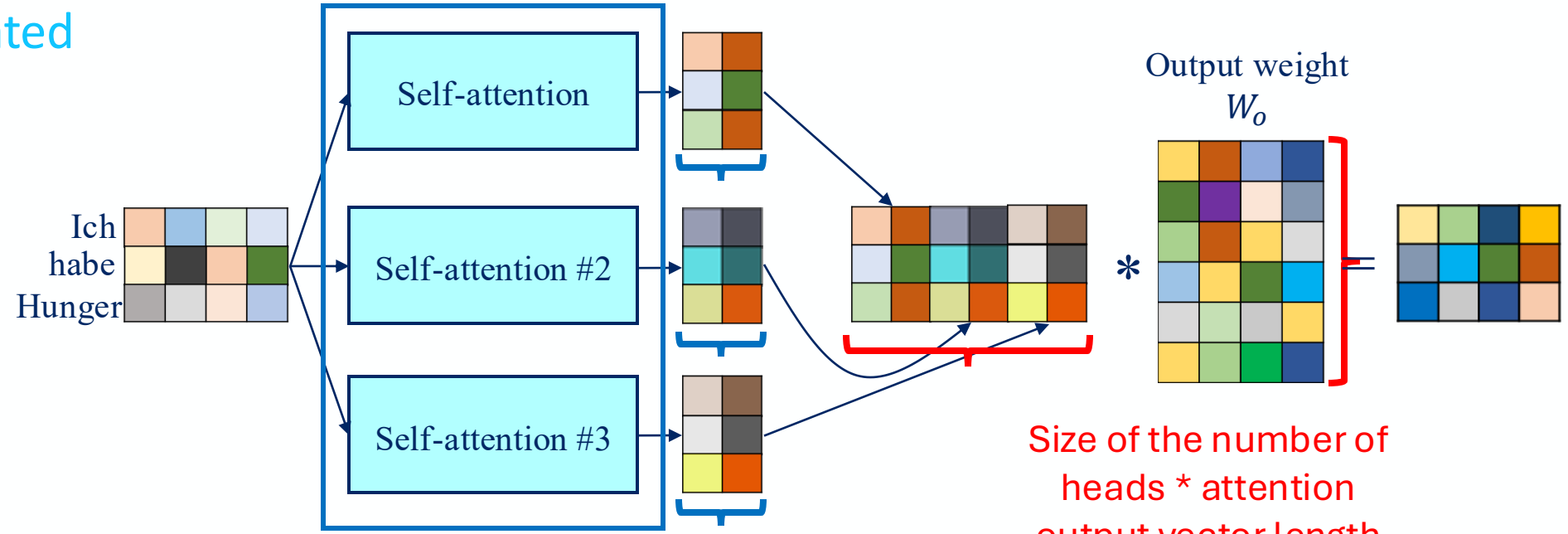
Multi-Head Attention

- More than 1 attentions, done in parallel
  - Originally, 8 attentions
  - Multiple attentions complementing each other → no shared weight
  - Concatenated
- Learnable output weight



Multi-Head Attention

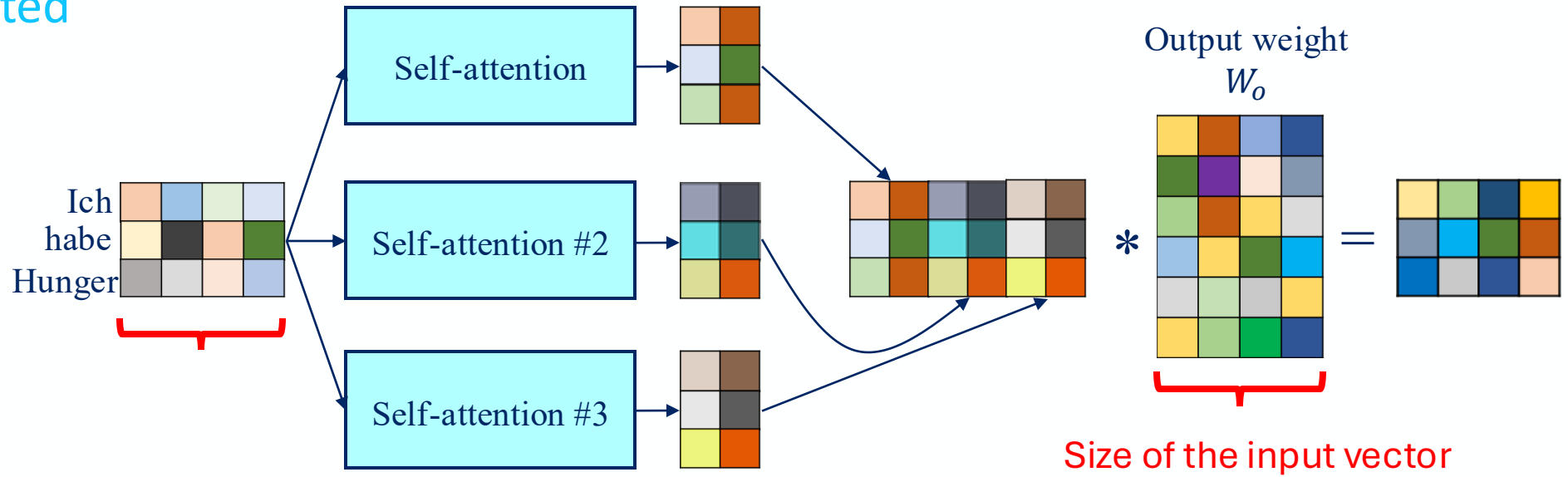
- More than 1 attentions, done in parallel
  - Originally, 8 attentions
  - Multiple attentions complementing each other → no shared weight
  - Concatenated
- Learnable output weight



Size of the number of heads * attention output vector length

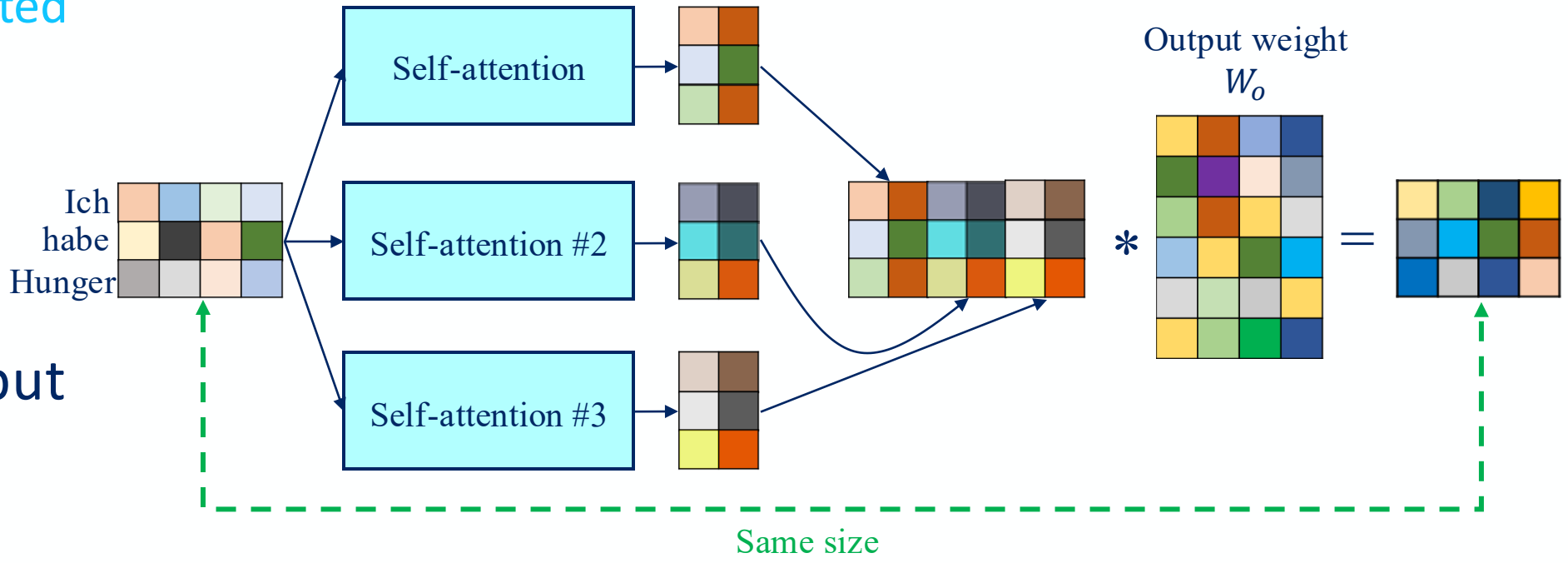
Multi-Head Attention

- More than 1 attentions, done in parallel
  - Originally, 8 attentions
  - Multiple attentions complementing each other → no shared weight
  - Concatenated
- Learnable output weight



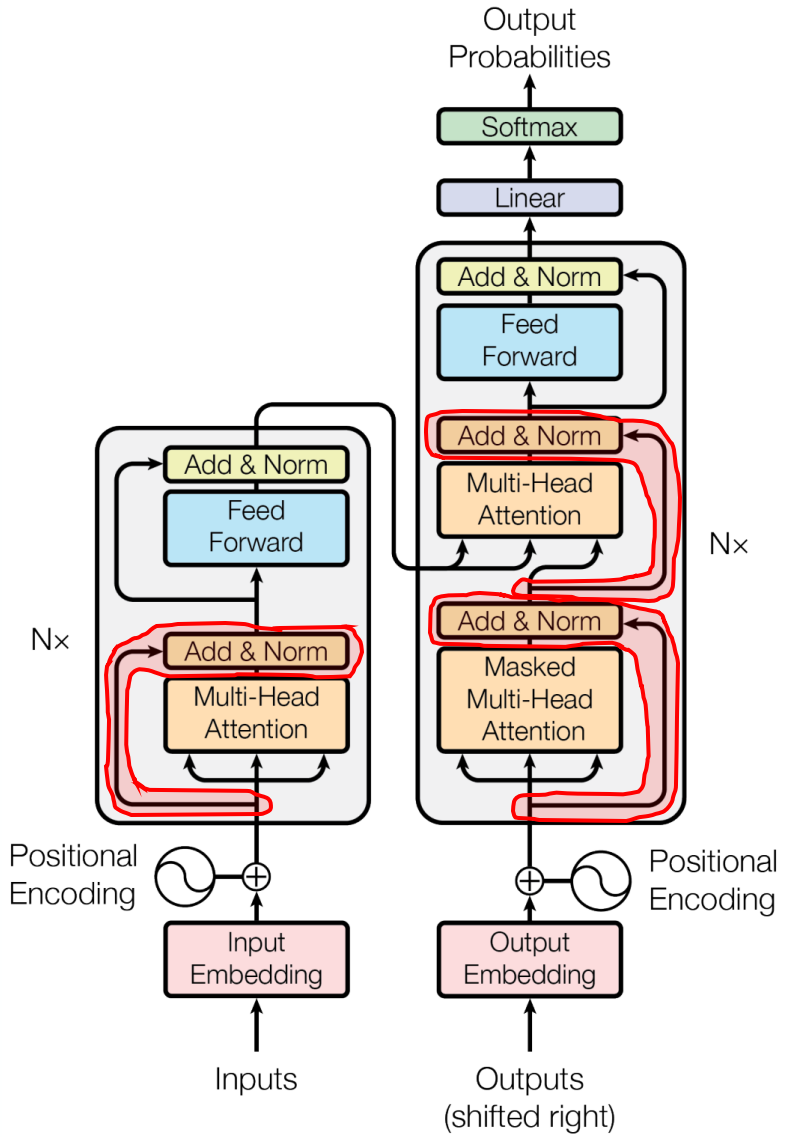
Multi-Head Attention

- More than 1 attentions, done in parallel
  - Originally, 8 attentions
  - Multiple attentions complementing each other → no shared weight
  - Concatenated
- Learnable output weight
- Same input & output size



Skip Connection

- Skip connection in each multi-head attention
  - Attention input size need to be same with attention output size

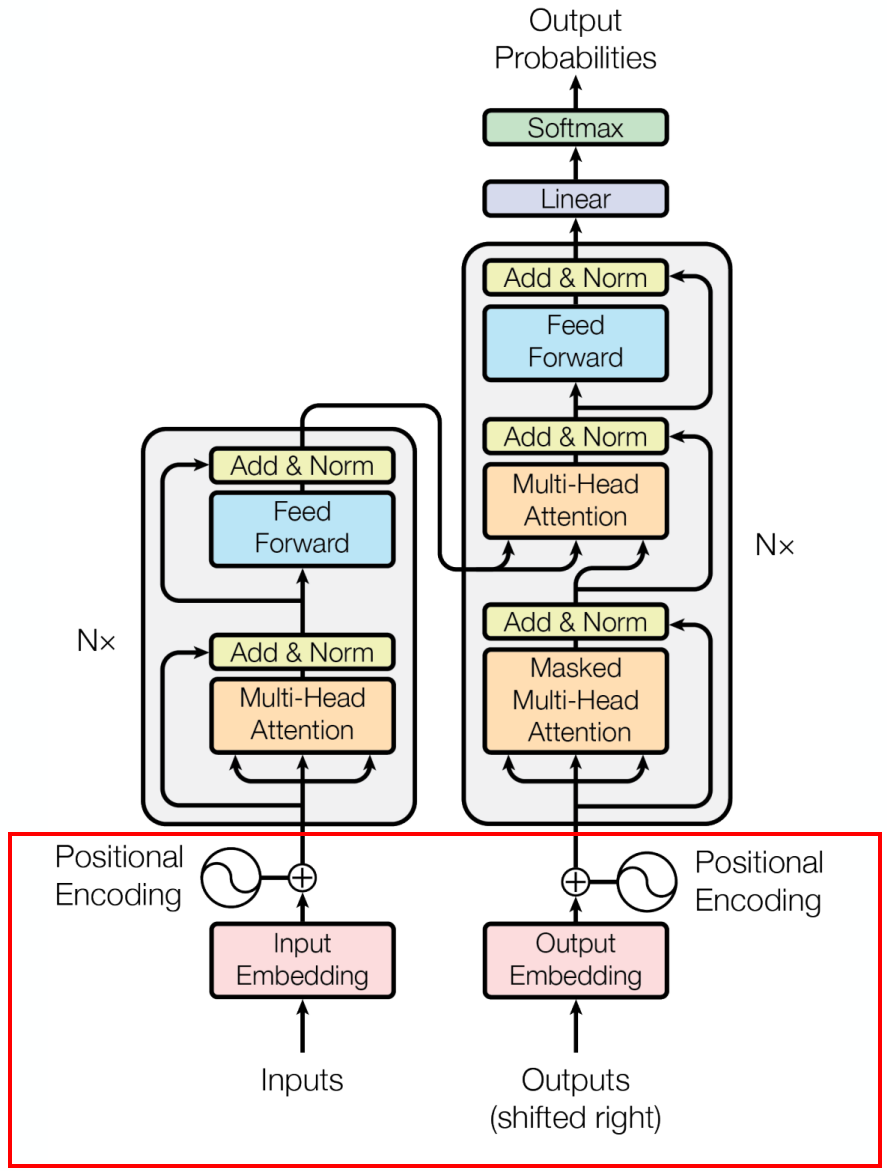


Outline

1. Overview
2. Preprocessing
3. Attention mechanism
4. Putting it all together
5. Variants of Transformer

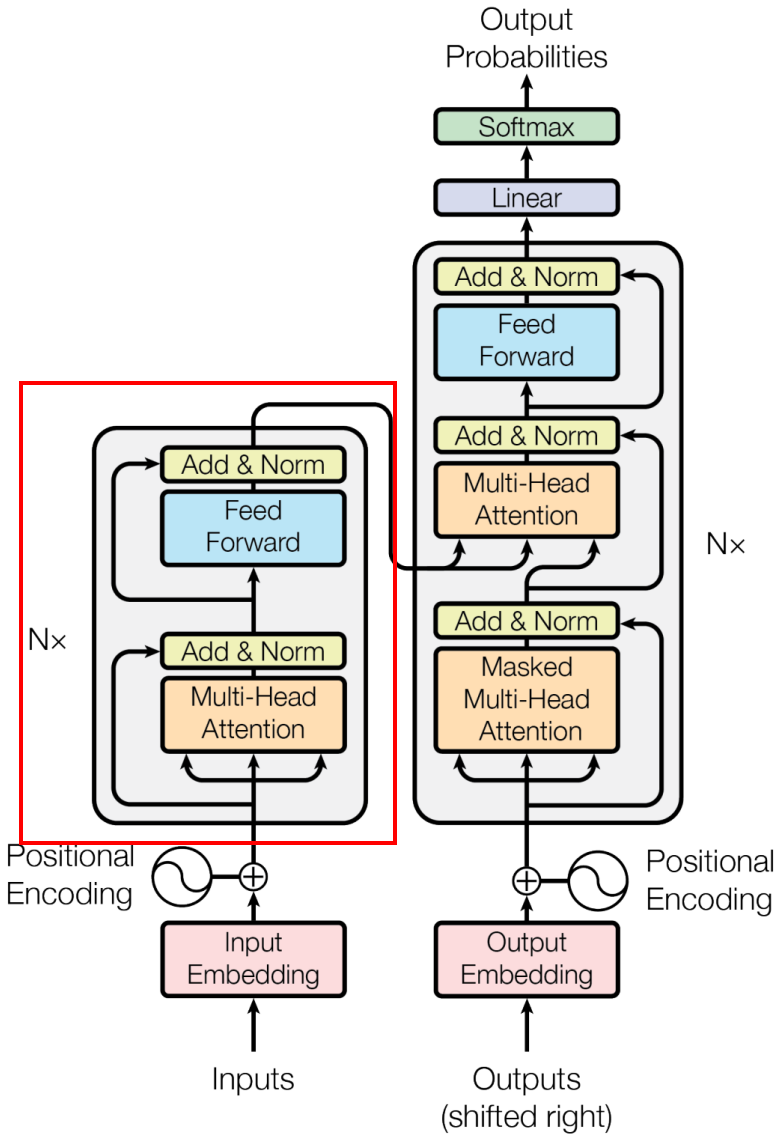
Transformer

- Preprocessing
  - Word embedding
  - Positional encoding



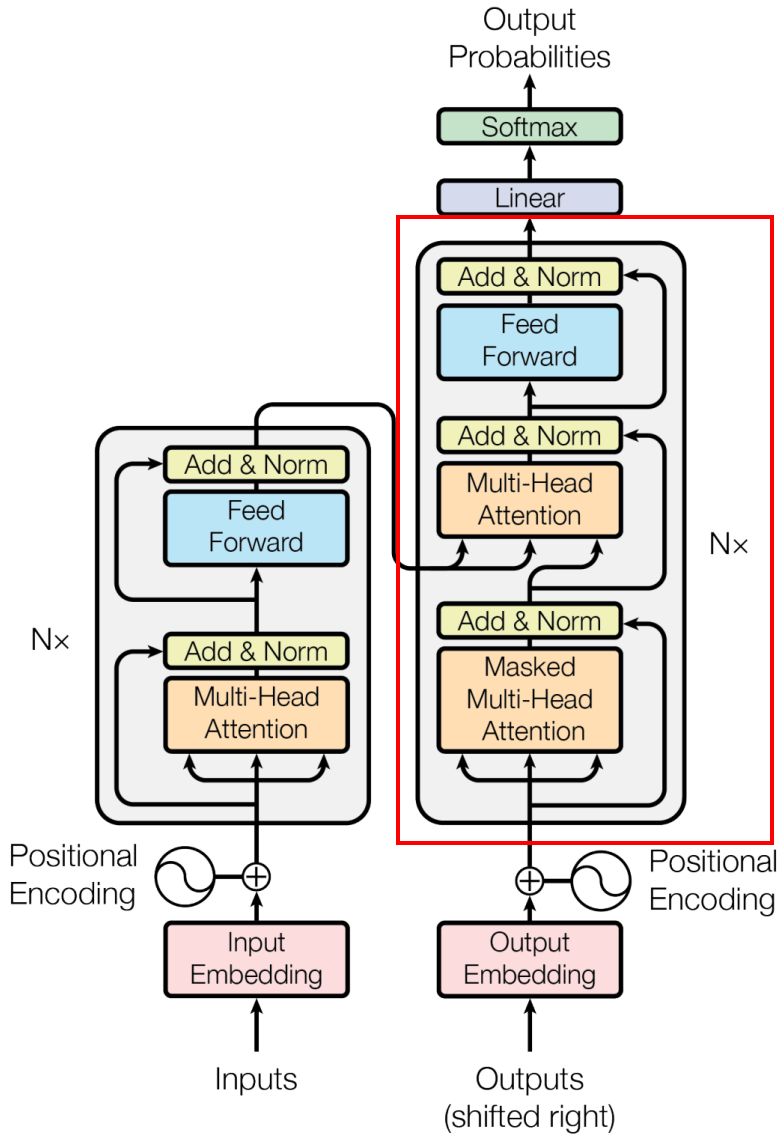
Transformer

- Preprocessing
  - Word embedding
  - Positional encoding
- Encoder



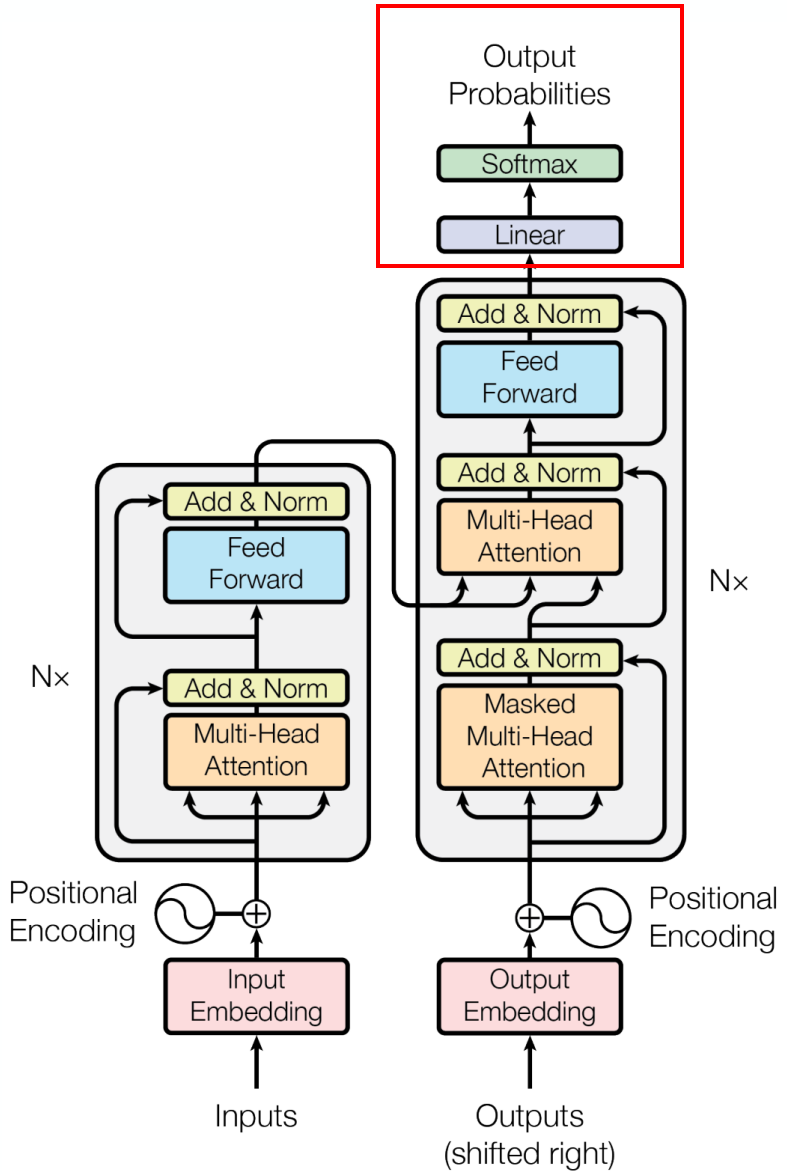
Transformer

- Preprocessing
  - Word embedding
  - Positional encoding
- Encoder
- Decoder



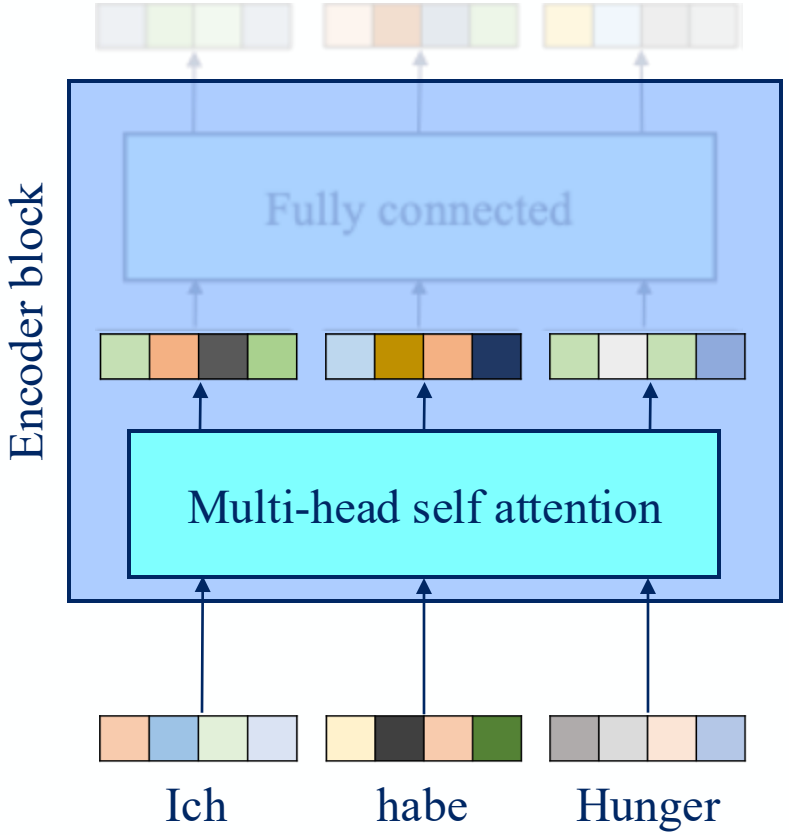
Transformer

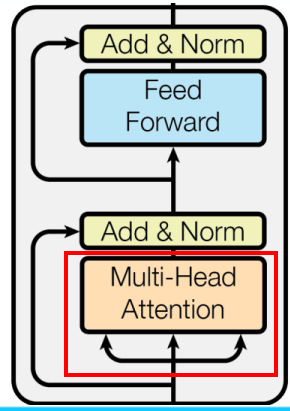
- Preprocessing
  - Word embedding
  - Positional encoding
- Encoder
- Decoder
- Output layers
  - Fully connected layers
  - Softmax



Encoder

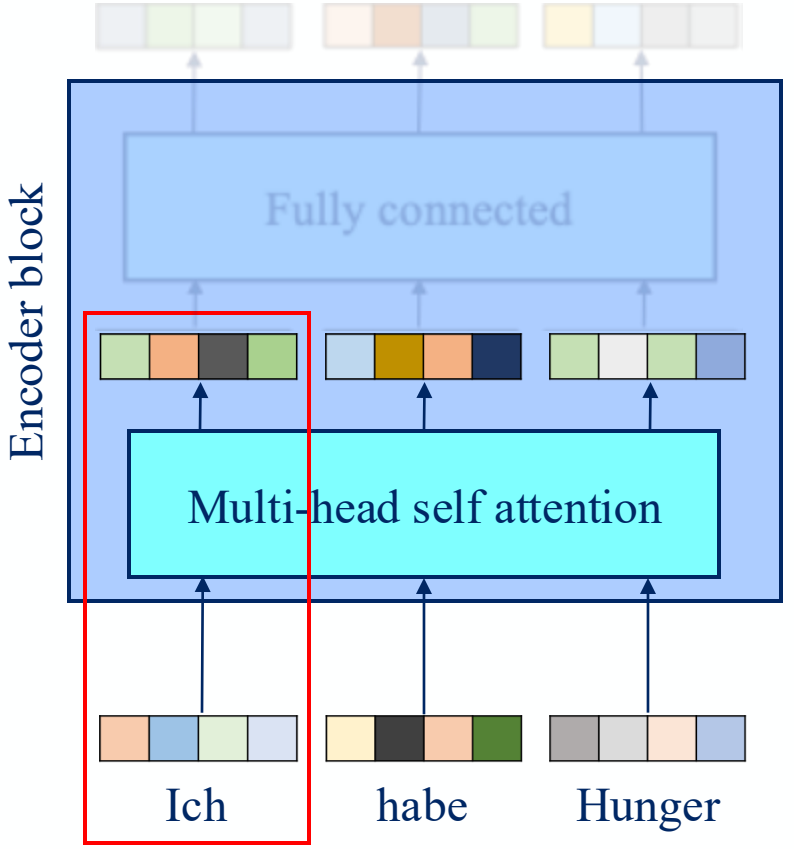
- Multi-head self-attention



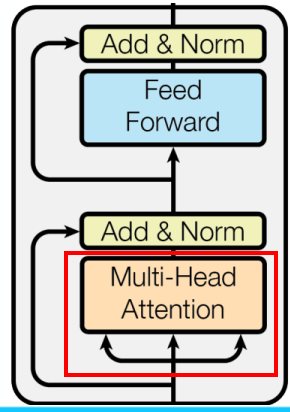


Encoder

- Multi-head self-attention

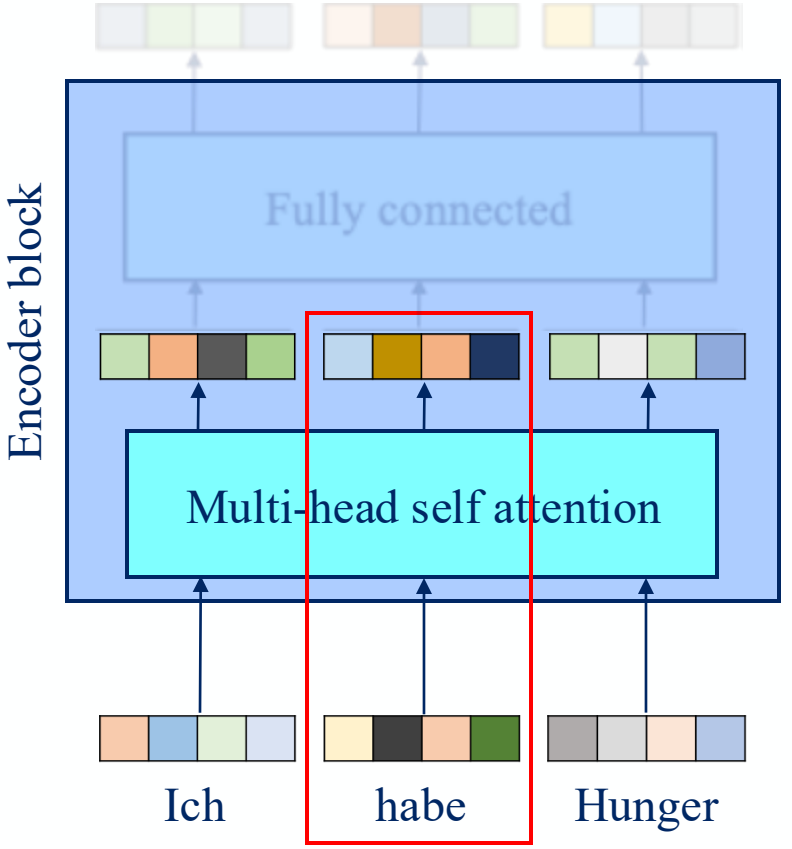


Finding connection between each word and all words

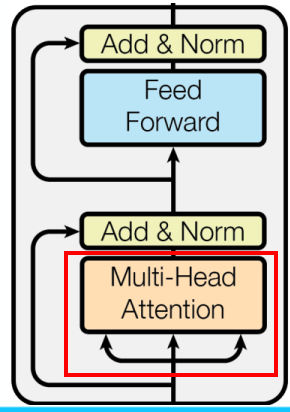


Encoder

- Multi-head self-attention

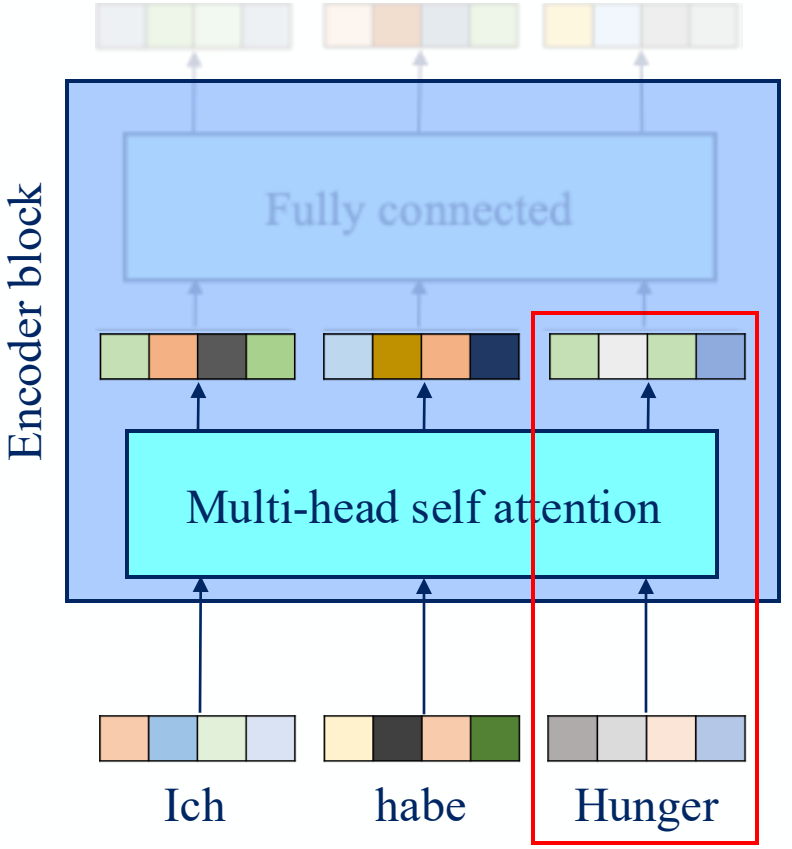


Finding connection between each word and all words

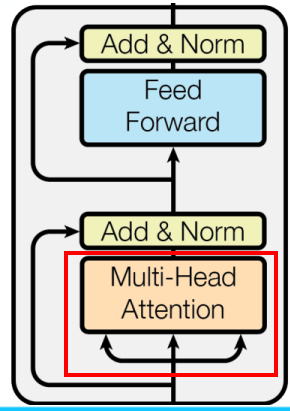


Encoder

- Multi-head self-attention

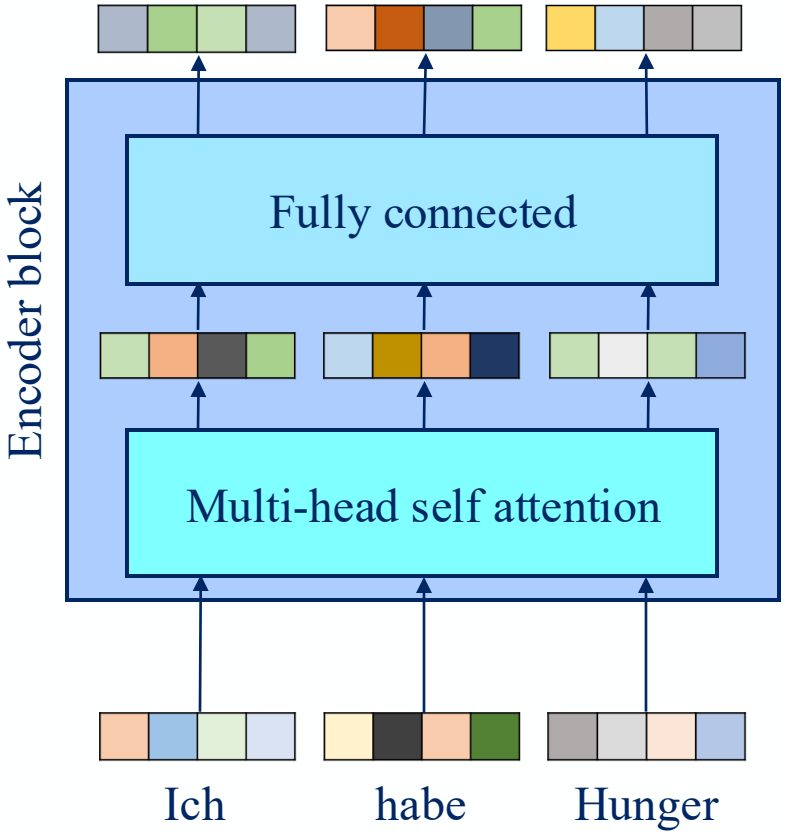


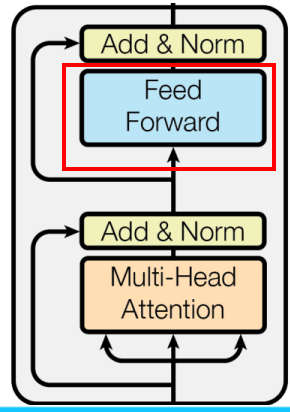
Finding connection between each word and all words



Encoder

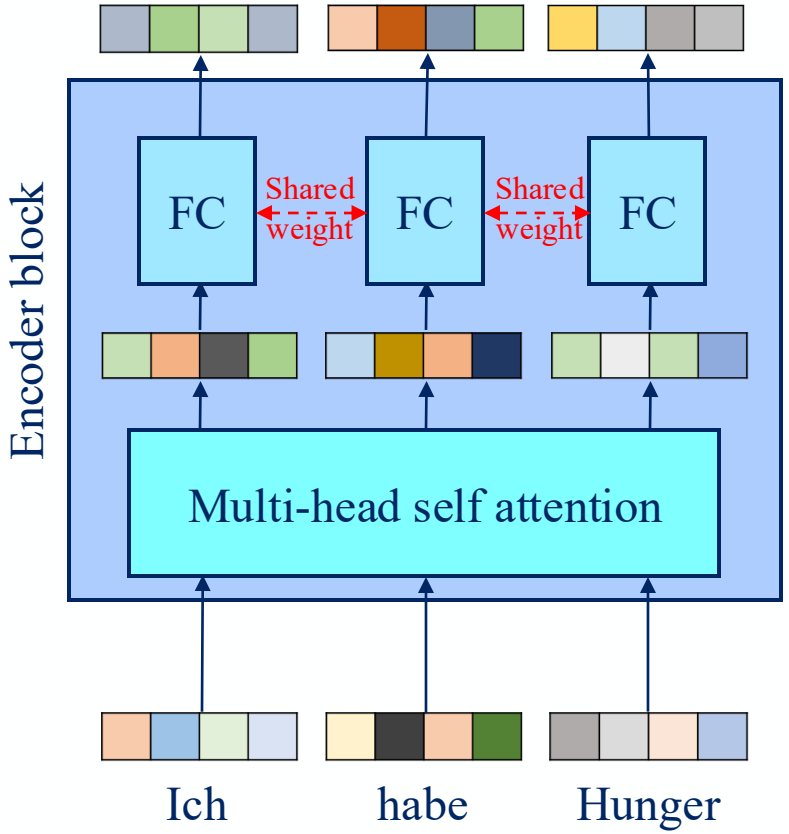
- Multi-head self-attention
- Feed forward/fully connected layer

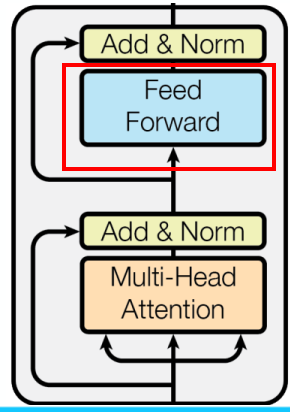




Encoder

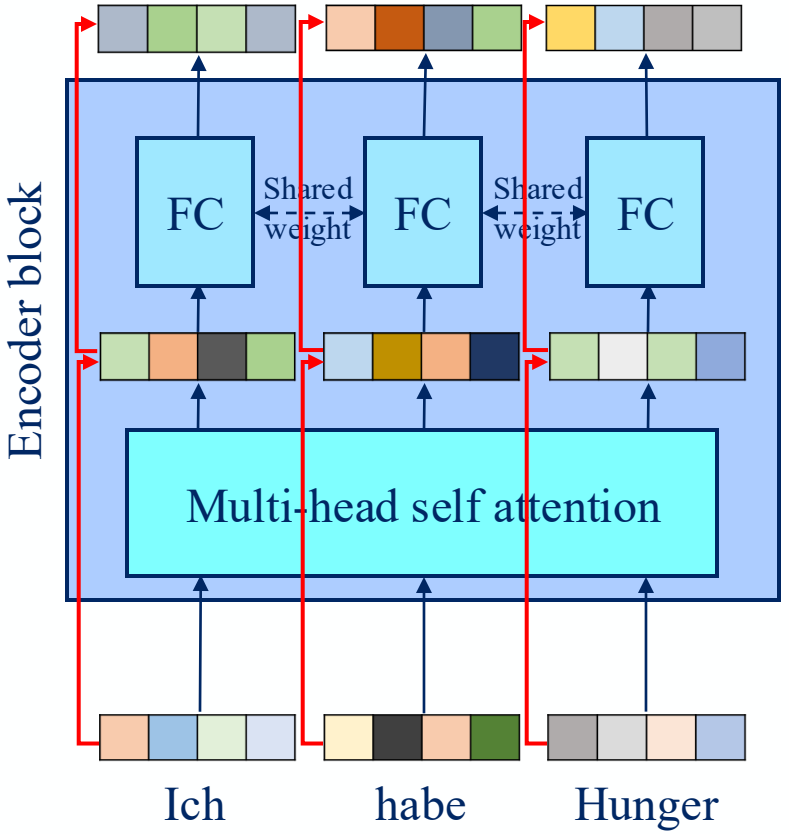
- Multi-head self-attention
- Feed forward/fully connected layer
  - Shared weight

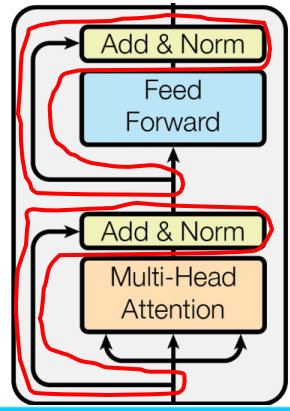




Encoder

- Multi-head self-attention
- Feed forward/fully connected layer
  - Shared weight
- Residual connection

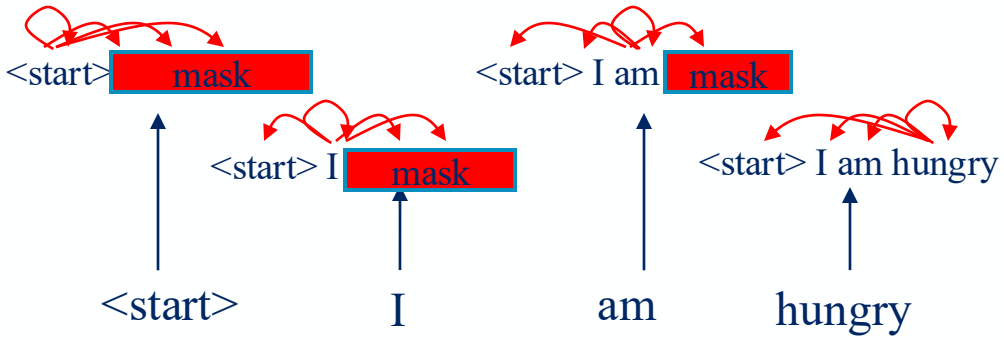


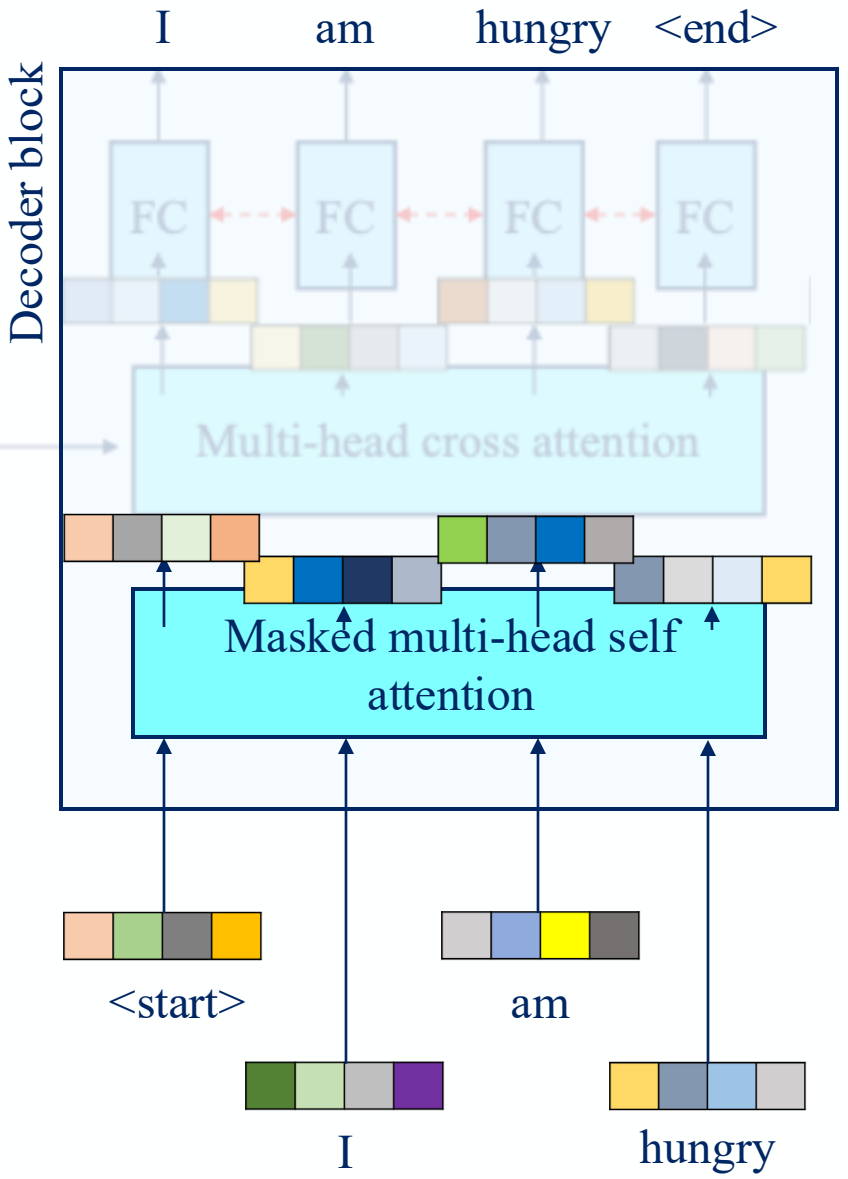


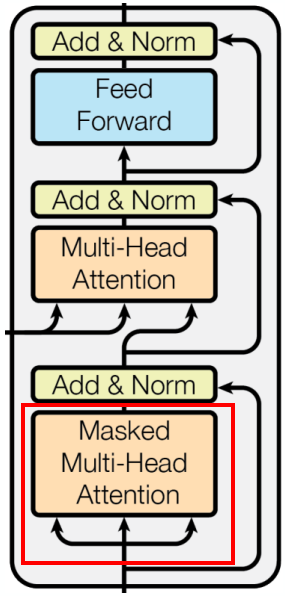
Decoder

- Masked multi-head self attention

Finding connection between each word and all current & past words



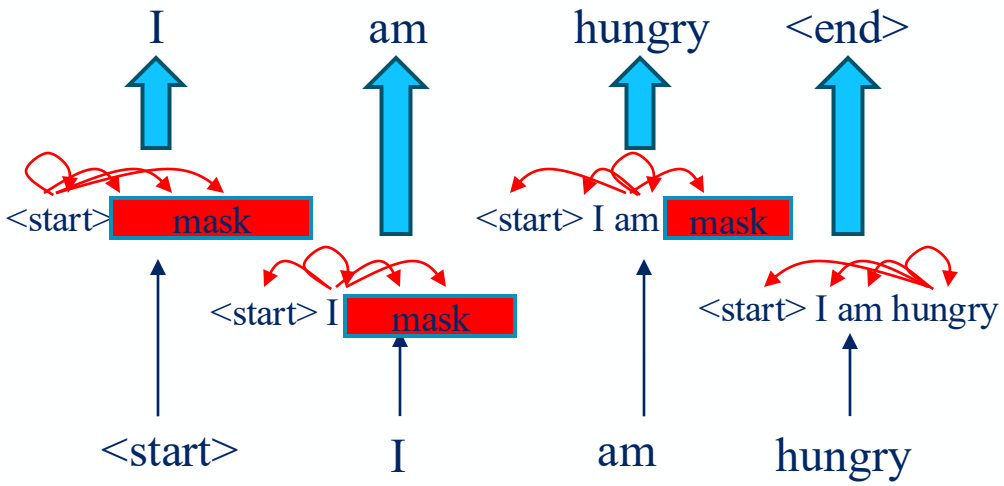


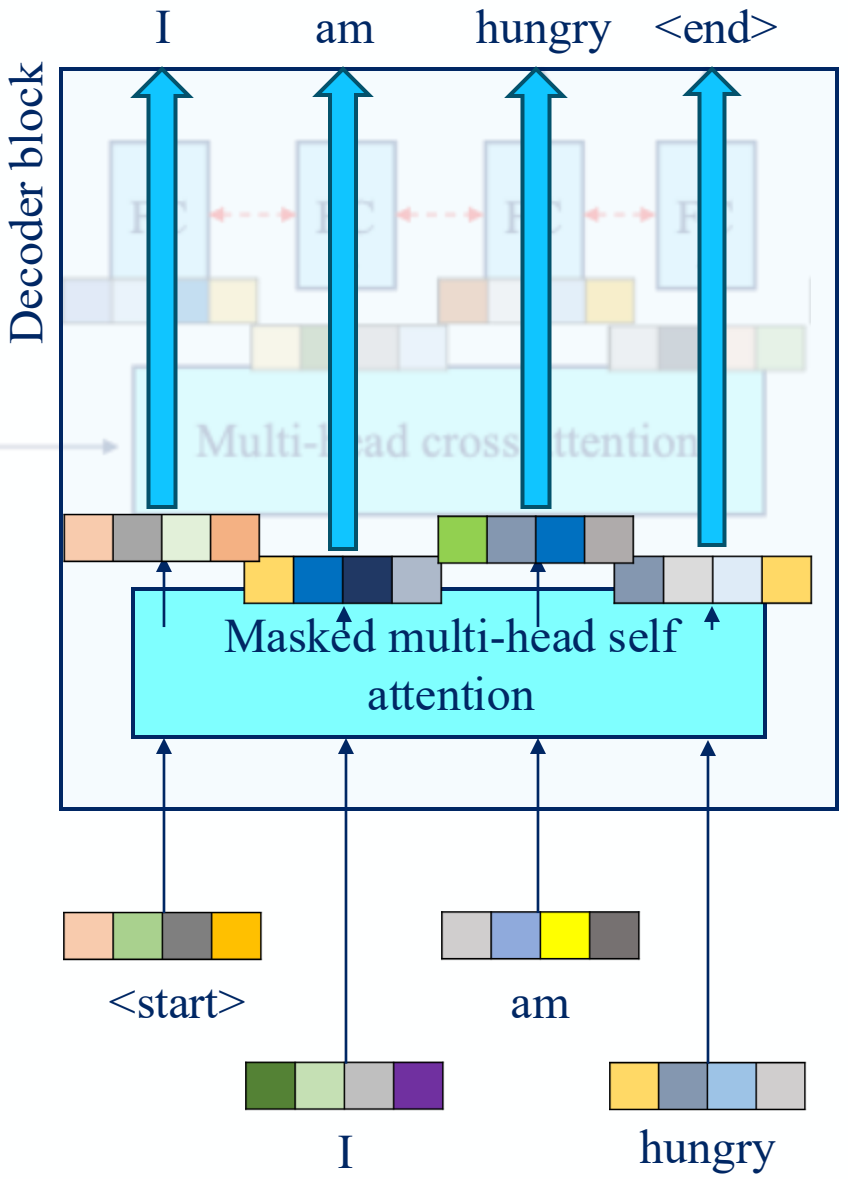


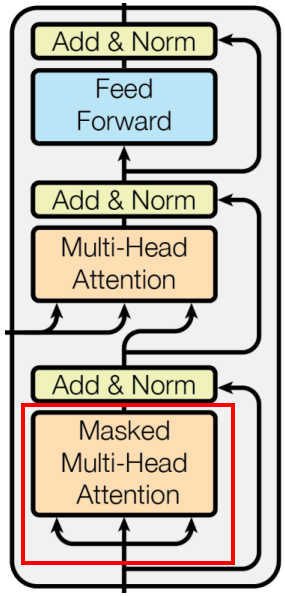
Decoder

- Masked multi-head self attention

So the rest of the decoder can predict the future word

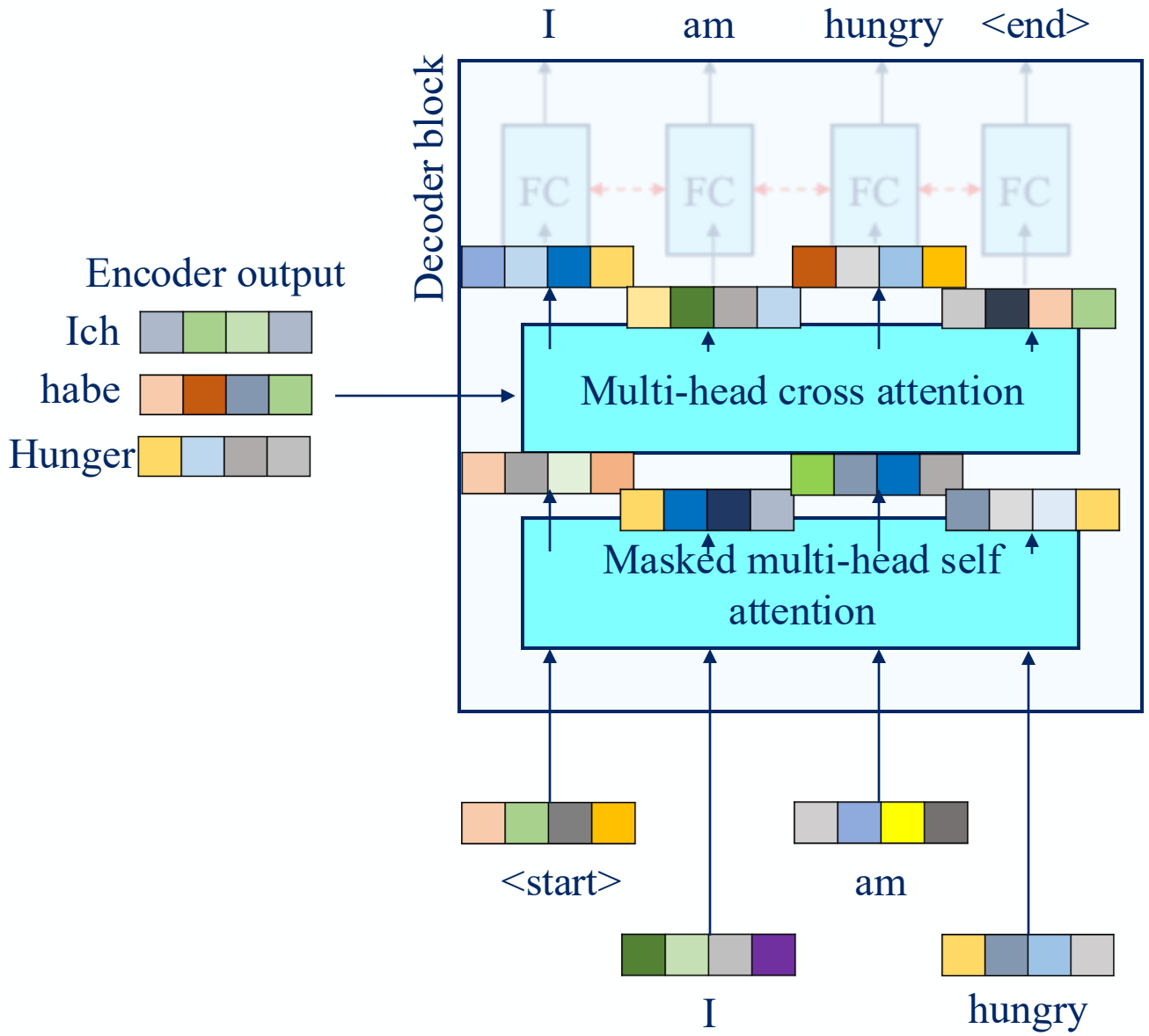


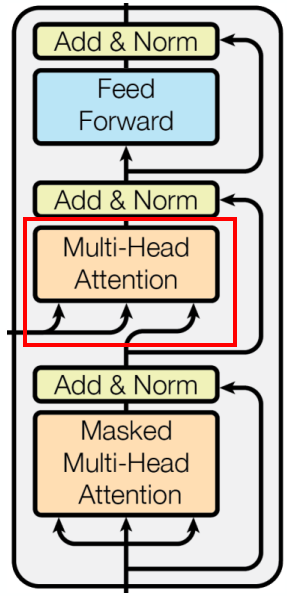




Decoder

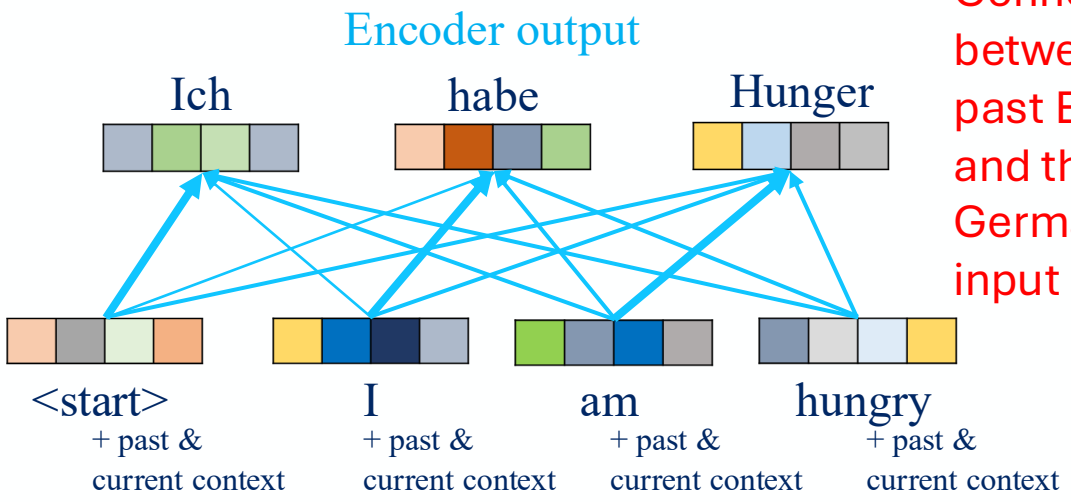
- Masked multi-head self attention
- Multi-head cross attention



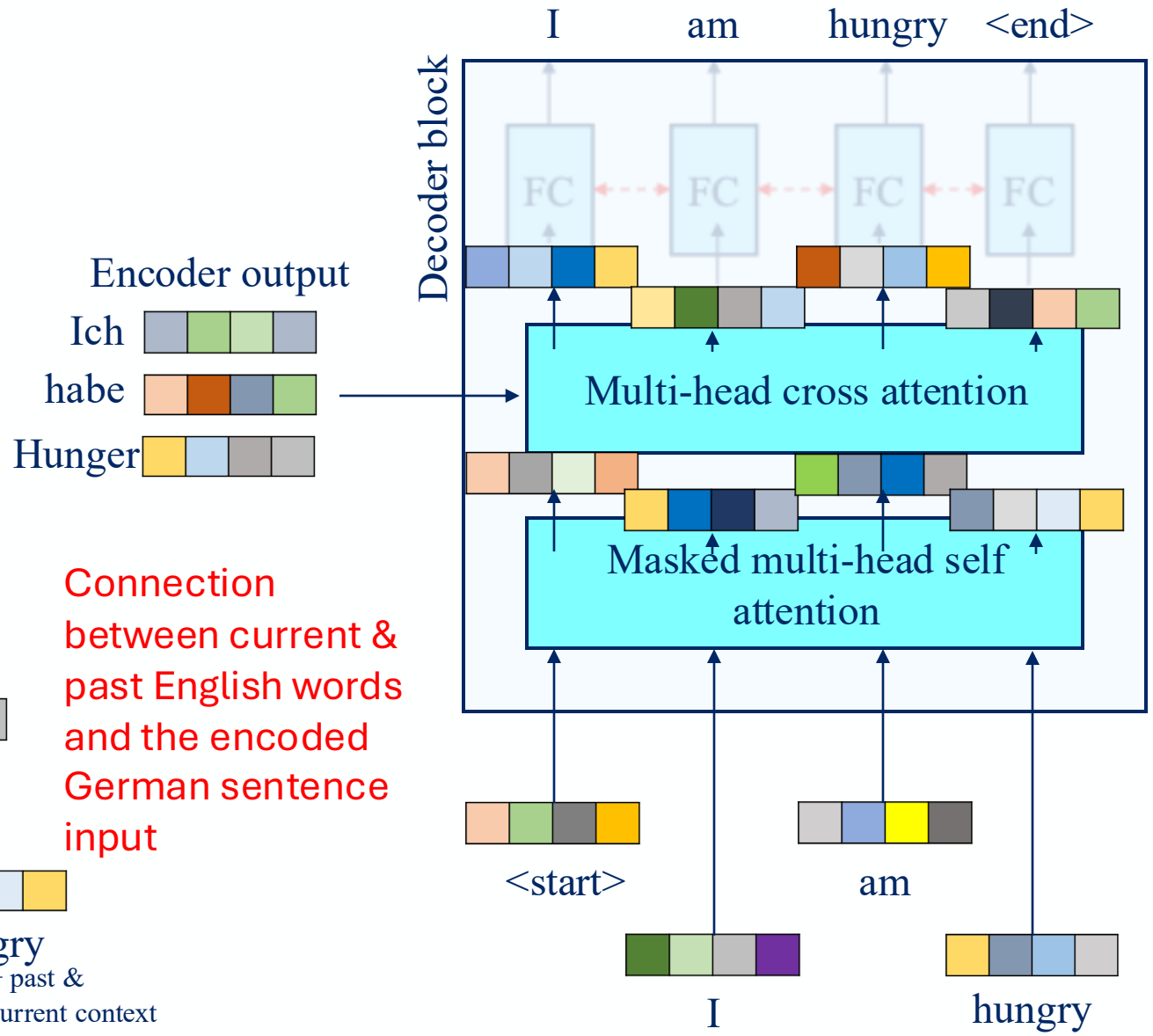


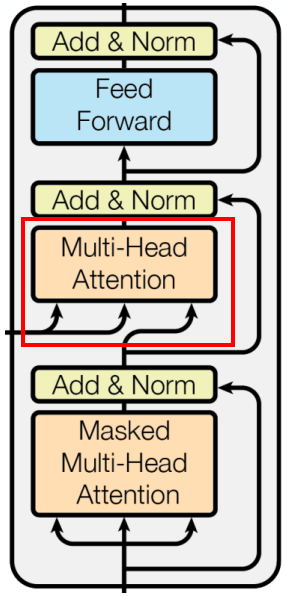
Decoder

- Masked multi-head self attention
- Multi-head cross attention



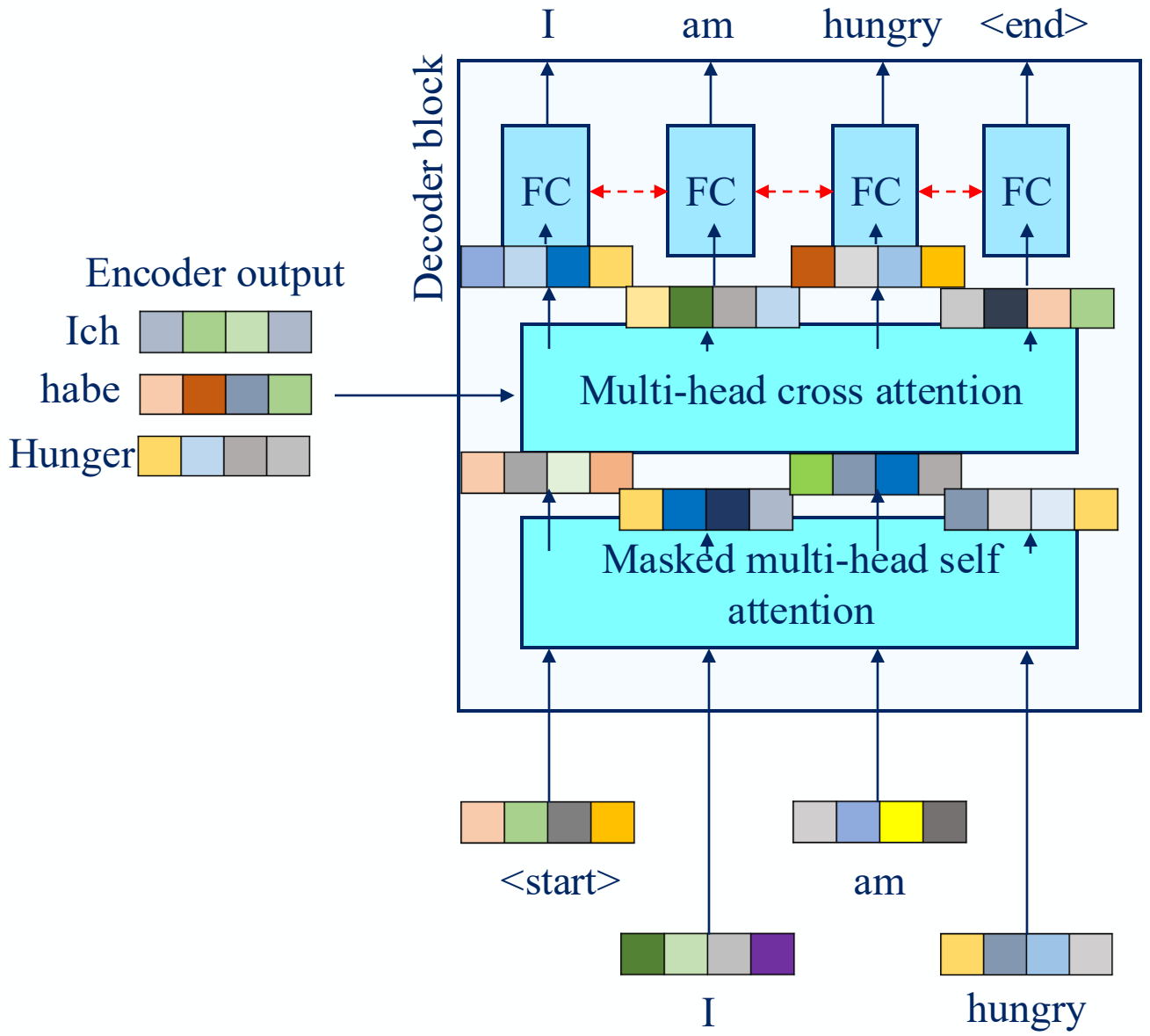
Connection between current & past English words and the encoded German sentence input

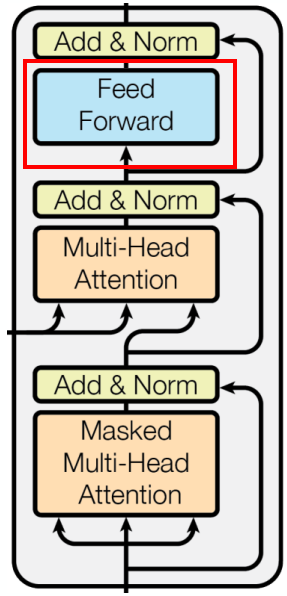




Decoder

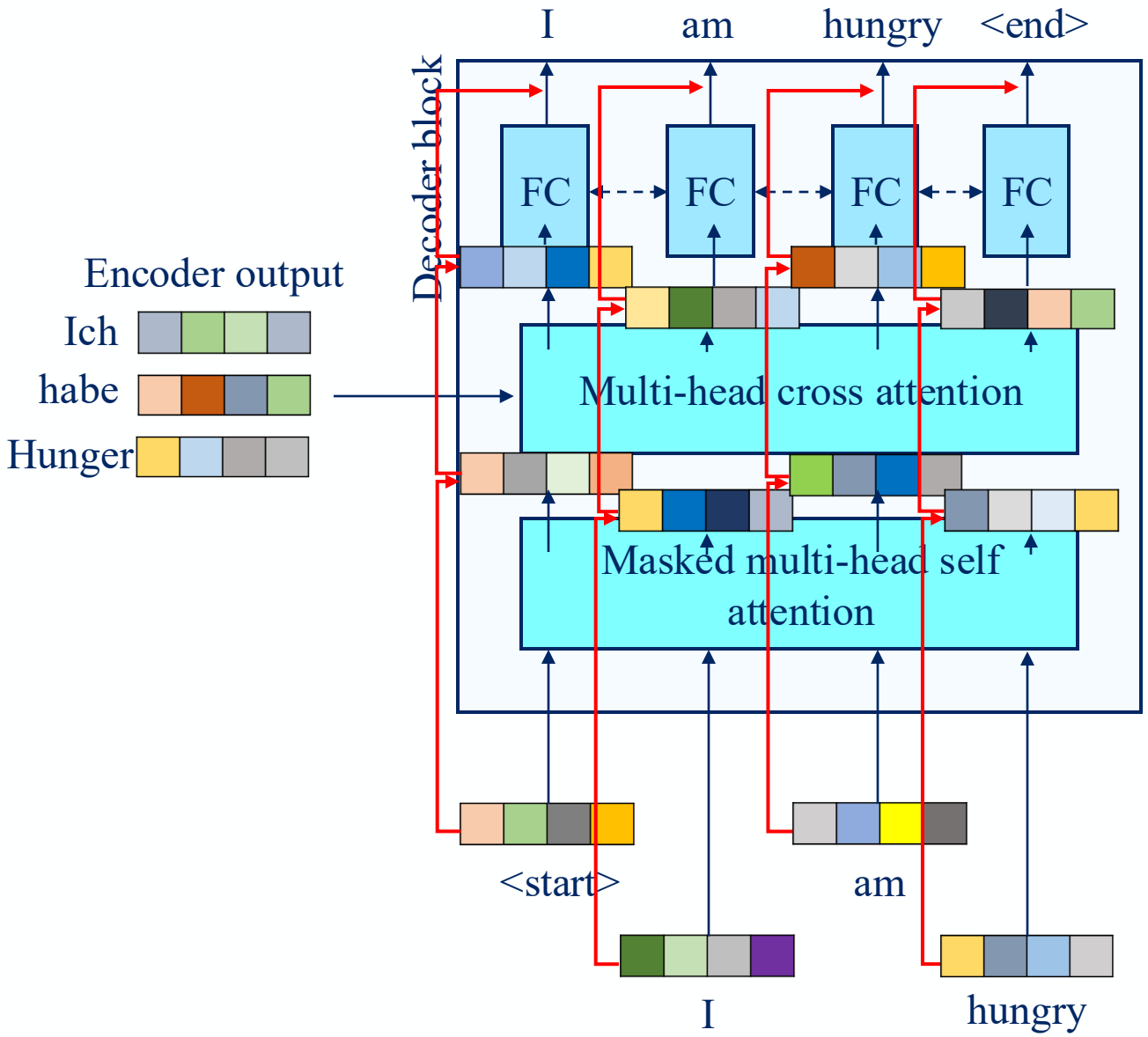
- Masked multi-head self attention
- Multi-head cross attention
- Feed forward/fully connected layer
  - Shared weight

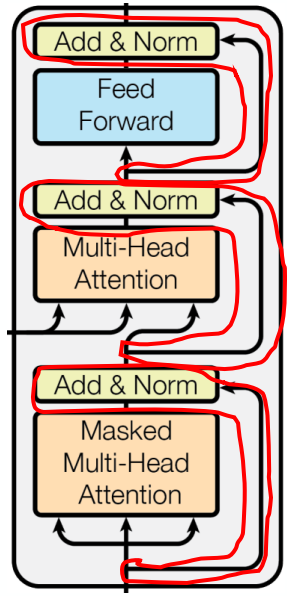




Decoder

- Masked multi-head self attention
- Multi-head cross attention
- Feed forward/fully connected layer
  - Shared weight
- Residual connection





Outline

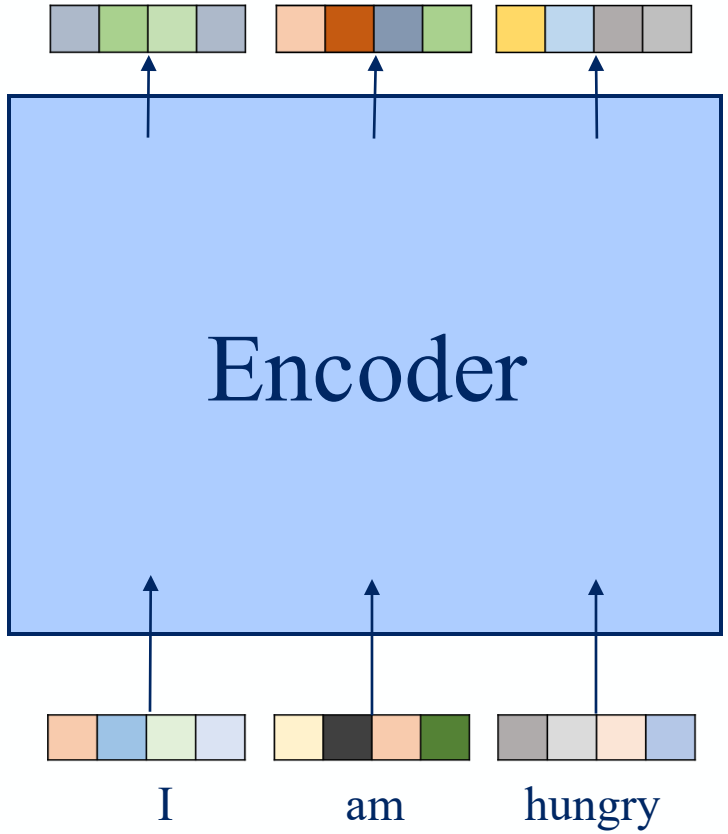
1. Overview
2. Preprocessing
3. Attention mechanism
4. Putting it all together
5. Variants of Transformer

Variants Of Transformer

- Bidirectional Encoder Representations from Transformers (BERT)
  - Encoder-only architecture
- Vision Transformer (ViT)
  - BERT, but for vision
- Generative Pre-trained Transformer (GPT)
  - Decoder-only architecture

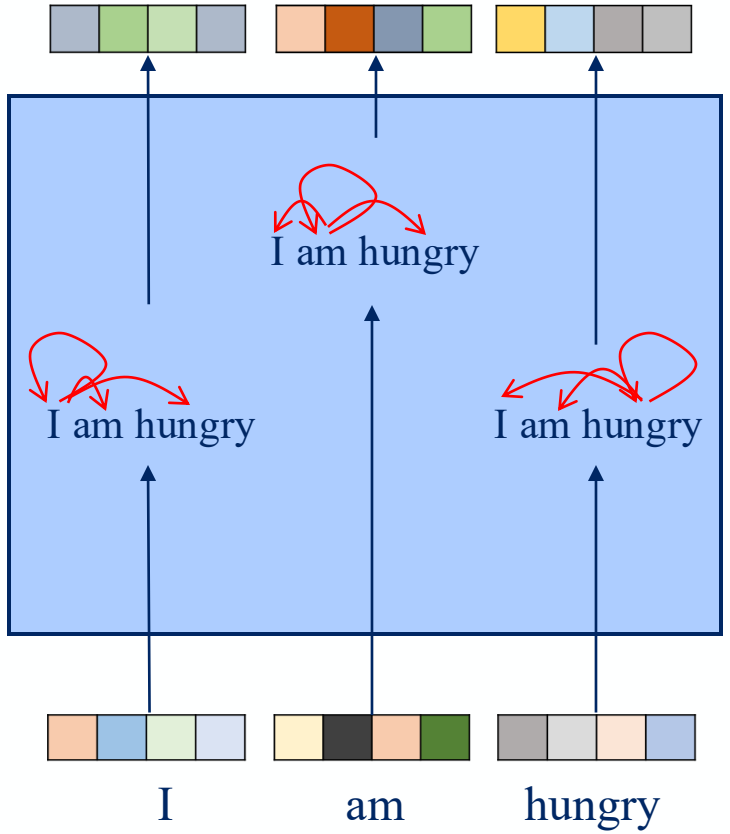
BERT

- Only encoder



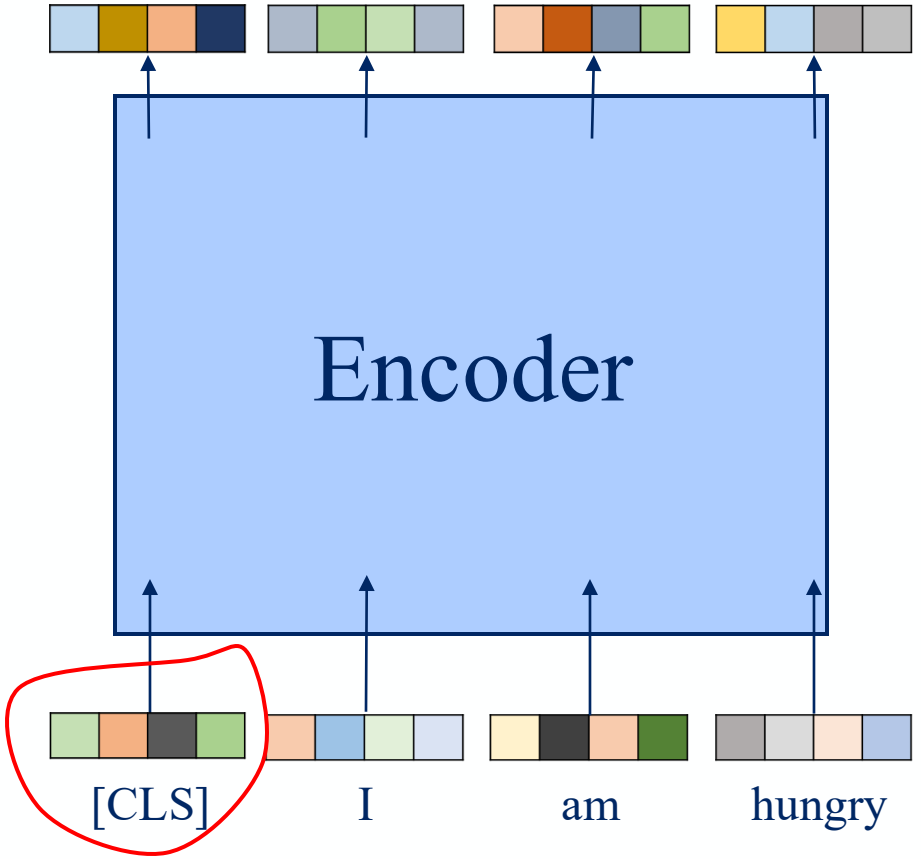
BERT

- Only encoder
- “Bidirectional” with self attention



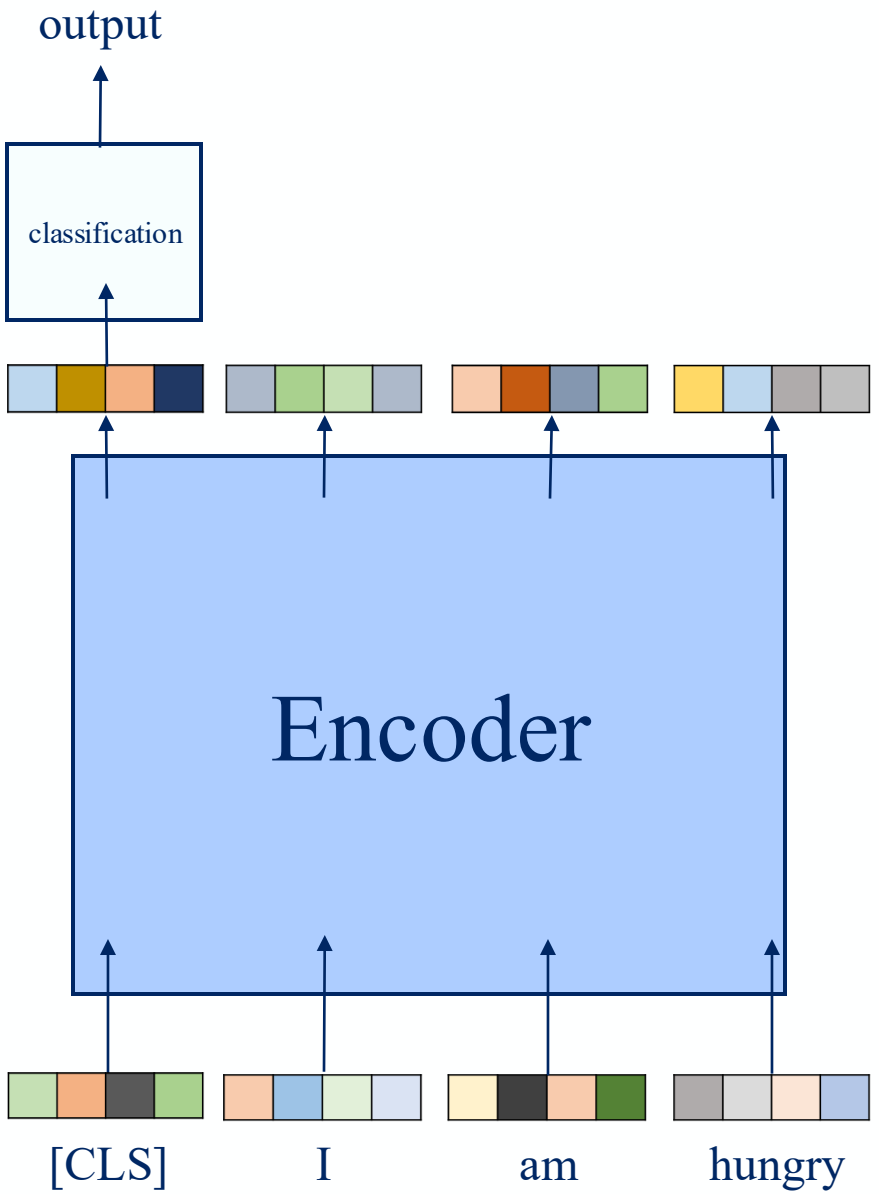
BERT

- Only encoder
- “Bidirectional” with self attention
- Add learnable classification token [CLS]



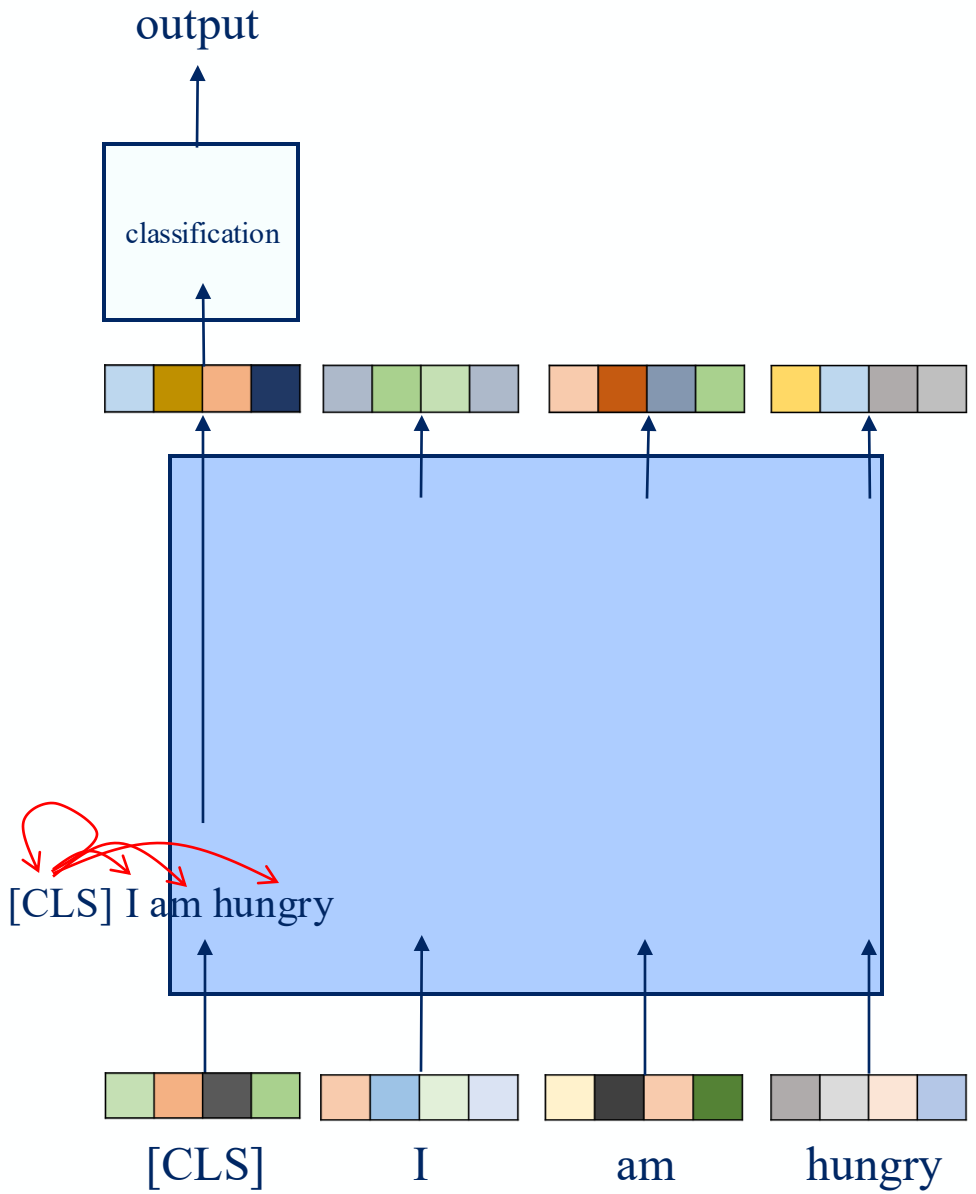
BERT

- Only encoder
- “Bidirectional” with self attention
- Add learnable classification token [CLS]
  - For classification task
    - E.g., spam/not spam, good/neutral/bad sentiment



BERT

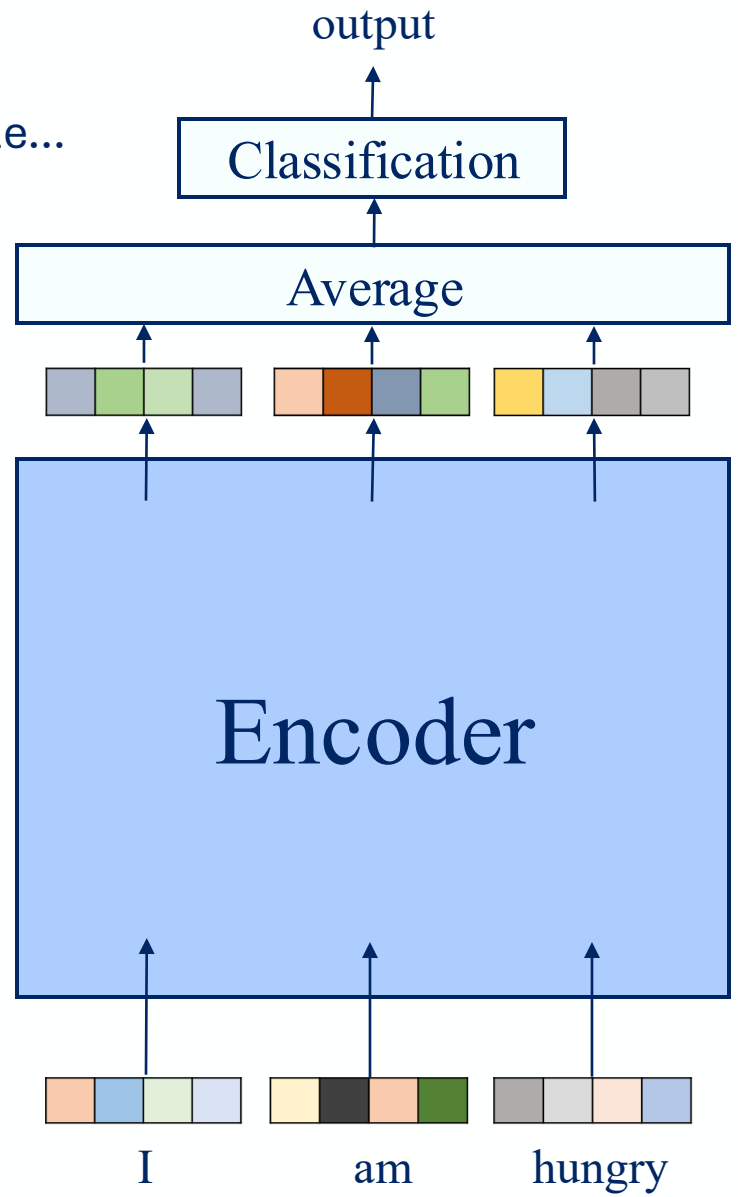
- Only encoder
- “Bidirectional” with self attention
- Add learnable classification token [CLS]
  - For classification task
  - Aggregate information using self attention



BERT

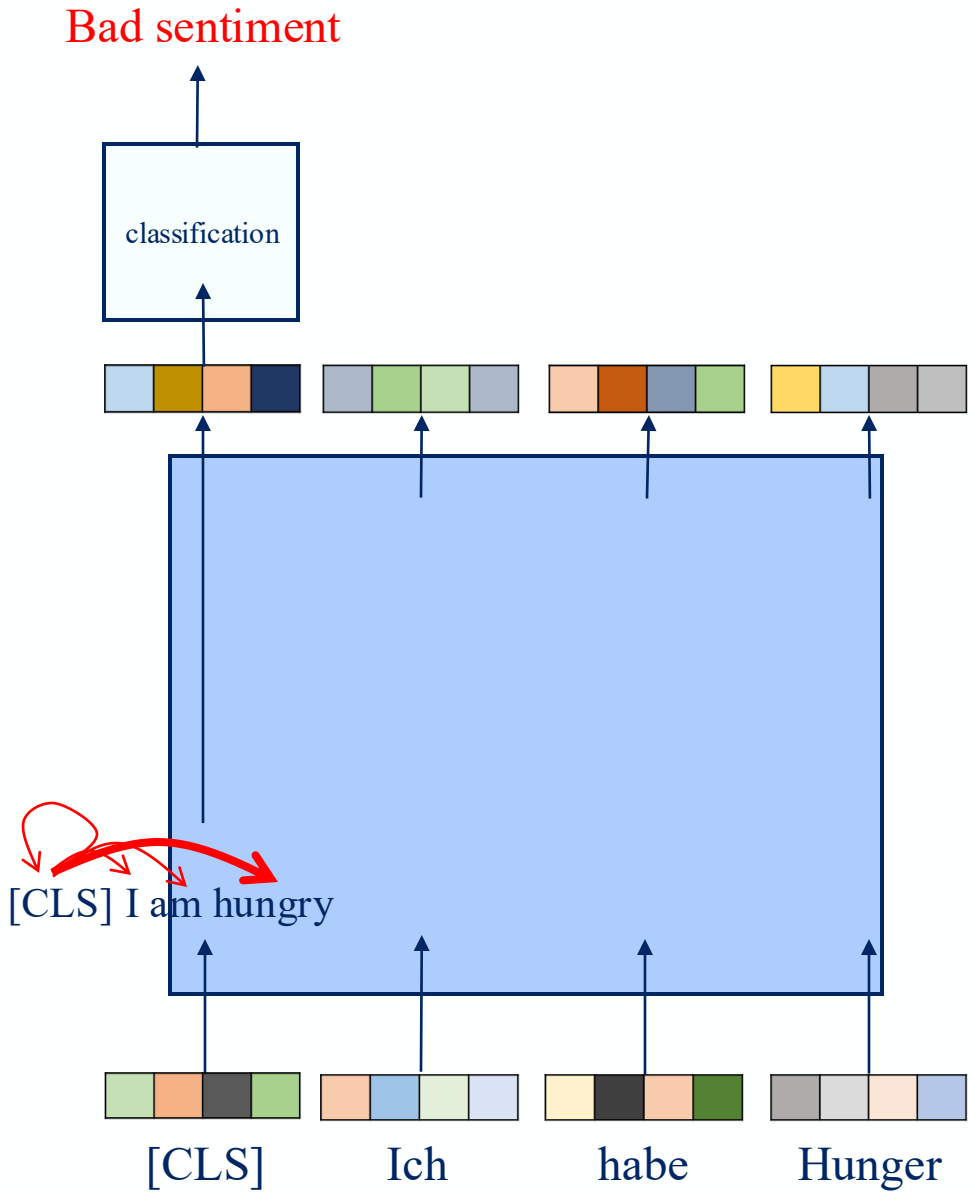
- Only encoder
- “Bidirectional” with self attention
- Add learnable classification token [CLS]
  - For classification task
  - Aggregate information using self attention

Technically possible...



BERT

- Only encoder
- “Bidirectional” with self attention
- Add learnable classification token [CLS]
  - For classification task
  - Aggregate information using self attention
    - Can focus/ignore some words

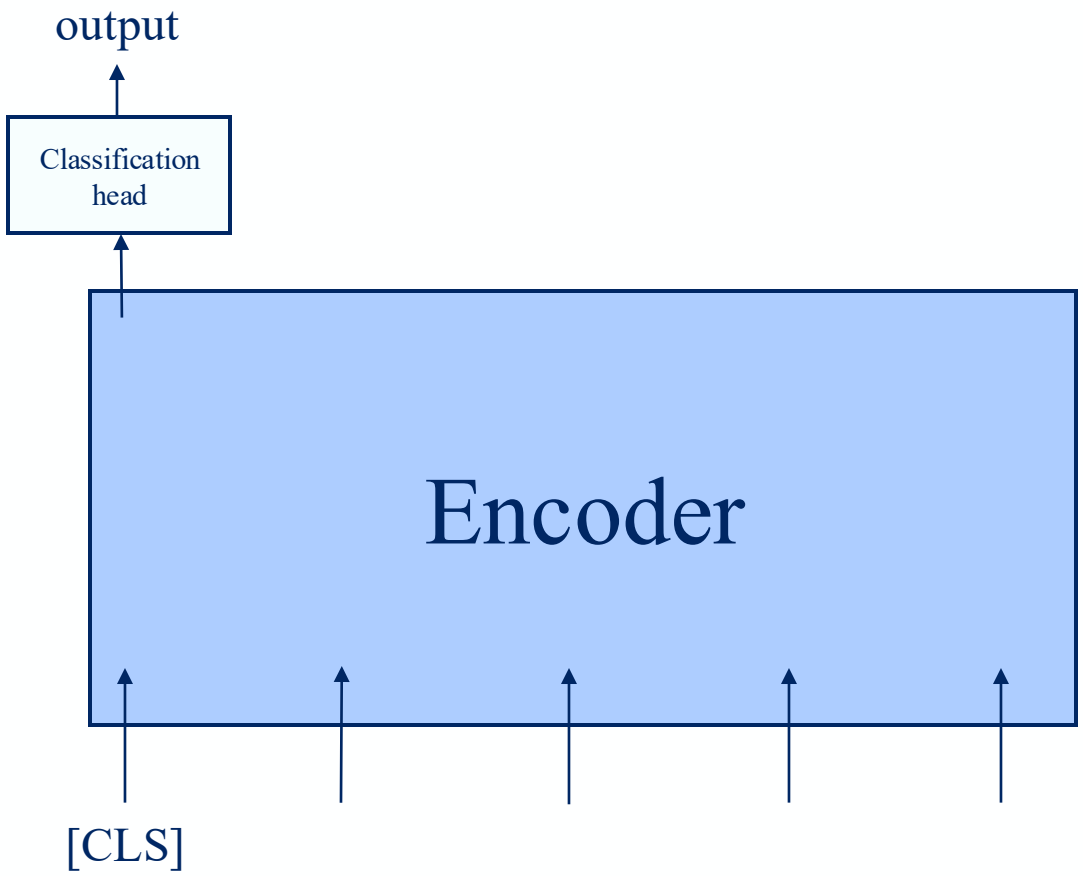


Variants Of Transformer

- Bidirectional Encoder Representations from Transformers (BERT)
  - Encoder-only architecture
- Vision Transformer (ViT)
  - BERT, but for vision
- Generative Pre-trained Transformer (GPT)
  - Decoder-only architecture

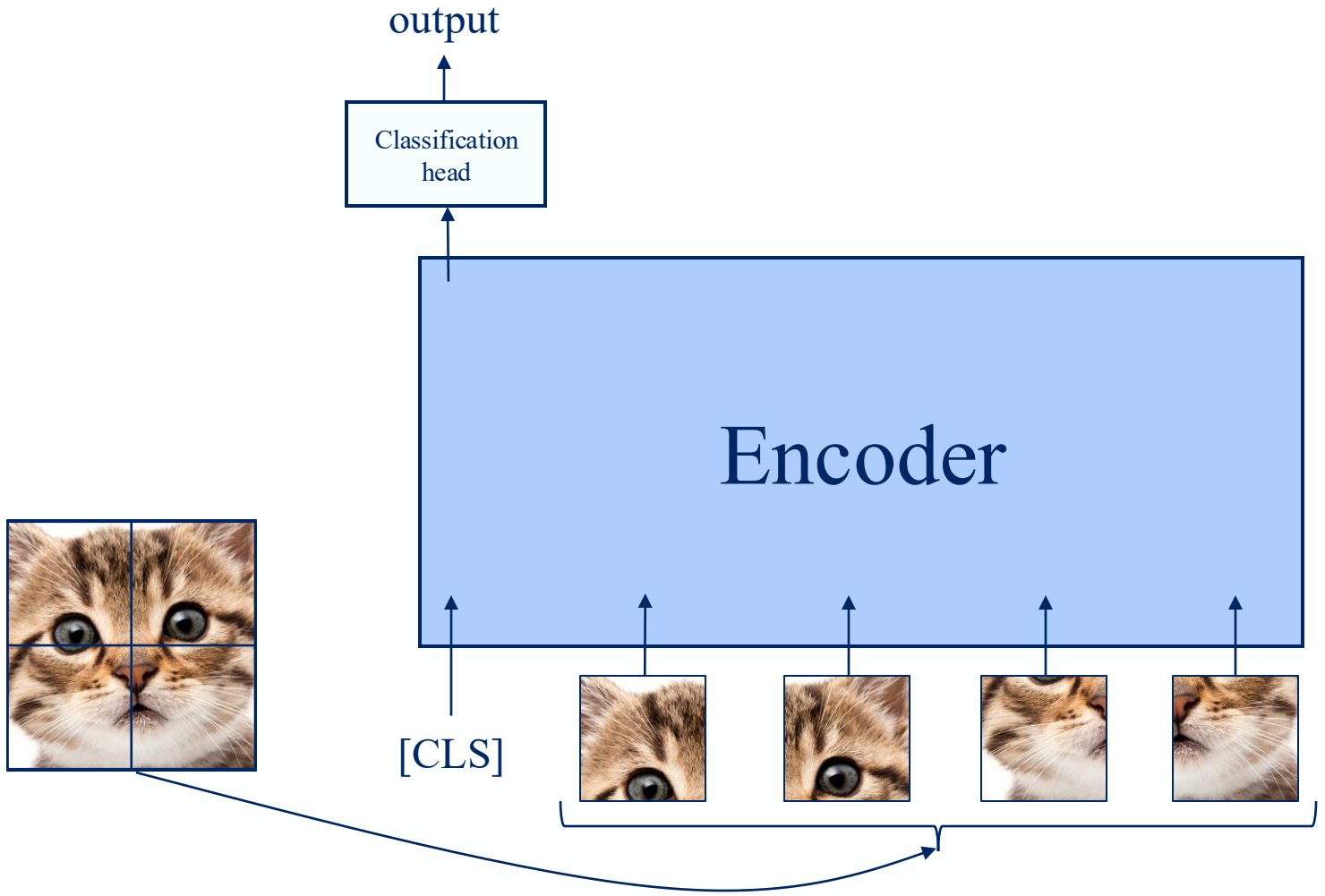
ViT

- Like BERT
  - Only the encoder
  - Learnable class token



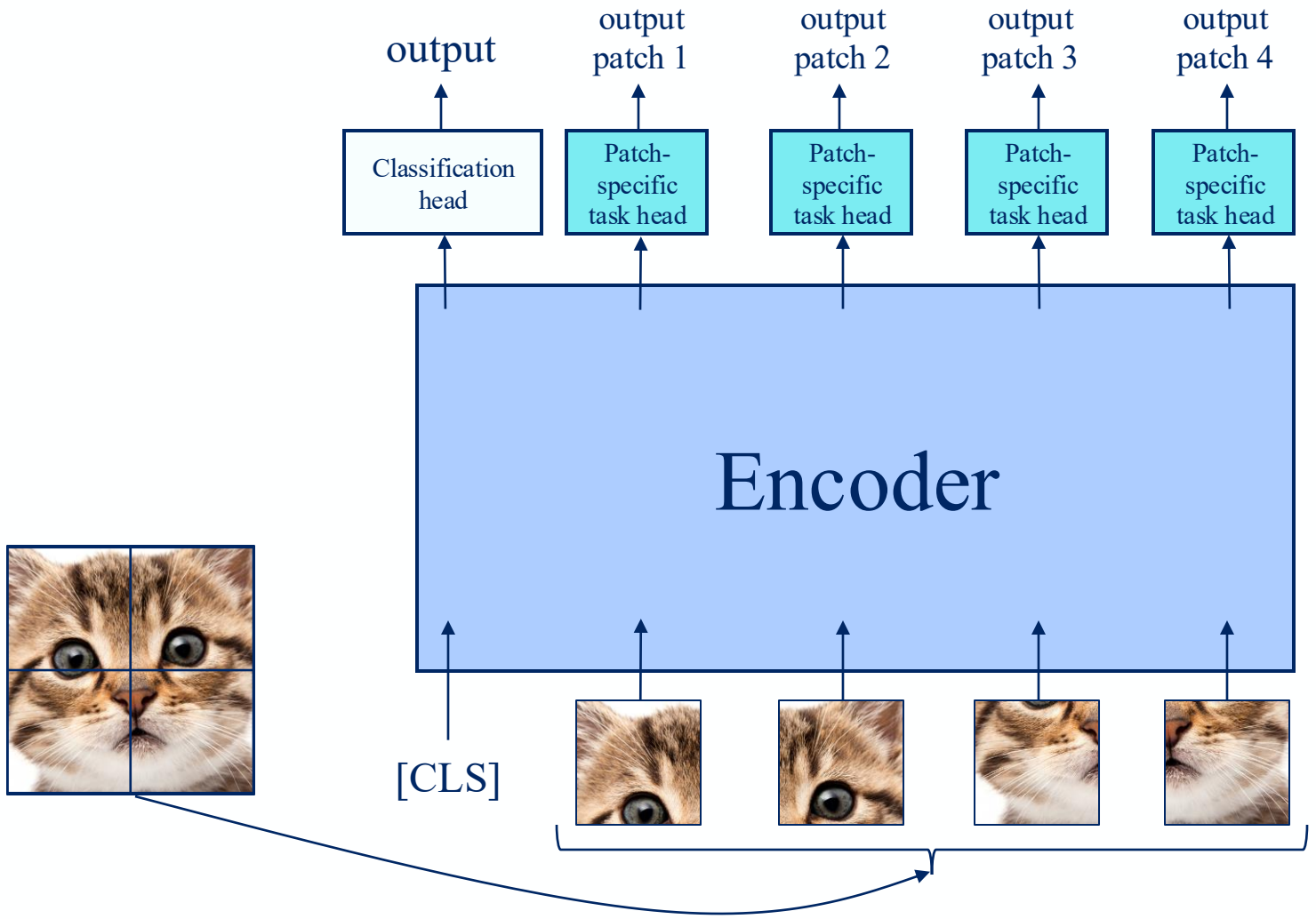
ViT

- Like BERT
  - Only the encoder
  - Learnable class token
- Use image patches



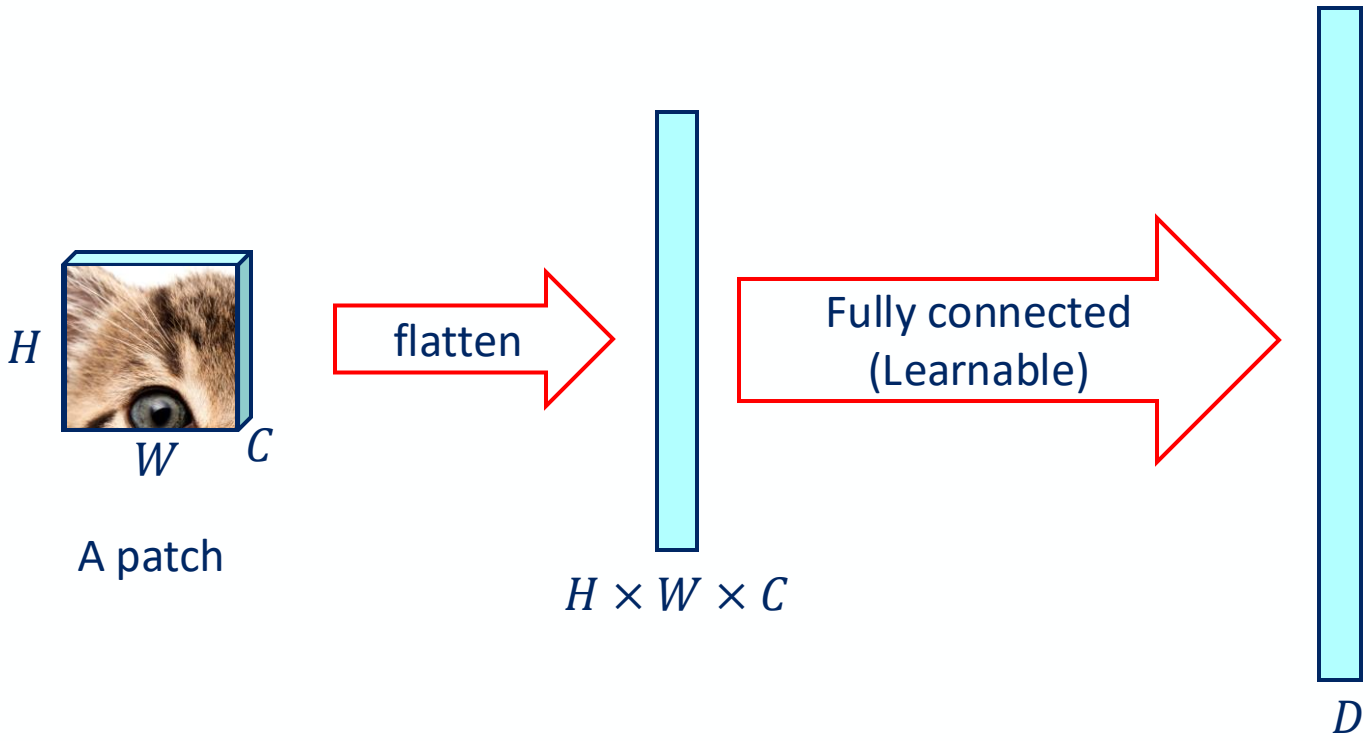
ViT

- Like BERT
  - Only the encoder
  - Learnable class token
- Use image patches
- Patch (not “word”) embedding can also be used
  - Not in original ViT
  - Depending on task



VIT's Patch Embedding

- Converting a patch to a vector
  - Like the word embedding

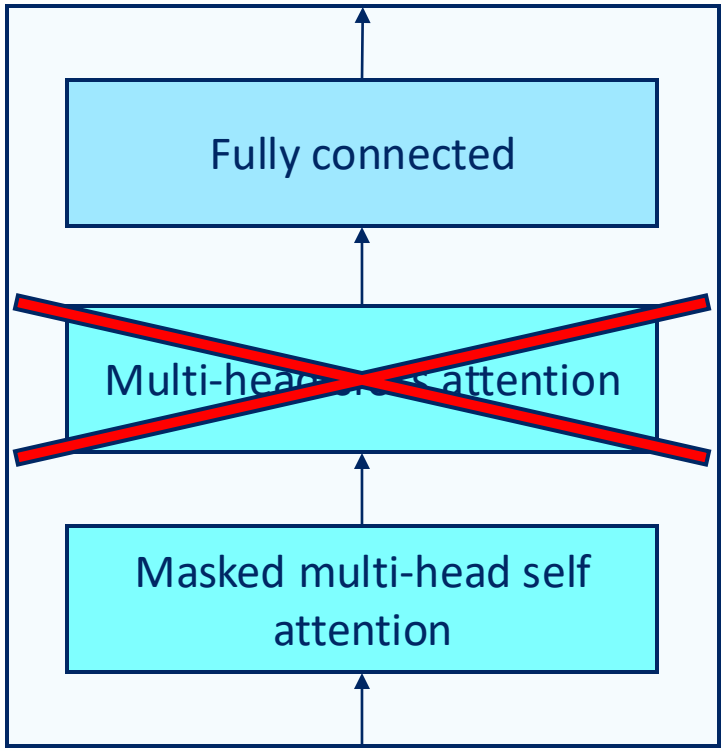


Variants Of Transformer

- Bidirectional Encoder Representations from Transformers (BERT)
  - Encoder-only architecture
- Vision Transformer (ViT)
  - BERT, but for vision
- Generative Pre-trained Transformer (GPT)
  - Decoder-only architecture

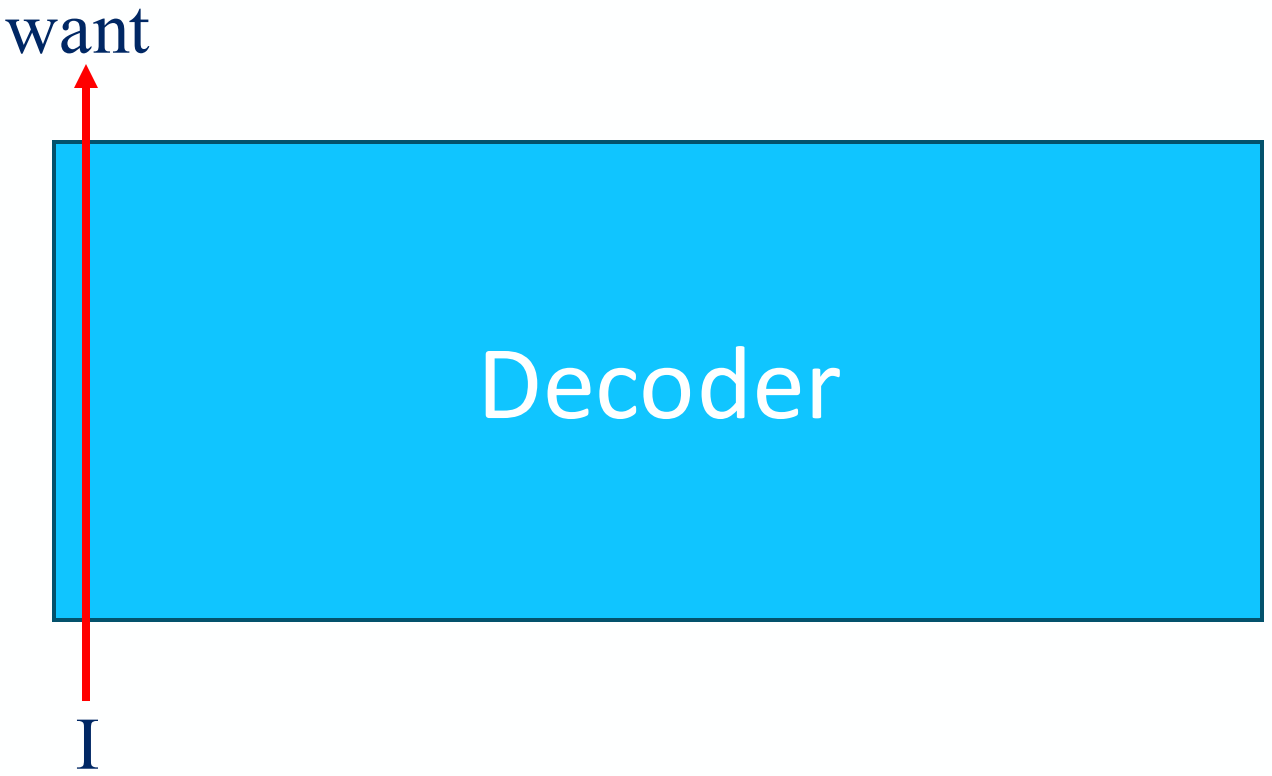
GPT

- Decoder-only architecture
  - Without cross attention as we don't have encoder



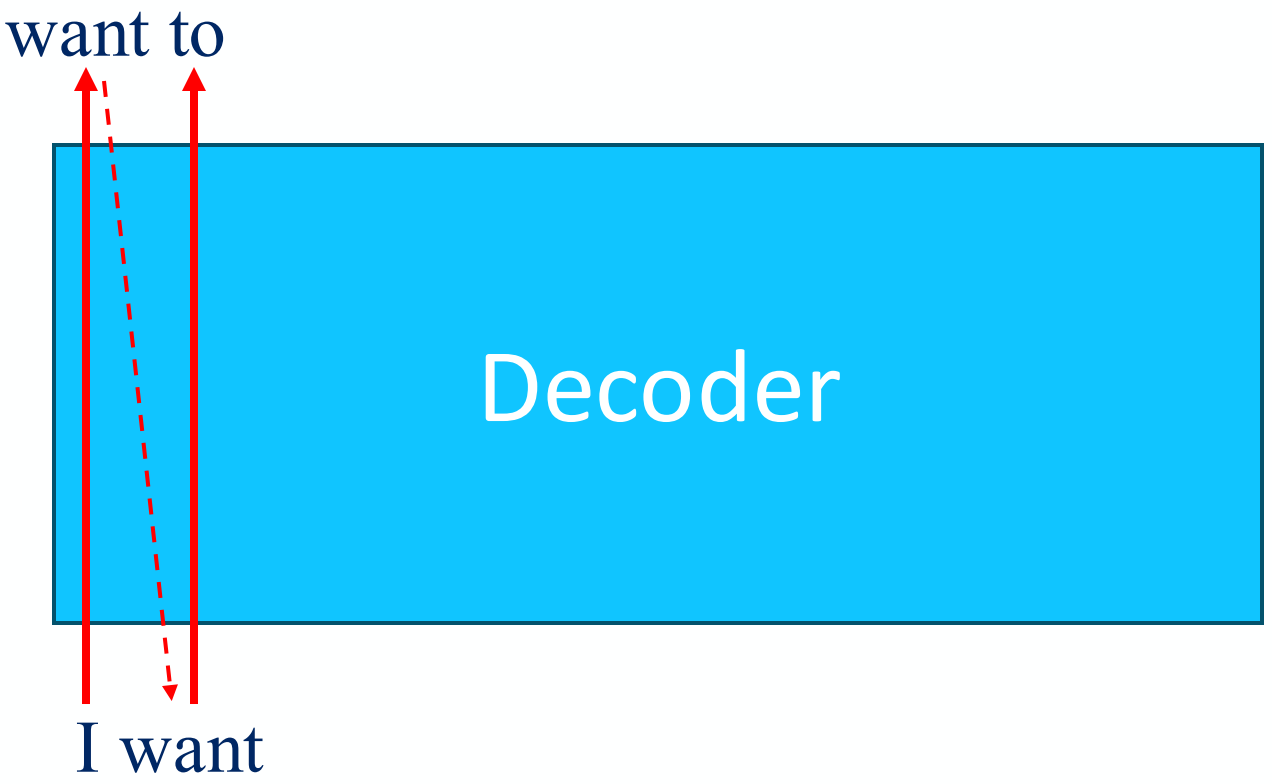
GPT

- For generative purpose
  - Given the past words, predict the next word



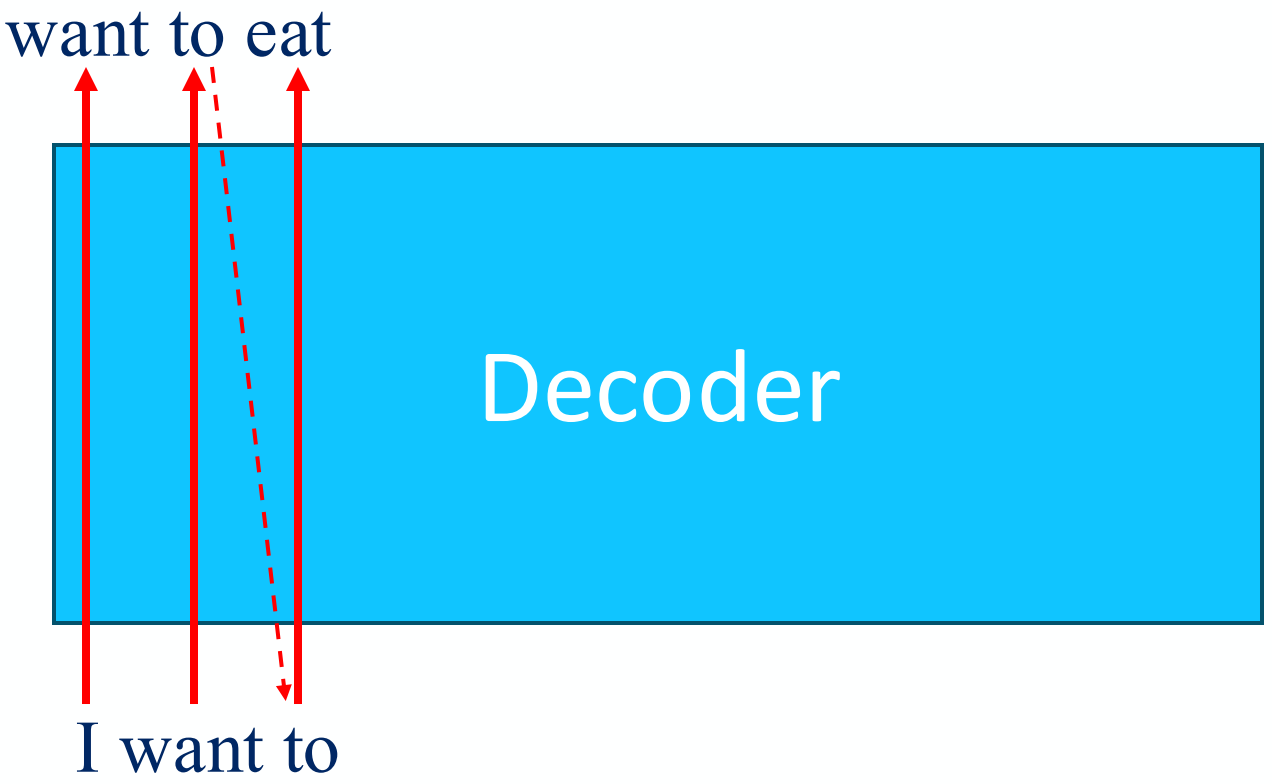
GPT

- For generative purpose
  - Given the past words, predict the next word



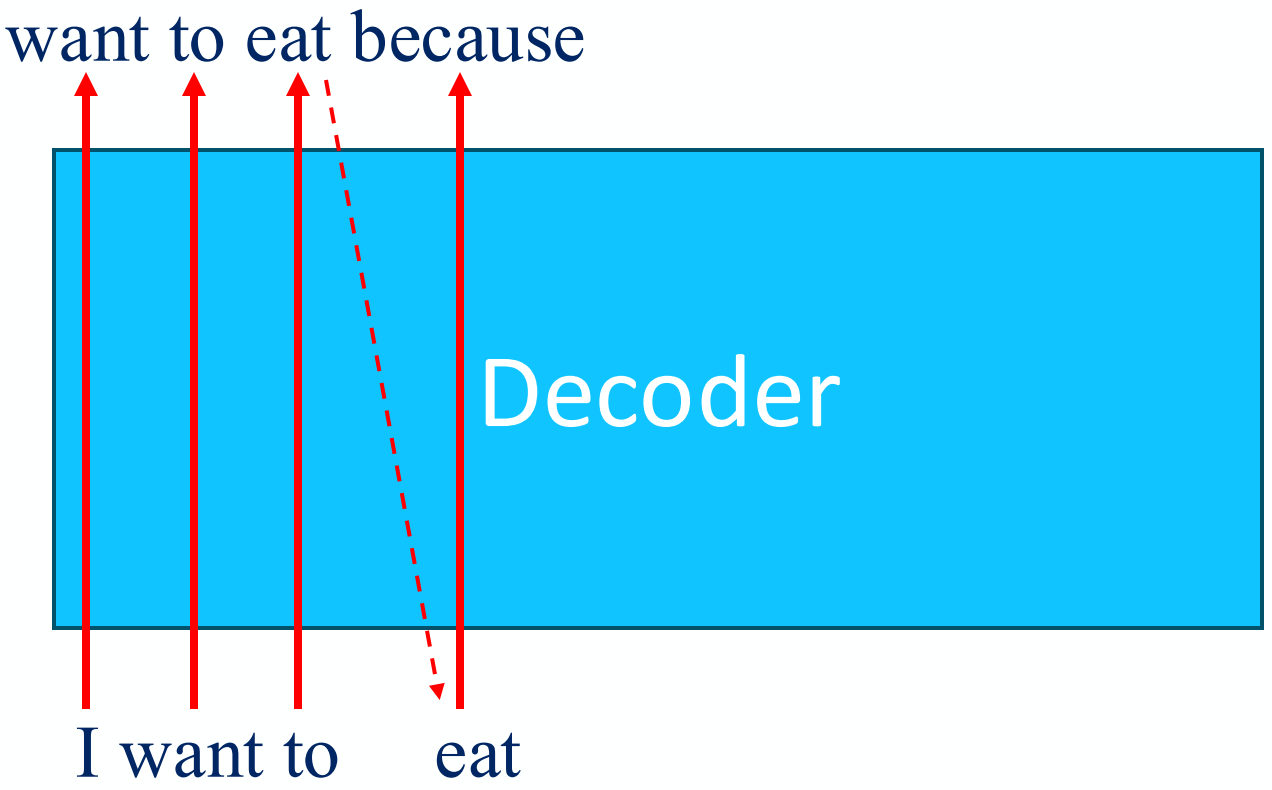
GPT

- For generative purpose
  - Given the past words, predict the next word



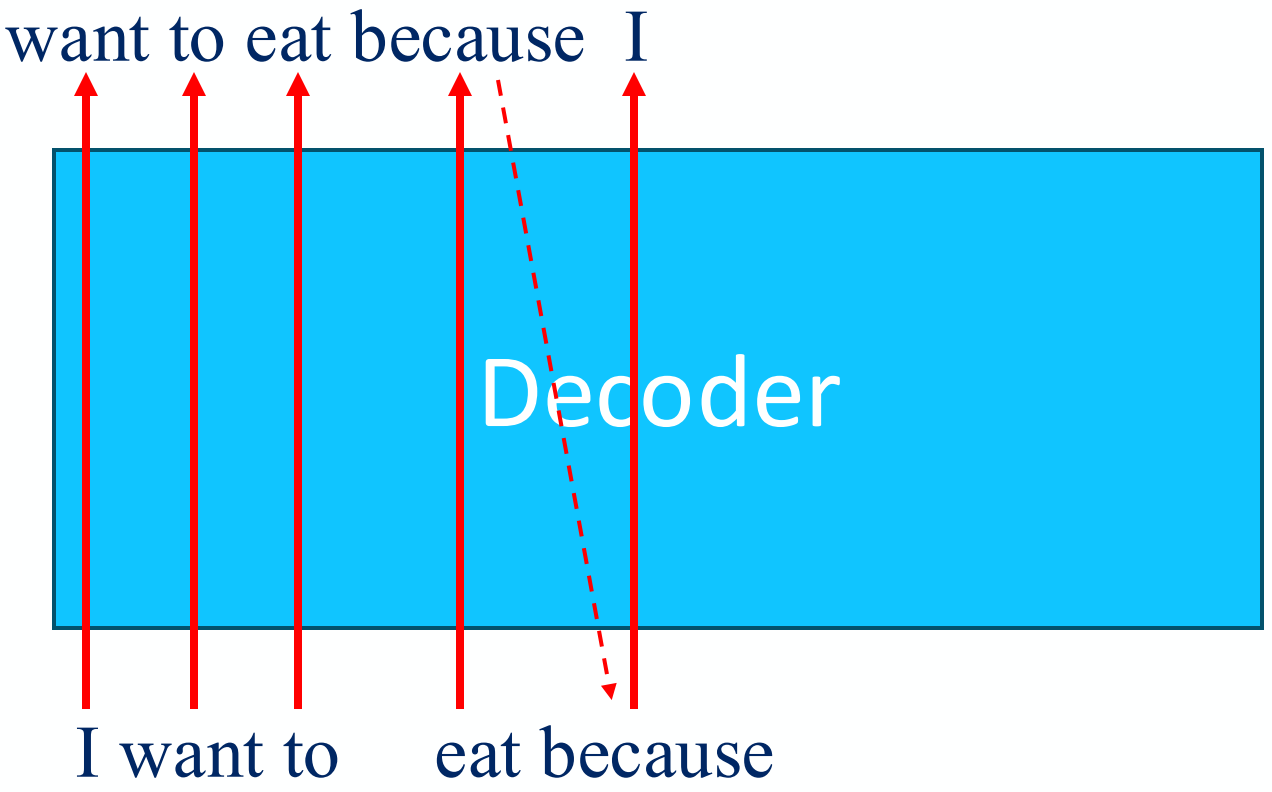
GPT

- For generative purpose
  - Given the past words, predict the next word



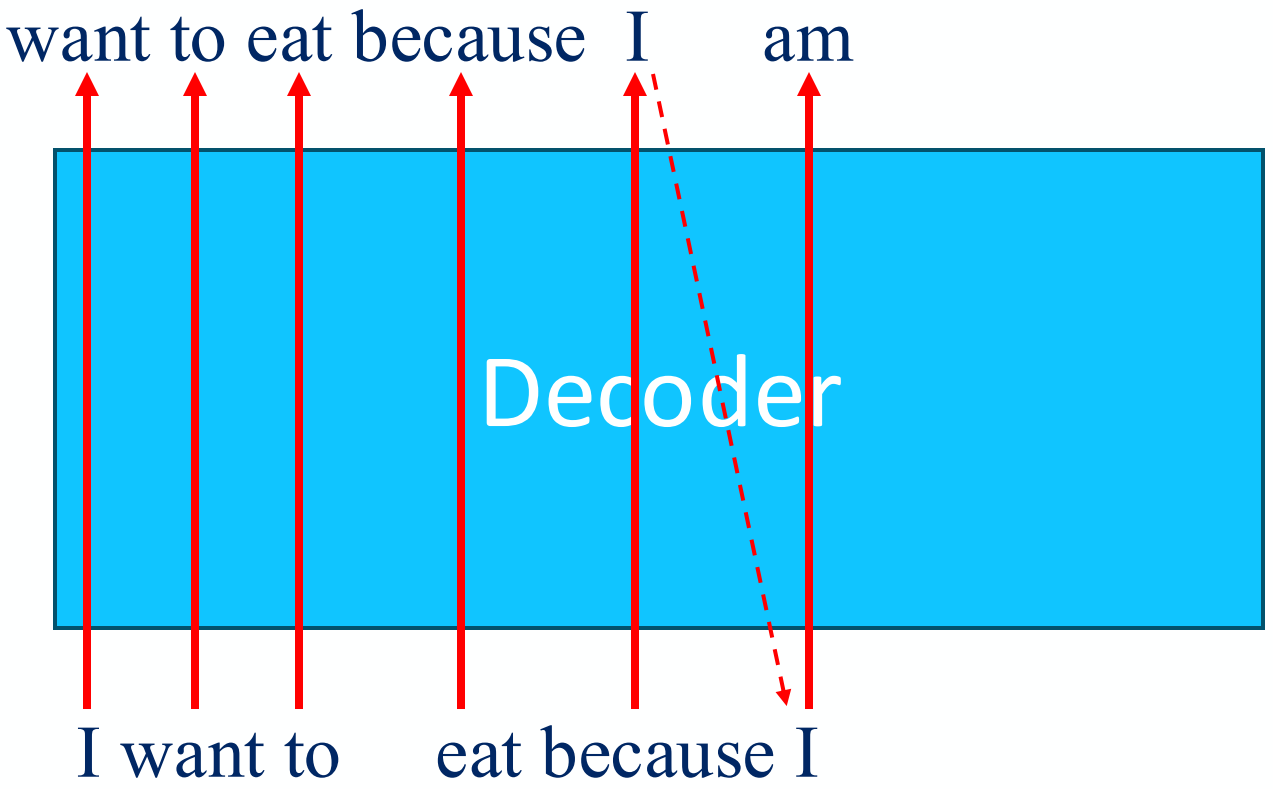
GPT

- For generative purpose
  - Given the past words, predict the next word



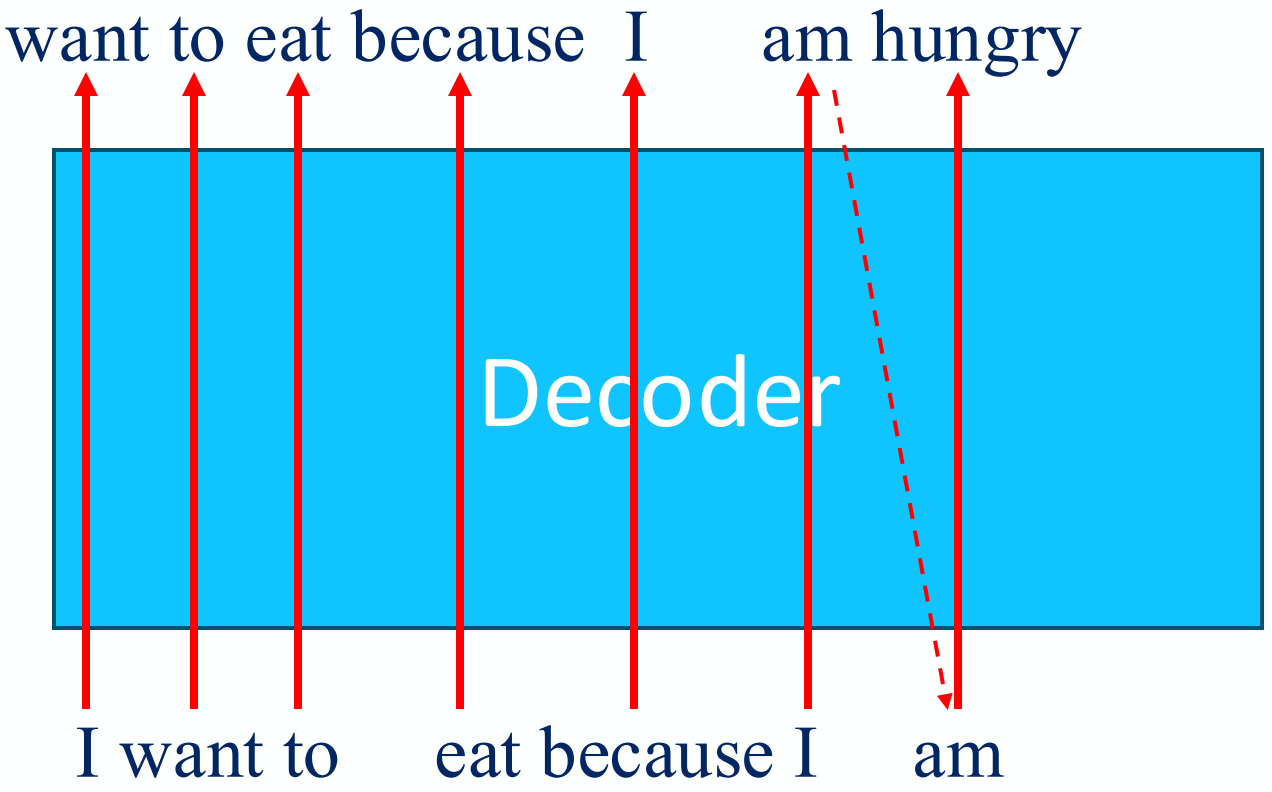
GPT

- For generative purpose
  - Given the past words, predict the next word



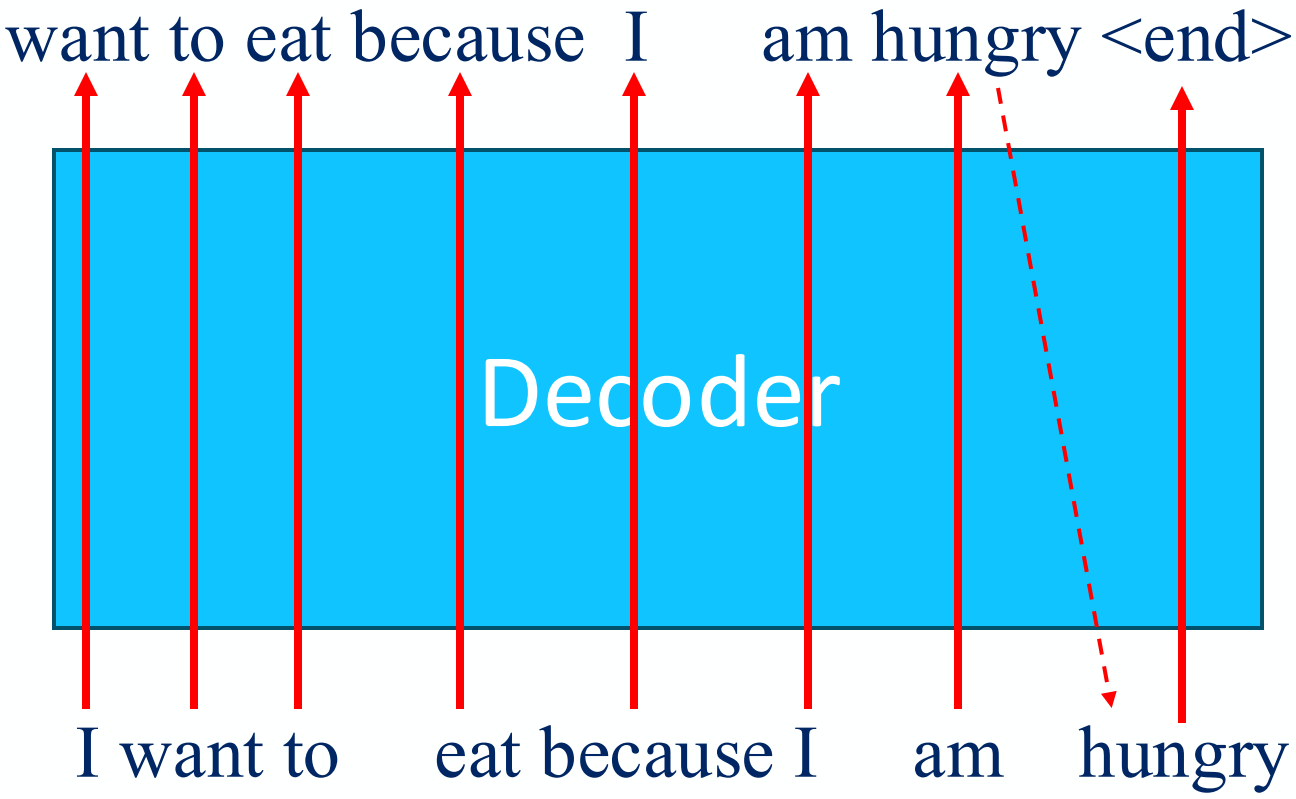
GPT

- For generative purpose
  - Given the past words, predict the next word



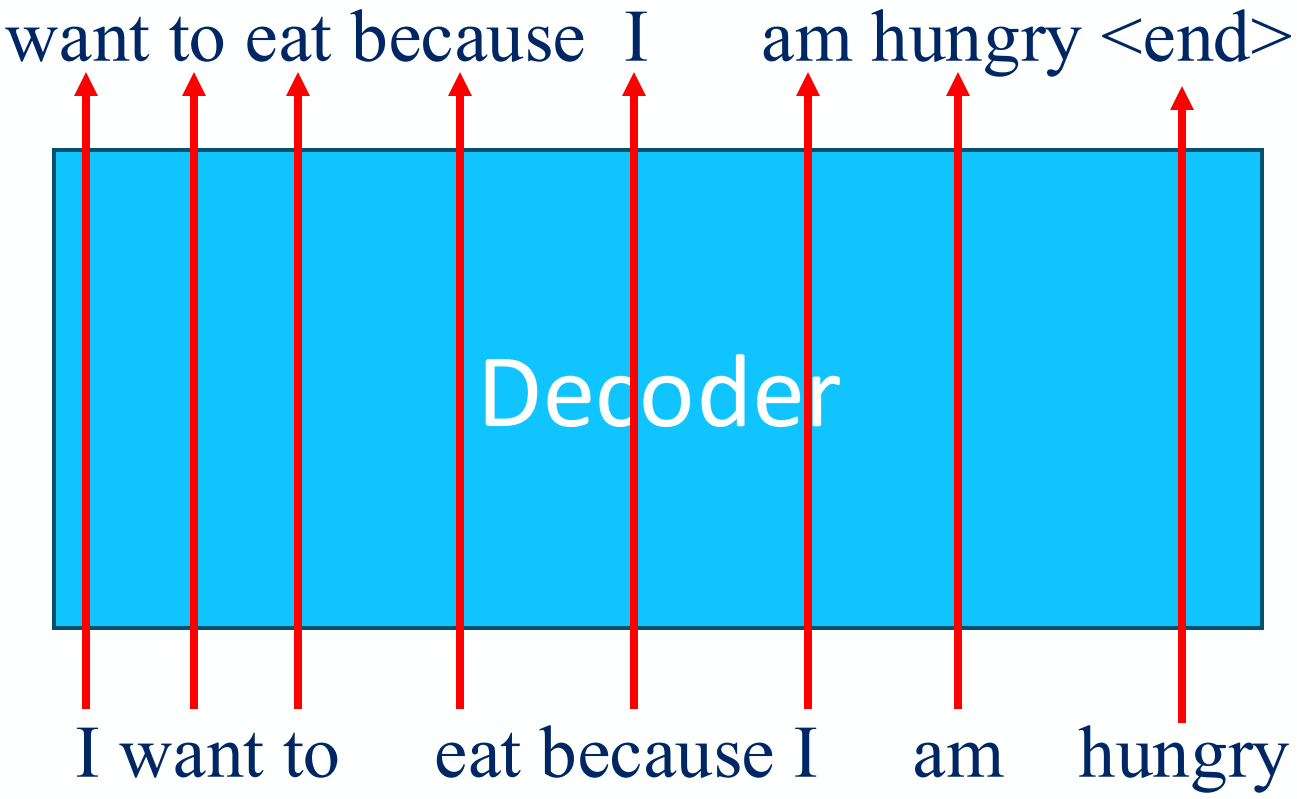
GPT

- For generative purpose
  - Given the past words, predict the next word



GPT

- For generative purpose
  - Given the past words, predict the next word
- Trained in self-supervised way
  - No human label
  - Predict next word of sentences from many articles/books/etc.

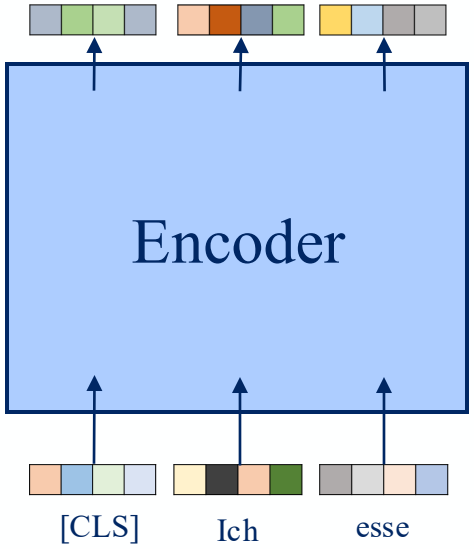


Agenda

09:00 - 10:30	Introduction to Language Modeling
10:30 - 10:45	Break
10:45 - 11:45	Blablador: concept, models description. Optional: integration in VScode
11:45 - 12:00	Q&A
12:00 - 13:00	Lunch Break
13:00 - 14:00	Attention, self-attention, transformer architecture
14:00 - 14:15	Pre-training vs fine-tuning and foundation models (intro to the hands-on session)
14:15 - 14:30	Break
14:30 - 15:30	Hands-on: fine-tune a model on a downstream task
15:30 - 16:00	Q&A, Wrap-up, and Conclusion

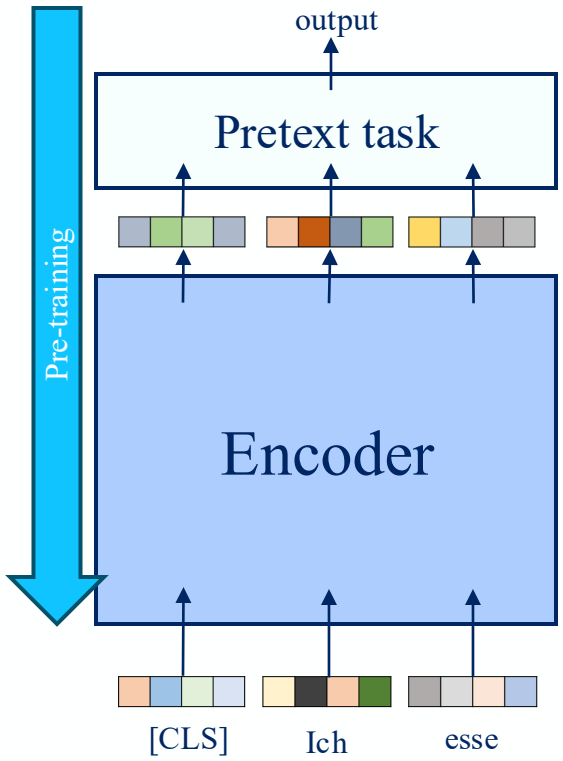
Bert pretraining and fine-tuning

- 2 stages:



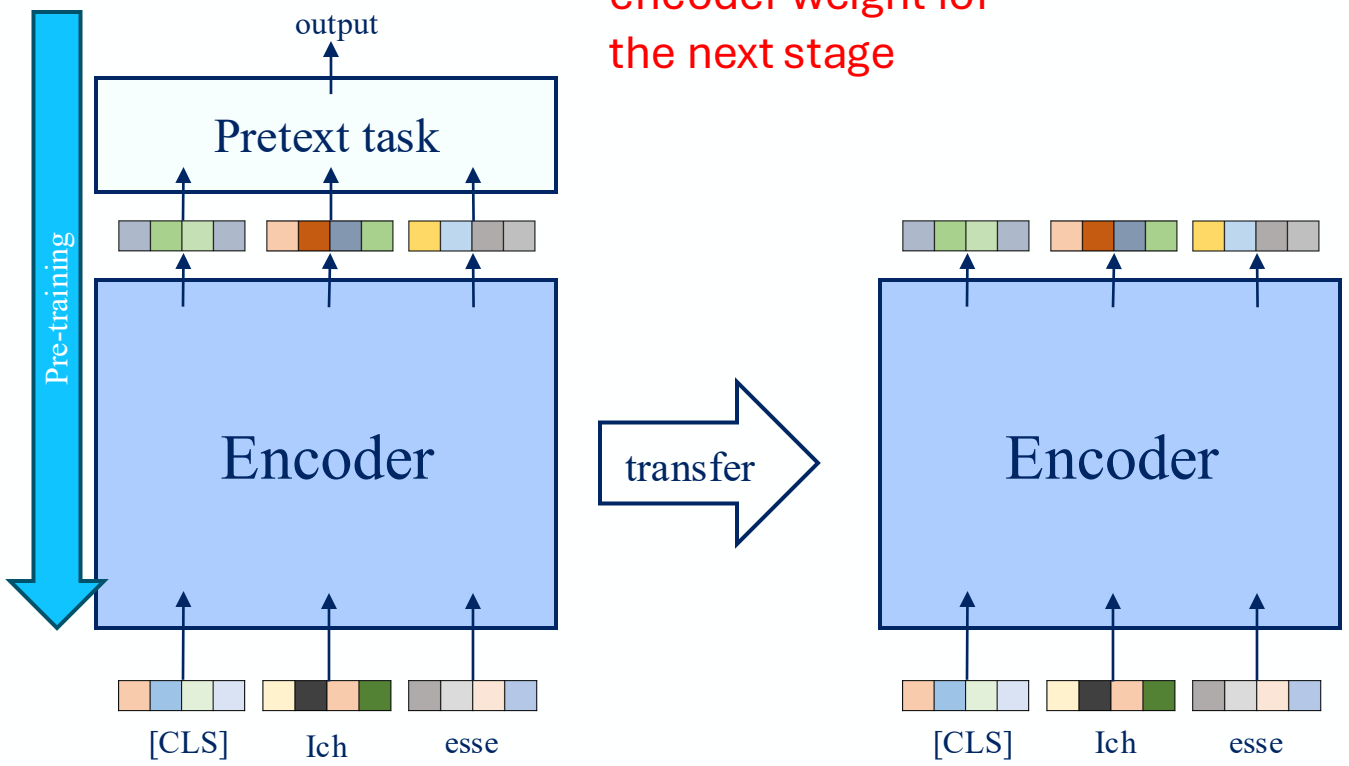
Bert pretraining and fine-tuning

- 2 stages:
  - 1st stage (pretraining): self-supervised learning
    - Without any human label using pretext task



Bert pretraining and fine-tuning

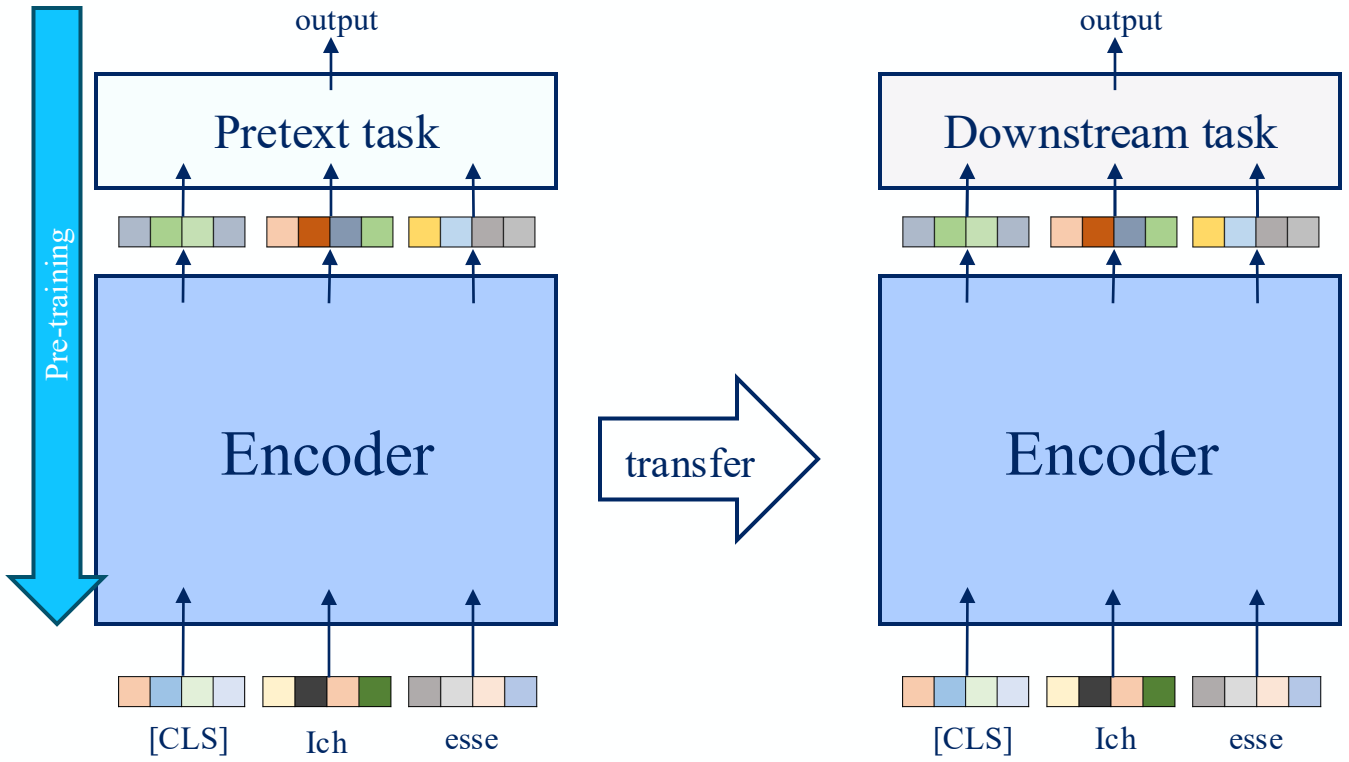
- 2 stages:
  - 1st stage (pretraining): self-supervised learning



Transfer the encoder weight for the next stage

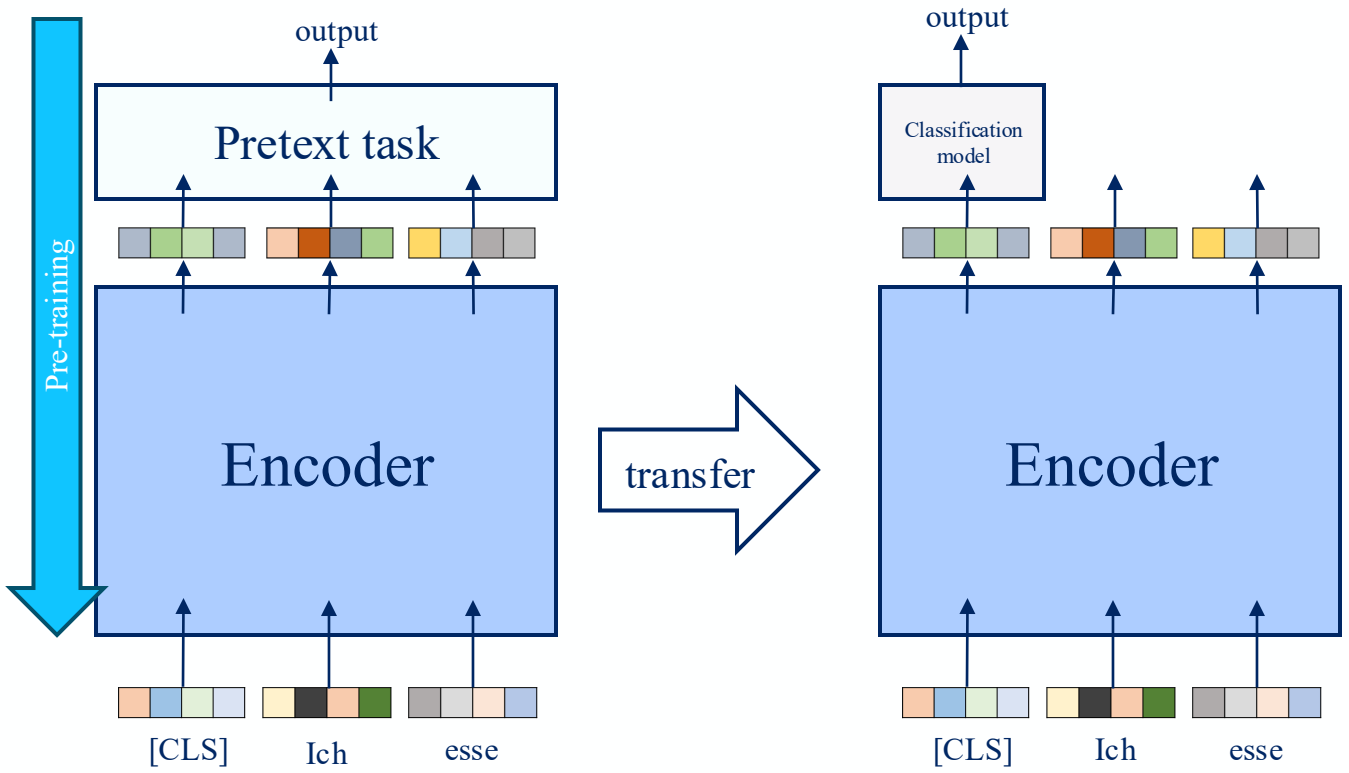
Bert pretraining and fine-tuning

- 2 stages:
  - 1st stage (pretraining): self-supervised learning
  - 2nd stage (fine-tuning): downstream task



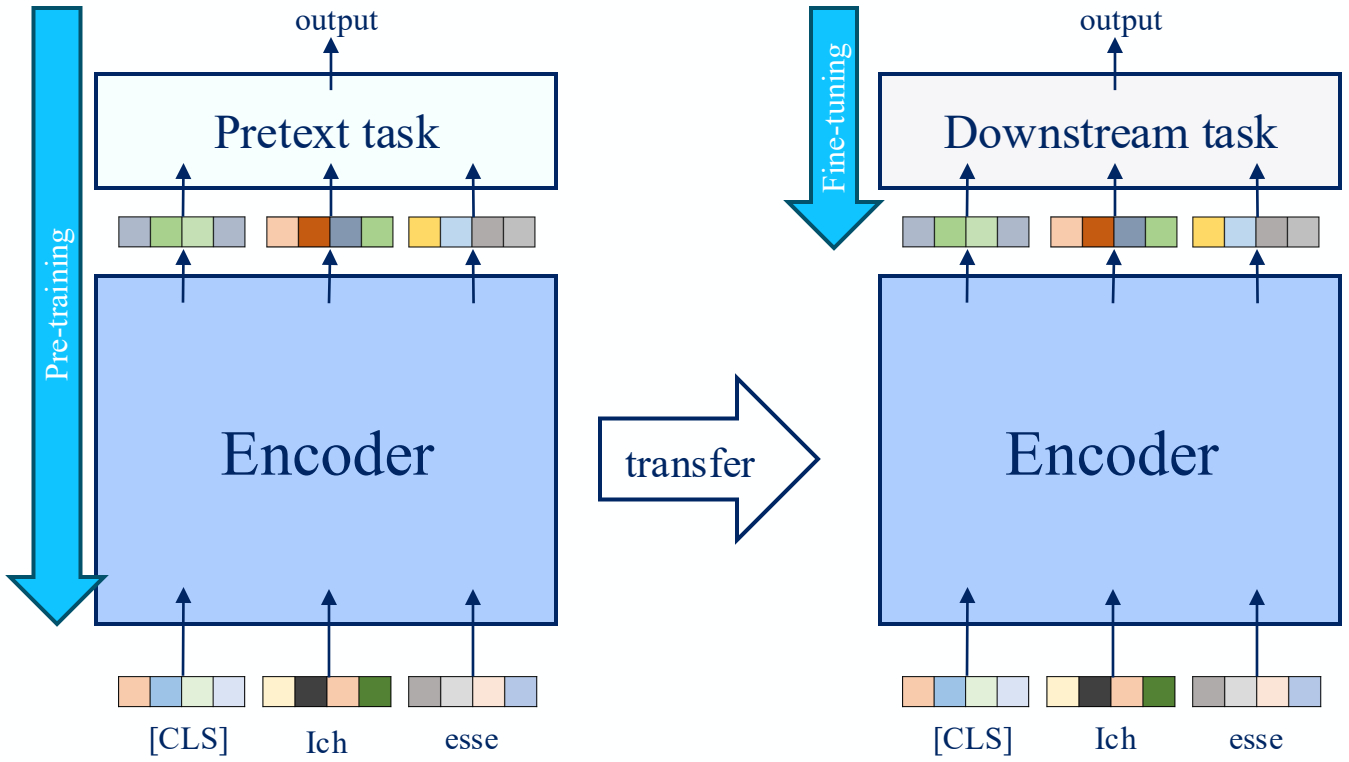
Bert pretraining and fine-tuning

- 2 stages:
  - 1st stage (pretraining): self-supervised learning
  - 2nd stage (fine-tuning): downstream task (e.g., classification)



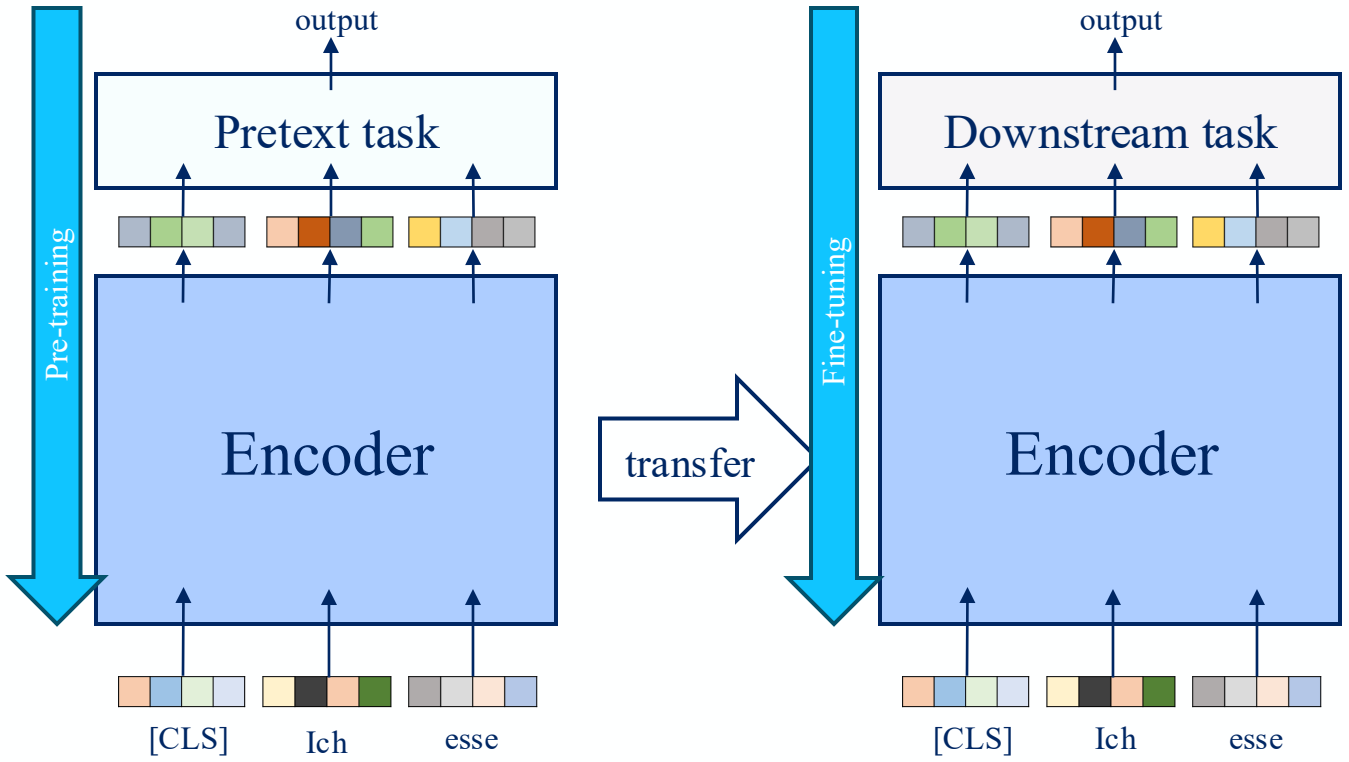
Bert pretraining and fine-tuning

- 2 stages:
  - 1st stage (pretraining): self-supervised learning
  - 2nd stage (fine-tuning): downstream task (e.g., classification)
    - Fine tune partially

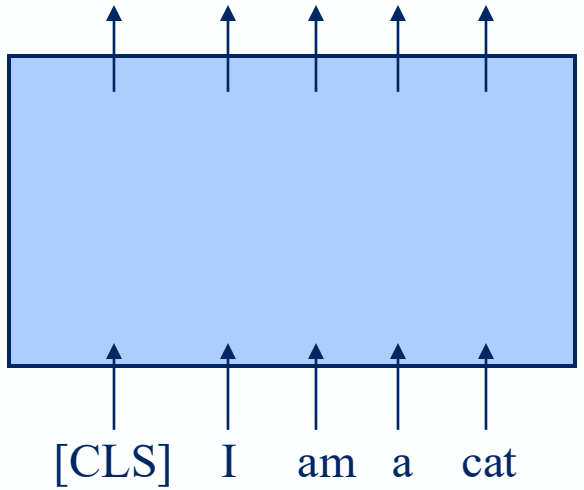


Bert pretraining and fine-tuning

- 2 stages:
  - 1st stage (pretraining): self-supervised learning
  - 2nd stage (fine-tuning): downstream task (e.g., classification)
    - Fine tune partially
    - Fine tune everything

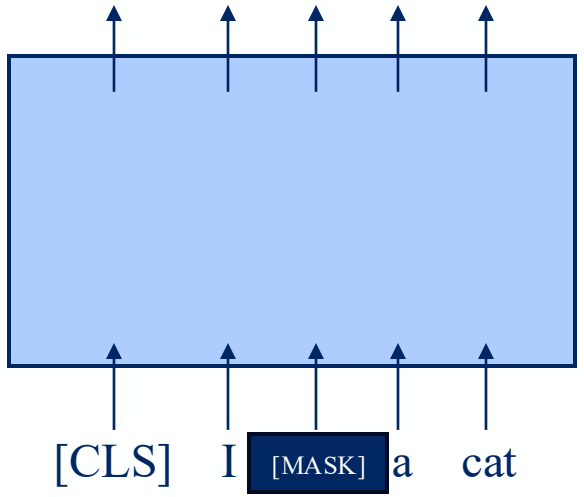


Pretext task: Masked Language Model



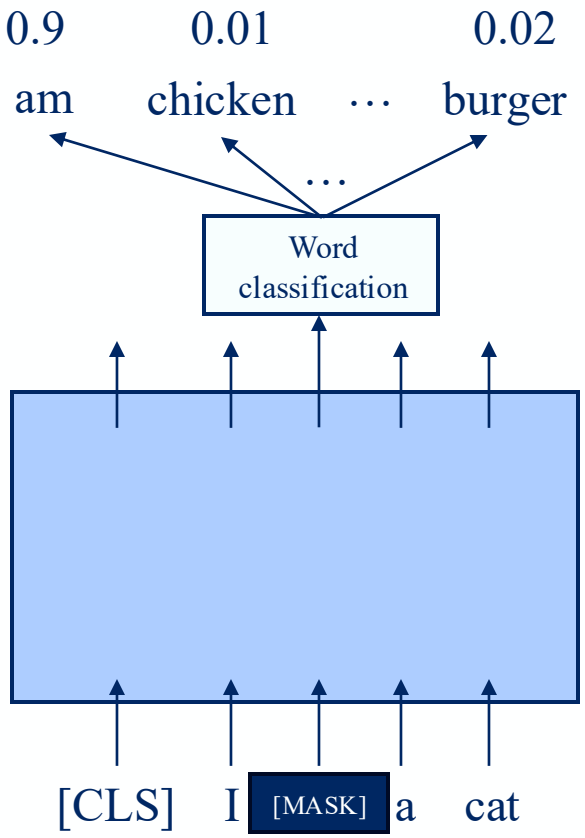
Pretext task: Masked Language Model

- Hide some input with [MASK] token
  - BERT paper: 15% of the input are masked



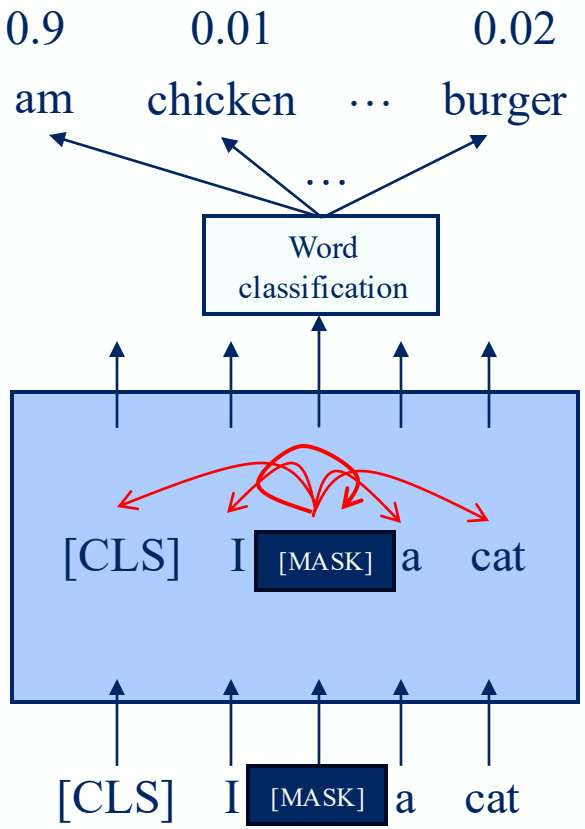
Pretext task: Masked Language Model

- Hide some input with [MASK] token
  - BERT paper: 15% of the input are masked
- Task: classify the masked word



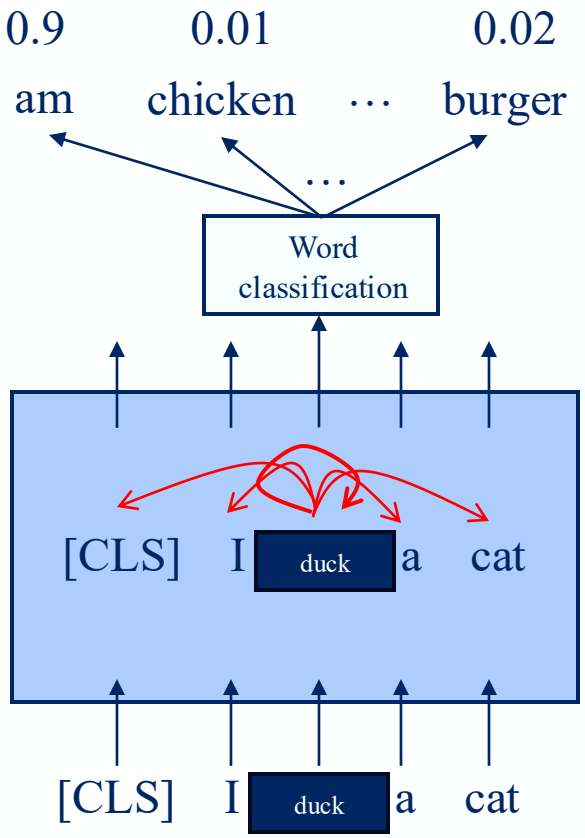
Pretext task: Masked Language Model

- Hide some input with [MASK] token
  - BERT paper: 15% of the input are masked
- Task: classify the masked word
- Model will learn how to see context



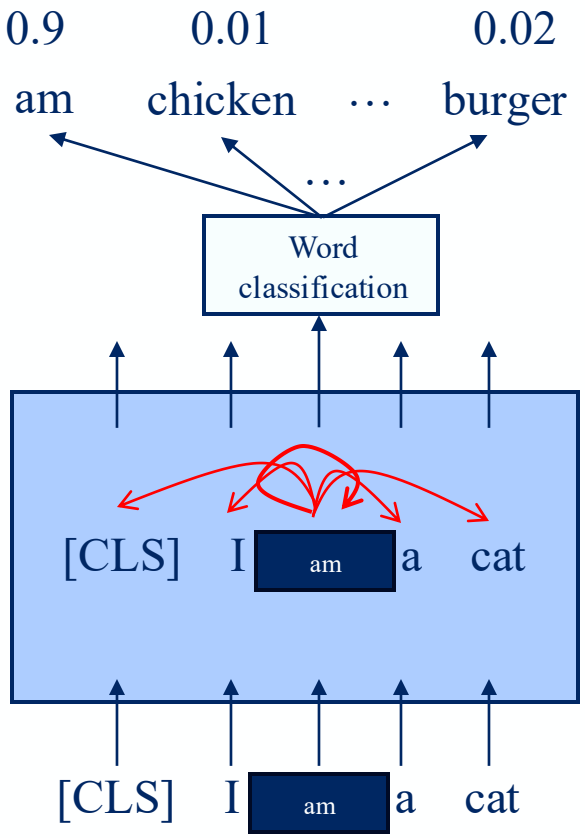
Pretext task: Masked Language Model

- Hide some input with [MASK] token
  - BERT paper: 15% of the input are masked
- Task: classify the masked word
- Model will learn how to see context
- Besides masking:
  - Change to another word

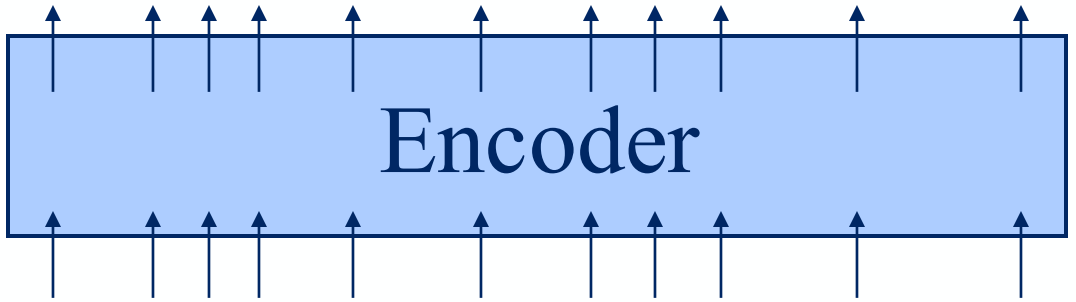


Pretext task: Masked Language Model

- Hide some input with [MASK] token
  - BERT paper: 15% of the input are masked
- Task: classify the masked word
- Model will learn how to see context
- Besides masking:
  - Change to another word
  - Unchanged
    - To not let the model learn trivial solution (i.e, if input token is A the output should not be A → trivial)

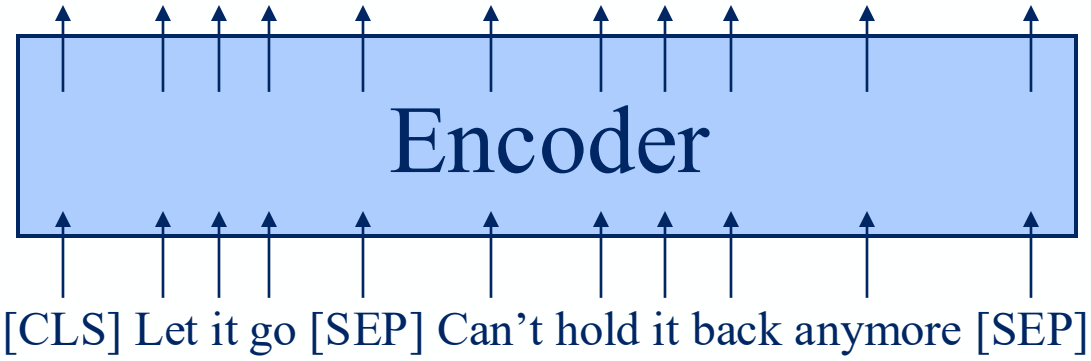


Pretext task: Next Sentence Prediction



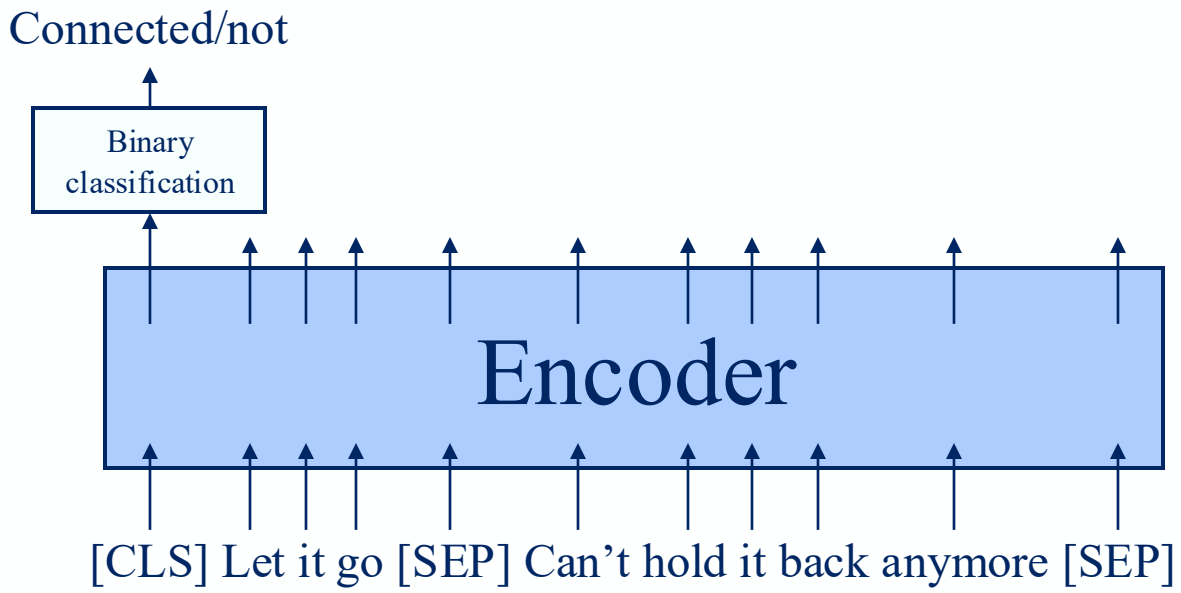
Pretext task: Next Sentence Prediction

- 2 sentences input, separated by [SEP] token



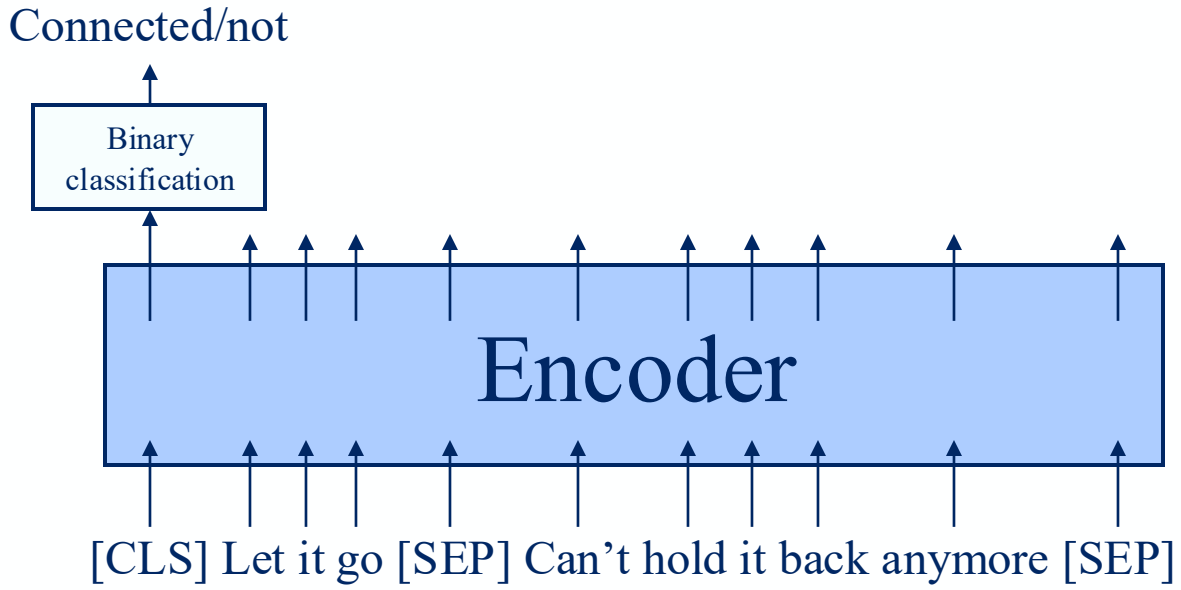
Pretext task: Next Sentence Prediction

- 2 sentences input, separated by [SEP] token
- Task: binary classification, are the 2 sentences connected?



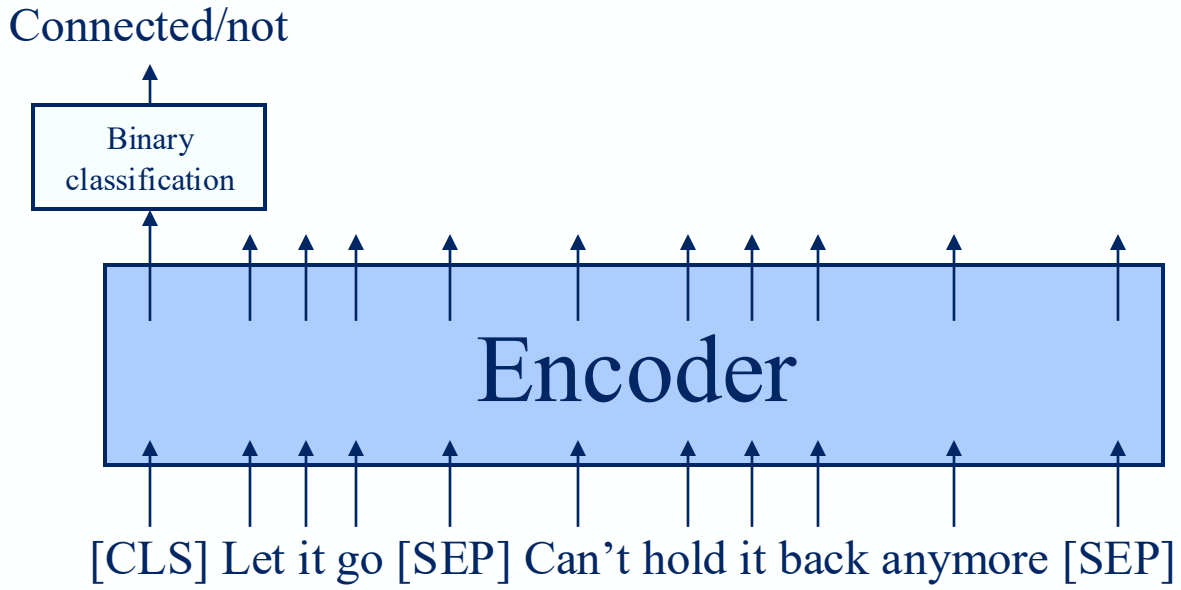
Pretext task: Next Sentence Prediction

- 2 sentences input, separated by [SEP] token
- Task: binary classification, are the 2 sentences connected?
- Model will learn to see connection between 2 sentences



Pretext task: Next Sentence Prediction

- 2 sentences input, separated by [SEP] token
- Task: binary classification, are the 2 sentences connected?
- Model will learn to see connection between 2 sentences
- Good downstream task: question answering
  - Question: 1st sentence
  - Candidate answer: 2nd sentence



Foundation models

Foundation models are *large, pretrained models* that serve as a base (foundation) for many downstream tasks.

- They are pre-trained on *broad and diverse unlabelled data* in a self-supervised manner.
- Extract a generalized representation of the data
- They can be *adapted* (via fine-tuning) to many specific downstream applications.

Bommasani, R., et. Al, *On the Opportunities and Risks of Foundation Models*.

Connection with transformers

Almost all modern foundation models are built using the Transformer architecture.

1. **Scalability:** Transformers train efficiently on massive data and self attention is parallelizable, making it efficient to run on modern GPUs.
2. **Transferability:** Pretrained Transformers adapt well to new task.

Link to the notebook

Next, open this notebook on Google Collab and dive into the hands-on session!

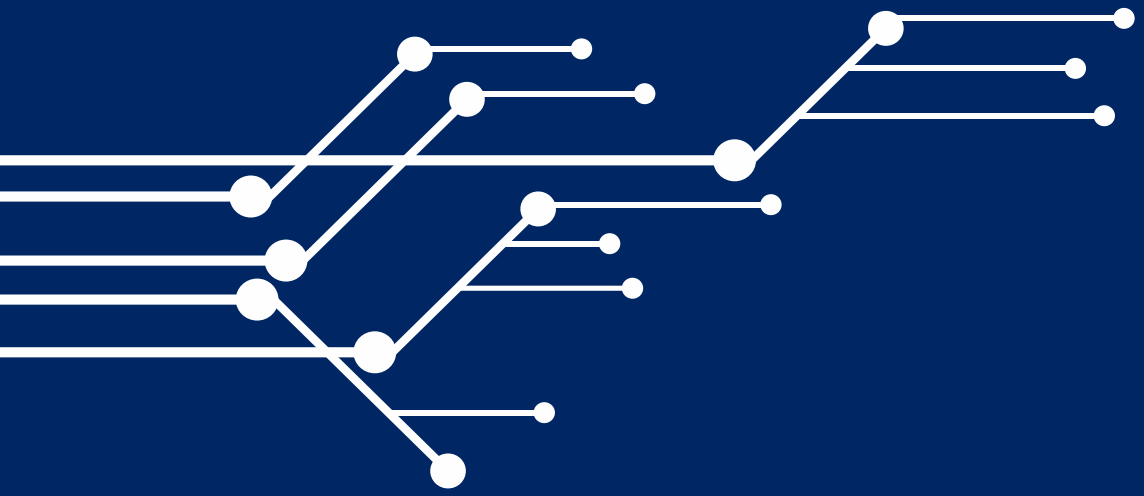
Agenda

09:00 - 10:30	Introduction to Language Modeling
10:30 - 10:45	Break
10:45 - 11:45	Blablador: concept, models description. Optional: integration in VScode
11:45 - 12:00	Q&A
12:00 - 13:00	Lunch Break
13:00 - 14:00	Attention, self-attention, transformer architecture
14:00 - 14:15	Pre-training vs fine-tuning and foundation models (intro to the hands-on session)
14:15 - 14:30	Break
14:30 - 15:30	Hands-on: fine-tune a model on a downstream task
15:30 - 16:00	Q&A, Wrap-up, and Conclusion

Agenda

09:00 - 10:30	Introduction to Language Modeling
10:30 - 10:45	Break
10:45 - 11:45	Blablador: concept, models description. Optional: integration in VScode
11:45 - 12:00	Q&A
12:00 - 13:00	Lunch Break
13:00 - 14:00	Attention, self-attention, transformer architecture
14:00 - 14:15	Pre-training vs fine-tuning and foundation models (intro to the hands-on session)
14:15 - 14:30	Break
14:30 - 15:30	Hands-on: fine-tune a model on a downstream task
15:30 - 16:00	Q&A, Wrap-up, and Conclusion

HELMHOLTZAI | ARTIFICIAL INTELLIGENCE COOPERATION UNIT



Practical Usage

Tips on application

- Start with pre-trained foundational models
  - ViT, BERT, GPT,
- Use existing pipelines for quick prototyping
  - HuggingFace pipeline
- Use mature frameworks to save on implementation time
  - HuggingFace, LangChain, OpenMMLab
- Adjust models for hardware limitations
  - Local computation vs cloud resources
  - Larger models can be overkill!

Challenges and Pitfalls

- Transformer models are larger in size
  - Often require GPUs for inference / training
- Large amount of data is usually necessary to train them
  - Fine-tuning is preferred on small datasets
- High computational cost through API calls
  - Always measure token count for cost estimation

Key Takeaways

- Transformers enabled a breakthrough in language processing.
- Attention + Parallelization are key features in Transformers.
- Transformers have a wide range of applications.
- While using them ensure:
  - You’re using the correct foundation model (task specific)
  - Fine tune the model on relevant data (domain specific)
  - Be aware of the hardware / data requirements of transformers
  - When using cloud services, estimate cost beforehand (measure tokens)

Questionnaire

- Please take a couple of minutes to provide feedback

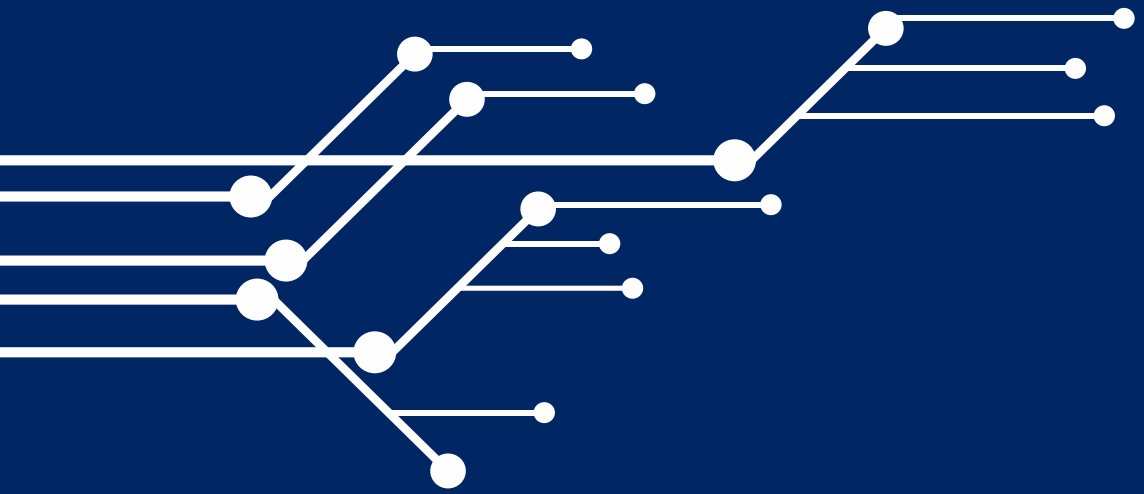
References

- https://xai-tutorials.readthedocs.io/en/latest/_ml_basics/transformer.html

Agenda

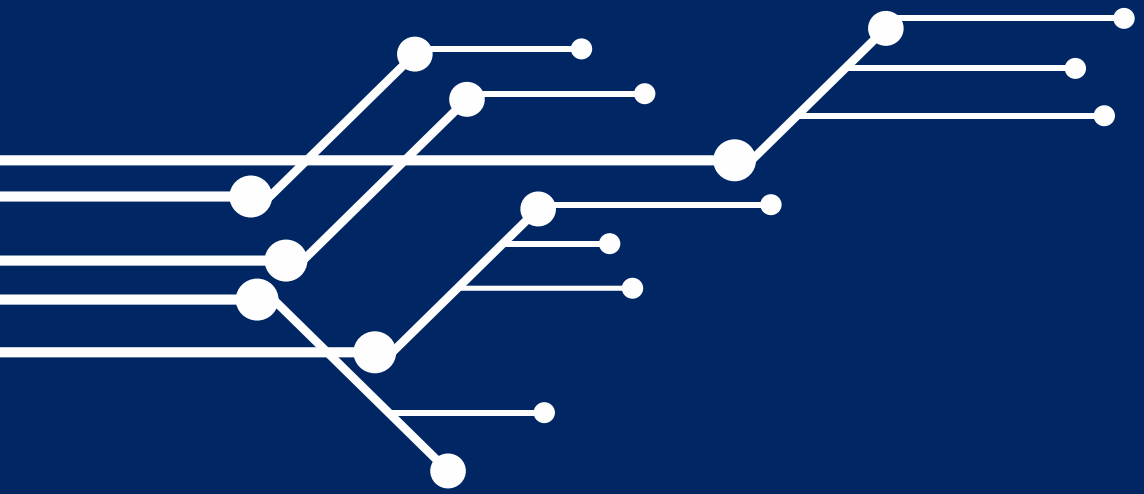
09:00 - 10:30	Introduction to Language Modeling
10:30 - 10:45	Break
10:45 - 11:45	Blablador: concept, models description. Optional: integration in VScode
11:45 - 12:00	Q&A
12:00 - 13:00	Lunch Break
13:00 - 14:00	Attention, self-attention, transformer architecture
14:00 - 14:15	Pre-training vs fine-tuning and foundation models (intro to the hands-on session)
14:15 - 14:30	Break
14:30 - 15:30	Hands-on: fine-tune a model on a downstream task
15:30 - 16:00	Q&A, Wrap-up, and Conclusion

HELMHOLTZAI | ARTIFICIAL INTELLIGENCE COOPERATION UNIT



Thank you

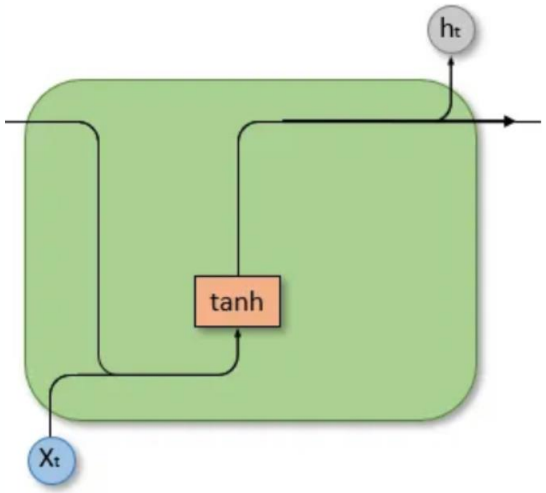
HELMHOLTZAI | ARTIFICIAL INTELLIGENCE COOPERATION UNIT



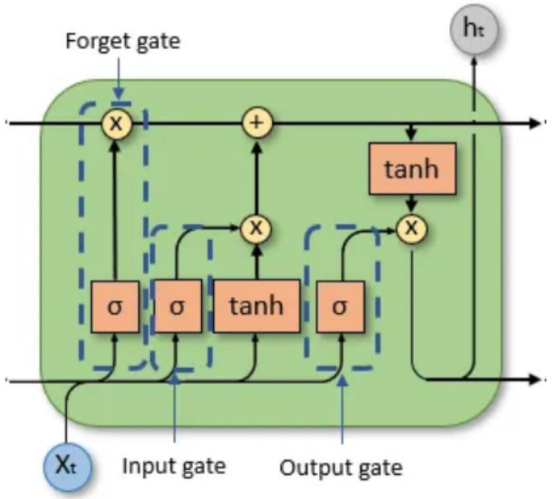
Appendix

Seq-2-seq models

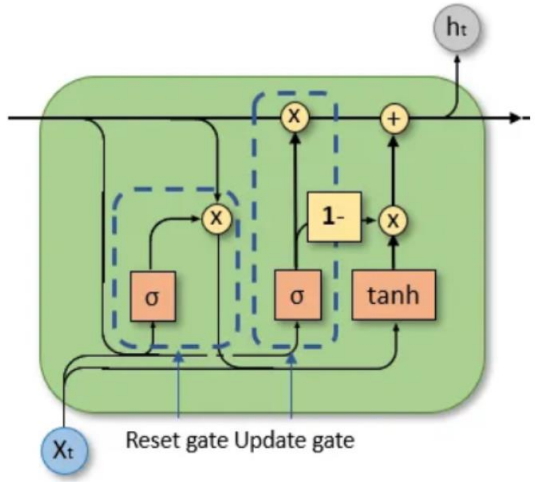
RNN



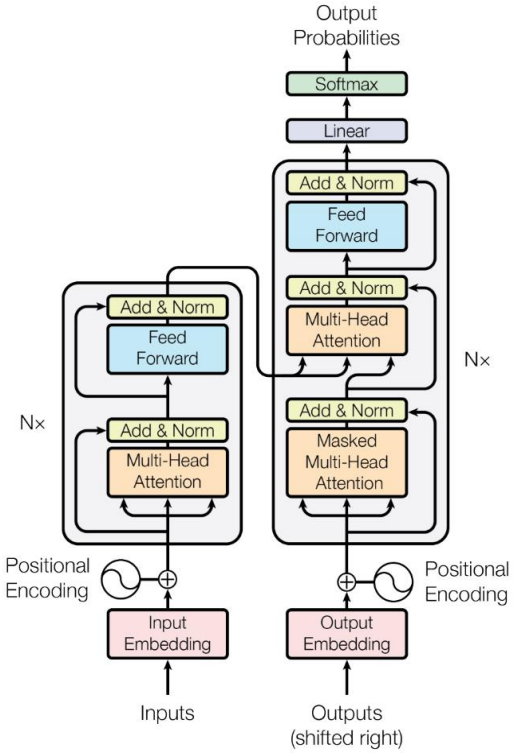
LSTM



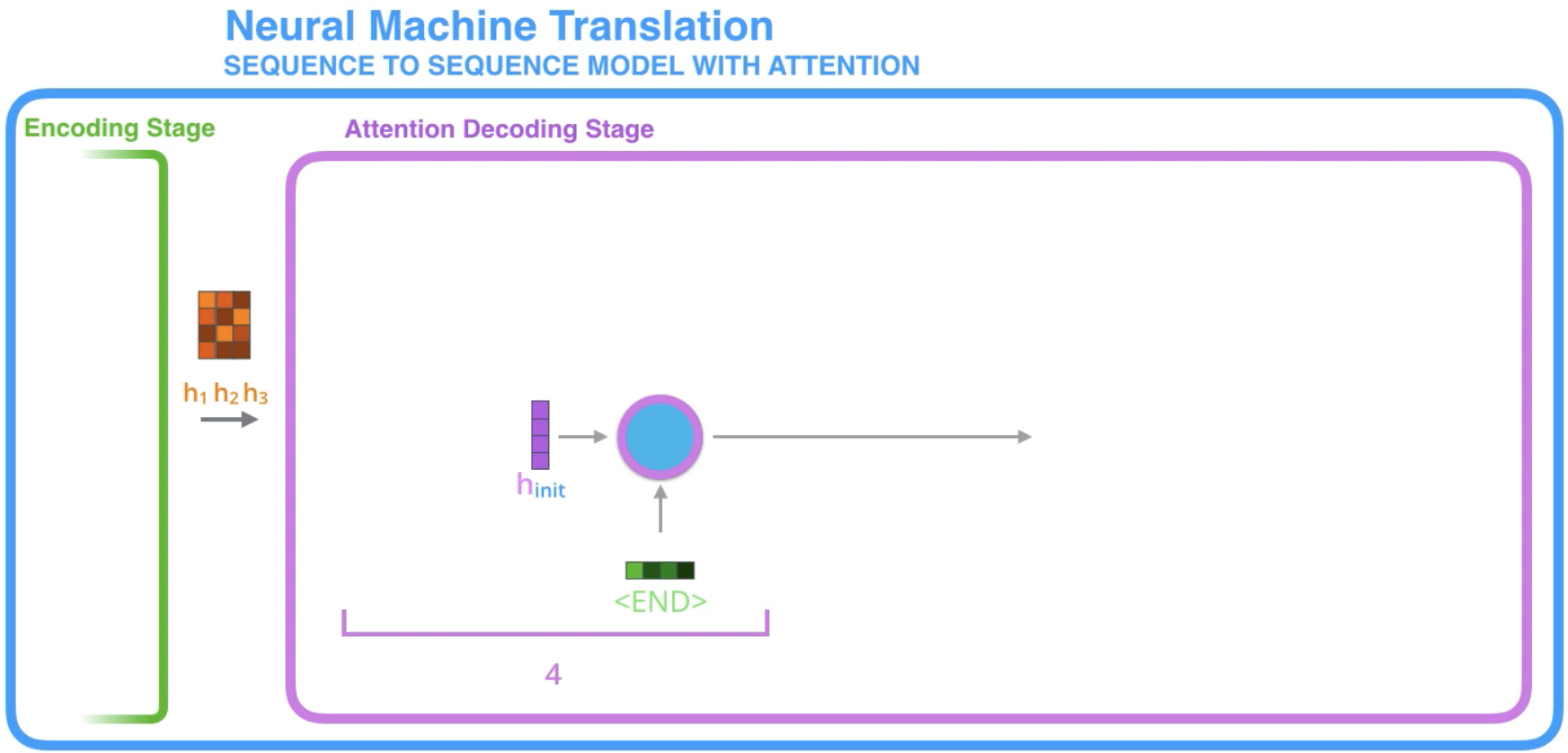
GRU



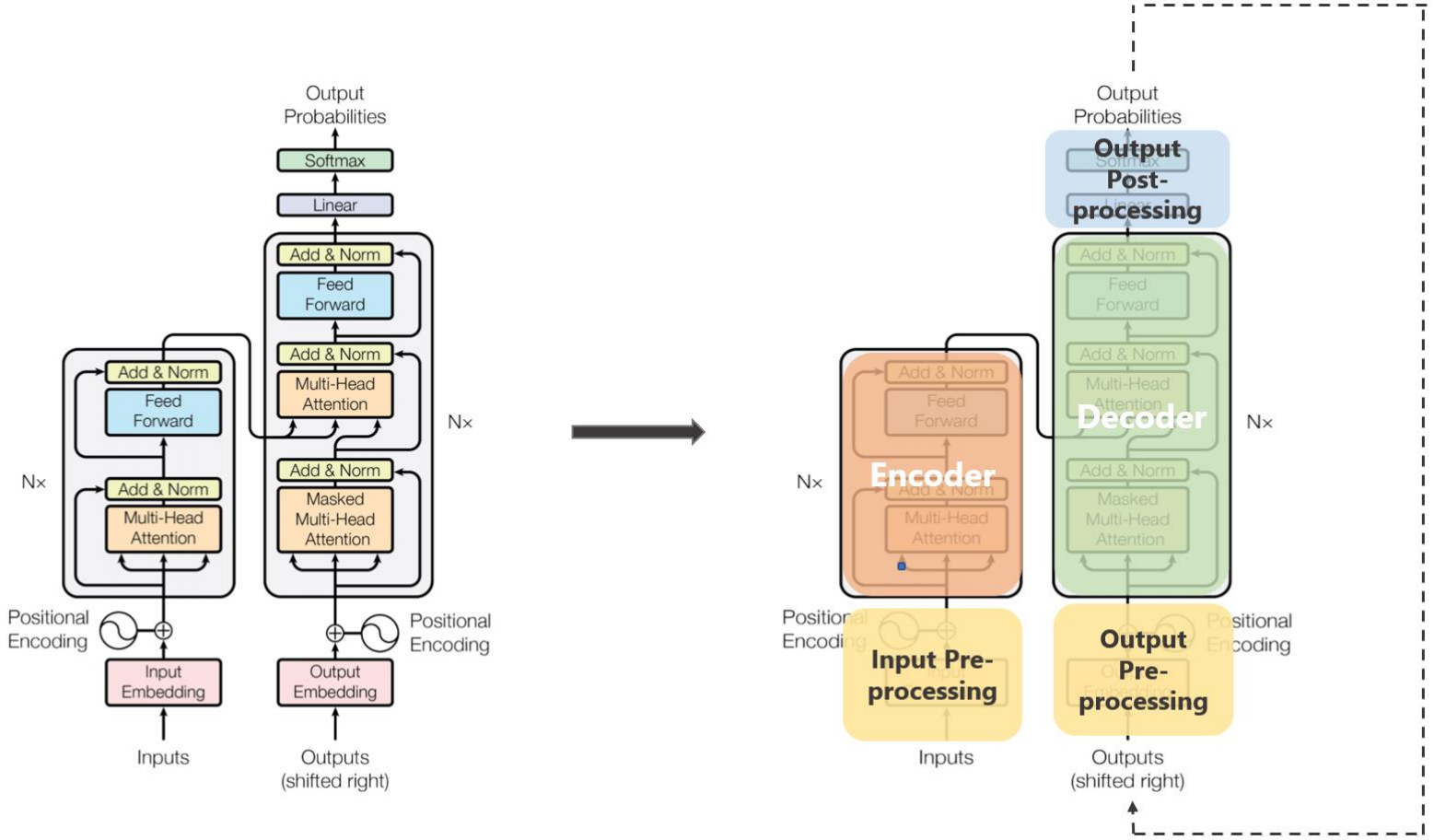
Transformers



Attention Mechanism in RNN



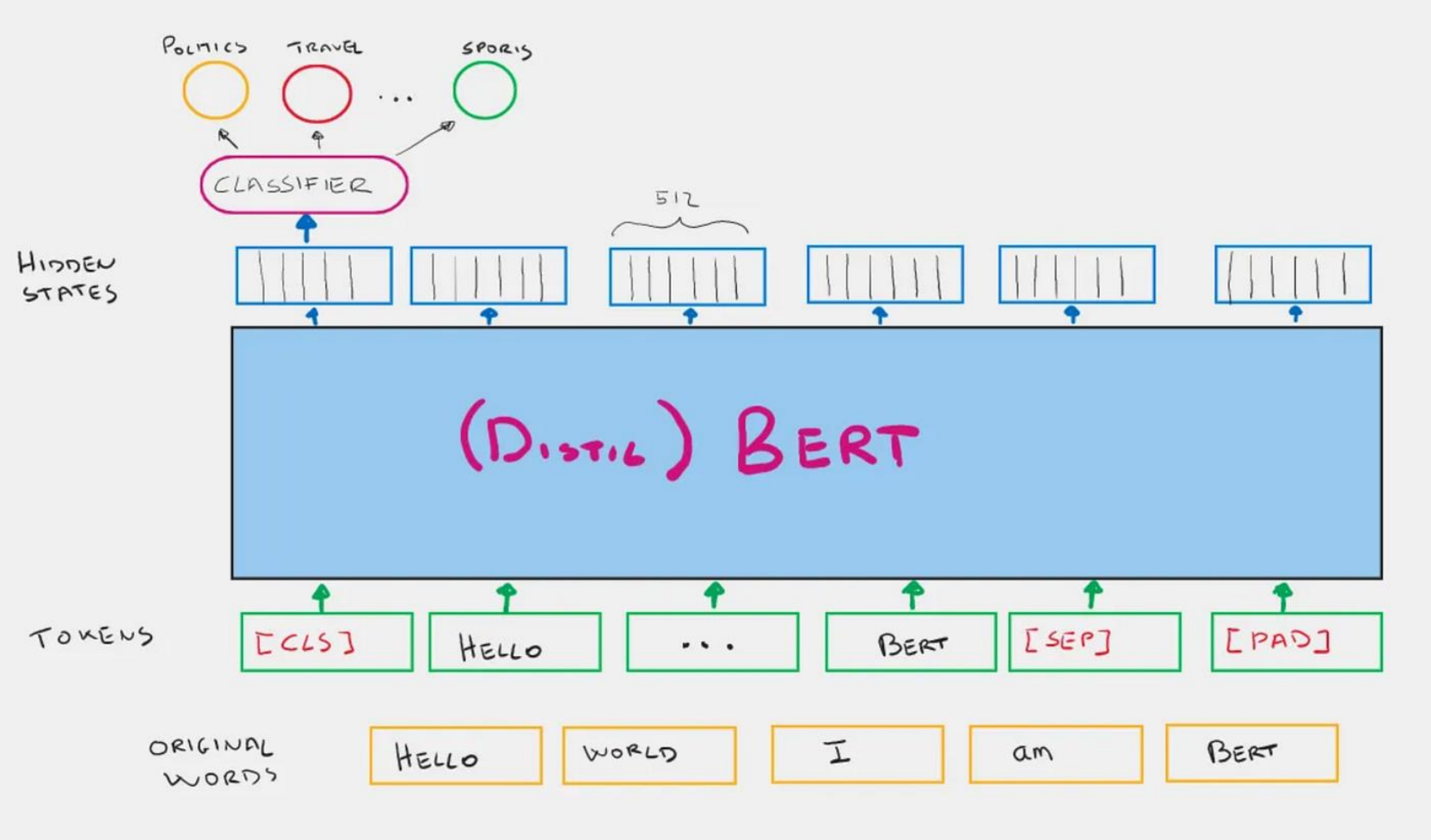
Easy transformer architecture



Question attention

- Attention can learn long term dependencies. Is there a limit to this?
  - How do large conversation models work…
- Why is the attention value scaled?
  - How is this related to batch normalization…
- What is the computation complexity of attention?
  - For a single run, which has higher computational complexity: transformer encoder or RNN encoder

[CLS] token in BERT



Fine-tuning techniques